When to use what

Need	Use
A few related values	Tuple / dict
Record with named fields, no logic	`@dataclass` or namedtuple
Data plus behaviour, state that changes	Class
Share behaviour across related types	Inheritance (or composition)
One function	Just a function

Common mistakes

Mistake	Fix
Forgetting `self` in the method signature	`def method(self, ...)`
Forgetting `self.` when reading an attribute	`self.balance`, not `balance`
Mutable class attribute shared by all objects (`items = []`)	Create it in `__init__`
Overusing inheritance	Prefer composition and small functions
Missing `super().__init__()` in the child	Call it to initialise parent state
Comparing objects without `__eq__`	Compares identity, not content; use a dataclass

Summary

KEY TAKEAWAYS

- A class is a blueprint; `__init__` sets up each object; `self` refers to the current object.
- Dunder methods (`__str__` , `__repr__` , `__eq__` , `__lt__` , `__len__`) make objects behave like built-ins.
- Inheritance reuses and overrides parent behaviour with `super()` ; polymorphism lets one call work across types.
- `@property` guards attributes; `@classmethod` builds alternative constructors; `@staticmethod` groups helpers.
- `@dataclass` removes boilerplate for data holders. Prefer composition over deep inheritance.

Quick quiz

6 questions · tap an answer to check it

1. What does `self` refer to?

A The class

B The specific object the method is called on

C The module

D The parent class

Answer: B. Python passes the instance automatically.

2. Which method runs when you create an object?

A `__str__`

B `__init__`

C `__new__` only

D `__call__`

Answer: B. `__init__` initialises the new instance.

3. What does `super().__init__(owner, balance)` do in a child class?

A Deletes the parent

B Runs the parent's constructor to set up inherited attributes

C Creates a new class

D Nothing

Answer: B. It reuses the parent's initialisation.

4. Which decorator gives a read-only computed attribute?

A `@staticmethod`

B `@property`

C `@classmethod`

D `@cache`

Answer: B. Access without parentheses.

5. What does `@dataclass` generate automatically?

A Only `__str__`

B `__init__` , `__repr__` and `__eq__`

C A database table

D Nothing

Answer: B. It removes boilerplate for data holders.

6. What is polymorphism?

A Having many parents

B Different classes responding to the same method name in their own way

C Copying a class

D Hiding attributes

Answer: B. The loop `for acc in accounts: acc.withdraw()` works for any account type.

EXERCISE **OOP problems**

(1) Create a `Rectangle` class with `area()` and `perimeter()`, and a `Square` subclass. (2) Add `__str__` and `__eq__` to a `Point(x, y)` class. (3) Build a `Stack` class (push, pop, peek, `__len__`). (4) Make a dataclass `Product` with price and `with_gst()`, then sort a list of products by price. (5) Write a `Counter`-like class that counts calls using a class attribute.

Show solution (worked answer)

Python

```
class Rectangle:
    def __init__(self, w, h): self.w, self.h = w, h
    def area(self): return self.w * self.h
    def perimeter(self): return 2 * (self.w + self.h)
class Square(Rectangle):
    def __init__(self, side): super().__init__(side, side)
print(Rectangle(3, 4).area(), Square(5).perimeter())
class Point:
    def __init__(self, x, y): self.x, self.y = x, y
    def __str__(self): return f"({self.x}, {self.y})"
    def __eq__(self, o): return (self.x, self.y) == (o.x, o.y)
print(Point(1, 2), Point(1, 2) == Point(1, 2))
class Stack:
    def __init__(self): self._d = []
    def push(self, x): self._d.append(x)
    def pop(self): return self._d.pop()
    def peek(self): return self._d[-1]
    def __len__(self): return len(self._d)
st = Stack(); st.push(1); st.push(2); print(st.pop(), len(st))
from dataclasses import dataclass
@dataclass(order=True)
class Product:
    price: float
    def with_gst(self): return round(self.price * 1.18, 2)
print(sorted([Product(500), Product(120)]), Product(100).with_gst())
```

INTERVIEW CORNER

Questions you may be asked

Q: Class attribute vs instance attribute? A class attribute is shared by all instances (defined in the class body); an instance attribute belongs to one object (set in `__init__`).

Q: What is the difference between `__str__` and `__repr__`? `__str__` is the readable form for users (`print`); `__repr__` is the unambiguous form for developers, ideally something that could recreate the object.

Q: Explain the four OOP pillars. Encapsulation (bundle and protect data), inheritance (reuse), polymorphism (same interface, different behaviour), abstraction (expose what, hide how).

Q: Inheritance or composition? Use inheritance for a true is-a relationship; use composition (objects holding other objects) for flexibility and lower coupling.

PYTHON FOR DATA SCIENCE · LESSON 12

Files, Exceptions, JSON and CSV

Read and write text, CSV and JSON files safely with pathlib and with-blocks, handle errors with try/except/else/finally, raise your own exceptions, and log what happened, using a small "load a messy file" project.

Python class 10 | Python 3.13 | Intermediate | 110 min

By the end of this lesson you can

- Open, read and write text files with with-blocks and pathlib
- Read and write CSV with the csv module and JSON with the json module
- Handle errors with try, except, else and finally, and catch specific exceptions
- Raise exceptions and define custom exception classes
- Use logging instead of print for programs that run unattended

Files and paths

Use `pathlib.Path` for paths (works on Windows, macOS and Linux) and a `with` block to open files: the file is closed automatically, even if an error happens.

Python

```
from pathlib import Path
folder = Path("demo_files"); folder.mkdir(exist_ok=True)
notes = folder / "notes.txt"                              # / joins path parts
with open(notes, "w", encoding="utf-8") as f:              # "w" overwrites, "a" appends, "r"
reads (default)
    f.write("Line one\n")
    f.writelines(["Line two\n", "Line three\n"])
with open(notes, "a", encoding="utf-8") as f: print("Line four", file=f)
print(notes.read_text(encoding="utf-8"))
print(notes.exists(), notes.name, notes.stem, notes.suffix, notes.stat().st_size, "bytes")
```

OUTPUT

```
Line one
Line two
Line three
Line four
```

```
True notes.txt notes .txt 39 bytes
```

Python

```python
with open(notes, encoding="utf-8") as f:
    for i, line in enumerate(f, 1):                    # iterate line by line: constant memory
for huge files
        print(i, line.rstrip("\n"))
with open(notes, encoding="utf-8") as f: lines = f.read().splitlines()
print(lines, sorted(p.name for p in folder.glob("*.txt")))
```

OUTPUT

```
1 Line one
2 Line two
3 Line three
4 Line four
['Line one', 'Line two', 'Line three', 'Line four'] ['notes.txt']
```

Mode	Meaning
`"r"`	Read (error if missing)
`"w"`	Write (creates or **overwrites**)
`"a"`	Append to the end
`"x"`	Create; error if it exists
`"b"` suffix	Binary (`"rb"` , `"wb"`) for images, Excel, Parquet

> **TIP** **Always give the encoding**
>
> `encoding="utf-8"` avoids garbled ₹, é or Hindi text and avoids differences between Windows and Linux defaults.

CSV files

The standard `csv` module handles quoting and commas inside values. (For analysis you will usually use `pandas.read_csv` , Lesson 16.)

Python

```
import csv
rows = [{"id": 1, "city": "Pune", "note": "fast, friendly"}, {"id": 2, "city": "Delhi", "note":
'said "ok"'}]
with open(folder / "people.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=["id", "city", "note"])
    w.writeheader(); w.writerows(rows)
print((folder / "people.csv").read_text(encoding="utf-8"))
with open(folder / "people.csv", newline="", encoding="utf-8") as f:
    for r in csv.DictReader(f): print(r)                # every value is a string
```

OUTPUT

```
id,city,note
1,Pune,"fast, friendly"
2,Delhi,"said ""ok"""

{'id': '1', 'city': 'Pune', 'note': 'fast, friendly'}
{'id': '2', 'city': 'Delhi', 'note': 'said "ok"'}
```

Python

```
# Stream a real 6,000-line CSV without pandas: total sales per region
from collections import defaultdict
totals = defaultdict(float)
with open("superstore_sales.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f): totals[row["region"]] += float(row["sales"])
print({k: round(v / 1e6, 1) for k, v in sorted(totals.items())})
```

OUTPUT

```
{'East': 46.5, 'North': 49.0, 'South': 54.0, 'West': 52.0}
```

JSON files

JSON maps to Python like this: object ↔ dict, array ↔ list, string ↔ str, number ↔ int/float, true/false ↔ True/False, null ↔ None.

Python

```
import json
config = {"source": "superstore_sales.csv", "regions": ["North", "South"], "min_sales": 1000,
"debug": False, "owner": None}
with open(folder / "config.json", "w", encoding="utf-8") as f: json.dump(config, f, indent=2)
with open(folder / "config.json", encoding="utf-8") as f: loaded = json.load(f)
print(loaded == config, loaded["regions"], json.dumps({"price": 4500.5, "ok": True, "x": None}))
print(json.dumps({"city": "Pune", "₹": 1}, ensure_ascii=False))
```

OUTPUT

```
True ['North', 'South'] {"price": 4500.5, "ok": true, "x": null}
{"city": "Pune", "₹": 1}
```

`json.dump` / `json.load` work with files; `json.dumps` / `json.loads` work with strings (the "s" is for string). Dates and sets are not JSON types: convert them first (`d.isoformat()` , `list(s)`).

Exceptions

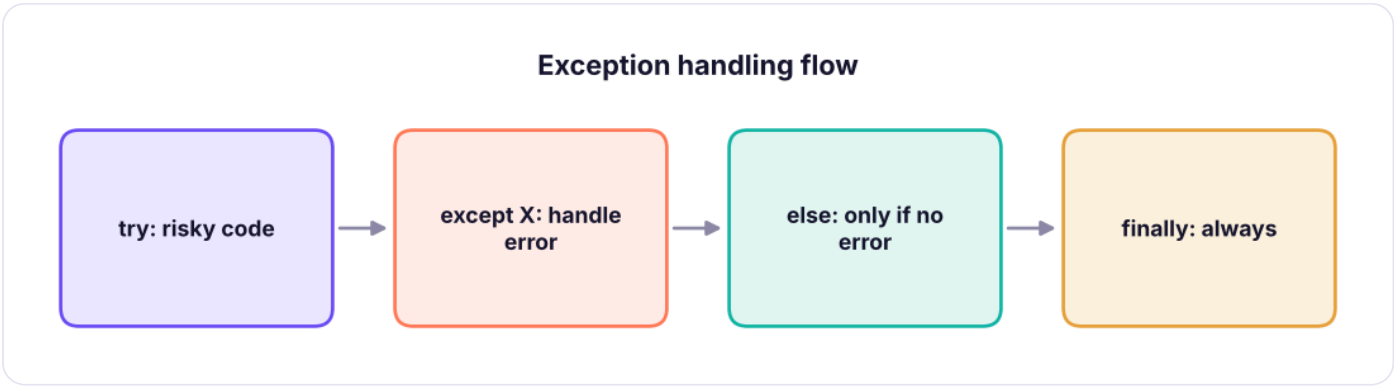
An **exception** is Python's way of reporting an error. Unhandled, it stops the program and prints a traceback. Handled, your program can recover.

Python

```python
def safe_ratio(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("  cannot divide by zero")
        return None
    except TypeError as e:
        print("  wrong types:", e)
        return None
    else:                       # runs only when no exception happened
        return round(result, 2)
    finally:                    # always runs: clean-up
        print("  checked", a, b)
print(safe_ratio(10, 4)); print(safe_ratio(1, 0)); print(safe_ratio("10", 2))
```

OUTPUT

```
  checked 10 4
2.5
  cannot divide by zero
  checked 1 0
None
  wrong types: unsupported operand type(s) for /: 'str' and 'int'
  checked 10 2
None
```



try / except / else / finally: which blocks run.

Common built-in exceptions

Exception	Typical cause
`ValueError`	Right type, wrong value: `int("abc")`
`TypeError`	Wrong type: `"a" + 1`
`KeyError`	Missing dict key
`IndexError`	List index out of range
`FileNotFoundError`	Missing file
`ZeroDivisionError`	Divide by zero
`AttributeError`	Object has no such attribute or method
`ImportError` / `ModuleNotFoundError`	Package not installed

Python

```
tests = [lambda: int("abc"), lambda: "a" + 1, lambda: {}["x"], lambda: [1][5], lambda:
open("missing.csv"), lambda: None.upper()]
for t in tests:
    try: t()
    except Exception as e: print(f"{type(e).__name__:<18} {e}")
```

OUTPUT

```
ValueError         invalid literal for int() with base 10: 'abc'
TypeError          can only concatenate str (not "int") to str
KeyError           'x'
IndexError         list index out of range
FileNotFoundError  [Errno 2] No such file or directory: 'missing.csv'
AttributeError     'NoneType' object has no attribute 'upper'
```

WATCH OUT **Do not swallow every error**

`except: pass` or `except Exception: pass` hides bugs. Catch the **specific** exceptions you expect, and let unexpected ones surface (or log them and re-raise).

Raising your own exceptions

Python

```
class DataQualityError(Exception):
    """Raised when an input file fails validation."""

def validate(df):
    missing = {"order_id", "sales"} - set(df.columns)
    if missing: raise DataQualityError(f"missing columns: {sorted(missing)}")
    if (df["sales"] < 0).any(): raise ValueError("negative sales found")
    return True

import pandas as pd
for d in (pd.DataFrame({"order_id": [1], "sales": [5]}), pd.DataFrame({"order_id": [1]}),
pd.DataFrame({"order_id": [1], "sales": [-2]})):
    try: print("valid:", validate(d))
    except (DataQualityError, ValueError) as e: print(type(e).__name__, "→", e)
```

OUTPUT

```
valid: True
DataQualityError → missing columns: ['sales']
ValueError → negative sales found
```

Use `raise ... from e` to wrap a low-level error in a clearer one while keeping the original cause.

Logging

`print` is for interactive work. Programs that run on a schedule (ETL jobs, APIs) should use `logging`, which adds timestamps and levels and can write to files.

Python

```
import logging, io
stream = io.StringIO()
log = logging.getLogger("etl"); log.handlers.clear(); log.setLevel(logging.INFO)
h = logging.StreamHandler(stream); h.setFormatter(logging.Formatter("%(levelname)s | %(name)s |
%(message)s")); log.addHandler(h)
log.debug("not shown: below INFO"); log.info("loaded 6,323 rows"); log.warning("12 rows had
blank city")
try: 1 / 0
except ZeroDivisionError: log.exception("step failed")       # logs the traceback too
print(stream.getvalue().splitlines()[:3])
```

OUTPUT

```
['INFO | etl | loaded 6,323 rows', 'WARNING | etl | 12 rows had blank city', 'ERROR | etl
| step failed']
```

Levels from low to high: `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`.

Project: load a messy file safely

Python

```
messy = """order_id,amount,city
1,1200,Pune
2,abc,Delhi
3,,Kochi
4,950,
5,-40,Goa
"""
(folder / "messy.csv").write_text(messy, encoding="utf-8")

def load_orders(path):
    good, bad = [], []
    with open(path, newline="", encoding="utf-8") as f:
        for lineno, row in enumerate(csv.DictReader(f), start=2):
            try:
                amount = float(row["amount"])
                if amount < 0: raise ValueError("negative amount")
                if not row["city"]: raise ValueError("blank city")
                good.append({"order_id": int(row["order_id"]), "amount": amount, "city":
row["city"]})
            except ValueError as e:
                bad.append((lineno, str(e)))
    return good, bad

good, bad = load_orders(folder / "messy.csv")
print("good:", good); print("rejected:", bad)
with open(folder / "rejected.json", "w", encoding="utf-8") as f: json.dump(bad, f)
import shutil; shutil.rmtree(folder)                    # tidy up the demo folder
```

OUTPUT

```
good: [{'order_id': 1, 'amount': 1200.0, 'city': 'Pune'}]
rejected: [(3, "could not convert string to float: 'abc'"), (4, "could not convert string
to float: ''"), (5, 'blank city'), (6, 'negative amount')]
```

Real pipelines do exactly this: keep the good rows, quarantine and report the bad ones, never crash on one bad line.

Summary

KEY TAKEAWAYS

- Use `pathlib.Path` and `with open(path, mode, encoding="utf-8")` ; iterate a file line by line for big files.
- `csv.DictReader` / `DictWriter` handle quoting (values are strings); `json.load` / `dump` convert JSON ↔ Python.
- `try` / `except SpecificError` / `else` / `finally` ; never use a bare `except: pass` .
- `raise` your own exceptions (subclass `Exception`) to signal bad data clearly.
- Use `logging` with levels for unattended programs.

Quick quiz

6 questions · tap an answer to check it

1. Why use `with open(...) as f:` ?

A It is faster

B The file is closed automatically, even if an error occurs

C It reads the whole file

D It is required for CSV

Answer: B. The context manager handles clean-up.

2. What does mode `"w"` do to an existing file?

A Appends

B Overwrites it

C Raises an error

D Reads it

Answer: B. Use `"a"` to append and `"x"` to refuse overwriting.

3. When does the `else` block of a `try` statement run?

A Always

B Only if no exception was raised in `try`

C Only after an exception

D Never

Answer: B. `finally` is the block that always runs.

4. What does `json.loads` take?

A A file

B A string

C A dictionary

D A path

Answer: B. `load` takes a file object; `loads` takes a string.

5. What exception does `int("abc")` raise?

- **A** TypeError
- **B** ValueError
- **C** KeyError
- **D** SyntaxError

Answer: B. The type (str) is acceptable, the value is not.

6. Why prefer `logging` over `print` for scheduled jobs?

- **A** It is shorter
- **B** It adds levels and timestamps and can write to files and monitoring tools
- **C** print is deprecated
- **D** logging is faster

Answer: B. You can filter by level and keep a history.

EXERCISE **File and error problems**

(1) Write a function that counts lines, words and characters in a text file. (2) Read `superstore_sales.csv` with `csv.DictReader` and count rows per `category`. (3) Save that result as JSON and load it back. (4) Write `to_int(text, default=0)` that returns the default on `ValueError`. (5) Raise a custom `EmptyFileError` when a file has no data rows.

Show solution (worked answer)

Python

```
def wc(path):
    text = Path(path).read_text(encoding="utf-8")
    return len(text.splitlines()), len(text.split()), len(text)
from collections import Counter
with open("superstore_sales.csv", newline="", encoding="utf-8") as f:
    counts = Counter(r["category"] for r in csv.DictReader(f))
Path("cat.json").write_text(json.dumps(counts), encoding="utf-8")
print(json.loads(Path("cat.json").read_text(encoding="utf-8")))
def to_int(text, default=0):
    try: return int(text)
    except ValueError: return default
class EmptyFileError(Exception): pass
def check(path):
    with open(path, encoding="utf-8") as f:
        if len(f.read().splitlines()) < 2: raise EmptyFileError(path)
```

INTERVIEW CORNER **Questions you may be asked**

Q: How do you read a 10 GB file in Python? Iterate it line by line (`for line in f`), or read in chunks (`pd.read_csv(..., chunksize=...)`), so memory stays constant.

Q: Difference between `except Exception` and a bare `except`? A bare `except` also catches `KeyboardInterrupt` and `SystemExit`; catch specific exceptions where possible.

Q: What is `finally` used for? Clean-up that must always happen, such as closing connections or releasing locks.

Q: How do you handle bad rows in an ETL job? Validate each row, keep good rows, write rejects with the reason to a quarantine file or table, log counts, and alert if the reject rate is too high.

PYTHON FOR DATA SCIENCE · LESSON 13

Modules, Packages, Virtual Environments and Git

Organise code into modules and packages, install libraries with pip or uv inside virtual environments, pin dependencies, use the standard library, and version-control your projects with Git and GitHub.

Python class 10 | ✦ Python 3.13 · pip 25 · uv · Git 2.5x | Intermediate | ⏱ 90 min

By the end of this lesson you can

- Import modules in the different ways and explain if **name** == "**main**"
- Create a package with **init**.py and a sensible project layout
- Create virtual environments and install and pin packages with pip or uv
- Use key standard-library modules (os, pathlib, datetime, random, math, statistics, collections)
- Use the everyday Git workflow: init, add, commit, branch, push, pull request

Modules

A **module** is any `.py` file. Importing it gives you its functions, classes and variables.

Python

```
import math                                  # whole module, use with prefix
from statistics import mean, median          # specific names
import datetime as dt                        # alias
from collections import Counter as C
print(math.sqrt(144), math.pi, mean([3, 5, 10]), median([3, 5, 10]), dt.date(2025, 4,
1).strftime("%d %b %Y"), C("aab"))
print(math.ceil(4.2), math.floor(4.8), math.log10(1000), round(math.exp(1), 4), math.comb(5, 2))
```

OUTPUT

```
12.0 3.141592653589793 6 5 01 Apr 2025 Counter({'a': 2, 'b': 1})
5 4 3.0 2.7183 10
```

Style	When
`import pandas as pd`	Standard aliases (`np` , `pd` , `plt` , `sns`)
`from pathlib import Path`	A few names used often
`from module import *`	**Avoid**: hides where names come from

Writing your own module

Python

```
from pathlib import Path
Path("metrics.py").write_text('''"""Reusable business metrics."""
GST = 0.18

def margin_pct(profit, sales):
    return round(100 * profit / sales, 1) if sales else None

def with_gst(amount):
    return round(amount * (1 + GST), 2)

if __name__ == "__main__":          # runs only when executed directly, not when imported
    print("self-test:", margin_pct(25, 200))
''', encoding="utf-8")
import importlib, sys
if "" not in sys.path: sys.path.insert(0, "")        # notebooks and scripts already search the
current folder
import metrics
importlib.reload(metrics)
print(metrics.margin_pct(30, 120), metrics.with_gst(1000), metrics.__doc__)
import subprocess, sys
print(subprocess.run([sys.executable, "metrics.py"], capture_output=True,
text=True).stdout.strip())
Path("metrics.py").unlink()
```

OUTPUT

```
25.0 1180.0 Reusable business metrics.
self-test: 12.5
```

`__name__` is `"__main__"` when the file is run as a script and the module name when imported. The guard lets one file be both a library and a script.

Where does Python look?

The current folder, then the folders in `sys.path` (including `site-packages` , where pip installs libraries). `ModuleNotFoundError` usually means: wrong environment, package not installed, or a typo. Never name your own file `pandas.py` or `random.py` : it will shadow the real library.

Packages and project layout

A **package** is a folder of modules (with an `__init__.py`). A clean data project:

Text

```
churn-project/
├── pyproject.toml          # project metadata and dependencies
├── README.md
├── .gitignore              # venv, data, secrets, __pycache__
├── data/                   # raw/ and processed/ (usually not committed)
├── notebooks/              # exploration
├── src/churn/
│   ├── __init__.py
│   ├── load.py             # read and validate data
│   ├── features.py         # feature engineering
│   └── train.py            # model training
└── tests/
    └── test_features.py
```

Import with `from churn.features import add_tenure_band`. Notebooks for exploring; `.py` modules for anything reused or scheduled.

The standard library tour

Python

```
import os, random, statistics, datetime as dt, itertools, textwrap, uuid, time
random.seed(42)
print(random.randint(1, 6), random.choice(["N", "S", "E"]), random.sample(range(100), 3),
round(random.random(), 3))
print(statistics.stdev([2, 4, 4, 4, 5, 5, 7, 9]), statistics.mode([1, 2, 2, 3]))
today = dt.date(2025, 10, 27); due = today + dt.timedelta(days=45)
print(due, due.strftime("%A"), (dt.date(2026, 1, 1) - today).days, "days to new year")
print(os.cpu_count() > 0, os.getenv("HOME") is not None, textwrap.shorten("A very long product
description here", 20), len(str(uuid.uuid4())))
```

OUTPUT

```
6 N [3, 94, 35] 0.245
2.138089935299395 2
2025-12-11 Thursday 66 days to new year
True True A very long [...] 36
```

Module	Use
`pathlib`, `os`, `shutil`, `glob`	Files and folders
`datetime`, `zoneinfo`, `time`	Dates, time zones, timing
`math`, `statistics`, `random`, `decimal`	Numbers (`decimal` for money)
`collections`, `itertools`, `functools`	Data structures and iteration helpers
`csv`, `json`, `sqlite3`, `pickle`	Data formats and storage
`re`	Regular expressions
`logging`, `argparse`, `subprocess`	Scripts and tools
`urllib`, `http`	Web (most people use `requests` / `httpx`)
`concurrent.futures`, `asyncio`	Parallel and asynchronous work

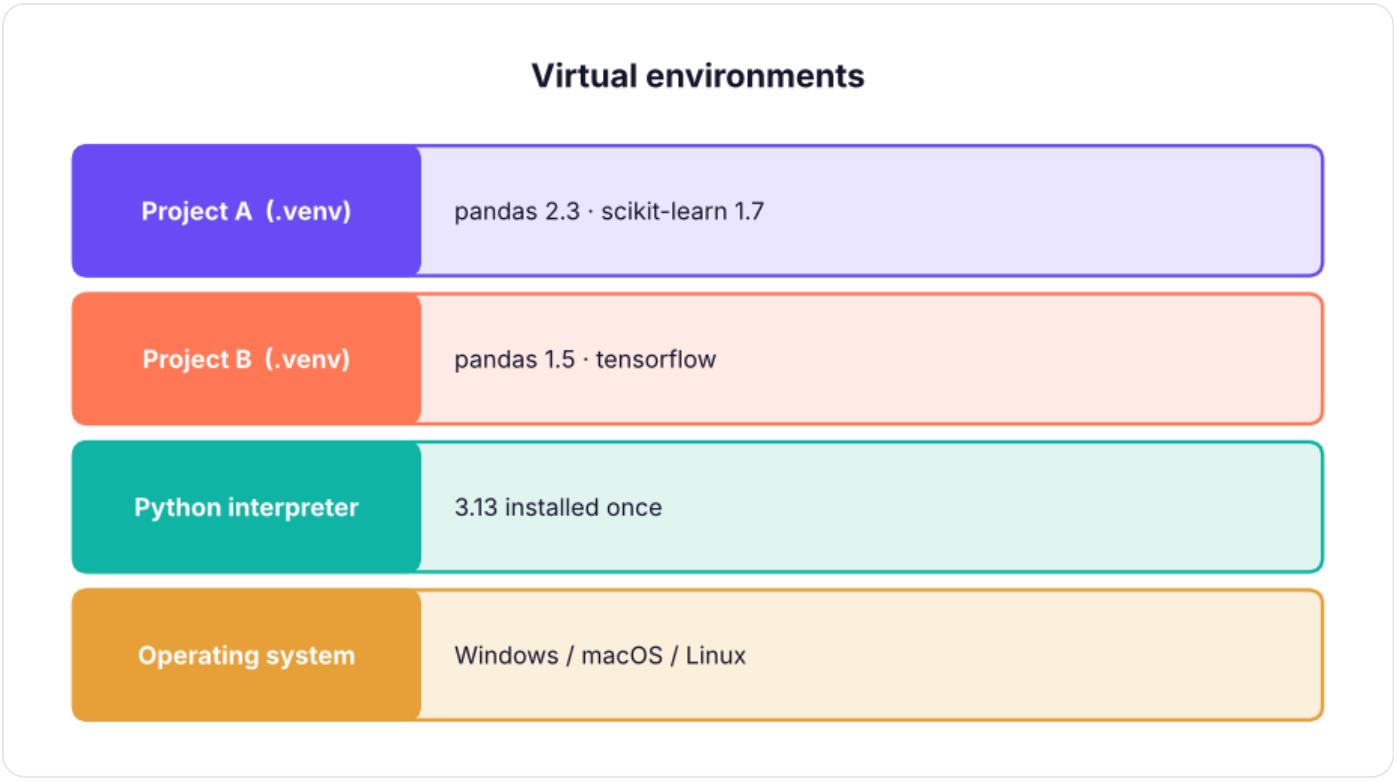
pip, virtual environments and uv

Each project should have its **own virtual environment** so its library versions do not clash with other projects.

```text
# Create and activate an environment (built-in venv)
python -m venv .venv
source .venv/bin/activate          # macOS/Linux
.venv\Scripts\activate             # Windows

# Install, list, pin, reproduce
pip install pandas scikit-learn matplotlib
pip list
pip freeze > requirements.txt
pip install -r requirements.txt

# uv: a much faster modern alternative (by Astral)
uv init churn-project && cd churn-project
uv add pandas scikit-learn
uv run python src/churn/train.py
```



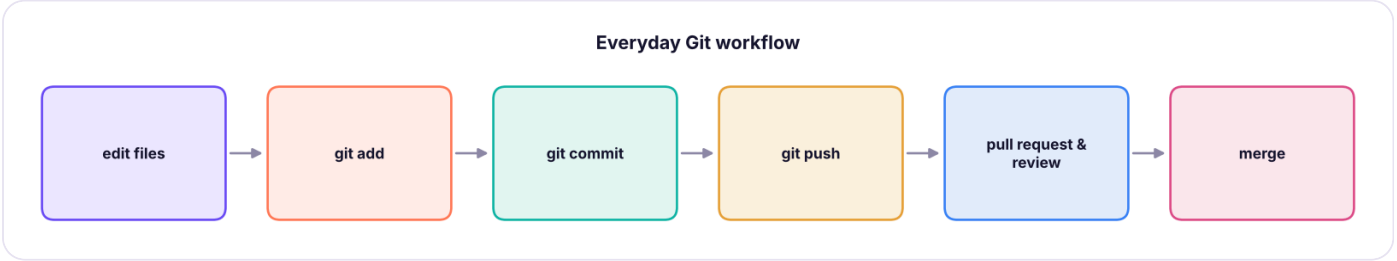
The project isolation model: each project gets its own environment and its own library versions.

TIP **Conda, Anaconda, Colab**

Anaconda/Miniconda bundle Python with data libraries and the `conda` environment manager; popular in data science courses. **Google Colab** and **Kaggle Notebooks** run Python in the browser with libraries pre-installed: ideal for practice. Pick one tool per project and stick to it.

Git and GitHub essentials

Git tracks changes to files; **GitHub** hosts repositories online for backup, collaboration and your portfolio.



The everyday Git cycle.

```text
git init                              # start tracking a folder
git status                            # what changed?
git add analysis.py                   # stage a file  (git add . = everything)
git commit -m "Add churn EDA"         # save a snapshot with a message
git log --oneline                     # history
git switch -c feature/rfm             # new branch
git push -u origin feature/rfm        # upload the branch to GitHub
git pull                              # download others' changes
git clone https://github.com/user/repo.git
git diff                              # unstaged changes
git restore file.py                   # discard local changes to a file
```

A `.gitignore` keeps junk and secrets out:

```text
.venv/
__pycache__/
.ipynb_checkpoints/
data/raw/
*.env
.DS_Store
```

WATCH OUT **Never commit secrets**

API keys, passwords and tokens belong in environment variables or a `.env` file listed in `.gitignore`. Read them with `os.getenv("OPENAI_API_KEY")`. If a key is ever pushed, revoke it immediately: deleting the commit is not enough.

Summary

KEY TAKEAWAYS

- A module is a `.py` file; a package is a folder of modules. Use standard aliases and avoid `import *`.
- `if __name__ == "__main__":` separates script code from importable code.
- One virtual environment per project; pin versions in `requirements.txt` or `pyproject.toml` (pip, uv or conda).
- The standard library covers files, dates, maths, data formats, logging and more.
- Git: add → commit → push → pull request → merge; use `.gitignore` and never commit secrets.

Quick quiz

6 questions · tap an answer to check it

1. What does `if __name__ == "__main__":` do?

- **A** Imports main
- **B** Runs the block only when the file is executed directly
- **C** Defines the main function
- **D** Speeds up the program

Answer: B. When imported, `__name__` is the module name.

2. Why use a virtual environment?

- **A** It makes Python faster
- **B** It isolates each project's library versions
- **C** It is required by Git
- **D** It encrypts the code

Answer: B. Different projects may need different versions.

3. How do you record the exact installed versions for others?

- **A** `pip list`
- **B** `pip freeze > requirements.txt`
- **C** `git add .`
- **D** `pip update`

Answer: B. Others recreate with `pip install -r requirements.txt`.

4. Why should you not name your file `pandas.py`?

A It is too long

B It shadows the real pandas library on import

C Git rejects it

D It is reserved

Answer: B. Python finds your file first.

5. Which Git command uploads your commits to GitHub?

A `git commit`

B `git push`

C `git add`

D `git log`

Answer: B. Commit saves locally; push uploads.

6. Where should API keys go?

A In the code

B In environment variables or an ignored .env file

C In the README

D In a commit message

Answer: B. Never commit secrets.

EXERCISE

Setup checklist

(1) Create a project folder with a virtual environment and install pandas and matplotlib. (2) Write a module `cleaning.py` with two functions and import it from a notebook. (3) Create a GitHub repository, add a `.gitignore` and a README, and push. (4) Create a branch, change the README and open a pull request to yourself.

Show solution (worked answer)

Follow the commands in this lesson. Check with `python -c "import pandas; print(pandas.__version__)"` and `git log --oneline`. A good README states the question, the data source, how to run the code and the key findings with one chart.

INTERVIEW CORNER

Questions you may be asked

Q: Difference between a module, a package and a library? A module is one file, a package is a folder of modules, and a library is a collection of packages distributed together (e.g. on PyPI).

Q: How do you make a Python project reproducible? Virtual environment, pinned dependencies, a README with run instructions, fixed random seeds, version control, and data lineage.

Q: git merge vs git rebase? Merge combines histories with a merge commit; rebase replays your commits on top of another branch for a linear history. Never rebase shared branches.

PYTHON FOR DATA SCIENCE · LESSON 14

Debugging, Testing, Regular Expressions and Code Quality

Find and fix bugs systematically, write automated tests with pytest, clean text with regular expressions, and keep code readable with PEP 8, type hints, linters and formatters.

Python class 10 · Python 3.13 · pytest 9 · ruff · Intermediate · 100 min

By the end of this lesson you can

- Read a traceback and debug with print, breakpoints and pdb
- Write and run unit tests with assert and pytest, including parametrised tests
- Use regular expressions to search, extract, validate and replace text
- Apply PEP 8 naming and layout, type hints, and automatic tools (ruff, black, mypy)
- Use AI assistants to debug and review code responsibly

Reading a traceback

Read a traceback **from the bottom up**: the last line names the error, the lines above show where it happened.

Python

```
def average(values):
    return sum(values) / len(values)
def report(data):
    return f"Average: {average(data['scores']):.1f}"
print(report({"scores": []}))
```

OUTPUT

```
ZeroDivisionError: division by zero
```

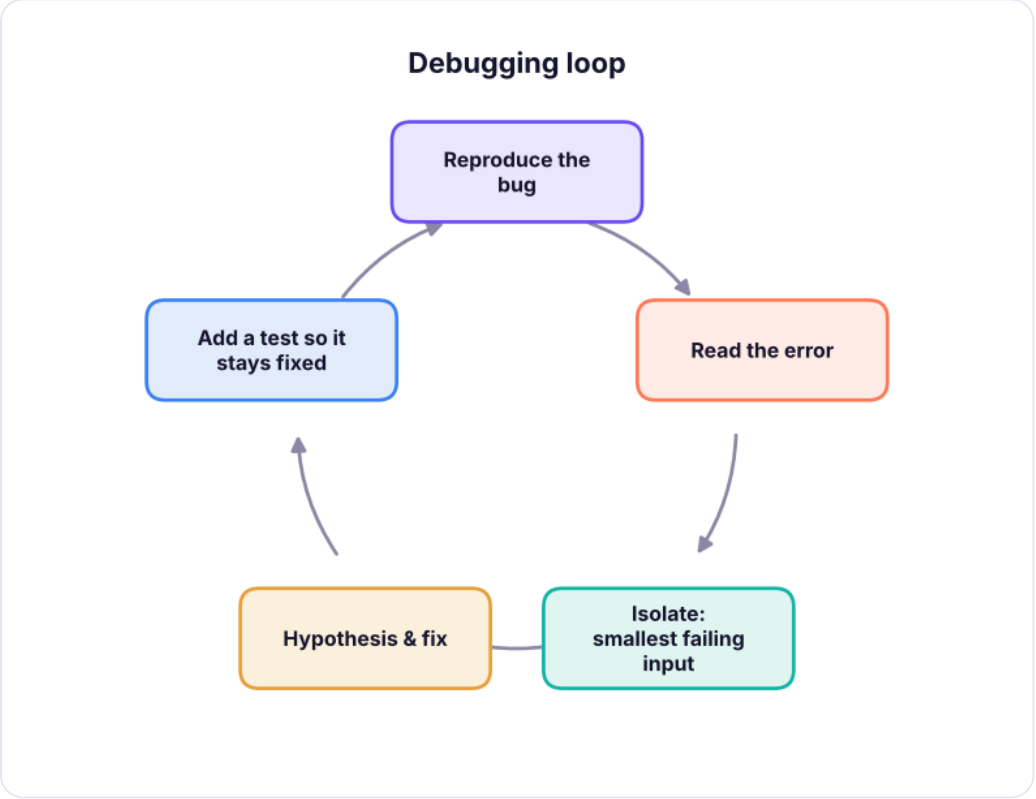
The last line says `ZeroDivisionError`; the frame above points at `average`, called from `report`. The fix is a guard for empty input:

```python
def average(values):
    return sum(values) / len(values) if values else None
print(average([]), average([4, 6]))
```

OUTPUT

```
None 5.0
```

A debugging routine



A systematic debugging loop beats random changes.

Technique	How
Print / f-string debugging	`print(f"{i=} {total=}")` shows names and values
`breakpoint()`	Pauses and opens the `pdb` debugger at that line
IDE debugger	VS Code / PyCharm: breakpoints, step over/into, watch variables
Jupyter `%debug`	Post-mortem debugging after an error in a cell
Rubber-duck	Explain the code line by line to someone (or an AI)
Bisect	Comment out halves of the code to find the failing part

Python

```python
total = 0
for i, v in enumerate([5, 10, 15]):
    total += v
    print(f"{i=} {v=} {total=}")          # the = specifier prints the name and the value
```

OUTPUT

```
i=0 v=5 total=5
i=1 v=10 total=15
i=2 v=15 total=30
```

pdb commands: `n` (next line), `s` (step into), `c` (continue), `p var` (print), `l` (list code), `q` (quit).

Testing

A **test** is code that checks other code. Tests catch regressions when you change things later.

Python

```python
def clean_city(raw):
    return " ".join(raw.split()).title() if raw else None

assert clean_city("  new   delhi ") == "New Delhi"
assert clean_city("") is None
print("asserts passed")
```

OUTPUT

```
asserts passed
```

pytest

`pytest` finds files named `test_*.py` and runs functions named `test_*`. Plain `assert` statements give detailed failure messages.

Python

```
from pathlib import Path
import subprocess, sys
Path("tests_demo").mkdir(exist_ok=True)
Path("tests_demo/cleaning.py").write_text('''
def clean_city(raw):
    return " ".join(raw.split()).title() if raw else None

def to_inr(value):
    return f"₹{value:,.0f}"
''', encoding="utf-8")
Path("tests_demo/test_cleaning.py").write_text('''
import pytest
from cleaning import clean_city, to_inr

@pytest.mark.parametrize("raw, expected", [("  pune", "Pune"), ("NEW  DELHI", "New Delhi"), ("",
None), (None, None)])
def test_clean_city(raw, expected):
    assert clean_city(raw) == expected

def test_to_inr():
    assert to_inr(1234567) == "₹1,234,567"

def test_bad_input():
    with pytest.raises(ValueError):          # a str cannot take the ',' number format
        to_inr("abc")

def test_deliberately_failing():
    assert to_inr(1000) == "₹1000"          # wrong expectation, to show a failure report
''', encoding="utf-8")
r = subprocess.run([sys.executable, "-m", "pytest", "-q", "--no-header", "-p",
"no:cacheprovider", "tests_demo"], capture_output=True, text=True)
lines = r.stdout.strip().splitlines()
print("\n".join([l for l in lines if "assert" in l or "passed" in l or "failed" in l][-4:]))
import shutil; shutil.rmtree("tests_demo")
```

OUTPUT

```
>       assert to_inr(1000) == "₹1000"          # wrong expectation, to show a failure
report
E       AssertionError: assert '₹1,000' == '₹1000'
1 failed, 6 passed in 0.02s
```

The report says which test failed and shows both values: `'₹1,000' == '₹1000'`. The function is right, so the test expectation is fixed.

TIP **What to test in data work**

Cleaning functions, metric calculations, edge cases (empty, missing, negative, duplicates), schema checks on loaded data (columns, types, ranges), and that a pipeline's row counts reconcile. Tools such as **pandera** and **Great Expectations** validate DataFrames.

Regular expressions

A **regular expression** (regex) is a pattern for text. Python's `re` module searches, extracts, validates and replaces.

Pattern	Matches
`\d` / `\D`	A digit / a non-digit
`\w` / `\s`	Word character (letter, digit, _) / whitespace
`.`	Any character except newline
`[A-Z]` , `[^0-9]`	Character set / negated set
`*` , `+` , `?`	0 or more, 1 or more, 0 or 1
`{3}` , `{2,4}`	Exactly 3, between 2 and 4
`^` , `$`	Start, end of string
`( )`	Capture group; `(?P<name> )` named group
`|`	Or

Python

```
import re
text = "Call 98765-43210 or +91 91234 56789; email asha.k@gmail.com, ravi@skillnavora.in. Order #A-1029 on 2025-04-07."
print(re.findall(r"\d{5}[- ]?\d{5}", text))                                  # phone numbers
print(re.findall(r"[\w.+-]+@[\w-]+(?:\.[\w-]+)+", text))                     # emails (no trailing full stop)
m = re.search(r"(?P<y>\d{4})-(?P<m>\d{2})-(?P<d>\d{2})", text)
print(m.group(), m["y"], m.groupdict())
print(re.sub(r"\d", "X", "PAN ABCDE1234F"), re.split(r"[;,]\s*", "a, b;c ,d"))
print(bool(re.fullmatch(r"[A-Z]{5}\d{4}[A-Z]", "ABCDE1234F")), bool(re.fullmatch(r"[A-Z]{5}\d{4}[A-Z]", "ABCD1234F")))    # PAN format
```

OUTPUT

```
['98765-43210', '91234 56789']
['asha.k@gmail.com', 'ravi@skillnavora.in']
2025-04-07 2025 {'y': '2025', 'm': '04', 'd': '07'}
```

```
PAN ABCDEXXXXF ['a', 'b', 'c ', 'd']
True False
```

Always use **raw strings** `r"..."` for patterns, so `\d` is not treated as an escape. Test patterns at regex101.com.

Regex on a DataFrame

Python

```python
import pandas as pd
c = pd.DataFrame({"name": ["Asha", "Ravi", "Meera", "Kabir", "Zoya"],
                  "email": ["asha@gmail.com", "ravi@company", "MEERA@Yahoo.co.in", "kabir
@gmail.com", None],
                  "phone": ["98765 43210", "+91-91234-56789", "12345", "9123456789",
"0987654321"]})
c["email_ok"] = c["email"].str.fullmatch(r"[\w.+-]+@[\w-]+(?:\.[\w-]+)+", na=False)
c["domain"] = c["email"].str.lower().str.extract(r"@([\w.-]+)$")[0]
digits = c["phone"].str.replace(r"\D", "", regex=True).str[-10:]
c["phone_ok"] = digits.str.fullmatch(r"[6-9]\d{9}")
print(c[["name", "email_ok", "domain", "phone_ok"]])
```

OUTPUT

```
    name  email_ok       domain  phone_ok
0   Asha      True    gmail.com      True
1   Ravi     False      company      True
2  Meera      True  yahoo.co.in     False
3  Kabir     False    gmail.com      True
4   Zoya     False          NaN     False
```

pandas string methods (`.str.contains` , `.str.extract` , `.str.replace` , `.str.fullmatch`) accept regex and run on the whole column.

Code quality

PEP 8 is Python's style guide. The essentials:

Rule	Example
`snake_case` for variables and functions	`total_sales` , `clean_city()`
`PascalCase` for classes	`CleaningPipeline`
`UPPER_CASE` for constants	`GST_RATE = 0.18`
4-space indentation, lines ≤ 79–100 chars	
Spaces around operators, after commas	`x = a + b` , `f(a, b)`
Imports at the top, grouped std / third-party / local	
Meaningful names over comments	`days_since_last_order` , not `d`

Let tools do the work:

```text
pip install ruff mypy
ruff check .          # lint: unused imports, undefined names, bad patterns
ruff format .         # auto-format (black-compatible)
mypy src/             # check type hints
pre-commit install    # run them automatically before each commit
```

NOTE **Using AI to debug**

Paste the **full traceback and the smallest code that reproduces it** into an AI assistant (Claude, ChatGPT, Copilot, Gemini). Ask it to explain the cause before proposing a fix, then run the fix and add a test. Never paste secrets or confidential data, and check that suggested library functions really exist.

Summary

KEY TAKEAWAYS

- Read tracebacks bottom-up; reproduce, isolate, hypothesise, fix, then add a test.
- Debug with `print(f"{x=}")` , `breakpoint()` , an IDE debugger or `%debug` .
- `pytest` runs `test_*` functions with plain `assert` ; use `parametrize` for many cases and `pytest.raises` for errors.
- Regex: `re.search` , `findall` , `fullmatch` , `sub` , `split` ; raw strings; pandas `.str` methods accept regex.
- Follow PEP 8 and automate it with ruff, a formatter and mypy.

Quick quiz

6 questions · tap an answer to check it

1. Where do you start reading a Python traceback?

- **A** The first line
- **B** The last line, which names the error
- **C** The middle
- **D** The import lines

Answer: B. Then read upward to find where it happened.

2. What does `print(f"{total=}")` print when total is 5?

- **A** 5
- **B** total=5
- **C** {total=5}
- **D** An error

Answer: B. The `=` specifier is a debugging shortcut.

3. Which file name will pytest discover by default?

- **A** cleaning_tests.py
- **B** test_cleaning.py
- **C** tests.txt
- **D** cleaning.py

Answer: B. Files `test_*.py` or `*_test.py`, functions `test_*`.

4. What does `\d{3}` match?

A The letter d three times

B Exactly three digits

C Up to three digits

D Any three characters

Answer: B. `\d` is a digit; `{3}` repeats it exactly three times.

5. Why use `r"..."` for regex patterns?

A It runs faster

B Backslashes are kept literally instead of being treated as escapes

C It makes the pattern case-insensitive

D It is required by pandas

Answer: B. Raw strings avoid double escaping.

6. Which naming follows PEP 8 for a function?

A CalculateMargin

B calculate_margin

C CALCULATE_MARGIN

D calculateMargin

Answer: B. snake_case for functions and variables.

EXERCISE **Practice**

(1) Write a regex that extracts all hashtags from a tweet. (2) Validate Indian PIN codes (6 digits, not starting with 0). (3) Write three pytest tests for a `percent_change` function, including the zero case. (4) Run `ruff check` on one of your notebooks exported as `.py` and fix the warnings.

Show solution (worked answer)

Python

```
import re
print(re.findall(r"#\w+", "Loving #Python and #DataScience!"))
pin = re.compile(r"[1-9]\d{5}")
print(bool(pin.fullmatch("411001")), bool(pin.fullmatch("011001")))
# test_metrics.py
# def test_up(): assert percent_change(100, 120) == 20
# def test_down(): assert percent_change(100, 80) == -20
# def test_zero(): assert percent_change(0, 5) is None
```

INTERVIEW CORNER **Questions you may be asked**

Q: How do you debug a function that gives wrong output only for some inputs? Reproduce with the smallest failing input, inspect intermediate values (prints or debugger), form a hypothesis, fix, and add the case as a regression test.

Q: Unit test vs integration test? A unit test checks one small function in isolation; an integration test checks that components work together (e.g. load → transform → write).

Q: Write a regex for a 10-digit Indian mobile number. `^[6-9]\d{9}$` (optionally allowing a `+91` prefix: `^(?:\+91[- ]?)?[6-9]\d{9}$`).

PYTHON FOR DATA SCIENCE · LESSON 15

NumPy: Fast Arrays, Vectorisation and Broadcasting

Learn the array library under pandas, scikit-learn and PyTorch. Create arrays, understand shape and dtype, index and slice in 1-D and 2-D, filter with boolean masks, compute with vectorised operations, aggregations along axes and broadcasting, generate random data and do basic linear algebra.

Python class 11 · NumPy 2.x · Intermediate · 120 min

By the end of this lesson you can

- Create arrays and describe their shape, ndim, dtype and size
- Index, slice and filter 1-D and 2-D arrays with boolean masks and fancy indexing
- Use vectorised arithmetic, universal functions and aggregations along an axis
- Apply broadcasting rules, reshape, stack and split arrays
- Generate reproducible random data and perform matrix operations

Why NumPy?

A Python list holds pointers to separate objects; a NumPy **array** stores numbers of one type in one contiguous block of memory and runs operations in compiled C. The result: code that is shorter **and** usually many times faster (the gap grows with heavier maths).

Python

```
import numpy as np, time
n = 1_000_000
py = list(range(n)); arr = np.arange(n)
t0 = time.perf_counter(); sq_list = [x * x for x in py]; t1 = time.perf_counter(); sq_arr = arr * arr; t2 = time.perf_counter()
print(f"list comprehension {1000*(t1-t0):.1f} ms | numpy {1000*(t2-t1):.1f} ms | same result: {sq_list[-1] == sq_arr[-1]}")
print(np.__version__)
```

OUTPUT

```
list comprehension 44.7 ms | numpy 34.8 ms | same result: True
2.5.3
```

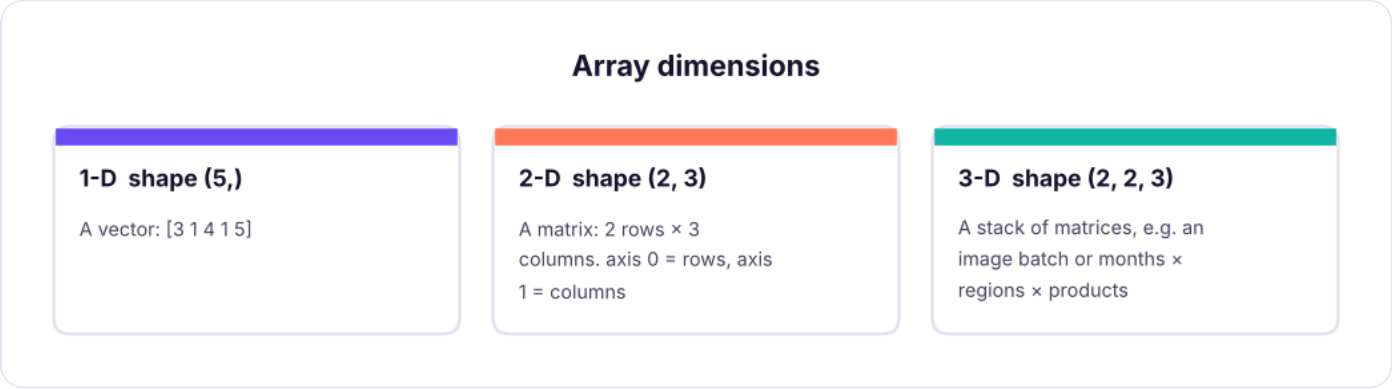
Creating arrays

Python

```
a = np.array([3, 1, 4, 1, 5])
m = np.array([[1, 2, 3], [4, 5, 6]])
print(a, a.dtype, a.shape, a.ndim, a.size)
print(m, m.shape, m.ndim, m.dtype)
print(np.zeros(3), np.ones((2, 2)), np.full(3, 7), np.eye(2))
print(np.arange(0, 10, 2.5), np.linspace(0, 1, 5), np.array([1, 2.5]).dtype, np.array([1, 2],
dtype=np.float32).dtype)
```

OUTPUT

```
[3 1 4 1 5] int64 (5,) 1 5
[[1 2 3]
 [4 5 6]] (2, 3) 2 int64
[0. 0. 0.] [[1. 1.]
 [1. 1.]] [7 7 7] [[1. 0.]
 [0. 1.]]
[0.  2.5 5.  7.5] [0.   0.25 0.5  0.75 1.  ] float64 float32
```



Shapes of 1-D, 2-D and 3-D arrays. axis 0 runs down the rows, axis 1 across the columns.

All items share one **dtype** (`int64` , `float64` , `bool` , `<U5` strings...). Mixing ints and floats upcasts to float.

Indexing and slicing

Python

```
a = np.array([10, 20, 30, 40, 50, 60])
print(a[0], a[-1], a[1:4], a[::2], a[::-1])
m = np.arange(1, 13).reshape(3, 4)
print(m)
print(m[0, 2], m[1], m[:, 1], m[1:, 2:], m[-1, -1])          # [row, column]
```

OUTPUT

```
10 60 [20 30 40] [10 30 50] [60 50 40 30 20 10]
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
3 [5 6 7 8] [ 2  6 10] [[ 7  8]
 [11 12]] 12
```

WATCH OUT **Slices are views**

`b = a[1:4]` does **not** copy; changing `b` changes `a`. Use `a[1:4].copy()` when you need an independent array. (Python lists behave the other way: slicing copies.)

Boolean masks and fancy indexing

Python

```
sales = np.array([120, 45, 300, 80, 210, 15])
mask = sales > 100
print(mask, sales[mask], sales[(sales > 50) & (sales < 250)], sales[~mask])    # & | ~ with
brackets
print(sales[[0, 2, 4]], np.where(sales > 100, "high", "low"), np.where(sales > 100)[0])
sales[sales < 50] = 0                                  # conditional assignment
print(sales, np.count_nonzero(sales), (sales > 100).mean())
```

OUTPUT

```
[ True False  True False  True False] [120 300 210] [120  80 210] [45 80 15]
[120 300 210] ['high' 'low' 'high' 'low' 'high' 'low'] [0 2 4]
[120   0 300  80 210   0] 4 0.5
```

Vectorised maths and aggregations

Python

```
price = np.array([100.0, 250.0, 80.0]); qty = np.array([3, 1, 10])
print(price * qty, (price * qty).sum(), price * 1.18, np.round(np.sqrt(price), 2),
np.log(price).round(2))
m = np.array([[5, 2, 9], [1, 7, 3]])
print(m.sum(), m.sum(axis=0), m.sum(axis=1), m.max(axis=1), m.argmax(axis=1), m.mean(axis=0))
x = np.array([2, 4, 4, 4, 5, 5, 7, 9])
print(x.mean(), np.median(x), x.std(), x.std(ddof=1).round(3), np.percentile(x, [25, 75]),
np.cumsum(x), np.diff(x))
```

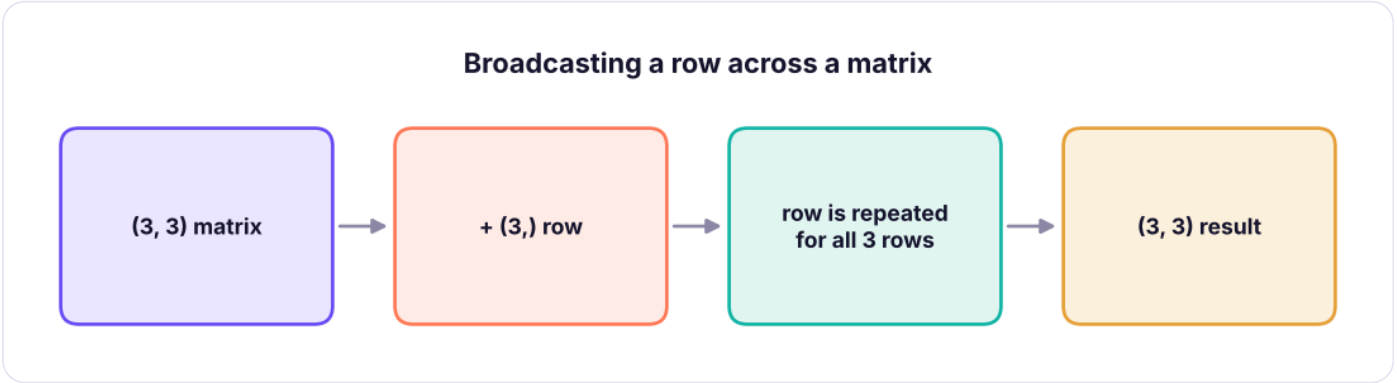
OUTPUT

```
[300. 250. 800.] 1350.0 [118.  295.   94.4] [10.   15.81  8.94] [4.61 5.52 4.38]
27 [ 6  9 12] [16 11] [9 7] [2 1] [3.  4.5 6. ]
5.0 4.5 2.0 2.138 [4.  5.5] [ 2  6 10 14 19 24 31 40] [2 0 0 1 0 2 2]
```

`axis=0` collapses rows (one result per column); `axis=1` collapses columns (one result per row). `ddof=1` gives the **sample** standard deviation that pandas uses by default.

Broadcasting

When shapes differ, NumPy **stretches** the smaller array along dimensions of size 1, without copying data.



Broadcasting: a (3, 3) matrix plus a (3,) row is applied to every row.

Python

```python
m = np.arange(1, 10).reshape(3, 3)
print(m + np.array([10, 20, 30]))                    # add to each row
print(m * np.array([[1], [10], [100]]))              # multiply each row by its own factor (column vector)
scores = np.array([[70, 80, 90], [60, 75, 85], [88, 92, 79]], dtype=float)
z = (scores - scores.mean(axis=0)) / scores.std(axis=0)   # standardise each column in one line
print(z.round(2))
```

OUTPUT

```
[[11 22 33]
 [14 25 36]
 [17 28 39]]
[[  1   2   3]
 [ 40  50  60]
 [700 800 900]]
[[-0.23 -0.33  1.19]
 [-1.09 -1.03  0.07]
 [ 1.32  1.36 -1.26]]
```

Rule: compare shapes from the right; dimensions are compatible if equal or one of them is 1.

Reshaping, stacking, sorting

Python

```
a = np.arange(12)
print(a.reshape(3, 4).shape, a.reshape(2, -1).shape, a.reshape(3, 4).T.shape, a.reshape(3,
4).ravel()[:5])
x, y = np.array([1, 2]), np.array([3, 4])
print(np.vstack([x, y]), np.hstack([x, y]), np.column_stack([x, y]), np.concatenate([x, y]))
v = np.array([30, 10, 20, 10])
print(np.sort(v), np.argsort(v), np.unique(v, return_counts=True), v[np.argsort(-v)][:2])
```

OUTPUT

```
(3, 4) (2, 6) (4, 3) [0 1 2 3 4]
[[1 2]
 [3 4]] [1 2 3 4] [[1 3]
 [2 4]] [1 2 3 4]
[10 10 20 30] [1 3 2 0] (array([10, 20, 30]), array([2, 1, 1])) [30 20]
```

Random numbers

Use the modern generator API with a seed so results are reproducible.

Python

```
rng = np.random.default_rng(42)
print(rng.integers(1, 7, size=10))                      # dice
print(rng.normal(loc=50, scale=10, size=5).round(1))    # normal distribution
print(rng.choice(["A", "B"], size=8, p=[0.7, 0.3]), rng.uniform(0, 1, 3).round(3))
sample = rng.normal(50, 10, 100_000)
print(round(sample.mean(), 2), round(sample.std(), 2), round(((sample > 40) & (sample <
60)).mean(), 3))   # about 68% within 1 sd
```

OUTPUT

```
[1 5 4 3 3 6 1 5 2 1]
[37.  51.3 46.8 49.8 41.5]
['A' 'B' 'A' 'B' 'A' 'A' 'A' 'A'] [0.828 0.632 0.758]
49.96 10.04 0.682
```

Linear algebra basics

Machine-learning models are built from these operations.

Python

```
A = np.array([[2, 1], [1, 3]]); b = np.array([5, 10])
print(A @ np.array([1, 2]), A.T, np.linalg.det(A).round(2), np.linalg.inv(A).round(3))
print(np.linalg.solve(A, b))                              # solves 2x + y = 5, x + 3y = 10
w = np.array([0.5, -0.2]); X = np.array([[1.0, 2.0], [3.0, 1.0], [2.0, 2.0]])
print(X @ w)                                               # predictions of a linear model y = Xw
```

OUTPUT

```
[4 7] [[2 1]
 [1 3]] 5.0 [[ 0.6 -0.2]
 [-0.2  0.4]]
[1. 3.]
[0.1 1.3 0.6]
```

Real data: house prices

Python

```
import pandas as pd
h = pd.read_csv("house_prices.csv")
area, price = h["area_sqft"].to_numpy(), h["price_lakh"].to_numpy()
print("missing values:", np.isnan(area).sum(), "area,", np.isnan(price).sum(), "price")     # real data has gaps
ok = ~np.isnan(area) & ~np.isnan(price)
area, price = area[ok], price[ok]                                                            # keep complete rows
per_sqft = price * 1e5 / area
print(f"{len(price)} houses | median price ₹{np.median(price):.1f} lakh | median ₹{np.median(per_sqft):,.0f}/sq ft")
print("price percentiles 10/50/90:", np.percentile(price, [10, 50, 90]).round(1))
big = area > 1500
print(f"{big.mean():.0%} are over 1,500 sq ft; their median price is ₹{np.median(price[big]):.1f} lakh vs ₹{np.median(price[~big]):.1f} lakh")
r = np.corrcoef(area, price)[0, 1]
slope, intercept = np.polyfit(area, price, 1)
print(f"correlation area-price r = {r:.2f}; fitted line: price ≈ {intercept:.1f} + {slope*100:.2f} lakh per 100 sq ft")
```

OUTPUT

```
missing values: 25 area, 0 price
1175 houses | median price ₹78.7 lakh | median ₹6,258/sq ft
price percentiles 10/50/90: [ 40.3  78.7 119.9]
27% are over 1,500 sq ft; their median price is ₹110.0 lakh vs ₹68.1 lakh
correlation area-price r = 0.79; fitted line: price ≈ -7.5 + 7.05 lakh per 100 sq ft
```

`np.polyfit` fits a straight line by least squares, which is the same idea as linear regression (ML track).

Summary

KEY TAKEAWAYS

- Arrays hold one dtype in contiguous memory; operations are vectorised and much faster than Python loops.
- Know `shape`, `ndim`, `dtype`, `size`; index with `[row, col]`; slices are views (use `.copy()`).
- Boolean masks (`a[a > 0]`, `&`, `|`, `~`) and `np.where` filter and recode.
- Aggregate with `axis=0` (per column) or `axis=1` (per row); `ddof=1` for sample standard deviation.
- Broadcasting stretches size-1 dimensions; reshape with `-1` to infer a dimension.
- Use `np.random.default_rng(seed)` and `@` for matrix multiplication.

Quick quiz

6 questions · tap an answer to check it

1. What is `np.arange(12).reshape(3, -1).shape` ?

A (3, 3)

B (3, 4)

C (4, 3)

D Error

Answer: B. -1 lets NumPy infer the missing dimension: 12 / 3 = 4.

2. What does `m.sum(axis=0)` return for a 2 × 3 matrix?

A One number

B Three numbers: the sum of each column

C Two numbers: the sum of each row

D A 2 × 3 matrix

Answer: B. axis=0 collapses the rows.

3. Which expression keeps values between 10 and 20?

A `a[a > 10 and a < 20]`

B `a[(a > 10) & (a < 20)]`

C `a[10:20]`

D `a.between(10, 20)`

Answer: B. Use `&` with brackets for element-wise conditions.

4. After `b = a[0:3]; b[0] = 99`, what happens to `a[0]`?

A Unchanged

B It becomes 99 because a slice is a view

C Error

D It becomes 0

Answer: B. Use `.copy()` to avoid this.

5. Can a (3, 4) array be added to a (4,) array?

A Yes, the (4,) array is broadcast across the rows

B No, shapes must match exactly

C Only with a loop

D Only if both are float

Answer: A. Trailing dimensions match (4 and 4).

6. How do you make random results reproducible?

A Use `np.random.random()`

B Create a generator with a seed: `np.random.default_rng(42)`

C Run the cell twice

D Use floats

Answer: B. The same seed gives the same numbers.

EXERCISE **NumPy practice**

(1) Create a 5 × 5 array of 1..25 and select the diagonal and the last column. (2) Replace all negative values in an array with 0. (3) Standardise the `area_sqft` column (z-scores) and count values beyond ±2. (4) Simulate 10,000 rolls of two dice and estimate P(sum = 7). (5) Solve 3x + 2y = 12, x − y = 1.

Show solution (worked answer)

Python

```
g = np.arange(1, 26).reshape(5, 5); print(np.diag(g), g[:, -1])
v = np.array([3, -1, 4, -5]); v[v < 0] = 0; print(v)
z = (area - area.mean()) / area.std(); print((np.abs(z) > 2).sum())
rng = np.random.default_rng(0); rolls = rng.integers(1, 7, (10_000, 2)).sum(axis=1);
print((rolls == 7).mean())   # about 1/6
print(np.linalg.solve([[3, 2], [1, -1]], [12, 1]))   # x = 2.8, y = 1.8
```

INTERVIEW CORNER **Questions you may be asked**

Q: Why is NumPy faster than Python lists? Contiguous memory of one dtype, operations implemented in C over the whole array (vectorisation), and no per-element Python overhead.

Q: What is broadcasting? NumPy's rule for operating on arrays of different shapes by virtually repeating dimensions of size 1, aligning shapes from the right.

Q: View vs copy? Basic slices return views that share memory; fancy/boolean indexing and `.copy()` return copies.

Q: Difference between `np.std(x)` and `pandas.Series.std()`? NumPy defaults to population std (`ddof=0`); pandas defaults to sample std (`ddof=1`).

PYTHON FOR DATA SCIENCE · LESSON 16

pandas I: Series, DataFrames, Reading Data, Selecting and Filtering

Meet the most-used data library. Create Series and DataFrames, read CSV and Excel files, inspect data with head, info and describe, select columns and rows with loc and iloc, filter with conditions and query, sort, add columns and save results, using the Superstore sales data.

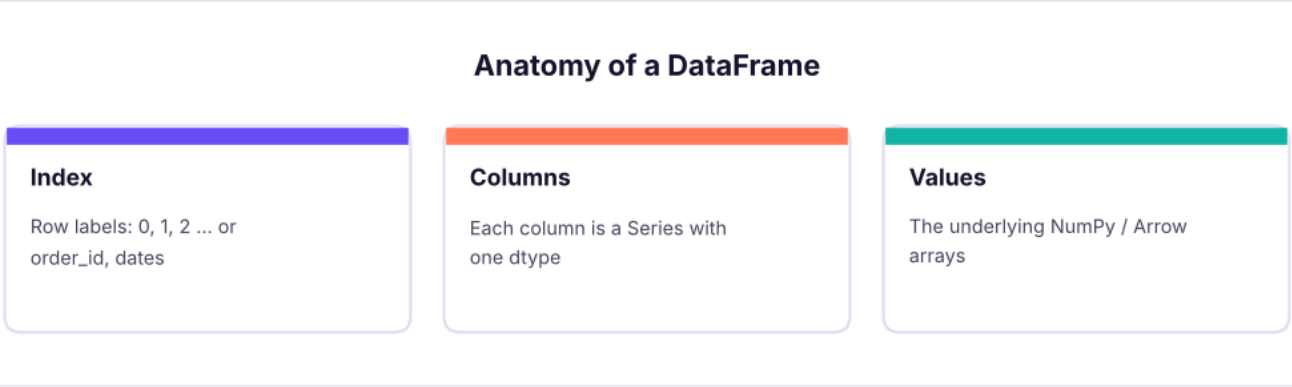
Python class 12 · pandas 2.3 / 3.0 · Intermediate · 120 min

By the end of this lesson you can

- Create Series and DataFrames and explain the index
- Read CSV and Excel files with the right options and write results back out
- Inspect a dataset with shape, dtypes, head, info, describe and value_counts
- Select with [], loc and iloc and filter rows with boolean conditions, isin, between and query
- Sort, rank, add and rename columns, and avoid the SettingWithCopy pitfall

Series and DataFrame

pandas gives Python spreadsheet-like tables. A **Series** is one labelled column; a **DataFrame** is a table of Series sharing one **index** (row labels).



A DataFrame is a set of Series (columns) that share an index (row labels).

Python

```
import pandas as pd, numpy as np
pd.set_option("display.max_columns", 20); pd.set_option("display.width", 160)   # show wide tables in full
s = pd.Series([120, 95, 140], index=["North", "South", "East"], name="sales")
print(s, s["South"], s.max(), s.idxmax(), sep="\n")
df = pd.DataFrame({"city": ["Pune", "Delhi", "Kochi"], "sales": [120, 95, 140], "target": [100, 110, 130]})
df["hit"] = df["sales"] ⩾ df["target"]
print(df)
print(df.shape, list(df.columns), df.index, df.dtypes.to_dict(), pd.__version__)
```

OUTPUT

```
North    120
South     95
East     140
Name: sales, dtype: int64
95
140
East
    city  sales  target    hit
0   Pune    120     100   True
1  Delhi     95     110  False
2  Kochi    140     130   True
(3, 4) ['city', 'sales', 'target', 'hit'] RangeIndex(start=0, stop=3, step=1) {'city': dtype('O'), 'sales': dtype('int64'), 'target': dtype('int64'), 'hit': dtype('bool')}
2.3.3
```

Reading data

Python

```
s = pd.read_csv("superstore_sales.csv", parse_dates=["order_date", "ship_date"])
print(s.shape)
print(s.head(3).T)
```

OUTPUT

```
(6323, 19)
                              0                   1                   2
row_id                        1                   2                   3
order_id             IN-2023-00001       IN-2023-00001       IN-2023-00001
order_date     2023-01-01 00:00:00 2023-01-01 00:00:00 2023-01-01 00:00:00
ship_date      2023-01-05 00:00:00 2023-01-05 00:00:00 2023-01-05 00:00:00
ship_mode             Second Class        Second Class        Second Class
customer_id                 C0765               C0765               C0765
customer_name         Ishaan Gupta        Ishaan Gupta        Ishaan Gupta
```

segment	Consumer	Consumer	Consumer
city	Bengaluru	Bengaluru	Bengaluru
state	Karnataka	Karnataka	Karnataka
region	South	South	South
product_id	P0059	P0050	P0011
category	Technology	Office Supplies	Furniture
sub_category	Phones	Appliances	Tables
product_name	Nova Phone 223	Orbit Appliance 798	Orbit Table 248
sales	36000.0	3850.0	62080.0
quantity	2	1	4
discount	0.0	0.0	0.0
profit	3472.55	1169.54	6044.47

Option	Use
`sep=";"`	Other delimiters (`"\t"` for TSV)
`usecols=[...]`	Read only some columns (faster, less memory)
`dtype={"pin": str}`	Keep leading zeros and IDs as text
`parse_dates=[...]`	Convert date columns
`na_values=["NA", "-"]`	Extra markers for missing
`nrows=1000` , `chunksize=100_000`	Sample or stream big files
`encoding="latin-1"`	Fix decode errors in older files

Other readers: `pd.read_excel("file.xlsx", sheet_name="Sales")` (needs `openpyxl`), `pd.read_json`, `pd.read_parquet`, `pd.read_sql` (Lesson 23), `pd.read_html`, `pd.read_clipboard`.

Inspecting data

Run these five on every new dataset:

Python

```
print(s.info(memory_usage="deep"))
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6323 entries, 0 to 6322
Data columns (total 19 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   row_id         6323 non-null   int64
 1   order_id       6323 non-null   object
```

```
 2   order_date     6323 non-null   datetime64[ns]
 3   ship_date      6323 non-null   datetime64[ns]
 4   ship_mode      6323 non-null   object
 5   customer_id    6323 non-null   object
 6   customer_name  6323 non-null   object
 7   segment        6323 non-null   object
 8   city           6323 non-null   object
 9   state          6323 non-null   object
 10  region         6323 non-null   object
 11  product_id     6323 non-null   object
 12  category       6323 non-null   object
 13  sub_category   6323 non-null   object
 14  product_name   6323 non-null   object
 15  sales          6323 non-null   float64
 16  quantity       6323 non-null   int64
 17  discount       6323 non-null   float64
 18  profit         6323 non-null   float64
dtypes: datetime64[ns](2), float64(3), int64(2), object(12)
memory usage: 4.6 MB
None
```

Python

```python
print(s.describe(include="number").round(1).T)
print(s["region"].value_counts(), s["category"].value_counts(normalize=True).round(3), sep="\n")
print(s["customer_id"].nunique(), "customers |", s["order_date"].min().date(), "to",
s["order_date"].max().date())
print(s.isna().sum().sum(), "missing cells |", s.duplicated().sum(), "duplicate rows")
```

OUTPUT

```
             count     mean      std      min     25%     50%      75%       max
row_id      6323.0   3162.0   1825.4      1.0  1581.5  3162.0   4742.5    6323.0
sales       6323.0  31854.1  60414.2     42.0  1564.0  8430.0  32592.0  619290.0
quantity    6323.0      2.6      1.6      1.0     1.0     2.0      3.0       7.0
discount    6323.0      0.1      0.1      0.0     0.0     0.0      0.2       0.4
profit      6323.0   2902.4  11002.0 -75873.8   -17.9   266.3   2270.1  149424.9
region
South    1656
West     1614
North    1530
East     1523
Name: count, dtype: int64
category
Office Supplies    0.398
Furniture          0.303
Technology         0.299
Name: proportion, dtype: float64
```

```
785 customers | 2023-01-01 to 2025-12-31
0 missing cells | 0 duplicate rows
```

Selecting columns and rows

Python

```python
print(s["sales"].head(3).tolist())                    # one column → Series
print(s[["order_id", "region", "sales"]].head(3))     # list of columns → DataFrame
print(s.loc[0, "customer_name"], s.iloc[0, 6])        # loc = labels, iloc = positions
print(s.loc[0:2, "region":"product_id"])               # loc slices INCLUDE the end label
print(s.iloc[0:2, 10:12])                              # iloc slices EXCLUDE the end, like
Python
```

OUTPUT

```
[36000.0, 3850.0, 62080.0]
        order_id region    sales
0  IN-2023-00001  South  36000.0
1  IN-2023-00001  South   3850.0
2  IN-2023-00001  South  62080.0
Ishaan Gupta Ishaan Gupta
  region product_id
0  South      P0059
1  South      P0050
2  South      P0011
  region product_id
0  South      P0059
1  South      P0050
```

Tool	Selects by	Example
`df["col"]` / `df[["a","b"]]`	Column names	`s["sales"]`
`df.loc[rows, cols]`	**Labels** and boolean masks	`s.loc[s.profit < 0, ["order_id", "profit"]]`
`df.iloc[rows, cols]`	**Integer positions**	`s.iloc[:5, :3]`
`df.at[r, c]` / `df.iat[i, j]`	One value, fast	`s.at[0, "sales"]`

Filtering rows

Python

```
loss = s[s["profit"] < 0]
print(len(loss), "loss-making lines")
tech_south = s[(s["category"] == "Technology") & (s["region"] == "South")]           # & | ~
need brackets
print(len(tech_south), round(tech_south["sales"].sum() / 1e6, 2), "M")
print(s[s["city"].isin(["Pune", "Kochi"])]["city"].value_counts().to_dict())
print(len(s[s["discount"].between(0.1, 0.2)]), len(s[s["product_name"].str.contains("Chair")]))
print(s.query("region == 'West' and sales > 50000 and discount == 0")[["order_id",
"sales"]].head(3))
big = 50000
print(len(s.query("sales > @big")))                      # @ refers to a Python variable
```

OUTPUT

```
1682 loss-making lines
495 41.89 M
{'Pune': 529, 'Kochi': 417}
1943 493
         order_id    sales
7   IN-2023-00003  69280.0
22  IN-2023-00013  72000.0
98  IN-2023-00051  64450.0
1126
```

Sorting and ranking

Python

```
top = s.sort_values("sales", ascending=False)[["order_id", "product_name", "sales"]].head(5)
print(top)
print(s.sort_values(["region", "profit"], ascending=[True, False]).groupby("region").head(1)
[["region", "product_name", "profit"]])
print(s.nlargest(3, "profit")[["product_name", "profit"]], s.nsmallest(2, "profit")
[["product_name", "profit"]], sep="\n")
```

OUTPUT

```
          order_id       product_name     sales
3669  IN-2024-01857   Orbit Copier 746  619290.0
2087  IN-2024-01040  Vertex Copier 262  551530.0
1142  IN-2023-00564  Vertex Copier 262  551530.0
5696  IN-2025-02880   Orbit Copier 746  530820.0
1653  IN-2023-00809   Orbit Copier 746  526396.5
     region       product_name     profit
4219   East   Orbit Copier 746  135564.21
```

```
3669  North   Orbit Copier 746  149424.89
3893  South    Apex Copier 347  107757.96
481    West  Zenith Copier 719   95485.58
         product_name     profit
3669  Orbit Copier 746  149424.89
4219  Orbit Copier 746  135564.21
3893   Apex Copier 347  107757.96
            product_name    profit
5979    Orbit Copier 746 -75873.84
639   Pioneer Copier 366 -67392.75
```

Adding, changing and removing columns

Python

```python
e = pd.read_csv("employees_small.csv", parse_dates=["join_date"])
e["bonus"] = e["salary"] * 0.1                                    # vectorised arithmetic
e["band"] = np.where(e["salary"] ⩾ 100000, "Senior", "Standard")      # IF
e["grade"] = pd.cut(e["salary"], bins=[0, 70000, 100000, np.inf], labels=["C", "B", "A"])   # bins
e["tenure_yrs"] = ((pd.Timestamp("2025-10-27") - e["join_date"]).dt.days / 365.25).round(1)
e = e.assign(name_upper=e["name"].str.upper()).rename(columns={"rating": "perf"}).drop(columns=["name_upper"])
e["salary_rank"] = e["salary"].rank(ascending=False, method="dense").astype(int)
print(e.head(6))
```

OUTPUT

```
  emp_id              name department       city  join_date  salary  perf    bonus
band grade  tenure_yrs  salary_rank
0   E101         Kavya Iyer         HR     Mumbai 2016-02-28   97000     4   9700.0
Standard     B         9.7            8
1   E102        Imran Reddy    Finance      Delhi 2019-12-14  157000     3  15700.0
Senior     A         5.9            1
2   E103  Harpreet Kulkarni    Finance     Mumbai 2024-10-05  131000     4  13100.0
Senior     A         1.1            4
3   E104          Diya Iyer         HR  Bengaluru 2024-01-02   90000     4   9000.0
Standard     B         1.8            9
4   E105        Simran Iyer         HR  Bengaluru 2017-08-03  139000     4  13900.0
Senior     A         8.2            3
5   E106         Saanvi Rao         HR      Delhi 2025-02-18   83000     4   8300.0
Standard     B         0.7           10
```

WATCH OUT **SettingWithCopyWarning**

`sub = df[df.x > 0]; sub["y"] = 1` may warn because `sub` might be a view. Use `df.loc[df.x > 0, "y"] = 1` to change the original, or `sub = df[df.x > 0].copy()` for an independent table. pandas 3 makes copy-on-write the default, which removes the ambiguity.

Saving results

Python

```
out = s.groupby("region", as_index=False)["sales"].sum()
out.to_csv("region_sales.csv", index=False)
print(open("region_sales.csv").read())
import os; os.remove("region_sales.csv")
```

OUTPUT

```
region,sales
East,46484119.0
North,49018161.0
South,53952768.5
West,51958520.5
```

Also: `to_excel("out.xlsx", sheet_name="Summary", index=False)`, `to_parquet` (fast, typed, compressed), `to_json(orient="records")`, `to_sql`.

pandas vs Excel vs SQL

Task	Excel	SQL	pandas
Filter	AutoFilter	`WHERE`	`df[df.x > 5]` / `query`
New column	Formula	`SELECT a*b AS c`	`df["c"] = df.a * df.b`
Sort	Sort	`ORDER BY`	`sort_values`
Lookup	XLOOKUP	`JOIN`	`merge` (Lesson 18)
Pivot	PivotTable	`GROUP BY`	`groupby`, `pivot_table` (Lesson 18)

Summary

KEY TAKEAWAYS

- A DataFrame is a table of Series sharing an index; read files with `read_csv` / `read_excel` and the right options.
- Inspect first: `shape`, `info()`, `describe()`, `value_counts()`, `isna().sum()`, `duplicated().sum()`.
- Select with `[]` (columns), `loc` (labels, end-inclusive) and `iloc` (positions, end-exclusive).
- Filter with boolean masks using `&`, `|`, `~` and brackets, plus `isin`, `between`, `str.contains`, `query`.
- Add columns with vectorised maths, `np.where`, `pd.cut`; use `.loc` or `.copy()` to avoid SettingWithCopy issues.

Quick quiz

6 questions · tap an answer to check it

1. What does `s[["region", "sales"]]` return?

A A Series

B A DataFrame with two columns

C A list

D An error

Answer: B. Double brackets pass a list of columns.

2. Which is end-inclusive?

A `df.loc[0:2]`

B `df.iloc[0:2]`

C Both

D Neither

Answer: A. `loc` uses labels and includes the end label.

3. Which filter is correct?

A `df[df.a > 1 and df.b < 5]`

B `df[(df.a > 1) & (df.b < 5)]`

C `df[df.a > 1 & df.b < 5]`

D `df.where(a > 1, b < 5)`

Answer: B. Use `&` with brackets around each condition.

4. How do you keep leading zeros in a PIN or ID column when reading?

A `parse_dates`

B `dtype={"pin": str}`

C `usecols`

D `index_col`

Answer: B. Otherwise pandas reads it as a number.

5. Which command shows column types and non-null counts?

A `describe()`

B `info()`

C `head()`

D `shape`

Answer: B. `describe()` gives statistics.

6. How do you reference a Python variable inside `query`?

A `{var}`

B `@var`

C `$var`

D `var()`

Answer: B. `df.query("sales > @big")`.

EXERCISE **pandas practice (Superstore)**

(1) How many orders (unique `order_id`) are there? (2) Show the 5 most profitable sub-categories by total profit. (3) Filter Consumer-segment orders in 2025 with discount above 20%. (4) Add `margin_pct` and list the 3 lowest-margin product names among lines with sales above 20,000. (5) Save the Technology rows to `tech.csv`.

Show solution (worked answer)

Python

```
print(s["order_id"].nunique())
print(s.groupby("sub_category")["profit"].sum().nlargest(5))
f = s[(s.segment == "Consumer") & (s.order_date.dt.year == 2025) & (s.discount > 0.2)];
print(len(f))
s["margin_pct"] = 100 * s.profit / s.sales
print(s[s.sales > 20000].nsmallest(3, "margin_pct")[["product_name", "margin_pct"]])
s[s.category == "Technology"].to_csv("tech.csv", index=False)
```

INTERVIEW CORNER

Questions you may be asked

Q: loc vs iloc? `loc` selects by label (and boolean masks), end-inclusive; `iloc` selects by integer position, end-exclusive.

Q: How do you find and handle duplicates? `df.duplicated(subset=[...]).sum()` to count, `drop_duplicates(subset=..., keep="first")` to remove, after deciding which columns define a duplicate.

Q: How do you read a file that does not fit in memory? `usecols` and `dtype` to shrink it, `chunksize` to stream it, or use DuckDB/Polars/Parquet.

PYTHON FOR DATA SCIENCE · LESSON 17

pandas II: Data Cleaning: Missing Values, Types, Duplicates, Text and Outliers

Turn messy data into analysis-ready data. Detect and treat missing values, fix data types, remove duplicates, standardise text, parse dates, handle outliers with IQR and caps, and document every step in a reusable cleaning function.

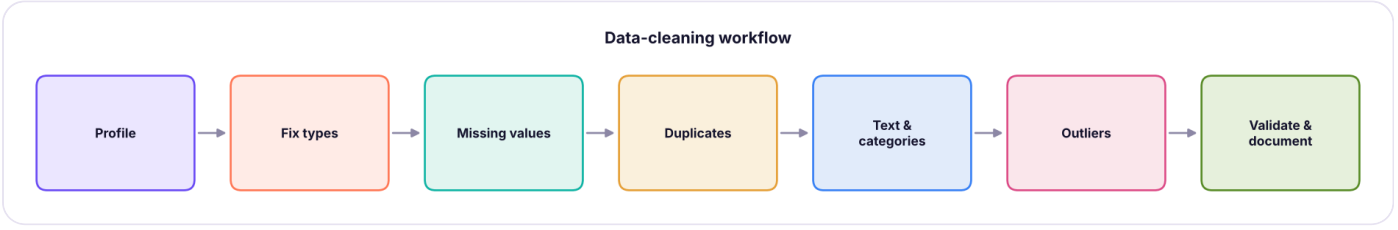
Python class 13 | pandas 2.3 / 3.0 | Intermediate | 120 min

By the end of this lesson you can

- Profile a dataset for missing values, wrong types, duplicates and inconsistent categories
- Choose and apply missing-value strategies (drop, fill with statistic, group-wise fill, flag)
- Convert types safely with to_numeric, to_datetime and astype, including categories
- Standardise text columns and map inconsistent labels
- Detect outliers with the IQR rule and z-scores and decide how to treat them

Why cleaning matters

Analysts often say 60–80% of project time goes into preparing data. Wrong types, gaps, duplicates and typos silently break totals, joins and models: **garbage in, garbage out**.



A repeatable cleaning workflow.

We will clean a deliberately messy extract:

Python

```
import pandas as pd, numpy as np
pd.set_option("display.max_columns", 20); pd.set_option("display.width", 160)
raw = pd.DataFrame({
    "order_id":  ["A1", "A2", "A3", "A3", "A4", "A5", "A6", "A7", "A8"],
    "city":      [" pune", "Pune ", "DELHI", "DELHI", "New Delhi", "kochi", None, "Bengaluru",
"Bangalore"],
    "amount":    ["1,200", "950", "₹3,400", "₹3,400", "abc", "780", "15000", "2,050", None],
    "order_date":["2025-01-05", "05/01/2025", "2025-01-07", "2025-01-07", "2025-02-30", "2025-
01-09", "2025-01-10", "2025-01-11", "2025-01-12"],
    "qty":       [2, 1, 3, 3, None, 1, 50, 2, 1],
})
print(raw)
print(raw.dtypes.to_dict())
```

OUTPUT

```
  order_id       city  amount  order_date   qty
0       A1       pune   1,200  2025-01-05   2.0
1       A2      Pune       950  05/01/2025   1.0
2       A3      DELHI  ₹3,400  2025-01-07   3.0
3       A3      DELHI  ₹3,400  2025-01-07   3.0
4       A4  New Delhi     abc  2025-02-30   NaN
5       A5      kochi     780  2025-01-09   1.0
6       A6        NaN   15000  2025-01-10  50.0
7       A7  Bengaluru   2,050  2025-01-11   2.0
8       A8  Bangalore     NaN  2025-01-12   1.0
{'order_id': <StringDtype(na_value=nan)>, 'city': <StringDtype(na_value=nan)>, 'amount':
<StringDtype(na_value=nan)>, 'order_date': <StringDtype(na_value=nan)>, 'qty':
dtype('float64')}
```

Step 1: Profile

Python

```
def profile(df):
    return pd.DataFrame({"dtype": df.dtypes.astype(str), "missing": df.isna().sum(),
"missing_%": (100 * df.isna().mean()).round(1),
                         "unique": df.nunique(), "example": df.iloc[0]})
print(profile(raw))
print("duplicate rows:", raw.duplicated().sum())
```

OUTPUT

```
             dtype  missing  missing_%  unique     example
order_id       str        0        0.0       8          A1
city           str        1       11.1       7        pune
amount         str        1       11.1       7       1,200
order_date     str        0        0.0       8  2025-01-05
```

```
qty          float64        1       11.1       4          2.0
duplicate rows: 1
```

Step 2: Fix types

Python

```python
df = raw.drop_duplicates().copy()                                    # exact duplicates first
df["amount"] = pd.to_numeric(df["amount"].str.replace(r"[₹,]", "", regex=True), errors="coerce")
# bad values → NaN
df["order_date"] = pd.to_datetime(df["order_date"], format="%Y-%m-%d", errors="coerce")
# strict format: others → NaT
df["qty"] = df["qty"].astype("Int64")                                # nullable integer keeps NaN
print(df[["order_id", "amount", "order_date", "qty"]])
print(df.dtypes.to_dict())
```

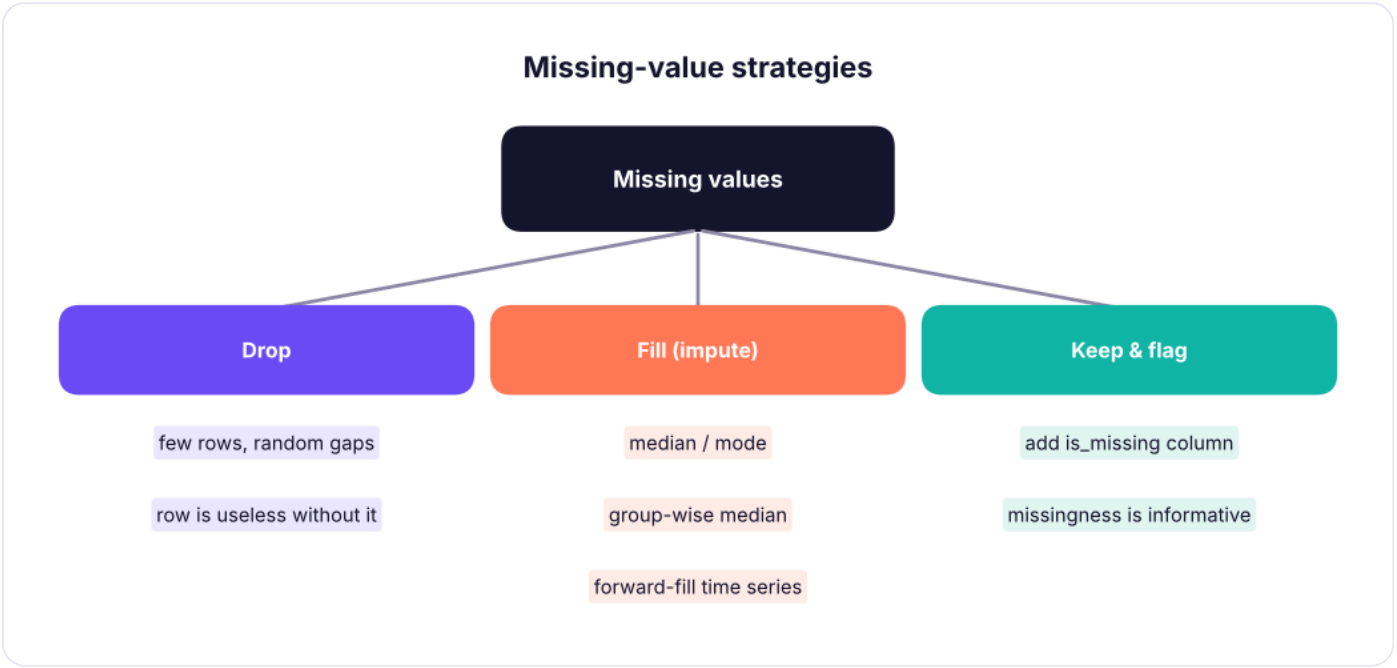
OUTPUT

```
  order_id   amount order_date   qty
0       A1   1200.0 2025-01-05     2
1       A2    950.0        NaT     1
2       A3   3400.0 2025-01-07     3
4       A4      NaN        NaT  <NA>
5       A5    780.0 2025-01-09     1
6       A6  15000.0 2025-01-10    50
7       A7   2050.0 2025-01-11     2
8       A8      NaN 2025-01-12     1
{'order_id': <StringDtype(na_value=nan)>, 'city': <StringDtype(na_value=nan)>, 'amount': dtype('float64'), 'order_date': dtype('<M8[us]'), 'qty': Int64Dtype()}
```

`errors="coerce"` turns unparseable values into `NaN` / `NaT` so you can count and inspect them instead of crashing. Always check how many values were coerced.

Conversion	Use
`pd.to_numeric(s, errors="coerce")`	Text with numbers
`pd.to_datetime(s, format=..., dayfirst=True)`	Dates (give the format when known)
`s.astype("category")`	Repeated labels: less memory, ordered categories
`s.astype("Int64")` , `"boolean"` , `"string"`	Nullable types that allow missing values

Step 3: Missing values



Choosing a strategy for missing values.

Python

```
print(df.isna().sum().to_dict())
df["amount_missing"] = df["amount"].isna()                    # flag before filling
df["amount"] = df["amount"].fillna(df["amount"].median())
df["qty"] = df["qty"].fillna(1)
df = df.dropna(subset=["order_date"])                         # a row without a valid
date is unusable here
print(df[["order_id", "amount", "amount_missing", "qty", "order_date"]])
```

OUTPUT

```
{'order_id': 0, 'city': 1, 'amount': 2, 'order_date': 2, 'qty': 1}
  order_id   amount  amount_missing  qty order_date
0       A1   1200.0           False    2 2025-01-05
2       A3   3400.0           False    3 2025-01-07
5       A5    780.0           False    1 2025-01-09
6       A6  15000.0           False   50 2025-01-10
7       A7   2050.0           False    2 2025-01-11
8       A8   1625.0            True    1 2025-01-12
```

Real data: `house_prices.csv` has gaps in `area_sqft`. A **group-wise** fill uses similar houses (same number of bedrooms) instead of one global value:

Python

```
h = pd.read_csv("house_prices.csv")
print("missing area:", h["area_sqft"].isna().sum())
by_bed = h.groupby("bedrooms")["area_sqft"].median().round(0)
print(by_bed.to_dict())
h["area_filled"] = h["area_sqft"].fillna(h.groupby("bedrooms")["area_sqft"].transform("median"))
print("after group-wise fill:", h["area_filled"].isna().sum(), "| global median would give",
h["area_sqft"].median())
```

OUTPUT

```
missing area: 25
{1: 760.0, 2: 1115.0, 3: 1430.0, 4: 1715.0, 5: 1955.0}
after group-wise fill: 0 | global median would give 1240.0
```

NOTE **Missing-data types**

MCAR (completely at random) → dropping is safe. **MAR** (depends on other columns) → impute using those columns. **MNAR** (depends on the missing value itself, e.g. high earners hide income) → flag it; imputation can bias results. Never fill target variables for modelling.

Step 4: Duplicates

Python

```
orders = pd.DataFrame({"order_id": ["B1", "B2", "B2", "B3"], "status": ["new", "new", "shipped",
"new"], "updated": pd.to_datetime(["2025-01-01", "2025-01-01", "2025-01-03", "2025-01-02"])})
print(orders.duplicated().sum(), orders.duplicated(subset="order_id").sum())
latest = orders.sort_values("updated").drop_duplicates(subset="order_id", keep="last")   # keep
the latest version per key
print(latest)
```

OUTPUT

```
0 1
  order_id   status    updated
0       B1      new 2025-01-01
3       B3      new 2025-01-02
2       B2  shipped 2025-01-03
```

Decide which columns define "the same record"; exact duplicates and key duplicates are different problems.

Step 5: Text and categories

Python

```
df["city"] = df["city"].str.strip().str.title()
fix = {"New Delhi": "Delhi", "Bangalore": "Bengaluru"}
df["city"] = df["city"].replace(fix).fillna("Unknown")
print(df["city"].value_counts().to_dict())
print(pd.Series(["  Data  Science ", "MACHINE learning"]).str.split().str.join("
").str.capitalize().tolist())
```

OUTPUT

```
{'Bengaluru': 2, 'Pune': 1, 'Delhi': 1, 'Kochi': 1, 'Unknown': 1}
['Data science', 'Machine learning']
```

Useful `.str` methods: `strip`, `lower/upper/title`, `replace`, `contains`, `startswith`, `extract`, `split`, `len`, `zfill`, `slice`. For fuzzy spellings, libraries such as **rapidfuzz** score string similarity.

Step 6: Outliers

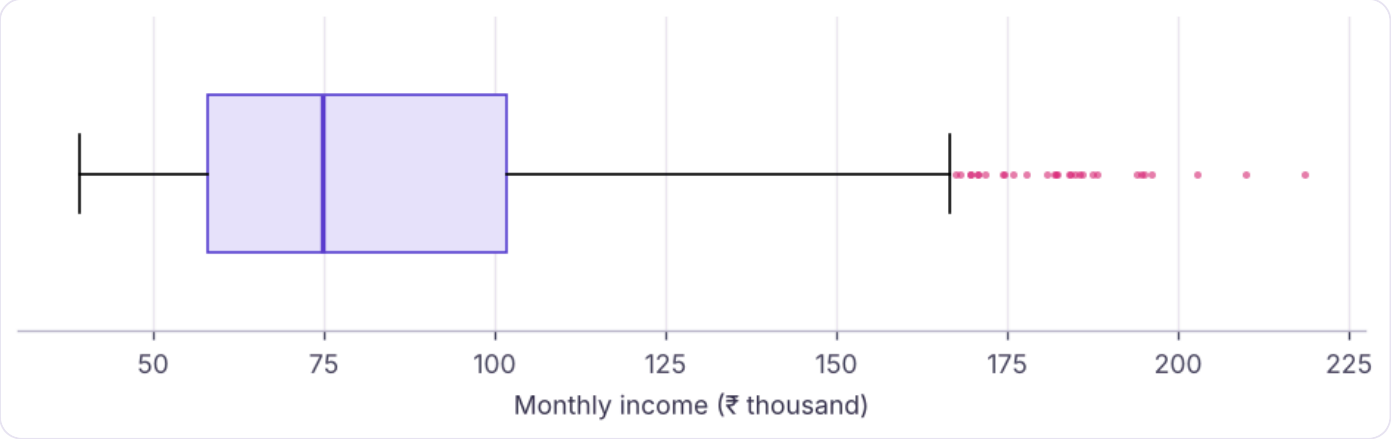
An **outlier** is far from most values. It may be an error (qty = 50 typed instead of 5) or real (one bulk order). Detect first, then decide.

Python

```
hr = pd.read_csv("hr_employees.csv")
inc = hr["monthly_income"]
q1, q3 = inc.quantile([0.25, 0.75]); iqr = q3 - q1
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(f"Q1 {q1:,.0f} | Q3 {q3:,.0f} | IQR fences {lo:,.0f} to {hi:,.0f}")
out = hr[(inc < lo) | (inc > hi)]
print(len(out), "IQR outliers;", out["job_title"].value_counts().head(3).to_dict())
z = (inc - inc.mean()) / inc.std()
print((z.abs() > 3).sum(), "rows with |z| > 3")
capped = inc.clip(lower=inc.quantile(0.01), upper=inc.quantile(0.99))    # winsorise at 1st/99th
percentile
print(f"max before {inc.max():,.0f} after capping {capped.max():,.0f}")
```

OUTPUT

```
Q1 58,062 | Q3 101,833 | IQR fences -7,594 to 167,489
29 IQR outliers; {'Sales Manager': 6, 'Finance Manager': 5, 'Operations Manager': 5}
19 rows with |z| > 3
max before 218,417 after capping 182,086
```



Monthly income: the box plot marks values beyond the IQR fences as points.

Senior roles earn more, so high incomes here are **real**, not errors. Options: keep them, analyse by job level, use medians, log-transform, or cap for models that are sensitive to extremes. Delete only values that are clearly impossible (negative age, qty 50 in a 1–5 item shop).

A reusable cleaning function

Python

```
def clean_orders(raw: pd.DataFrame) -> pd.DataFrame:
    """Return cleaned orders and print a short audit log."""
    df = raw.drop_duplicates().copy(); log = {"rows_in": len(raw), "dupes_removed": len(raw) -
len(df)}
    df["amount"] = pd.to_numeric(df["amount"].str.replace(r"[₹,]", "", regex=True),
errors="coerce")
    df["order_date"] = pd.to_datetime(df["order_date"], format="%Y-%m-%d", errors="coerce")
    log["bad_dates_dropped"] = int(df["order_date"].isna().sum()); df = df.dropna(subset=
["order_date"])
    log["amount_imputed"] = int(df["amount"].isna().sum()); df["amount"] =
df["amount"].fillna(df["amount"].median())
    df["city"] = df["city"].str.strip().str.title().replace({"New Delhi": "Delhi", "Bangalore":
"Bengaluru"}).fillna("Unknown")
    df["qty"] = df["qty"].fillna(1).clip(upper=10).astype(int); log["rows_out"] = len(df)
    print(log); return df
clean = clean_orders(raw)
assert clean["amount"].notna().all() and clean["order_id"].is_unique
print(clean)
```

OUTPUT

```
{'rows_in': 9, 'dupes_removed': 1, 'bad_dates_dropped': 2, 'amount_imputed': 1,
'rows_out': 6}
  order_id       city   amount order_date  qty
0       A1       Pune   1200.0 2025-01-05    2
2       A3      Delhi   3400.0 2025-01-07    3
5       A5      Kochi    780.0 2025-01-09    1
6       A6    Unknown  15000.0 2025-01-10   10
```

```
7       A7  Bengaluru   2050.0 2025-01-11    2
8       A8  Bengaluru   2050.0 2025-01-12    1
```

Summary

KEY TAKEAWAYS

- Profile first: dtypes, missing counts, uniques, duplicates, odd categories.
- Convert with `to_numeric` / `to_datetime(errors="coerce")` and count what was coerced; use nullable and category dtypes.
- Missing values: drop, fill (median, mode, group-wise `transform`), forward-fill, or flag; think about why they are missing.
- Define duplicates by key; `sort_values + drop_duplicates(keep="last")` keeps the latest record.
- Standardise text with `.str` and mapping dictionaries; detect outliers with IQR or z-scores and treat them deliberately.
- Wrap cleaning in a function with an audit log and assertions so it is repeatable.

Quick quiz

6 questions · tap an answer to check it

1. What does `pd.to_numeric(s, errors="coerce")` do with "abc"?

A Raises an error

B Converts it to NaN

C Keeps it as text

D Converts it to 0

Answer: B. You can then count and inspect the NaNs.

2. Which fill keeps group differences?

A `fillna(df.col.mean())`

B `fillna(df.groupby("g")["col"].transform("median"))`

C `dropna()`

D `fillna(0)`

Answer: B. `transform` returns a value per row from its group.

3. What are the IQR outlier fences?

A mean ± 2 sd

B Q1 − 1.5×IQR and Q3 + 1.5×IQR

C min and max

D median ± IQR

Answer: B. The IQR is Q3 − Q1.

4. How do you keep the latest record per order_id?

A `drop_duplicates()`

B Sort by update time, then `drop_duplicates(subset="order_id", keep="last")`

C `groupby().first()` without sorting

D `unique()`

Answer: B. Sorting decides which copy is "last".

5. Why add an `is_missing` flag before filling?

A Required by pandas

B The fact that a value was missing can itself be informative

C It speeds up filling

D To delete the column later

Answer: B. Models and analysts can use the flag.

6. Which dtype allows integers with missing values?

A int64

B Int64 (nullable)

C float32 only

D object

Answer: B. The capital-I `Int64` is pandas' nullable integer.

EXERCISE

Cleaning practice

Using `hr_employees.csv`: (1) profile every column. (2) Convert `hire_date` to datetime and compute tenure in years. (3) Check whether `years_at_company` agrees with the hire date. (4) Make `education` an ordered category. (5) Find income outliers per department using the IQR rule within each department.

Show solution (worked answer)

Python

```
hr = pd.read_csv("hr_employees.csv")
print(profile(hr))
hr["hire_date"] = pd.to_datetime(hr["hire_date"])
hr["tenure"] = ((pd.Timestamp("2025-12-31") - hr.hire_date).dt.days / 365.25).round(1)
print((hr["tenure"].round() - hr["years_at_company"]).abs().describe())
print(hr["education"].unique())
def iqr_flag(x):
    q1, q3 = x.quantile([.25, .75]); i = q3 - q1
    return (x < q1 - 1.5 * i) | (x > q3 + 1.5 * i)
hr["income_outlier"] = hr.groupby("department")["monthly_income"].transform(iqr_flag)
print(hr.groupby("department")["income_outlier"].sum())
```

INTERVIEW CORNER

Questions you may be asked

Q: How do you handle missing values? First find out why they are missing and how many. Then drop (few, random), impute (median/mode, group-wise, model-based, forward-fill for time series) or flag. Impute after the train/test split in ML to avoid leakage.

Q: How do you treat outliers? Check whether they are errors or real. Fix or remove errors; for real extremes use robust statistics (median), transformations (log), capping, or models robust to outliers.

Q: What checks do you run before trusting a dataset? Row counts vs source, uniqueness of keys, missing rates, value ranges, valid categories, date ranges, and totals reconciled to a trusted report.

PYTHON FOR DATA SCIENCE · LESSON 18

pandas III: GroupBy, Pivot Tables, Merge/Join and Reshape (melt/unpivot)

Summarise and combine data. Use groupby with agg, transform and filter, build pivot tables and crosstabs, join tables with merge (inner, left, right, outer, anti), stack with concat, and reshape between wide and long with melt, pivot, stack and unstack.

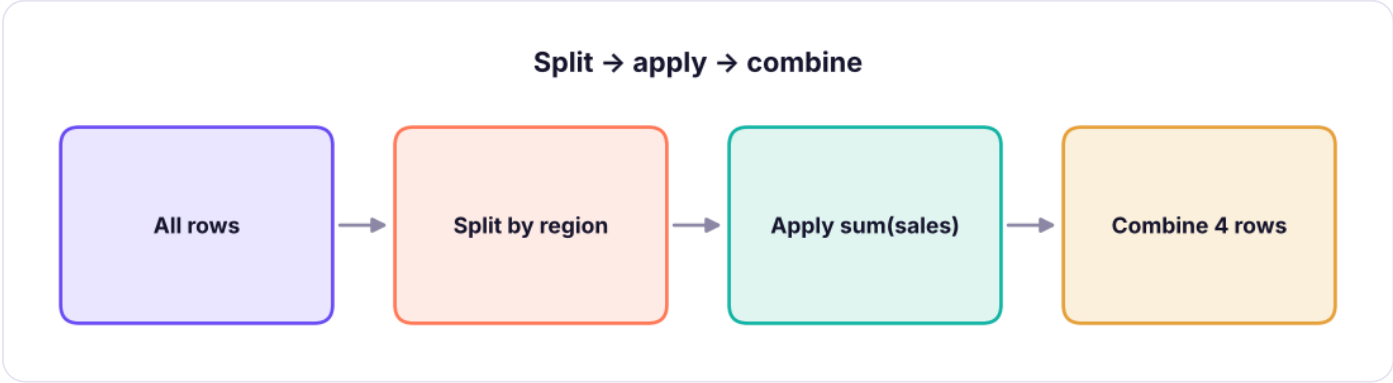
Python class 14 · pandas 2.3 / 3.0 · Intermediate · 120 min

By the end of this lesson you can

- Apply split-apply-combine with groupby, named aggregations and multiple keys
- Use transform for group-level columns and filter for group-level conditions
- Build pivot_table and crosstab summaries with totals and percentages
- Join DataFrames with merge, choose the join type, and validate the result
- Reshape wide ↔ long with melt (unpivot) and pivot, and combine with concat

Split–apply–combine

`groupby` **splits** rows into groups, **applies** a calculation to each group, and **combines** the results. It is SQL's `GROUP BY` and Excel's PivotTable.



groupby splits rows by key, applies an aggregation per group, and combines the answers.

Python

```
import pandas as pd, numpy as np
pd.set_option("display.max_columns", 20); pd.set_option("display.width", 160)
s = pd.read_csv("superstore_sales.csv", parse_dates=["order_date"])
print(s.groupby("region")["sales"].sum().sort_values(ascending=False))
print(s.groupby(["region", "category"])["profit"].sum().round(0).unstack())
```

OUTPUT

```
region
South    53952768.5
West     51958520.5
North    49018161.0
East     46484119.0
Name: sales, dtype: float64
category  Furniture  Office Supplies  Technology
region
East      -209426.0         434731.0   4149185.0
North     -236825.0         331848.0   4460806.0
South     -259193.0         349958.0   4611247.0
West      -112710.0         398257.0   4434286.0
```

Named aggregations

Python

```
summary = (s.groupby("category")
            .agg(orders=("order_id", "nunique"), lines=("row_id", "size"), sales=("sales",
"sum"),
                 profit=("profit", "sum"), avg_discount=("discount", "mean"), max_line=
("sales", "max"))
            .assign(margin_pct=lambda d: (100 * d.profit / d.sales).round(1))
            .sort_values("sales", ascending=False))
print(summary.round(2))
print(s.groupby("segment", as_index=False)["sales"].agg(["sum", "mean", "median",
"count"]).round(0))
```

OUTPUT

```
                orders  lines        sales       profit  avg_discount  max_line
margin_pct
category
Technology        1518   1892  156438412.5  17655523.58          0.07  619290.0
11.3
Furniture         1511   1917   34841853.0   -818153.31          0.13   93120.0
-2.3
Office Supplies   1870   2514   10133303.5   1514793.58          0.07   60830.0
14.9
       segment         sum     mean  median  count
```

```
0     Consumer  98603292.0  32066.0  8288.0   3075
1    Corporate  59113619.0  31427.0  8480.0   1881
2  Home Office  43696658.0  31965.0  9079.0   1367
```

Common aggregations: `sum`, `mean`, `median`, `min`, `max`, `count` (non-null), `size` (rows), `nunique`, `std`, `first`, `last`, or your own function.

transform and filter

`transform` returns a value **for every row** (the group result broadcast back), perfect for "share of group" and group-wise filling. `filter` keeps whole groups that meet a condition.

Python

```python
s["region_sales"] = s.groupby("region")["sales"].transform("sum")
s["share_of_region_%"] = (100 * s["sales"] / s["region_sales"]).round(3)
s["rank_in_region"] = s.groupby("region")["sales"].rank(ascending=False,
method="first").astype(int)
print(s.loc[s.rank_in_region ⩽ 2, ["region", "product_name", "sales", "share_of_region_%",
"rank_in_region"]].sort_values(["region", "rank_in_region"]))
loyal = s.groupby("customer_id").filter(lambda g: g["order_id"].nunique() ⩾ 8)
print(loyal["customer_id"].nunique(), "customers placed 8+ orders")
```

OUTPUT

```
     region       product_name     sales  share_of_region_%  rank_in_region
6244   East  Vertex Copier 262  472740.0              1.017               1
4219   East   Orbit Copier 746  442350.0              0.952               2
3669  North   Orbit Copier 746  619290.0              1.263               1
1653  North   Orbit Copier 746  526396.5              1.074               2
1142  South  Vertex Copier 262  551530.0              1.022               1
2087  South  Vertex Copier 262  551530.0              1.022               2
1779   West    Apex Copier 347  493080.0              0.949               1
3317   West   Orbit Copier 746  477738.0              0.919               2
41 customers placed 8+ orders
```

Pivot tables and crosstabs

Python

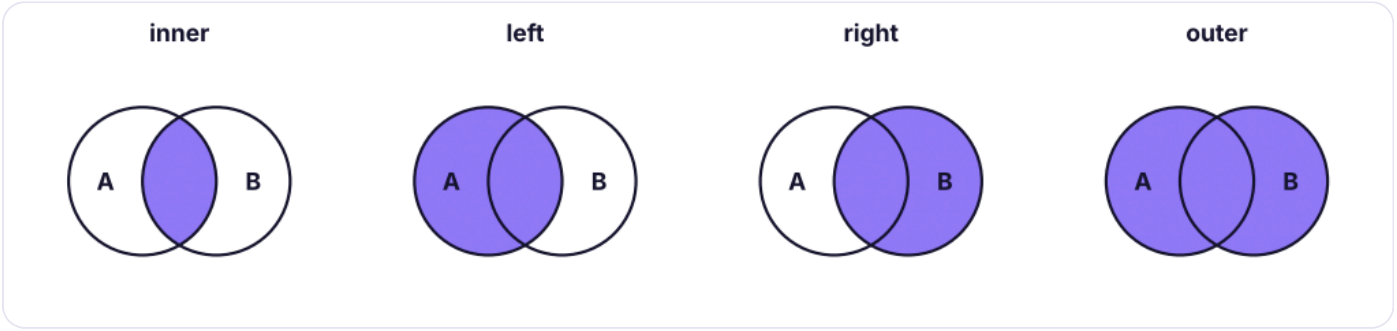
```python
s["year"] = s["order_date"].dt.year
pt = pd.pivot_table(s, values="sales", index="category", columns="year", aggfunc="sum",
margins=True, margins_name="Total")
print((pt / 1e6).round(2))
print(pd.crosstab(s["segment"], s["ship_mode"], normalize="index").mul(100).round(1))      #
row %
```

OUTPUT

```
year              2023   2024   2025   Total
category
Furniture         9.95  12.13  12.76   34.84
Office Supplies   2.61   3.55   3.97   10.13
Technology       45.08  49.72  61.65  156.44
Total            57.64  65.40  78.38  201.41
ship_mode    First Class  Same Day  Second Class  Standard Class
segment
Consumer            15.2       5.5          19.5            59.8
Corporate           13.2       5.0          20.3            61.5
Home Office         16.9       4.3          19.5            59.3
```

Merging (joins)

Real analysis needs several tables. `pd.merge` (or `df.merge`) joins on key columns like SQL joins.



Join types: which rows survive.

Python

```
orders = pd.DataFrame({"order_id": [1, 2, 3, 4], "cust": ["C1", "C2", "C2", "C9"], "amount":
[500, 250, 900, 120]})
cust = pd.DataFrame({"cust": ["C1", "C2", "C3"], "name": ["Asha", "Ravi", "Meera"], "city":
["Pune", "Delhi", "Kochi"]})
for how in ["inner", "left", "right", "outer"]:
    print(how, "→", len(orders.merge(cust, on="cust", how=how)), "rows")
print(orders.merge(cust, on="cust", how="left", indicator=True))
anti = orders.merge(cust, on="cust", how="left", indicator=True).query("_merge == 'left_only'")
print("orders with unknown customer:", anti["order_id"].tolist())
```

OUTPUT

```
inner → 3 rows
left → 4 rows
right → 4 rows
outer → 5 rows
   order_id cust  amount  name   city     _merge
0         1   C1     500  Asha   Pune       both
```

```
1         2   C2     250  Ravi  Delhi       both
2         3   C2     900  Ravi  Delhi       both
3         4   C9     120   NaN    NaN  left_only
orders with unknown customer: [4]
```

Argument	Use
`on="key"` / `on=["a", "b"]`	Same column name(s) in both
`left_on=` , `right_on=`	Different key names
`how=`	`inner` , `left` , `right` , `outer` , `cross`
`suffixes=("_x", "_y")`	Clashing column names
`indicator=True`	Adds `_merge` to see where each row came from
`validate="many_to_one"`	Raises an error if the key is not unique on the right

A real three-table join

Python

```python
products = pd.read_csv("products.csv"); customers = pd.read_csv("customers.csv")
lines = s[["order_id", "customer_id", "product_id", "sales", "quantity"]]
full = (lines.merge(products[["product_id", "list_price"]], on="product_id", how="left",
validate="many_to_one")
             .merge(customers[["customer_id", "city", "segment"]], on="customer_id", how="left",
validate="many_to_one"))
print(len(lines), len(full), full["list_price"].isna().sum(), "unmatched products")
full["realised_vs_list_%"] = (100 * full["sales"] / (full["list_price"] *
full["quantity"])).round(1)
print(full.groupby("segment")["realised_vs_list_%"].mean().round(1))
```

OUTPUT

```
6323 6323 0 unmatched products
segment
Consumer       91.0
Corporate      91.1
Home Office    90.4
Name: realised_vs_list_%, dtype: float64
```

WATCH OUT **Row explosion**

If the key is duplicated on **both** sides, a merge multiplies rows (many-to-many). Always compare row counts before and after, and use `validate=` to catch it.

concat: stacking tables

Python

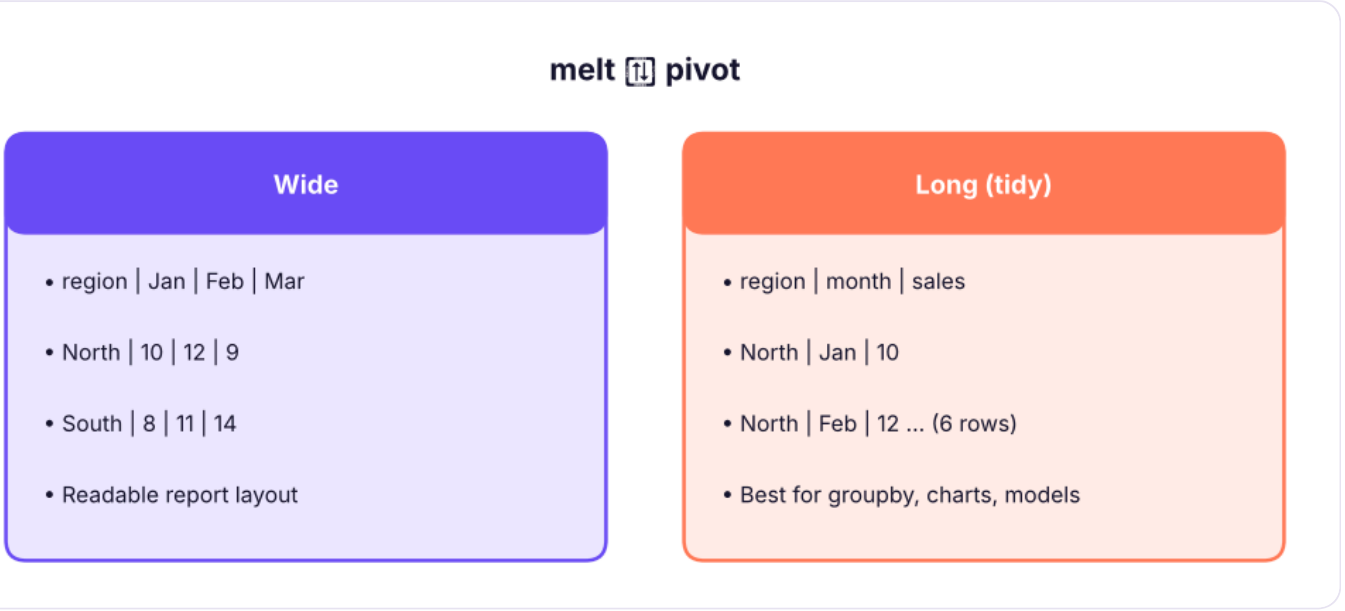
```
jan = pd.DataFrame({"store": ["A", "B"], "sales": [100, 80]}); feb = pd.DataFrame({"store":
["A", "C"], "sales": [120, 60]})
print(pd.concat([jan.assign(month="Jan"), feb.assign(month="Feb")], ignore_index=True))    #
rows (UNION ALL)
print(pd.concat([jan.set_index("store"), feb.set_index("store")], axis=1, keys=["Jan", "Feb"]))
# columns, aligned on index
```

OUTPUT

```
  store  sales month
0     A    100   Jan
1     B     80   Jan
2     A    120   Feb
3     C     60   Feb
         Jan    Feb
       sales  sales
store
A      100.0  120.0
B       80.0    NaN
C        NaN   60.0
```

Reshaping: wide ↔ long

Wide tables are easy to read (one column per month); **long** (tidy) tables are easy to analyse and chart (one row per observation). `melt` is pandas' **unpivot** (the same as Power Query's "Unpivot columns").



melt turns columns into rows; pivot turns rows into columns.

Python

```
wide = pd.DataFrame({"region": ["North", "South"], "Jan": [10, 8], "Feb": [12, 11], "Mar": [9, 14]})
long = wide.melt(id_vars="region", var_name="month", value_name="sales")
print(long)
back = long.pivot(index="region", columns="month", values="sales")[["Jan", "Feb", "Mar"]]
print(back)
st = back.stack(); print(st.head(3)); print(st.unstack().equals(back))
```

OUTPUT

```
  region month  sales
0  North   Jan     10
1  South   Jan      8
2  North   Feb     12
3  South   Feb     11
4  North   Mar      9
5  South   Mar     14
month   Jan  Feb  Mar
region
North    10   12    9
South     8   11   14
region  month
North   Jan      10
        Feb      12
        Mar       9
dtype: int64
True
```

`pivot` requires unique index/column pairs; `pivot_table` aggregates duplicates.

Python

```
monthly = (s.assign(month=s.order_date.dt.to_period("M").astype(str))
            .pivot_table(index="month", columns="category", values="sales",
aggfunc="sum").tail(3) / 1e6).round(2)
print(monthly)
print(monthly.reset_index().melt(id_vars="month", var_name="category",
value_name="sales_m").head(4))
```

OUTPUT

```
category  Furniture  Office Supplies  Technology
month
2025-10        1.15             0.31        4.68
2025-11        1.07             0.59        8.62
2025-12        1.31             0.40        5.94
     month         category  sales_m
0  2025-10        Furniture     1.15
```

```
1  2025-11        Furniture     1.07
2  2025-12        Furniture     1.31
3  2025-10  Office Supplies     0.31
```

Summary

KEY TAKEAWAYS

- `groupby(keys).agg(name=(col, func))` builds clean summaries; `transform` returns group values per row; `filter` keeps whole groups.
- `pivot_table` (with `margins=True`) and `crosstab` (with `normalize`) mirror Excel PivotTables.
- `merge(on=, how=)` joins tables; check row counts, use `indicator=True` for anti-joins and `validate=` to prevent row explosions.
- `concat` stacks rows (or aligns columns).
- `melt` = unpivot (wide → long); `pivot` / `pivot_table` = long → wide; `stack` / `unstack` move index levels.

Quick quiz

6 questions · tap an answer to check it

1. Which returns one value per original row?

A `groupby().agg()`

B `groupby().transform()`

C `groupby().size()`

D `pivot_table()`

Answer: B. transform broadcasts the group result back to each row.

2. Which merge keeps every order even when the customer is missing?

A inner

B left (orders on the left)

C right

D cross

Answer: B. Unmatched customer columns become NaN.

3. How do you find rows in A with no match in B?

A inner merge

B left merge with `indicator=True` and keep `_merge == "left_only"`

C outer merge without indicator

D concat

Answer: B. This is an anti-join.

4. What does `melt` do?

A Pivots long to wide

B Unpivots wide columns into rows (long format)

C Merges tables

D Deletes columns

Answer: B. `pivot` goes the other way.

5. A merge produced more rows than the left table. Most likely cause?

A A bug in pandas

B Duplicate keys on the right side

C Missing values

D Wrong dtype

Answer: B. Use `validate="many_to_one"` to catch this.

6. How do you add a grand-total row and column to a pivot table?

A `totals=True`

B `margins=True`

C `sum=True`

D `fill_value=0`

Answer: B. `margins_name` changes the label.

EXERCISE

GroupBy and merge practice

(1) Sales and profit by year and region, with margin %. (2) For each customer, the share of their spend in each category (use `transform`). (3) Top product per sub-category by profit. (4) Customers in `customers.csv` with no orders (anti-join). (5) Melt the year columns of a category × year pivot back to long format.

Show solution (worked answer)

Python

```
yr = s.groupby(["year", "region"]).agg(sales=("sales", "sum"), profit=("profit", "sum"))
yr["margin_%"] = (100 * yr.profit / yr.sales).round(1); print(yr)
cc = s.groupby(["customer_id", "category"])["sales"].sum().reset_index()
cc["share"] = cc.sales / cc.groupby("customer_id")["sales"].transform("sum")
top = s.groupby(["sub_category", "product_name"])
["profit"].sum().reset_index().sort_values("profit").groupby("sub_category").tail(1)
no_orders = customers.merge(s[["customer_id"]].drop_duplicates(), how="left",
indicator=True).query("_merge == 'left_only'")
print(len(no_orders))
pt = s.pivot_table(index="category", columns="year", values="sales", aggfunc="sum")
print(pt.reset_index().melt(id_vars="category", var_name="year", value_name="sales"))
```

INTERVIEW CORNER

Questions you may be asked

Q: Difference between `agg` and `transform`? `agg` reduces each group to one row; `transform` returns an output aligned to the original rows.

Q: How would you compute each product's share of its category's sales? `df.sales / df.groupby("category").sales.transform("sum")`.

Q: merge vs join vs concat? `merge` joins on columns (SQL-style); `join` is a shortcut that joins on the index; `concat` stacks along an axis without matching keys.

Q: What is tidy data? Each variable is a column, each observation a row, each value a cell; long format produced by `melt` is tidy.

PYTHON FOR DATA SCIENCE · LESSON 19

pandas IV: Dates, Time Series, Resampling and Rolling Windows

Work with time. Parse dates, extract parts with .dt, do date arithmetic, set a DatetimeIndex, resample to weeks, months and quarters, compute rolling and expanding windows, shifts and growth rates, and build a cohort-style monthly report with the daily sales data.

Python class 15 | pandas 2.3 / 3.0 | Intermediate | 110 min

By the end of this lesson you can

- Parse and format dates and extract year, month, weekday and quarter
- Do date arithmetic with Timedelta and DateOffset, including business days
- Resample time series and use rolling, expanding and exponentially weighted windows
- Compute period-over-period change, year-over-year growth and cumulative totals with shift, pct_change and cumsum
- Handle time zones and missing dates

Parsing dates

Python

```
import pandas as pd, numpy as np
pd.set_option("display.max_columns", 20); pd.set_option("display.width", 160)
d = pd.to_datetime(pd.Series(["2025-01-15", "2025-03-01", "2025-12-31"]))
print(d.dt.year.tolist(), d.dt.month_name().tolist(), d.dt.day_name().tolist(),
d.dt.quarter.tolist(), d.dt.isocalendar().week.tolist())
print(pd.to_datetime("05/01/2025", dayfirst=True).date(), pd.to_datetime("05/01/2025").date())
# 5 Jan vs 1 May!
print(pd.to_datetime("2025-01-15 14:30").strftime("%d %b %Y, %I:%M %p"), pd.Timestamp("2025-02-
28").is_month_end)
```

OUTPUT

```
[2025, 2025, 2025] ['January', 'March', 'December'] ['Wednesday', 'Saturday',
'Wednesday'] [1, 1, 4] [3, 9, 1]
2025-01-05 2025-05-01
15 Jan 2025, 02:30 PM True
```

WATCH OUT **Day-first dates**

Indian and European data often write 05/01/2025 for 5 January. Pass `dayfirst=True` or, better, an explicit `format="%d/%m/%Y"`. Check a few parsed values every time.

`.dt` accessor	Gives
`.year`, `.month`, `.day`, `.hour`	Parts
`.day_name()`, `.month_name()`	Text names
`.dayofweek` (Mon = 0), `.quarter`, `.isocalendar().week`	Calendar positions
`.to_period("M")`, `.to_period("Q")`	Month / quarter periods
`.floor("D")`, `.normalize()`	Truncate time
`.is_month_end`, `.days_in_month`	Calendar flags

Date arithmetic

Python

```
s = pd.read_csv("superstore_sales.csv", parse_dates=["order_date", "ship_date"])
s["ship_days"] = (s["ship_date"] - s["order_date"]).dt.days
print(s.groupby("ship_mode")["ship_days"].agg(["mean", "max"]).round(1))
t = pd.Timestamp("2025-01-31")
print(t + pd.Timedelta(days=30), t + pd.DateOffset(months=1), t + pd.offsets.MonthEnd(1), t +
pd.offsets.BDay(3))
print(np.busday_count("2025-10-01", "2025-11-01"), "working days in October 2025 (Mon-Fri)")
print(pd.date_range("2025-01-01", periods=4, freq="W-MON").date.tolist(), pd.date_range("2025-
01-01", "2025-06-30", freq="QE").date.tolist())
```

OUTPUT

```
               mean  max
ship_mode
First Class     1.5    2
Same Day        0.0    0
Second Class    3.0    4
Standard Class  5.6    7
2025-03-02 00:00:00 2025-02-28 00:00:00 2025-02-28 00:00:00 2025-02-05 00:00:00
23 working days in October 2025 (Mon-Fri)
[datetime.date(2025, 1, 6), datetime.date(2025, 1, 13), datetime.date(2025, 1, 20),
datetime.date(2025, 1, 27)] [datetime.date(2025, 3, 31), datetime.date(2025, 6, 30)]
```

`Timedelta` adds exact durations; `DateOffset` and offsets follow the calendar (month ends, business days).

A time series with a DatetimeIndex

Python

```
daily = pd.read_csv("daily_sales.csv", parse_dates=["date"]).set_index("date").sort_index()
print(daily.shape, daily.index.min().date(), "to", daily.index.max().date())
print(daily.loc["2023-03"].head(3))                    # partial-string indexing: all of March 2023
print(daily.loc["2023-12-24":"2023-12-27"])
full = pd.date_range(daily.index.min(), daily.index.max(), freq="D")
print("missing days:", len(full.difference(daily.index)))
```

OUTPUT

```
(1461, 1) 2022-01-01 to 2025-12-31
            units_sold
date
2023-03-01         260
2023-03-02         272
2023-03-03         234
            units_sold
date
2023-12-24         343
2023-12-25         270
2023-12-26         232
2023-12-27         257
missing days: 0
```

Resampling

`resample` is `groupby` for time: change the frequency and aggregate.

Python

```
monthly = daily["units_sold"].resample("ME").sum()
print(monthly.head(4))
q = daily["units_sold"].resample("QE").agg(["sum", "mean", "max"]).round(1)
print(q.tail(4))
wk = daily["units_sold"].resample("W-SUN").sum()
print(wk.describe().round(0))
print(daily.groupby(daily.index.day_name())
["units_sold"].mean().round(1).sort_values(ascending=False))
```

OUTPUT

```
date
2022-01-31    5998
2022-02-28    5797
```

```
2022-03-31    6756
2022-04-30    7156
Freq: ME, Name: units_sold, dtype: int64
              sum   mean  max
date
2025-03-31  30462  338.5  405
2025-06-30  34592  380.1  440
2025-09-30  34786  378.1  432
2025-12-31  33080  359.6  415
count     210.0
mean     2073.0
std       391.0
min       420.0
25%      1766.0
50%      2073.0
75%      2362.0
max      2801.0
Name: units_sold, dtype: float64
date
Sunday       323.3
Saturday     320.4
Friday       289.5
Tuesday      288.6
Thursday     288.5
Monday       287.9
Wednesday    287.3
Name: units_sold, dtype: float64
```

Frequencies: `D` day, `W` week, `ME` month end, `MS` month start, `QE` quarter, `YE` year, `h` hour, `min` minute.

Rolling and expanding windows

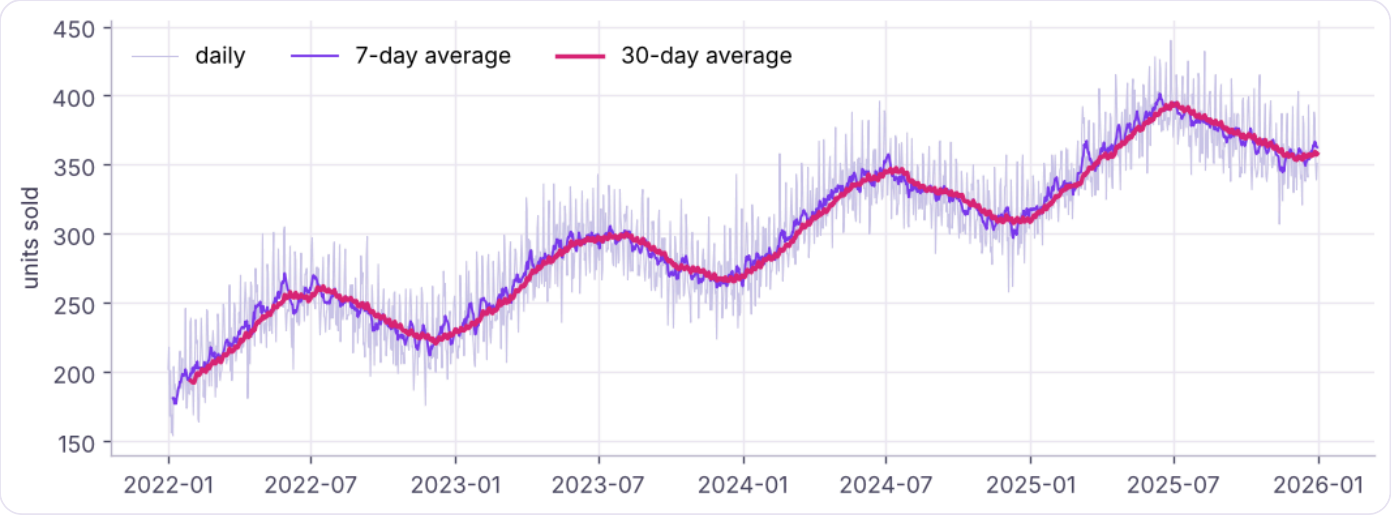
Python

```python
ts = daily["units_sold"]
df = pd.DataFrame({"units": ts, "ma_7": ts.rolling(7).mean(), "ma_30": ts.rolling(30).mean(),
"ewm_14": ts.ewm(span=14).mean(),
                   "max_so_far": ts.expanding().max(), "cum_units": ts.cumsum()})
print(df.iloc[[0, 6, 29, 400, -1]].round(1))
```

OUTPUT

```
            units   ma_7  ma_30  ewm_14  max_so_far  cum_units
date
2022-01-01    202    NaN    NaN   202.0       202.0        202
2022-01-07    154  181.1    NaN   176.0       218.0       1268
2022-01-30    239  198.0  193.7   203.2       246.0       5811
```

```
2023-02-05    273  252.6  244.1   251.9       305.0      93681
2025-12-31    351  362.6  357.9   359.0       440.0     435285
```



Daily units with 7-day and 30-day moving averages: averages smooth out daily noise and reveal trend and seasonality.

Shifts and growth

Python

```python
m = monthly.to_frame("units")
m["prev_month"] = m["units"].shift(1)
m["mom_%"] = (100 * m["units"].pct_change()).round(1)
m["yoy_%"] = (100 * m["units"].pct_change(12)).round(1)          # same month last year
m["ytd"] = m.groupby(m.index.year)["units"].cumsum()
print(m.tail(4))
yr = daily["units_sold"].resample("YE").sum()
print(yr, (100 * yr.pct_change()).round(1).tolist())
```

OUTPUT

```
            units  prev_month  mom_%  yoy_%     ytd
date
2025-09-30  11071     11798.0   -6.2   12.3   99840
2025-10-31  11291     11071.0    2.0   15.3  111131
2025-11-30  10721     11291.0   -5.0   14.7  121852
2025-12-31  11068     10721.0    3.2   15.1  132920
date
2022-12-31     85031
2023-12-31    100536
2024-12-31    116798
2025-12-31    132920
Freq: YE-DEC, Name: units_sold, dtype: int64 [nan, 18.2, 16.2, 13.8]
```

`shift(1)` is SQL's `LAG`; `shift(-1)` is `LEAD`; `pct_change(12)` on monthly data is year-over-year growth.

Time zones

Python

```
ts_utc = pd.Timestamp("2025-03-10 09:00", tz="UTC")
print(ts_utc.tz_convert("Asia/Kolkata"), ts_utc.tz_convert("America/New_York"))
naive = pd.Series(pd.to_datetime(["2025-03-10 09:00"]))
print(naive.dt.tz_localize("Asia/Kolkata").dt.tz_convert("UTC").tolist())
```

OUTPUT

```
2025-03-10 14:30:00+05:30 2025-03-10 05:00:00-04:00
[Timestamp('2025-03-10 03:30:00+0000', tz='UTC')]
```

Store timestamps in UTC; convert to local time only for reporting.

Mini case: monthly KPI report

Python

```
s["month"] = s["order_date"].dt.to_period("M")
kpi = s.groupby("month").agg(sales=("sales", "sum"), orders=("order_id", "nunique"), customers=
("customer_id", "nunique"))
kpi["aov"] = (kpi.sales / kpi.orders).round(0)
kpi["sales_3m_avg"] = kpi.sales.rolling(3).mean().round(0)
kpi["yoy_%"] = (100 * kpi.sales.pct_change(12)).round(1)
print((kpi.tail(6).assign(sales=lambda d: (d.sales / 1e6).round(2), sales_3m_avg=lambda d:
(d.sales_3m_avg / 1e6).round(2))))
first = s.groupby("customer_id")["order_date"].min().dt.to_period("M").rename("first_month")
s2 = s.join(first, on="customer_id")
new_vs_repeat = s2.assign(kind=np.where(s2.month == s2.first_month, "new",
"repeat")).groupby(["month", "kind"])["customer_id"].nunique().unstack().tail(3)
print(new_vs_repeat)
```

OUTPUT

```
         sales  orders  customers      aov  sales_3m_avg  yoy_%
month
2025-07   7.19      99         91  72627.0          5.86   49.2
2025-08   5.80      99         91  58608.0          5.65    0.8
2025-09   8.47     107        101  79141.0          7.15   58.9
2025-10   6.14     124        115  49518.0          6.80  -15.4
2025-11  10.28     152        139  67661.0          8.30   41.2
2025-12   7.65     134        121  57106.0          8.03   15.3
kind     new  repeat
```

```
month
2025-10  8.0   107.0
2025-11  4.0   135.0
2025-12  5.0   116.0
```

Summary

KEY TAKEAWAYS

- Parse with `pd.to_datetime` (give `format` or `dayfirst`); use `.dt` for parts and `to_period` for months/quarters.
- `Timedelta` = exact duration; `DateOffset` / `MonthEnd` / `BDay` = calendar-aware moves; `date_range` builds calendars.
- A sorted `DatetimeIndex` allows partial-string slicing (`df.loc["2025-03"]`) and `resample` .
- Rolling, expanding and EWM windows smooth series; `shift` = lag/lead, `pct_change(n)` = growth, `cumsum` = running totals.
- Keep timestamps in UTC, check for missing dates, and convert for display.

Quick quiz

6 questions · tap an answer to check it

1. What does `daily.resample("ME").sum()` produce?

A Daily totals

B Monthly totals (labelled at month end)

C A 7-day average

D An error

Answer: B. ME = month end frequency.

2. Which gives year-over-year growth on monthly data?

A `pct_change(1)`

B `pct_change(12)`

C `shift(-12)`

D `rolling(12).mean()`

Answer: B. Compare each month with the same month a year earlier.

3. `shift(1)` is equivalent to which SQL function?

A LAG

B LEAD

C ROW_NUMBER

D NTILE

Answer: A. `shift(-1)` is LEAD.

4. How do you parse "05/01/2025" as 5 January?

A `pd.to_datetime(x)`

B `pd.to_datetime(x, dayfirst=True)` or `format="%d/%m/%Y"`

C `.dt.day`

D `astype(str)`

Answer: B. The default assumes month first.

5. What does `ts.rolling(7).mean()` give for the first 6 rows?

A 0

B NaN (not enough values yet)

C The value itself

D An error

Answer: B. Use `min_periods=1` to allow partial windows.

6. What is the difference between `Timedelta(days=30)` and `DateOffset(months=1)` from 31 Jan?

A Timedelta adds exactly 30 days (2 Mar); DateOffset moves to the same day next month, clipped to 28 Feb

B They are identical

C Timedelta is calendar-aware

D DateOffset adds 30 days

Answer: A. Use calendar offsets for month logic.

EXERCISE

Time-series practice (daily_sales.csv)

(1) Which month had the highest total units? (2) Compute the 28-day rolling sum and its maximum. (3) Average units by weekday and month (a heatmap-ready pivot). (4) Year-over-year growth by quarter. (5) Flag days where units exceed the 30-day average by 50%.

Show solution (worked answer)

Python

```
print(monthly.idxmax(), monthly.max())
print(ts.rolling(28).sum().max())
pv = daily.assign(wd=daily.index.day_name(), m=daily.index.month).pivot_table(index="wd",
columns="m", values="units_sold", aggfunc="mean").round(0)
qq = ts.resample("QE").sum(); print((100 * qq.pct_change(4)).round(1).dropna())
spikes = daily[ts > 1.5 * ts.rolling(30).mean()]; print(len(spikes))
```

INTERVIEW CORNER

Questions you may be asked

Q: resample vs groupby? `resample` is a time-based groupby on a DatetimeIndex that also creates empty periods; `groupby` with `dt.to_period` is similar but only returns periods present in the data.

Q: How do you compute a 7-day moving average in pandas and in SQL? `s.rolling(7).mean()`; in SQL `AVG(x) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`.

Q: How do you handle missing dates in a daily series? Reindex to a full `date_range`, then decide: fill with 0 (no sales), forward-fill (stock levels), or interpolate (sensor readings).

PYTHON FOR DATA SCIENCE · LESSON 20

Data Visualisation I: Matplotlib and Choosing the Right Chart

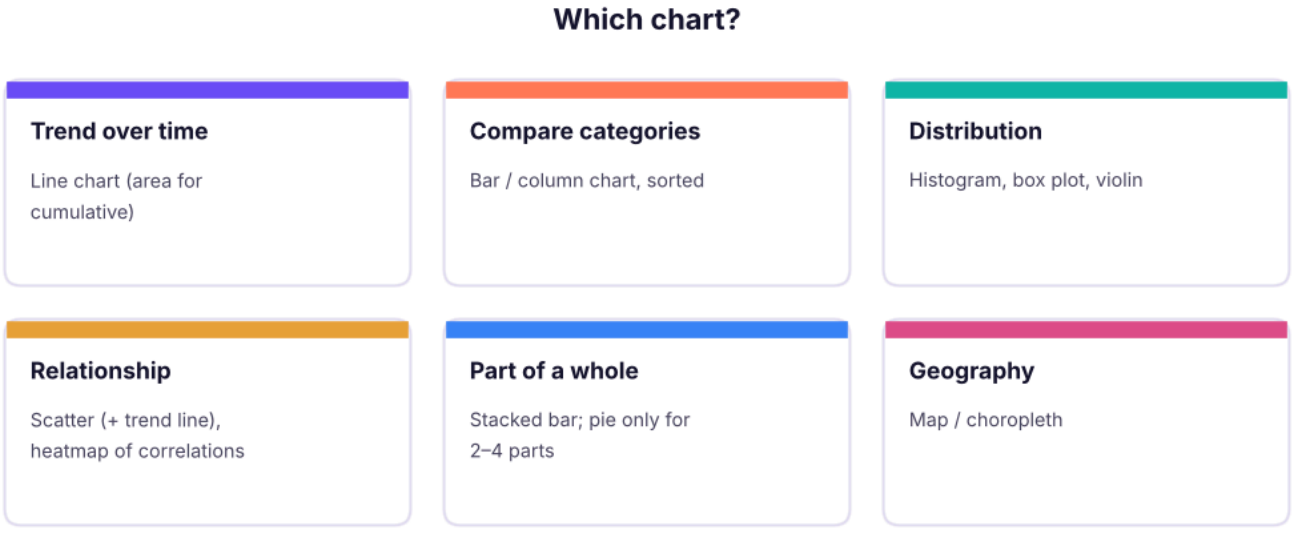
Pick the right chart for the question and build it with Matplotlib's object-oriented API. Line, bar, histogram, scatter, box and pie charts, subplots, annotations, styling and saving, plus design rules that make charts honest and readable.

Python class 16 · Matplotlib 3.10 · Intermediate · 110 min

By the end of this lesson you can

- Choose a chart type from the analytical question (trend, comparison, distribution, relationship, composition)
- Build charts with fig, ax = plt.subplots() and customise titles, labels, ticks, legends and colours
- Combine several charts with subplots and annotate key points
- Plot directly from pandas and save figures at publication quality
- Apply design rules: start bars at zero, remove clutter, label directly, use colour with purpose

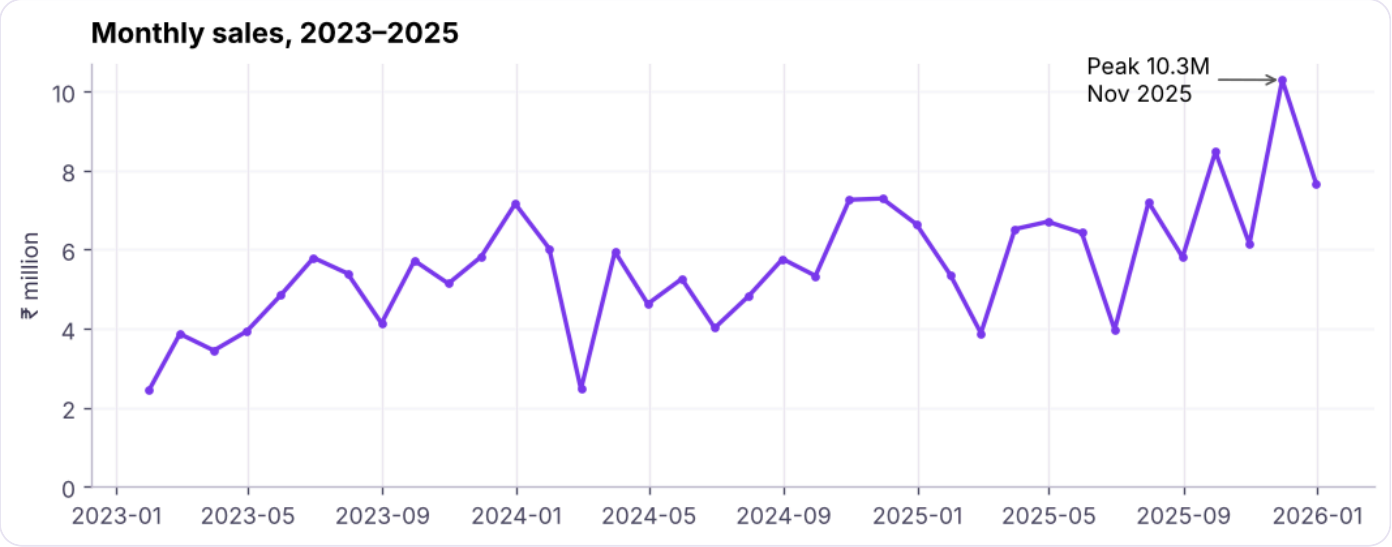
Choose the chart from the question



Start from the question, not the chart type.

The object-oriented API

Matplotlib has two styles. Use the **object-oriented** one: create a **Figure** (the canvas) and one or more **Axes** (plots), then call methods on the axes.



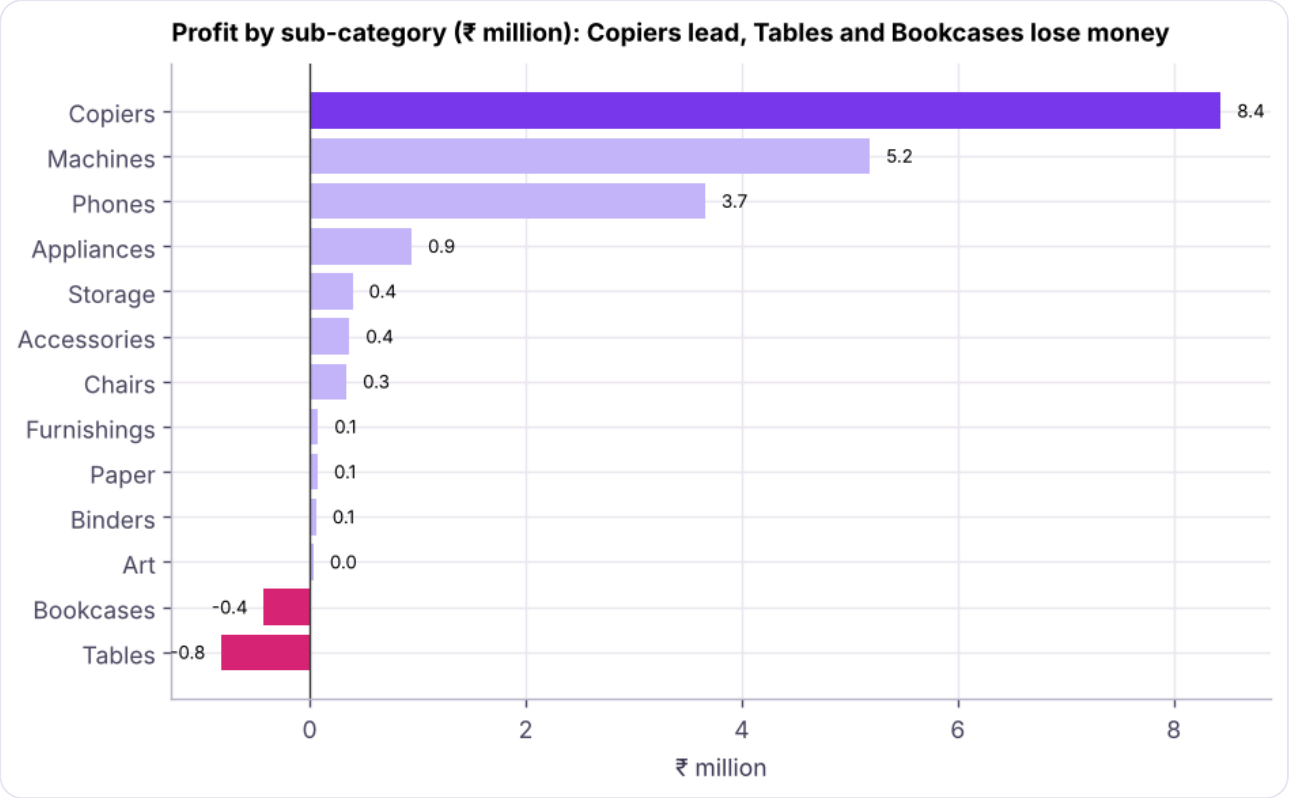
Monthly Superstore sales: a line chart shows the trend and the seasonal peaks.

Python

```python
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(9, 3.6))   # one figure, one axes
ax.plot(x, y)                              # draw
ax.set_title("..."); ax.set_xlabel("..."); ax.set_ylabel("...")
fig.savefig("chart.png", dpi=200, bbox_inches="tight")
plt.show()                                 # in scripts; notebooks display automatically
```

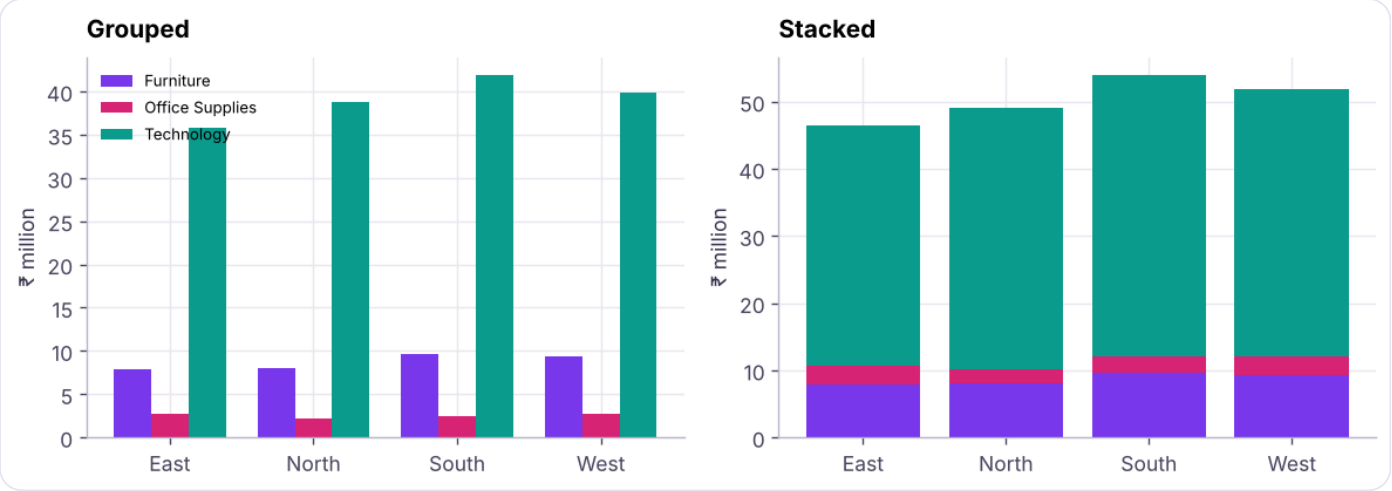
Element	Method
Title, axis labels	`ax.set_title` , `ax.set_xlabel` , `ax.set_ylabel`
Limits, ticks	`ax.set_ylim(0)` , `ax.set_xticks` , `ax.tick_params(rotation=45)`
Legend	`label="..."` in the plot call, then `ax.legend()`
Grid, spines	`ax.grid(axis="y")` , `ax.spines[["top","right"]].set_visible(False)`
Text, arrows	`ax.text` , `ax.annotate`
Reference lines	`ax.axhline(y)` , `ax.axvline(x)` , `ax.axvspan`
Number format	`ax.yaxis.set_major_formatter(...)`

Bar charts



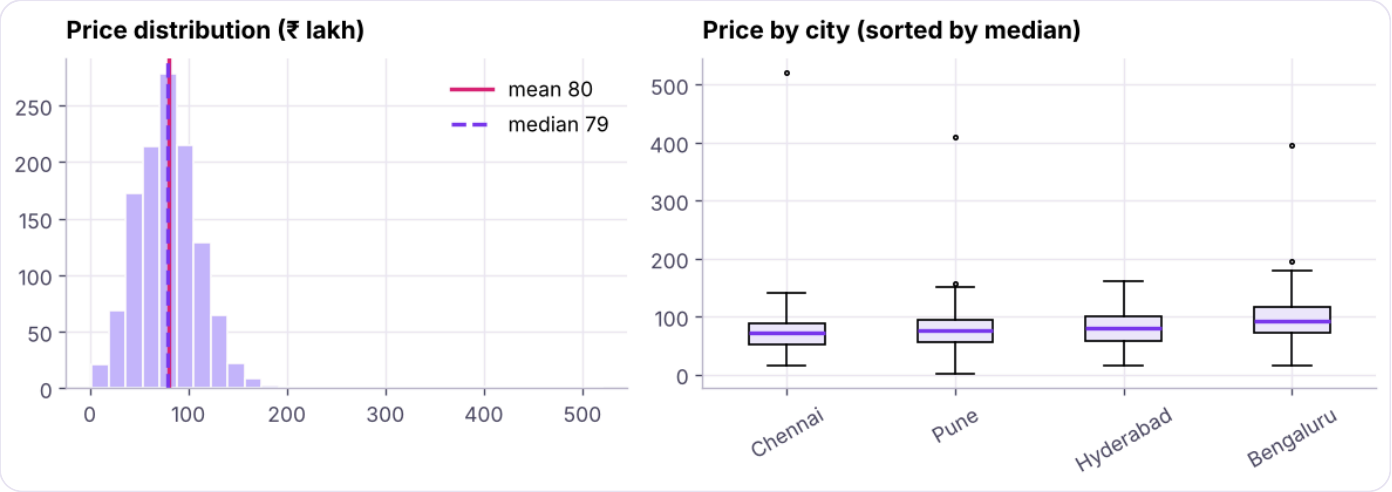
Sorted horizontal bars make category comparisons easy; one highlight colour draws the eye.

Grouped and stacked bars compare two dimensions:



Sales by region and category, as grouped bars (compare within region) and stacked bars (compare totals).

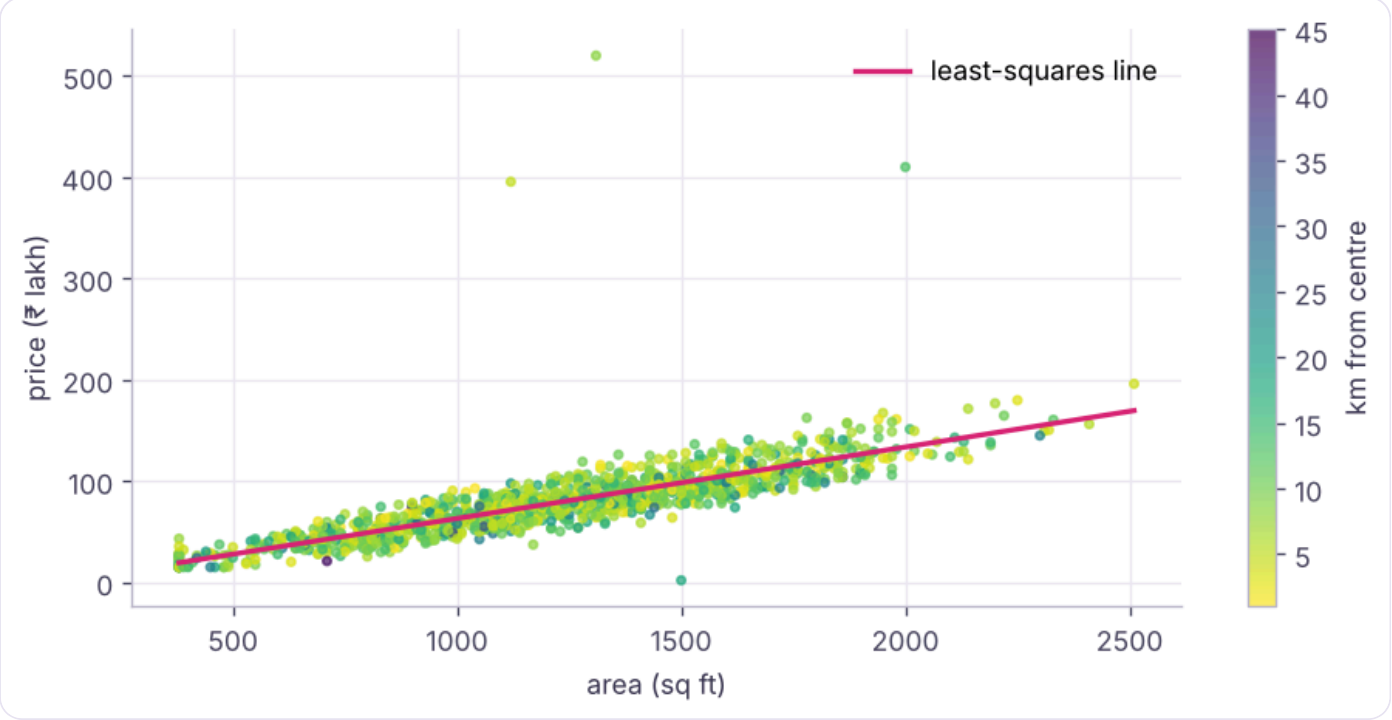
Distributions: histogram and box plot



House prices are right-skewed: the mean sits above the median. Box plots compare distributions across cities.

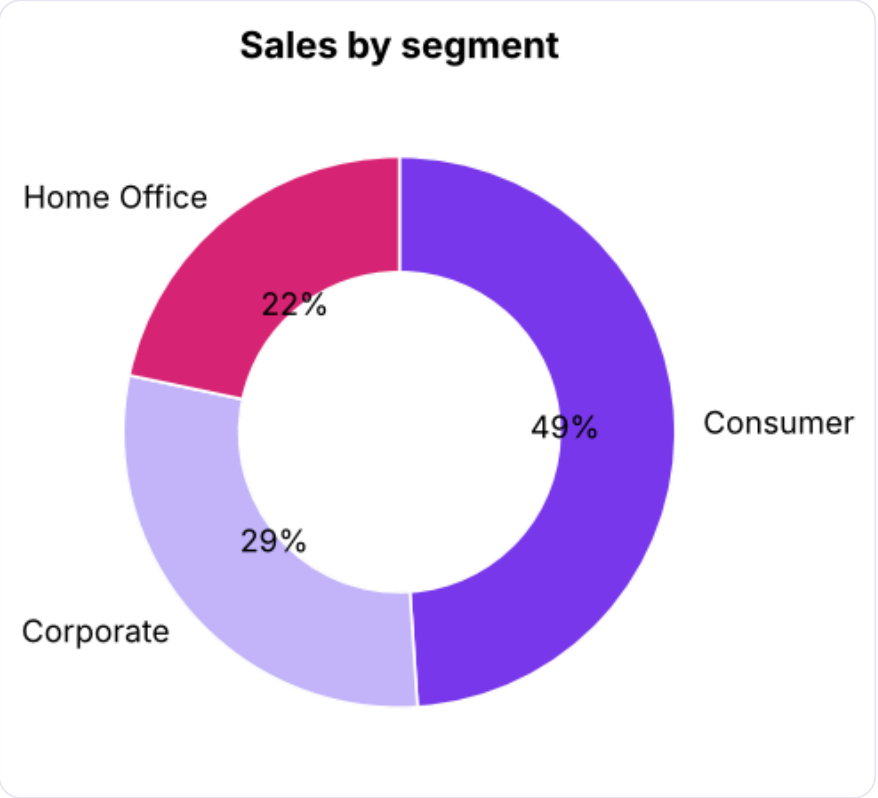
Bins matter: too few hide the shape, too many look noisy. Try `bins="auto"` or a few values.

Relationships: scatter



Larger houses cost more (r ≈ 0.8); colour shows a third variable, the distance to the centre.

Part of a whole

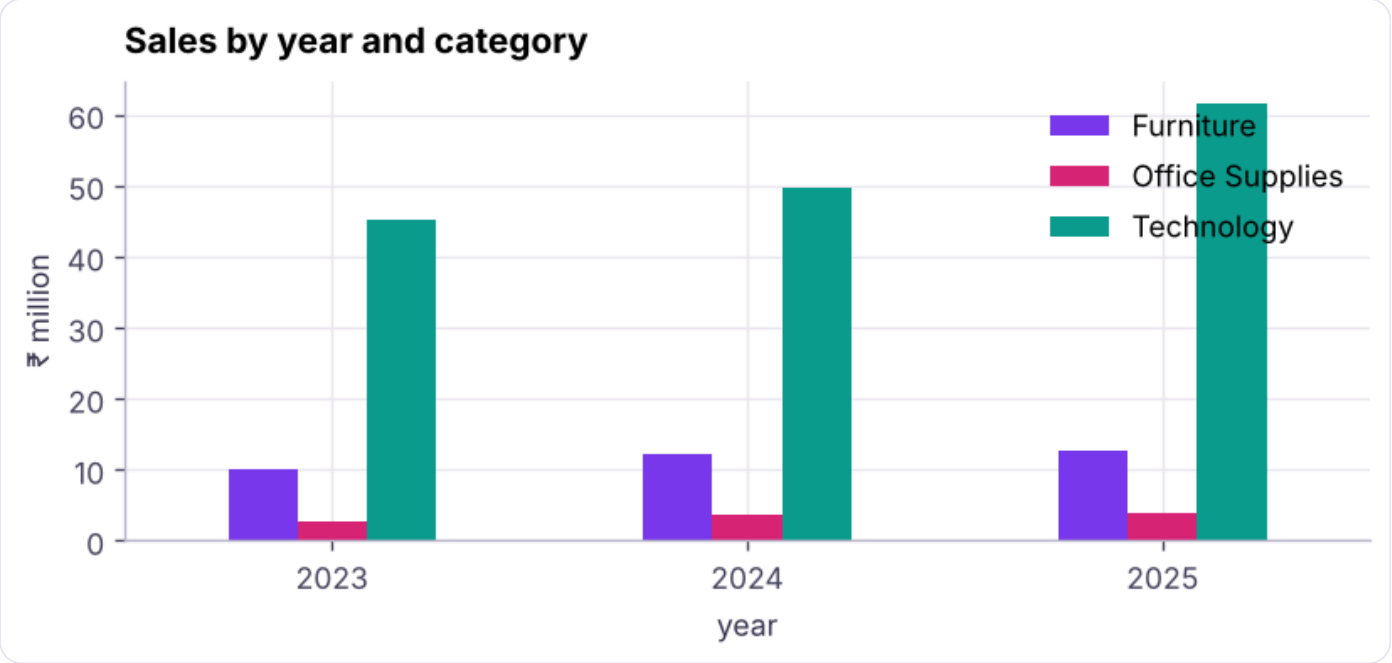


Pie charts work for a few parts; the donut variant leaves room for a label.

For more than 4 parts, or to compare parts across groups, use (stacked) bars.

Plotting from pandas

DataFrames and Series have `.plot()`, a quick wrapper around Matplotlib that returns an `ax` you can keep styling.



One line of pandas plotting, then a little Matplotlib styling.

`kind=` can be `line`, `bar`, `barh`, `hist`, `box`, `scatter` (with `x=`, `y=`), `area`, `pie`.

Subplots dashboards and saving

Python

```
fig, axes = plt.subplots(2, 2, figsize=(11, 7), constrained_layout=True)
axes[0, 0].plot(...); axes[0, 1].barh(...); axes[1, 0].hist(...); axes[1, 1].scatter(...)
fig.suptitle("Sales overview", fontsize=15, weight="bold")
fig.savefig("overview.png", dpi=200, bbox_inches="tight")   # also .svg / .pdf for crisp vector output
```

Design rules that make charts honest

Rule	Why
Bars start at zero	Truncated bars exaggerate differences
Sort categories by value	Faster comparison
Title states the insight	"Tables lose money" beats "Profit by sub-category"
Label directly, remove chart junk	Fewer eye movements
One highlight colour, grey for context	Colour carries meaning
Colour-blind safe palettes (viridis, Okabe-Ito)	About 1 in 12 men are colour-blind
Same scale when comparing panels	Avoid misleading comparisons
Avoid 3-D and dual axes when possible	They distort perception

Summary

KEY TAKEAWAYS

- Pick the chart from the question: trend → line, comparison → sorted bar, distribution → histogram/box, relationship → scatter, composition → stacked bar (pie only for a few parts).
- Use `fig, ax = plt.subplots()` and style the axes: titles, labels, limits, spines, legends, annotations.
- `plt.subplots(rows, cols)` builds small multiples; `fig.savefig(..., dpi=200, bbox_inches="tight")` exports.
- pandas `.plot()` is a fast start that returns a Matplotlib axes.
- Honest design: zero-based bars, sorted categories, insight titles, purposeful colour.

Quick quiz

6 questions · tap an answer to check it

1. Which chart best shows a trend over 36 months?

- **A** Pie
- **B** Line
- **C** Histogram
- **D** Box plot

Answer: B. Lines connect ordered time points.

2. Which chart shows the distribution of one numeric variable?

- **A** Scatter
- **B** Histogram
- **C** Pie
- **D** Stacked bar

Answer: B. Box and violin plots also work.

3. What does `fig, ax = plt.subplots(1, 2)` create?

- **A** One chart
- **B** One figure with two axes side by side
- **C** Two separate windows
- **D** A 2 × 2 grid

Answer: B. `ax` becomes an array of two axes.

4. Why should bar charts start at zero?

A Matplotlib requires it

B Bar length encodes value; a cut axis exaggerates differences

C It looks better

D It saves space

Answer: B. Line charts can zoom; bars should not.

5. When is a pie chart acceptable?

A For 12 categories

B For a few parts of one whole, when exact comparison is not needed

C For trends

D Never

Answer: B. Otherwise use bars.

6. How do you save a sharp image without cut-off labels?

A `plt.show()`

B `fig.savefig("c.png", dpi=200, bbox_inches="tight")`

C Screenshot

D `fig.close()`

Answer: B. SVG/PDF give vector output.

EXERCISE

Chart practice

(1) Line chart of monthly profit with a horizontal line at zero. (2) Sorted horizontal bar of sales by state (top 10). (3) Histogram of `discount`. (4) Scatter of `discount` vs `profit` (what do you see?). (5) A 2 × 2 dashboard of these four charts saved as PNG.

Show solution (worked answer)

Python

```
mp = s.set_index("order_date")["profit"].resample("ME").sum()
fig, axes = plt.subplots(2, 2, figsize=(11, 7), constrained_layout=True)
axes[0, 0].plot(mp.index, mp / 1e6); axes[0, 0].axhline(0, color="grey")
st = s.groupby("state")["sales"].sum().nlargest(10).sort_values(); axes[0,
1].barh(st.index, st / 1e6)
axes[1, 0].hist(s.discount, bins=9)
axes[1, 1].scatter(s.discount, s.profit, s=4, alpha=.3)   # profit falls as discount rises
fig.savefig("dashboard.png", dpi=150, bbox_inches="tight")
```

INTERVIEW CORNER

Questions you may be asked

Q: How do you choose a chart? Start from the question (trend, comparison, distribution, relationship, composition), the number of variables and the audience, then remove anything that does not help the message.

Q: What makes a chart misleading? Truncated bar axes, inconsistent scales, 3-D effects, cherry-picked ranges, dual axes with arbitrary scaling, and areas that do not match values.

Q: Matplotlib vs Seaborn vs Plotly? Matplotlib is the low-level foundation with full control; Seaborn adds statistical charts with good defaults on top; Plotly makes interactive web charts.

PYTHON FOR DATA SCIENCE · LESSON 21

Data Visualisation II: Seaborn Statistical Charts and Interactive Plotly

Make statistical charts quickly with Seaborn (distributions, categorical comparisons, relationships, correlation heatmaps, pair plots and facets) using the HR attrition data, and build interactive charts and simple dashboards with Plotly Express and Streamlit.

Python class 16 · Seaborn 0.13 · Plotly 6 · Intermediate · 100 min

By the end of this lesson you can

- Use Seaborn's figure-level and axes-level functions with hue, col and row
- Compare distributions across groups with histplot, kdeplot, boxplot, violinplot and countplot
- Show relationships with scatterplot, regplot, heatmaps of correlations and pairplot
- Build interactive charts with Plotly Express and know when to use Streamlit
- Turn charts into a clear data story

Seaborn in one idea

Seaborn works directly with **long DataFrames**: you name the columns for `x`, `y`, `hue` (colour), `col` / `row` (small multiples), and it groups, aggregates and adds legends for you.

Python

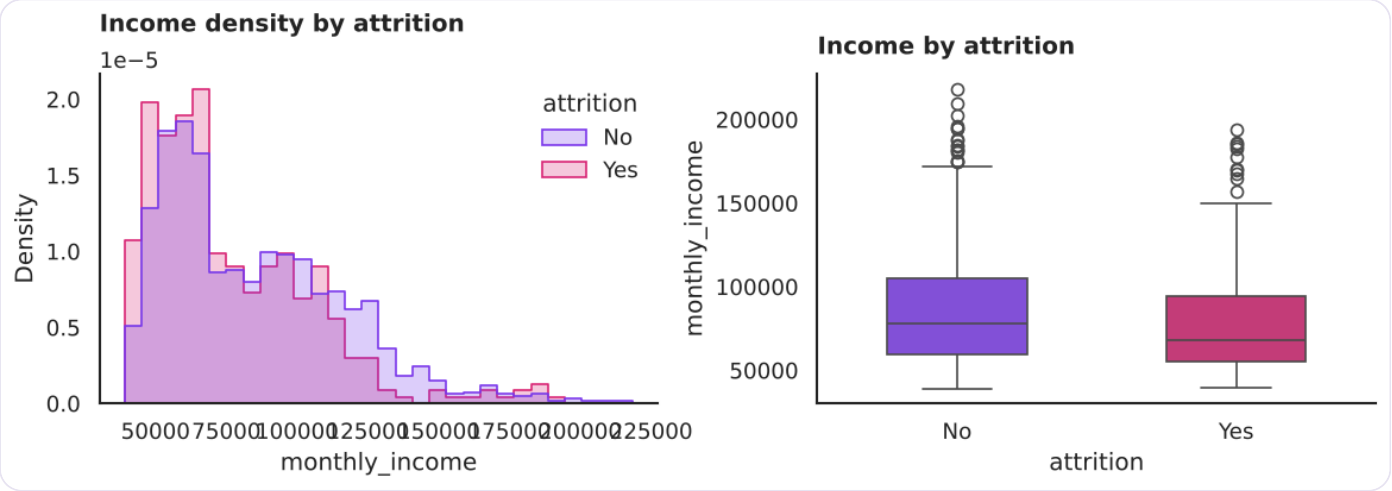
```
import pandas as pd, numpy as np, seaborn as sns, matplotlib.pyplot as plt
hr = pd.read_csv("hr_employees.csv")
hr["left"] = (hr["attrition"] == "Yes").astype(int)
print(hr.shape, f"attrition rate {hr.left.mean():.1%}", sns.__version__)
print(hr.groupby("overtime")["left"].mean().round(3).to_dict(), hr.groupby("department")
["left"].mean().round(3).sort_values().to_dict())
```

OUTPUT

```
(1500, 16) attrition rate 25.9% 0.13.2
{'No': 0.216, 'Yes': 0.367} {'Operations': 0.219, 'Finance': 0.227, 'HR': 0.229, 'IT':
0.243, 'R&D': 0.26, 'Sales': 0.304}
```

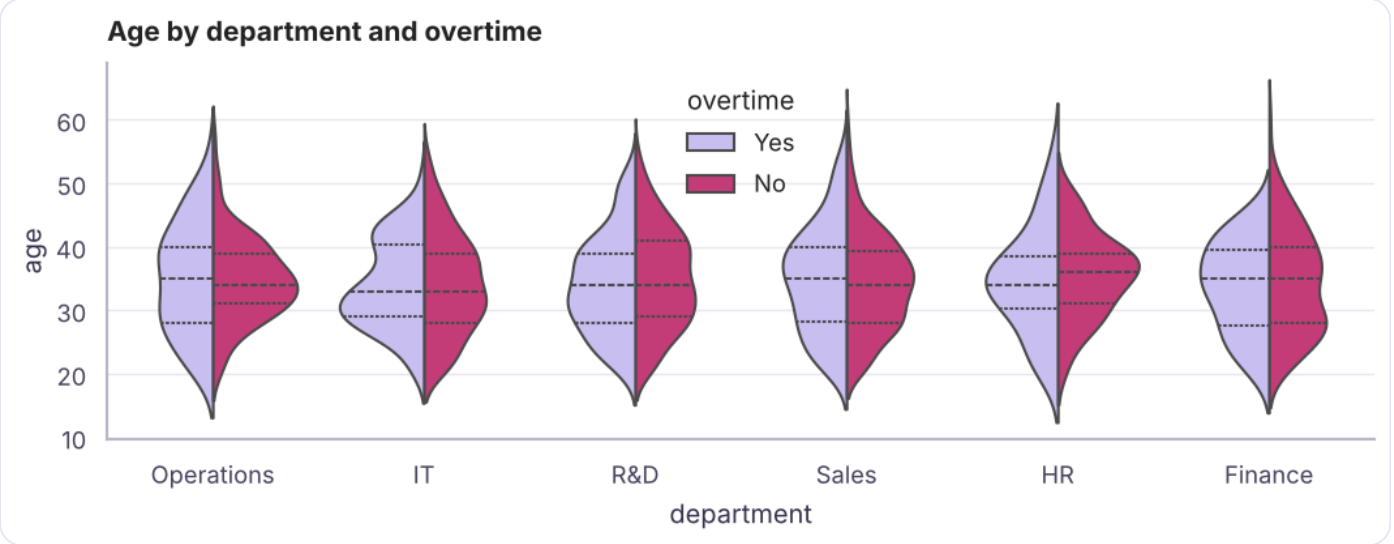
Question	Seaborn function
Distribution of one variable	`histplot`, `kdeplot`, `ecdfplot`
Distribution by group	`boxplot`, `violinplot`, `stripplot`, `boxenplot`
Counts of categories	`countplot`
Average per category (with CI)	`barplot`, `pointplot`
Two numeric variables	`scatterplot`, `regplot`, `lmplot`, `jointplot`
Many variables	`pairplot`, `heatmap` of `df.corr()`
Small multiples	`displot`, `catplot`, `relplot` with `col=` / `row=`

Distributions



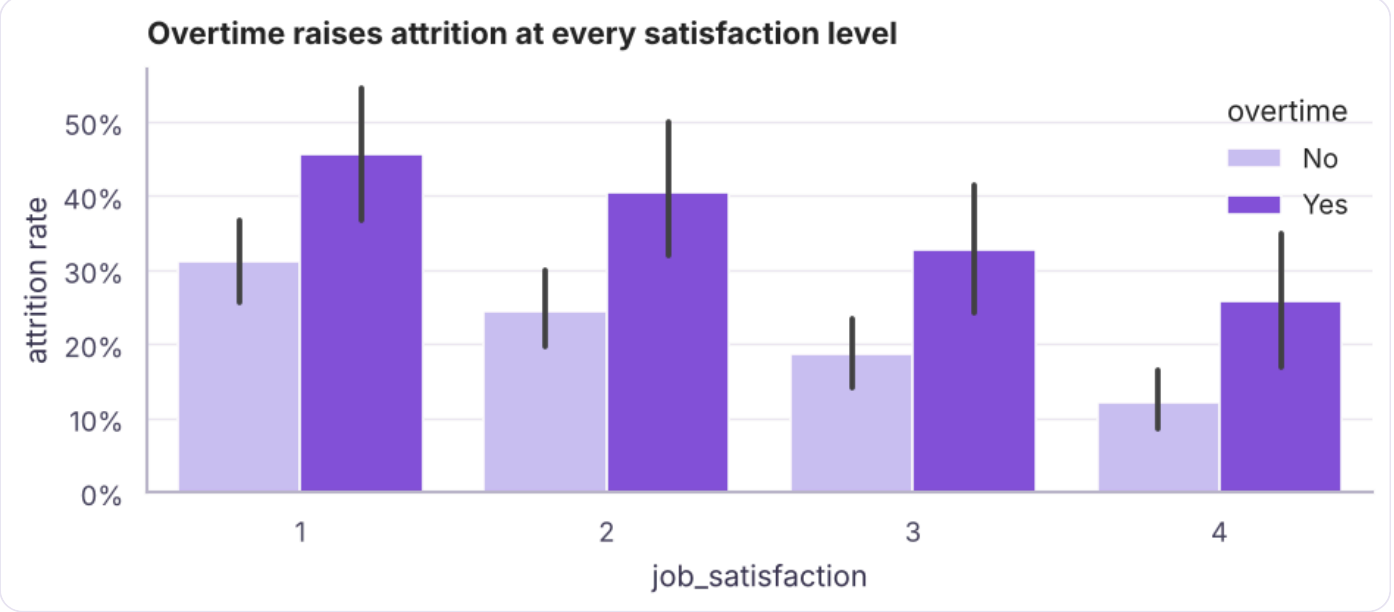
Income distribution for leavers vs stayers: leavers cluster at lower incomes.

`common_norm=False` scales each group separately, so a small group (leavers) is not dwarfed by a big one.



Violin plots combine a box plot with the distribution shape; split by overtime.

Categorical comparisons



Attrition rate by overtime and job satisfaction. Bars show the mean with a 95% confidence interval.

Python

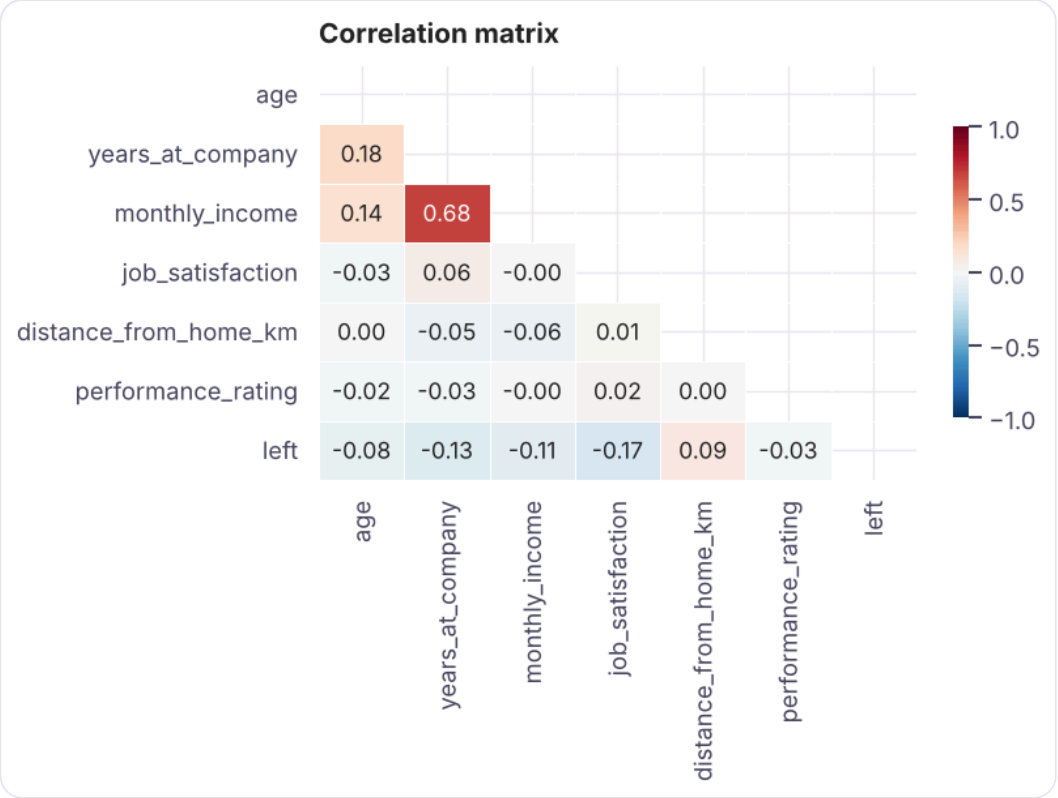
```
print((hr.pivot_table(index="job_satisfaction", columns="overtime", values="left",
aggfunc="mean") * 100).round(1))
```

OUTPUT

```
overtime            No   Yes
job_satisfaction
1                 31.1  45.5
2                 24.4  40.5
```

```
3                 18.7  32.8
4                 12.0  25.8
```

Relationships and correlation



Correlation heatmap of numeric HR variables (Pearson r). Only the lower triangle is shown.

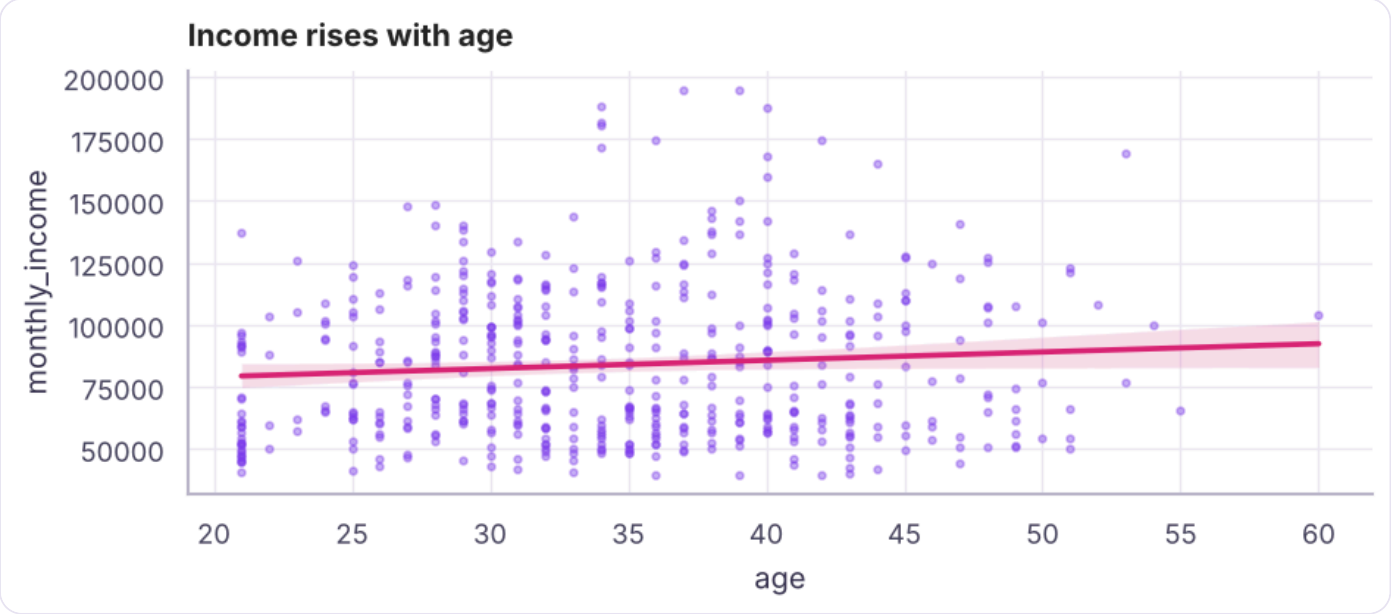
Python

```python
print(corr["left"].drop("left").sort_values())
```

OUTPUT

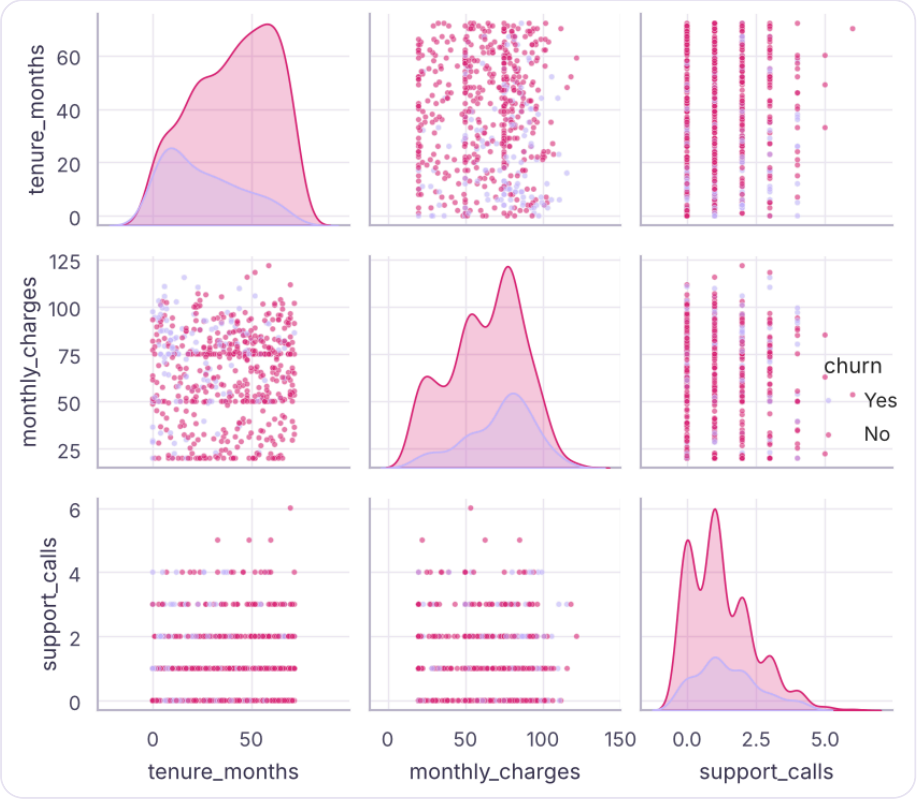
```
job_satisfaction        -0.17
years_at_company        -0.13
monthly_income          -0.11
age                     -0.08
performance_rating      -0.03
distance_from_home_km    0.09
Name: left, dtype: float64
```

Correlation measures **linear** association between −1 and +1. It does not prove causation, and it can miss non-linear patterns: always look at the scatter too.

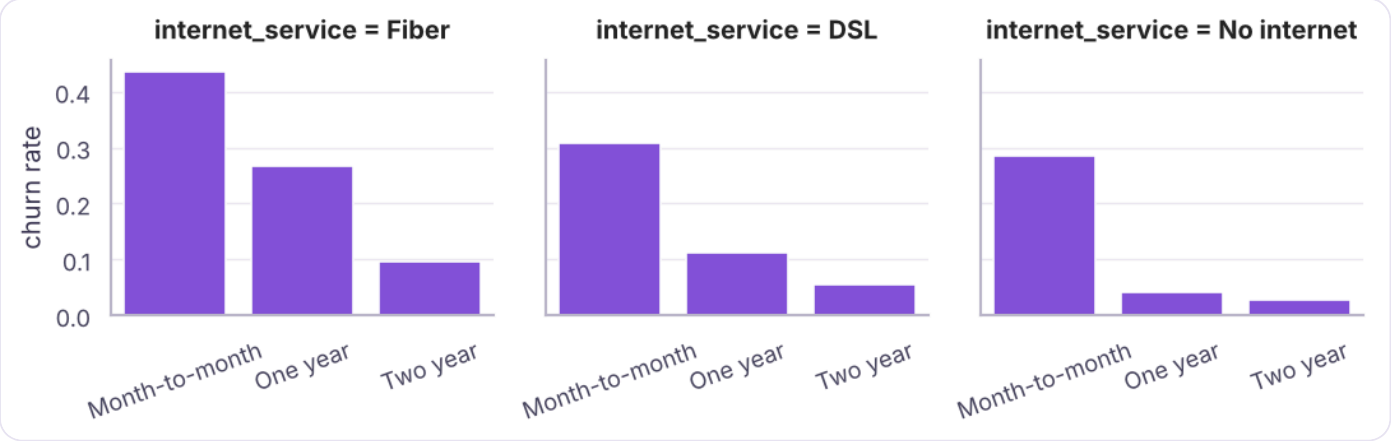


regplot adds a fitted line with a confidence band; lmplot repeats it per group.

Pair plots and facets



A pair plot shows every pairwise scatter and each distribution, coloured by churn (telecom data).



Facets (small multiples) with catplot: churn rate by contract type, split by internet service.

Python

```
print((tc.pivot_table(index="contract", columns="internet_service", values="churned",
aggfunc="mean") * 100).round(1))
```

OUTPUT

```
internet_service   DSL  Fiber  No internet
contract
Month-to-month    30.8   43.5         28.5
One year          11.1   26.7          4.1
Two year           5.6    9.5          2.8
```

Interactive charts with Plotly Express

Plotly makes charts you can hover, zoom and filter in a browser or notebook. The syntax mirrors Seaborn:

Python

```
import plotly.express as px
fig = px.scatter(tc, x="tenure_months", y="monthly_charges", color="churn",
                 hover_data=["contract", "payment_method"], opacity=.6,
                 title="Churn by tenure and charges")
fig.show()                          # interactive in notebooks
fig.write_html("churn.html")        # share as a standalone web page

px.line(monthly_df, x="month", y="sales", color="category")
px.bar(region_df, x="region", y="sales", text_auto=".2s")
px.histogram(hr, x="monthly_income", color="attrition", barmode="overlay")
px.choropleth(...)                  # maps
```

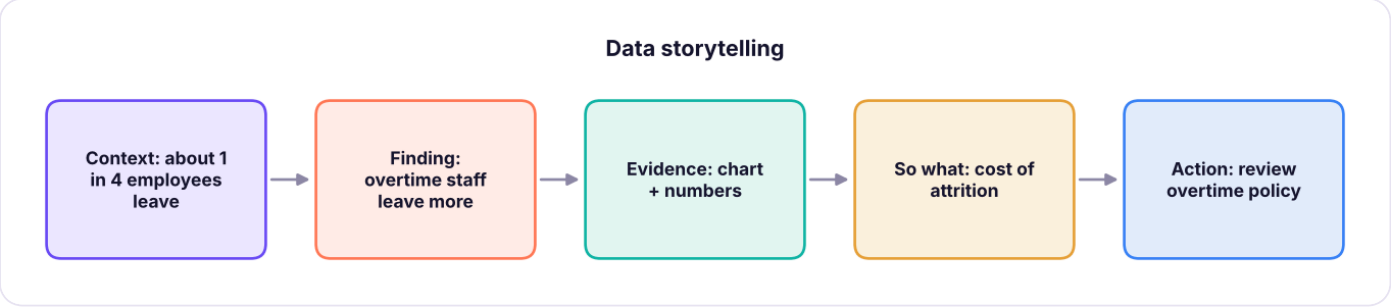
A mini dashboard app with Streamlit

Python

```python
# app.py  →  run with:  streamlit run app.py
import streamlit as st, pandas as pd, plotly.express as px
df = pd.read_csv("telecom_churn.csv")
contract = st.sidebar.multiselect("Contract", df.contract.unique(),
default=list(df.contract.unique()))
d = df[df.contract.isin(contract)]
st.title("Churn dashboard")
st.metric("Churn rate", f"{(d.churn == 'Yes').mean():.1%}")
st.plotly_chart(px.histogram(d, x="tenure_months", color="churn", barmode="overlay"))
```

Streamlit turns a Python script into a web app with filters, which is useful for sharing analyses and ML demos (see the GenAI track for chat apps).

From charts to a data story



The story arc: context, finding, evidence, recommendation.

1. **One message per chart**, written as the title.
2. Show the comparison that proves it (overtime vs not), not every chart you made.
3. Quantify impact (people, ₹, %).
4. End with a recommendation and its owner.

Summary

KEY TAKEAWAYS

- Seaborn plots long DataFrames by column name with `hue`, `col` and `row`; it sits on Matplotlib, so you can still style the axes.
- Distributions: `histplot` (with `common_norm=False`), `kdeplot`, `boxplot`, `violinplot`; categories: `countplot`, `barplot` with confidence intervals.
- Relationships: `scatterplot`, `regplot`, `pairplot`, correlation `heatmap` (correlation ≠ causation).
- Facets (`catplot`, `relplot`, `displot`) create small multiples.
- Plotly Express gives interactive charts; Streamlit turns scripts into dashboards. Tell a story: one message per chart.

Quick quiz

6 questions · tap an answer to check it

1. What does `hue="attrition"` do in Seaborn?

A Changes the theme

B Colours the marks by the attrition category

C Filters to attrition = Yes

D Adds a title

Answer: B. `col=` would create separate panels instead.

2. Which chart shows the mean per category with a confidence interval?

A countplot

B barplot

C histplot

D heatmap

Answer: B. `countplot` shows counts.

3. A correlation of −0.8 means...

A No relationship

B A strong negative linear relationship

C One variable causes the other to fall

D A weak relationship

Answer: B. Correlation does not imply causation.

4. Which function creates small multiples of bar charts?

A barplot

B catplot(kind="bar", col=...)

C heatmap

D pairplot

Answer: B. Figure-level functions accept `col` and `row`.

5. What is the main advantage of Plotly over Matplotlib?

A Faster to print

B Interactive hover, zoom and filtering in the browser

C More chart types for PDFs

D No code needed

Answer: B. Export with `write_html`.

6. Why use `common_norm=False` in a histplot with hue?

- **A** So each group's density is scaled to its own size and small groups stay visible
- **B** To remove the legend
- **C** To use counts
- **D** To normalise to 100

Answer: A. Otherwise the larger group dominates.

EXERCISE **Seaborn practice (telecom_churn.csv)**

(1) Histogram of `monthly_charges` by churn. (2) Box plot of `tenure_months` by contract. (3) Churn rate by payment method, sorted. (4) Correlation heatmap of numeric columns plus `churned`. (5) Write three insight titles from what you see.

Show solution (worked answer)

Python

```
sns.histplot(tc, x="monthly_charges", hue="churn", common_norm=False, stat="density")
sns.boxplot(tc, x="contract", y="tenure_months")
order = tc.groupby("payment_method")["churned"].mean().sort_values(ascending=False).index
sns.barplot(tc, x="payment_method", y="churned", order=order, errorbar=None)
sns.heatmap(tc.select_dtypes("number").corr(), annot=True, fmt=".2f", cmap="RdBu_r",
center=0)
# e.g. "Month-to-month customers churn far more", "Churners have short tenure", "Churn
rises with support calls"
```

INTERVIEW CORNER

Questions you may be asked

Q: How would you visualise the relationship between a categorical and a numeric variable? Box or violin plots per category (distribution), or bar/point plots of the mean with confidence intervals.

Q: How do you show correlations among many numeric variables? A correlation heatmap (masked upper triangle) followed by scatter plots for the interesting pairs; a pair plot for a handful of variables.

Q: How do you present findings to non-technical stakeholders? Lead with the recommendation, show one clear chart per point with an insight title, quantify impact, and keep methods in an appendix.

PYTHON FOR DATA SCIENCE · LESSON 22

Exploratory Data Analysis (EDA) Case Study: Telecom Customer Churn

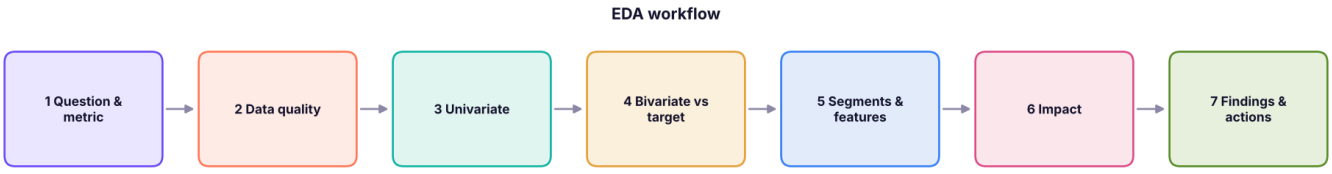
Run a complete, structured EDA on 3,000 telecom customers. Frame the question, check data quality, do univariate, bivariate and segment analysis, engineer features, quantify revenue at risk and finish with findings and recommendations: the template you will reuse for every dataset and for the ML churn model.

Python class 17 · pandas 3 · Seaborn 0.13 · Intermediate · 150 min

By the end of this lesson you can

- Follow a repeatable EDA checklist from question to recommendation
- Check data quality and consistency between related columns
- Analyse univariate distributions and bivariate relationships with the target
- Build segments and quantify business impact (revenue at risk)
- Write findings as clear, numbered insights with actions

The EDA checklist



A structured EDA moves from the business question to actions.

Business question. A telecom operator loses customers every month. *Which customers churn, why, and where should retention money go first?* **Target:** `churn` (Yes/No). **Metric:** churn rate = churned ÷ customers.

1. Load and first look

Python

```
import pandas as pd, numpy as np, seaborn as sns, matplotlib.pyplot as plt
pd.set_option("display.max_columns", 20); pd.set_option("display.width", 160)
tc = pd.read_csv("telecom_churn.csv")
tc["churned"] = (tc["churn"] == "Yes").astype(int)
print(tc.shape); print(tc.head(3))
print(tc.dtypes.astype(str).to_dict())
print(f"Overall churn rate: {tc.churned.mean():.1%} ({tc.churned.sum()} of {len(tc)})")
```

OUTPUT

```
(3000, 11)
  customer_id  senior_citizen  tenure_months        contract internet_service
payment_method  monthly_charges  total_charges  support_calls churn  churned
0      T00001               0             71        Two year            Fiber    Net
banking            81.28        5848.06              1    No        0
1      T00002               1             56  Month-to-month            Fiber     Auto-
debit          115.12        6482.44              0    No        0
2      T00003               0             38        Two year            Fiber     Auto-
debit           85.90        3186.34              1    No        0
{'customer_id': 'str', 'senior_citizen': 'int64', 'tenure_months': 'int64', 'contract':
'str', 'internet_service': 'str', 'payment_method': 'str', 'monthly_charges': 'float64',
'total_charges': 'float64', 'support_calls': 'int64', 'churn': 'str', 'churned': 'int64'}
Overall churn rate: 25.7% (772 of 3000)
```

2. Data quality

Python

```
print("missing:", tc.isna().sum().sum(), "| duplicate ids:", tc.customer_id.duplicated().sum())
print(tc.describe().round(1).T[["min", "50%", "max", "mean"]])
for c in ["contract", "internet_service", "payment_method"]: print(c,
tc[c].value_counts().to_dict())
gap = tc.total_charges - tc.tenure_months * tc.monthly_charges       # consistency check
between columns
print("total_charges - tenure × monthly: median", round(gap.median(), 1), "| 95% within ±",
round(gap.abs().quantile(.95), 0))
print("tenure 0 customers:", (tc.tenure_months == 0).sum())
```

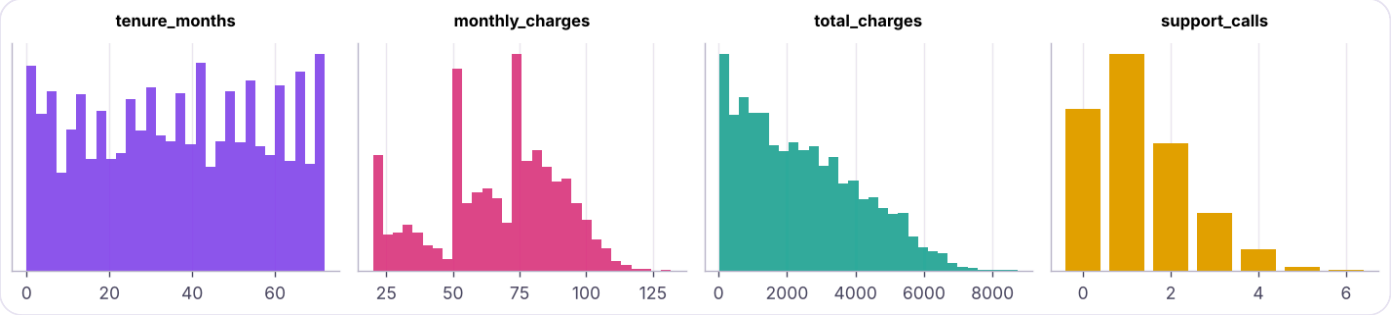
OUTPUT

```
missing: 0 | duplicate ids: 0
                  min     50%     max    mean
senior_citizen    0.0     0.0     1.0     0.2
tenure_months     0.0    36.0    72.0    36.1
monthly_charges  20.0    73.8   131.7    67.1
```

```
total_charges    19.6  2161.3  8724.4  2418.3
support_calls     0.0     1.0     6.0     1.3
churned           0.0     0.0     1.0     0.3
contract {'Month-to-month': 1690, 'One year': 739, 'Two year': 571}
internet_service {'Fiber': 1368, 'DSL': 1047, 'No internet': 585}
payment_method {'Card': 767, 'UPI': 753, 'Auto-debit': 746, 'Net banking': 734}
total_charges - tenure × monthly: median -0.8 | 95% within ± 189.0
tenure 0 customers: 45
```

No missing values or duplicate IDs. `total_charges` roughly equals tenure × monthly charges (small gaps from price changes), so the columns are consistent. A few customers have tenure 0 (new joiners), which is valid.

3. Univariate analysis



Distributions of the numeric variables. Tenure is spread across 0–72 months; monthly charges have a bump of low-price plans.

4. Bivariate: each driver vs churn

Python

```python
def churn_by(col):
    g = tc.groupby(col, observed=True)["churned"].agg(customers="size", churn_rate="mean")
    g["churn_rate"] = (100 * g.churn_rate).round(1); g["lift"] = (g.churn_rate / (100 *
tc.churned.mean())).round(2)
    return g
for col in ["contract", "internet_service", "payment_method", "senior_citizen"]:
print(churn_by(col), "\n")
```

OUTPUT

```
               customers  churn_rate  lift
contract
Month-to-month       1690        35.9  1.40
One year              739        16.9  0.66
Two year              571         7.0  0.27
```

```
                  customers  churn_rate  lift
internet_service
DSL                    1047        21.8  0.85
Fiber                  1368        32.2  1.25
No internet             585        17.6  0.68

                customers  churn_rate  lift
payment_method
Auto-debit            746        23.5  0.91
Card                  767        26.3  1.02
Net banking           734        28.5  1.11
UPI                   753        24.7  0.96

                customers  churn_rate  lift
senior_citizen
0                    2490        25.3  0.98
1                     510        27.8  1.08
```

Lift = segment churn ÷ overall churn. Lift 1.4 means 40% more likely to churn than average.

Python

```
tc["tenure_band"] = pd.cut(tc.tenure_months, [0, 6, 12, 24, 48, 72], include_lowest=True,
labels=["0-6", "7-12", "13-24", "25-48", "49-72"])
tc["calls_band"] = pd.cut(tc.support_calls, [-1, 0, 1, 2, 3, 10], labels=["0", "1", "2", "3",
"4+"])
tc["charge_band"] = pd.qcut(tc.monthly_charges, 4, labels=["Q1 low", "Q2", "Q3", "Q4 high"])
for col in ["tenure_band", "calls_band", "charge_band"]: print(churn_by(col), "\n")
```

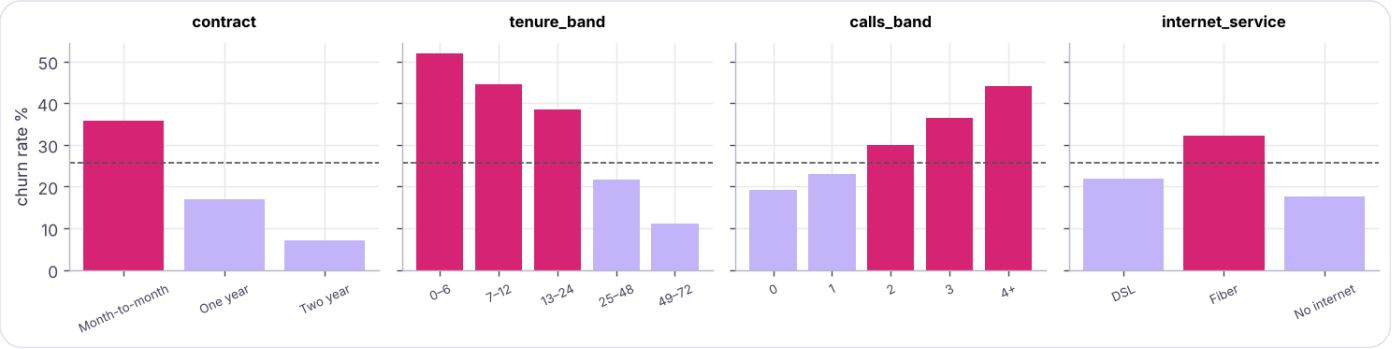
OUTPUT

```
             customers  churn_rate  lift
tenure_band
0-6                318        51.9  2.02
7-12               238        44.5  1.73
13-24              452        38.5  1.50
25-48             1011        21.6  0.84
49-72              981        11.1  0.43

            customers  churn_rate  lift
calls_band
0                 822        19.1  0.74
1                1101        23.1  0.90
2                 649        30.0  1.17
3                 294        36.4  1.41
4+                134        44.0  1.71

             customers  churn_rate  lift
charge_band
```

```
Q1 low              781         19.0  0.74
Q2                  719         20.2  0.78
Q3                  750         29.9  1.16
Q4 high             750         34.0  1.32
```



Churn rate by the four strongest drivers. The dashed line is the overall rate.

Numeric drivers

Python

```python
print(tc.groupby("churn")[["tenure_months", "monthly_charges", "total_charges",
"support_calls"]].median())
print(tc[["tenure_months", "monthly_charges", "support_calls", "senior_citizen",
"churned"]].corr()["churned"].drop("churned").round(3).sort_values())
```

OUTPUT

```
       tenure_months  monthly_charges  total_charges  support_calls
churn
No              42.0           68.010       2445.905            1.0
Yes             21.0           75.405       1357.910            1.0
tenure_months     -0.322
senior_citizen     0.022
monthly_charges    0.143
support_calls      0.153
Name: churned, dtype: float64
```

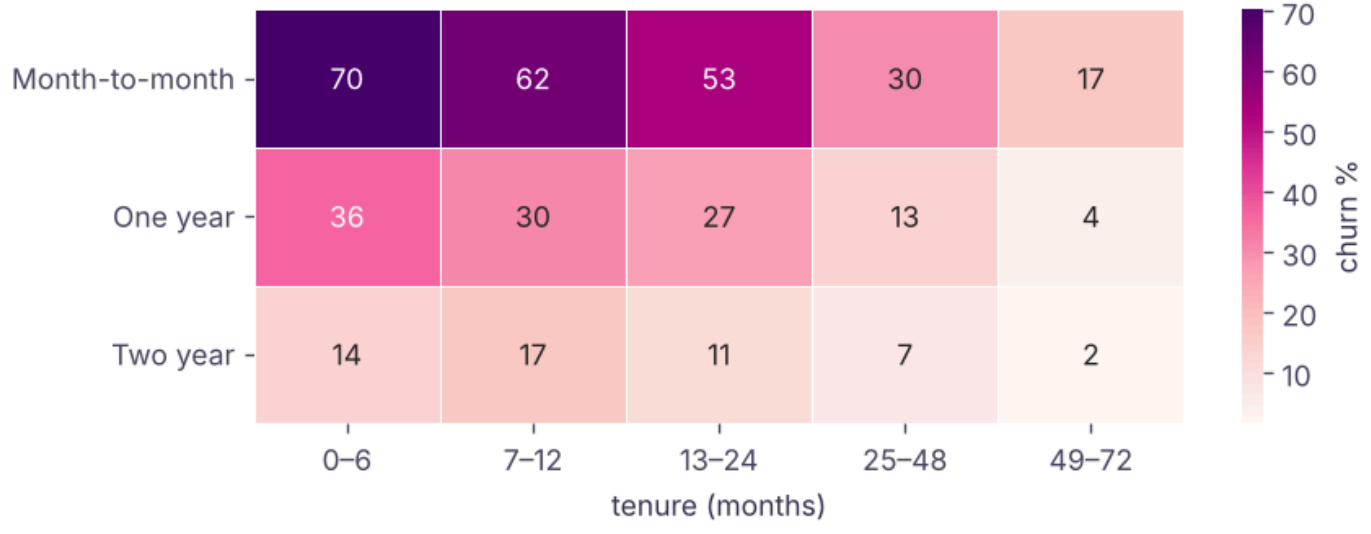
5. Segments: interactions matter

Python

```
seg = tc.pivot_table(index="contract", columns="tenure_band", values="churned", aggfunc="mean",
observed=True).mul(100).round(1)
print(seg)
cnt = tc.pivot_table(index="contract", columns="tenure_band", values="churned", aggfunc="size",
observed=True)
print(cnt)
```

OUTPUT

```
tenure_band      0-6  7-12  13-24  25-48  49-72
contract
Month-to-month  70.2  62.4   53.4   29.7   17.4
One year        36.5  30.3   26.9   13.4    4.3
Two year        13.5  17.0   10.7    6.8    1.5
tenure_band     0-6  7-12  13-24  25-48  49-72
contract
Month-to-month  181   125    249    583    552
One year         85    66    119    238    231
Two year         52    47     84    190    198
```



Churn rate (%) by contract and tenure: new month-to-month customers are the hot spot.

Python

```
hot = tc[(tc.contract == "Month-to-month") & (tc.tenure_months <= 12)]
fiber_calls = tc[(tc.internet_service == "Fiber") & (tc.support_calls >= 3)]
for name, d in [("Month-to-month, tenure ≤ 12", hot), ("Fiber with 3+ support calls",
fiber_calls)]:
    print(f"{name:32s} customers {len(d):4d} | churn {d.churned.mean():.1%} | lift
{d.churned.mean() / tc.churned.mean():.2f}")
```

OUTPUT

```
Month-to-month, tenure ≤ 12      customers  306 | churn 67.0% | lift 2.60
Fiber with 3+ support calls      customers  192 | churn 46.9% | lift 1.82
```

6. Business impact: revenue at risk

Python

```
tc["monthly_revenue_lost"] = tc.monthly_charges * tc.churned
lost = tc.monthly_revenue_lost.sum()
print(f"Monthly revenue of churned customers: {lost:,.0f}  ({lost /
tc.monthly_charges.sum():.1%} of all monthly revenue)")
imp = tc.groupby("contract").agg(churners=("churned", "sum"), revenue_lost=
("monthly_revenue_lost", "sum"))
imp["share_of_loss_%"] = (100 * imp.revenue_lost / lost).round(1)
print(imp.round(0))
```

OUTPUT

```
Monthly revenue of churned customers: 56,173  (27.9% of all monthly revenue)
                churners  revenue_lost  share_of_loss_%
contract
Month-to-month       607       43293.0             77.0
One year             125        9763.0             17.0
Two year              40        3117.0              6.0
```

Revenue numbers here are in the data's currency units per month; multiply by expected remaining lifetime to estimate lifetime value lost.

7. Findings and recommendations

KEY TAKEAWAYS

1. **About 1 in 4 customers churn** (25.7%), taking 27.9% of monthly revenue with them.
2. **Contract is the strongest driver**: month-to-month churn is about 36%, against about 7% on two-year contracts, and month-to-month customers account for 77% of the monthly revenue lost.
3. **The first year is critical**: churn is above 40% in the first 12 months and falls steadily with tenure.
4. **Support calls signal risk**: churn roughly doubles from 0 calls (about 19%) to 3 or more calls (about 36–44%).
5. **Fiber customers churn more** (about 32% against 22% on DSL), which is consistent with price or service-quality issues.
6. Payment method and senior status make only small differences.

Recommendations. (a) Offer month-to-month customers in their first year an incentive to move to an annual plan. (b) Trigger a proactive call after the second support ticket. (c) Investigate Fiber service quality and pricing. (d) Build a churn-score model (ML track, capstone) to rank customers each month and measure the retention campaign with an A/B test.

NOTE **EDA is not causal proof**

These are associations. Contract and churn are related, but customers who intend to leave may simply choose month-to-month plans. Experiments (A/B tests) or careful causal methods are needed before claiming that a change will cause lower churn.

Automated profiling tools

Libraries such as **ydata-profiling** (`ProfileReport(df)`), **sweetviz** and **D-Tale** generate an EDA report in one line, and AI assistants (Lesson ai-analytics) can draft charts. Use them for a quick first scan, then do the focused, question-driven analysis above.

Summary

KEY TAKEAWAYS

- EDA follows a checklist: question → quality → univariate → bivariate → segments → impact → findings.
- Check consistency between related columns, not just missing values.
- Compare churn rate and lift by category; band numeric variables with `cut` / `qcut`.
- Interactions (contract × tenure) reveal hot spots that single variables hide.
- Quantify impact in money and end with specific, owned recommendations; remember that association is not causation.

Quick quiz

6 questions · tap an answer to check it

1. What is "lift" in segment analysis?

A The number of customers in a segment

B Segment rate divided by the overall rate

C The difference between two means

D A chart type

Answer: B. Lift 2 = twice as likely as average.

2. Why band tenure into groups?

A pandas needs it

B To compare churn rates across ranges and spot non-linear patterns

C To delete outliers

D To save memory

Answer: B. `pd.cut` for fixed edges, `qcut` for equal-sized groups.

3. Which finding came from an interaction analysis?

A Overall churn is 25.7%

B New month-to-month customers have the highest churn

C Fiber churns more

D No missing values

Answer: B. It combines two variables.

4. Why check `total_charges` against tenure × monthly charges?

A To verify consistency between related columns

B To compute churn

C To fill missing values

D To make a chart

Answer: A. Inconsistent columns hint at data errors.

5. What should an EDA conclude with?

A All charts produced

B Numbered findings with quantified impact and recommendations

C The raw data

D A model

Answer: B. Stakeholders act on findings, not charts.

6. Can EDA alone prove that a two-year contract reduces churn?

A Yes

B No, it shows association; experiments or causal methods are needed

C Only with a heatmap

D Only if r > 0.5

Answer: B. Self-selection can explain the pattern.

EXERCISE **Extend the case study**

(1) Compute churn by payment method within month-to-month customers only. (2) Find the 10 highest-revenue customers at risk (month-to-month, tenure ≤ 12, 2+ support calls). (3) Plot churn rate against monthly charge deciles. (4) Write a five-bullet executive summary.

Show solution (worked answer)

Python

```
print(tc[tc.contract == "Month-to-
month"].groupby("payment_method").churned.mean().round(3))
risk = tc[(tc.contract == "Month-to-month") & (tc.tenure_months ≤ 12) & (tc.support_calls
≥ 2) & (tc.churned == 0)]
print(risk.nlargest(10, "monthly_charges")[["customer_id", "monthly_charges",
"support_calls"]])
dec = tc.groupby(pd.qcut(tc.monthly_charges, 10), observed=True).churned.mean()
dec.plot(marker="o")
```

INTERVIEW CORNER

Questions you may be asked

Q: Walk me through how you would do EDA on a new dataset. Clarify the question and target, check quality (types, missing, duplicates, ranges, consistency), look at each variable, compare each variable against the target, explore interactions and segments, quantify impact, and summarise findings with next steps.

Q: How do you decide which variables matter most for churn? Compare churn rates/lift by category, correlations and group medians for numeric variables, then confirm with a model (feature importance) and, for causality, with experiments.

Q: Which chart would you use to show churn by two categorical variables? A heatmap of churn rates (with counts available) or grouped bars.

PYTHON FOR DATA SCIENCE · LESSON 23

Python and SQL: sqlite3, DuckDB, SQLAlchemy and pandas.read_sql

Connect Python to databases. Create and query SQLite with the standard library, use parameterised queries safely, load DataFrames to tables and back, query CSV and DataFrames directly with DuckDB, connect to MySQL, PostgreSQL and SQL Server with SQLAlchemy, and decide what to do in SQL versus pandas.

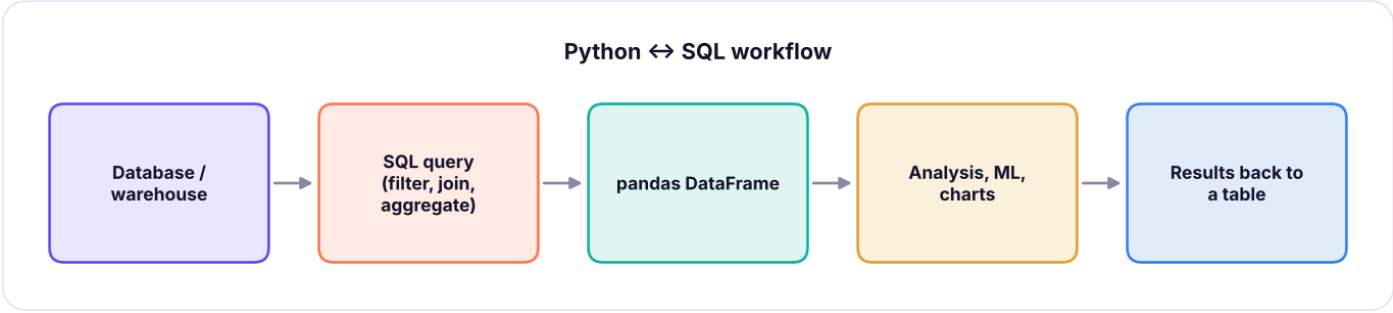
Python class 18 · DuckDB 1.x/2.x · SQLAlchemy 2 · Intermediate · 100 min

By the end of this lesson you can

- Create a SQLite database, run DDL/DML from Python and fetch results
- Write parameterised queries and explain SQL injection
- Move data between pandas and databases with to_sql and read_sql
- Query files and DataFrames with DuckDB
- Connect to server databases with SQLAlchemy connection URLs and secrets kept out of code

Why connect Python to SQL?

Company data lives in databases and warehouses (MySQL, PostgreSQL, SQL Server, Snowflake, BigQuery, Databricks). The usual pattern: **filter and aggregate in SQL** (close to the data), then **analyse, model and chart in Python**.



SQL pulls and summarises; Python analyses, models and presents.

sqlite3: a database in a file

SQLite ships with Python. It is perfect for learning, prototypes and small apps.

Python

```
import sqlite3, pandas as pd
con = sqlite3.connect(":memory:")                    # or "shop.db" for a file
cur = con.cursor()
cur.executescript("""
CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT NOT NULL, city TEXT);
CREATE TABLE orders (id INTEGER PRIMARY KEY, customer_id INTEGER REFERENCES customers(id),
amount REAL, order_date TEXT);
""")
cur.executemany("INSERT INTO customers (name, city) VALUES (?, ?)", [("Asha", "Pune"), ("Ravi",
"Delhi"), ("Meera", "Kochi")])
cur.executemany("INSERT INTO orders (customer_id, amount, order_date) VALUES (?, ?, ?)",
               [(1, 1200, "2025-01-05"), (1, 450, "2025-02-11"), (2, 3000, "2025-02-14"), (3,
780, "2025-03-01")])
con.commit()
cur.execute("SELECT c.name, COUNT(*), SUM(o.amount) FROM orders o JOIN customers c ON c.id =
o.customer_id GROUP BY c.name ORDER BY 3 DESC")
print(cur.fetchall())
cur.execute("SELECT * FROM customers WHERE city = ?", ("Pune",)); print(cur.fetchone())
```

OUTPUT

```
[('Ravi', 1, 3000.0), ('Asha', 2, 1650.0), ('Meera', 1, 780.0)]
(1, 'Asha', 'Pune')
```

Method	Use
`con.execute(sql, params)` / `cursor.execute`	Run one statement
`executemany(sql, rows)`	Insert many rows
`fetchone()`, `fetchall()`, `fetchmany(n)`	Read results
`con.commit()` / `con.rollback()`	Save / undo a transaction
`with con:`	Commits on success, rolls back on error

Parameterised queries and SQL injection

Python

```
user_input = "Pune' OR '1'='1"
unsafe = f"SELECT name FROM customers WHERE city = '{user_input}'"       # NEVER build SQL with
f-strings
print("unsafe returns:", con.execute(unsafe).fetchall())
print("safe returns:  ", con.execute("SELECT name FROM customers WHERE city = ?",
(user_input,)).fetchall())
```

OUTPUT

```
unsafe returns: [('Asha',), ('Ravi',), ('Meera',)]
safe returns:   []
```

The unsafe query returned **every** customer because the input changed the SQL. Placeholders (`?` in SQLite, `%s` or `:name` in other drivers) send values separately, so input can never become code.

Transactions

Python

```python
try:
    with con:                                                    # one transaction
        con.execute("UPDATE orders SET amount = amount - 500 WHERE id = 1")
        con.execute("INSERT INTO orders (customer_id, amount) VALUES (2, 500)")          #
second change
        raise ValueError("payment gateway down")                      # simulate a failure mid-
way
except ValueError as e:
    print("rolled back:", e)
print(con.execute("SELECT id, amount FROM orders WHERE id = 1").fetchone(), con.execute("SELECT
COUNT(*) FROM orders").fetchone())
```

OUTPUT

```
rolled back: payment gateway down
(1, 1200.0) (4,)
```

Both changes were undone together: that is atomicity (the A in ACID).

pandas ↔ database

Python

```python
s = pd.read_csv("superstore_sales.csv")
s.to_sql("sales", con, index=False, if_exists="replace")              # DataFrame → table
q = """
SELECT region, category, ROUND(SUM(sales) / 1e6, 2) AS sales_m, ROUND(100.0 * SUM(profit) /
SUM(sales), 1) AS margin_pct
FROM sales WHERE discount ⩽ :max_disc GROUP BY region, category ORDER BY sales_m DESC LIMIT 5
"""
print(pd.read_sql(q, con, params={"max_disc": 0.1}))                  # query → DataFrame
con.close()
```

OUTPUT

```
  region    category  sales_m  margin_pct
0  South  Technology    32.14        15.9
1   West  Technology    30.21        16.1
2  North  Technology    29.93        16.6
```

```
3    East   Technology    26.57        17.5
4    West    Furniture     5.36        10.3
```

For large tables, read in pieces with `chunksize=` or push the aggregation into SQL so only the summary travels.

DuckDB: SQL on files and DataFrames

DuckDB is an in-process analytical database: no server, very fast on columnar data, and it can query CSV, Parquet and pandas DataFrames directly.

Python

```python
import duckdb
print(duckdb.sql("""
    SELECT category, COUNT(*) AS lines, ROUND(SUM(sales)/1e6, 2) AS sales_m
    FROM 'superstore_sales.csv' GROUP BY category ORDER BY sales_m DESC
""").df())
customers = pd.read_csv("customers.csv")
top = duckdb.sql("""
    SELECT c.state, COUNT(DISTINCT s.customer_id) AS customers, ROUND(SUM(s.sales)/1e6, 2) AS
sales_m,
           RANK() OVER (ORDER BY SUM(s.sales) DESC) AS rnk
    FROM s JOIN customers c USING (customer_id)                    -- s and customers are
pandas DataFrames!
    GROUP BY c.state QUALIFY rnk ⩽ 5 ORDER BY rnk
""").df()
print(top, duckdb.__version__)
```

OUTPUT

```
          category  lines  sales_m
0       Technology   1892   156.44
1        Furniture   1917    34.84
2  Office Supplies   2514    10.13
            state  customers  sales_m  rnk
0     Maharashtra         93    25.67    1
1           Delhi         54    15.66    2
2          Kerala         48    14.71    3
3  Madhya Pradesh         57    14.41    4
4     West Bengal         54    13.89    5 2.0.0
```

DuckDB also writes Parquet (`COPY (SELECT ...) TO 'out.parquet'`), reads from S3, and powers tools such as MotherDuck and the Metabase/MinusX demos in the AI analytics track.

Server databases with SQLAlchemy

`SQLAlchemy` gives one interface to many databases. You need the database's driver package too.

Python

```python
from sqlalchemy import create_engine, text
import os, pandas as pd

# Connection URLs:  dialect+driver://user:password@host:port/database
url = os.environ["DB_URL"]          # e.g.
"postgresql+psycopg://analyst:***@db.company.com:5432/sales"
#   MySQL:       "mysql+pymysql://user:pw@host:3306/db"
#   SQL Server:  "mssql+pyodbc://user:pw@server/db?driver=ODBC+Driver+18+for+SQL+Server"
#   SQLite:      "sqlite:///shop.db"
engine = create_engine(url)

with engine.connect() as conn:
    df = pd.read_sql(text("SELECT * FROM orders WHERE order_date ⩾ :d"), conn, params={"d":
"2025-01-01"})

df_summary.to_sql("daily_summary", engine, if_exists="append", index=False, chunksize=10_000)
```

Warehouses have their own connectors (`snowflake-connector-python` , `google-cloud-bigquery` , `databricks-sql-connector`), and most also accept SQLAlchemy URLs.

WATCH OUT **Keep credentials out of code**

Read passwords from environment variables, a `.env` file (with `python-dotenv`) that is in `.gitignore` , or a secrets manager. Use a read-only database user for analysis.

SQL or pandas?

Do it in SQL when…	Do it in pandas when…
The data is large and lives in the database	The data fits in memory
Filtering, joining, aggregating	Complex reshaping, statistics, ML, charts
Many people need the same logic (views)	Exploratory, one-off analysis
You want the database's indexes and compute	You need Python libraries

Summary

KEY TAKEAWAYS

- `sqlite3` creates databases in a file or memory; `execute`, `executemany`, `fetchall`, `commit`; `with con:` wraps a transaction.
- Always use parameter placeholders, never f-strings, to prevent SQL injection.
- `df.to_sql` writes tables; `pd.read_sql(query, con, params=...)` reads results into DataFrames.
- DuckDB runs fast SQL directly on CSV/Parquet files and pandas DataFrames.
- SQLAlchemy connects to MySQL, PostgreSQL, SQL Server and warehouses with a URL; keep credentials in environment variables.

Quick quiz

6 questions · tap an answer to check it

1. How should user input be passed into a SQL query?

A With an f-string

B As parameters with placeholders (`?` , `%s` , `:name`)

C With string concatenation

D In a comment

Answer: B. Placeholders prevent SQL injection.

2. What does `with con:` do in sqlite3?

A Opens a file

B Commits if the block succeeds and rolls back on an exception

C Closes the database

D Creates a table

Answer: B. It makes the block a transaction.

3. Which writes a DataFrame to a database table?

A `pd.read_sql`

B `df.to_sql("table", con)`

C `df.to_csv`

D `con.fetchall`

Answer: B. `if_exists` controls replace/append.

4. What is special about DuckDB?

A It requires a server

B It runs in-process and can query CSV, Parquet and DataFrames directly with SQL

C It only stores text

D It replaces pandas entirely

Answer: B. It is optimised for analytics.

5. Where should a database password be stored?

A In the notebook

B In an environment variable or secrets manager

C In the SQL query

D On GitHub

Answer: B. Never commit credentials.

6. Why aggregate in SQL before loading into pandas?

A SQL is always faster to write

B Only the small summary travels over the network and into memory

C pandas cannot aggregate

D It is required

Answer: B. Push heavy work to the database.

EXERCISE **Practice**

(1) Load `customers.csv` and `superstore_sales.csv` into SQLite. (2) In SQL, find the top 5 cities by profit and read the result into pandas. (3) Repeat with DuckDB directly on the CSV files. (4) Write a function `get_sales(con, region)` that uses a parameterised query.

Show solution (worked answer)

Python

```
con = sqlite3.connect(":memory:")
pd.read_csv("customers.csv").to_sql("customers", con, index=False)
pd.read_csv("superstore_sales.csv").to_sql("sales", con, index=False)
print(pd.read_sql("SELECT city, SUM(profit) p FROM sales GROUP BY city ORDER BY p DESC
LIMIT 5", con))
print(duckdb.sql("SELECT city, SUM(profit) p FROM 'superstore_sales.csv' GROUP BY city
ORDER BY p DESC LIMIT 5").df())
def get_sales(con, region):
    return pd.read_sql("SELECT SUM(sales) AS sales FROM sales WHERE region = ?", con,
params=(region,))
print(get_sales(con, "West"))
```

INTERVIEW CORNER

Questions you may be asked

Q: What is SQL injection and how do you prevent it? Malicious input that changes a query's meaning; prevent it with parameterised queries (and least-privilege accounts, input validation, ORMs).

Q: How do you load a 50 GB table for analysis? Aggregate or sample in the database, read in chunks, or use a columnar engine (DuckDB, Spark, the warehouse itself) instead of pulling everything into pandas.

Q: What does ACID mean? Atomicity, Consistency, Isolation, Durability: transactions are all-or-nothing, keep rules valid, do not interfere, and survive crashes.

PYTHON FOR DATA SCIENCE · LESSON 24

APIs, JSON and Web Scraping with requests and BeautifulSoup

Collect data from the web. Understand HTTP, call REST APIs with requests (GET/POST, query parameters, headers, authentication, pagination, retries and rate limits), flatten nested JSON with json_normalize, and scrape HTML tables and pages responsibly with BeautifulSoup and pandas.read_html.

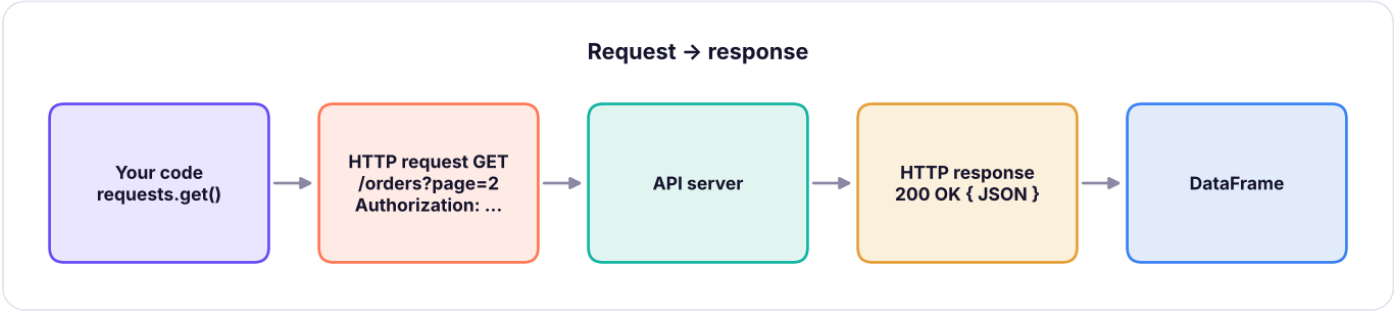
Python class 18 · ✦ requests 2.3x · httpx · BeautifulSoup 4 · Intermediate · ⏱ 100 min

By the end of this lesson you can

- Explain HTTP methods, status codes, headers and JSON responses
- Call REST APIs with parameters, headers and API keys stored safely
- Handle pagination, timeouts, retries and rate limits
- Flatten nested JSON into DataFrames with json_normalize
- Scrape HTML with BeautifulSoup and read_html, respecting robots.txt and terms of use

HTTP in five minutes

An **API** (Application Programming Interface) lets programs ask another system for data. Web APIs use **HTTP**: the client sends a **request** (method, URL, headers, optional body) and receives a **response** (status code, headers, body, usually JSON).



A REST API call.

Method	Meaning	Status code	Meaning
`GET`	Read	`200`	OK
`POST`	Create / send data	`201`	Created
`PUT` / `PATCH`	Replace / update	`400`	Bad request
`DELETE`	Remove	`401` / `403`	Not authenticated / not allowed
		`404`	Not found
		`429`	Too many requests (rate limit)
		`500` – `503`	Server error: retry later

A practice API

To keep this lesson runnable offline, we start a tiny local API that behaves like a real one (paginated orders, an API key, a rate limit). Every call below works the same way against real services.

Python

```
import json, threading, time
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlparse, parse_qs
ORDERS = [{"id": i, "customer": {"name": n, "city": c}, "items": [{"sku": f"P{i}", "qty": i % 3
+ 1, "price": 100.0 * i}], "status": s}
          for i, (n, c, s) in enumerate([("Asha", "Pune", "shipped"), ("Ravi", "Delhi", "new"),
("Meera", "Kochi", "shipped"), ("Kabir", "Goa", "cancelled"), ("Zoya", "Pune", "new")], 1)]
HITS = {"n": 0}
class Api(BaseHTTPRequestHandler):
    def log_message(self, *a): pass
    def send(self, code, body, ctype="application/json"):
        self.send_response(code); self.send_header("Content-Type", ctype); self.end_headers();
self.wfile.write(body.encode())
    def do_GET(self):
        u = urlparse(self.path); q = parse_qs(u.query)
        if u.path == "/page": return self.send(200, "<html><body><h1>Prices</h1><table id='p'>
<tr><th>Product</th><th>Price</th></tr><tr><td>Chair</td><td>₹4,500</td></tr><tr><td>Desk</td>
<td>₹12,000</td></tr></table><a href='/about'>About</a><p class='note'>Updated daily</p></body>
</html>", "text/html; charset=utf-8")
        if self.headers.get("X-API-Key") != "demo-key": return self.send(401,
json.dumps({"error": "missing or bad API key"}))
        HITS["n"] += 1
        if u.path == "/flaky" and HITS["n"] % 2: return self.send(503, json.dumps({"error":
"busy"}))
        if u.path in ("/orders", "/flaky"):
            page, size = int(q.get("page", [1])[0]), int(q.get("size", [2])[0]); st =
q.get("status", [None])[0]
            rows = [o for o in ORDERS if not st or o["status"] == st]
            return self.send(200, json.dumps({"data": rows[(page - 1) * size: page * size],
"page": page, "next": page + 1 if page * size < len(rows) else None}))
        self.send(404, json.dumps({"error": "not found"}))
    def do_POST(self):
        body = json.loads(self.rfile.read(int(self.headers["Content-Length"])))
        self.send(201, json.dumps({"created": True, "echo": body}))
srv = HTTPServer(("127.0.0.1", 0), Api); BASE = f"http://127.0.0.1:{srv.server_port}"
threading.Thread(target=srv.serve_forever, daemon=True).start()
print("practice API running on this machine at", BASE.rsplit(":", 1)[0])
```

OUTPUT

```
practice API running on this machine at http://127.0.0.1
```

GET requests with requests

Python

```
import requests, os
os.environ.setdefault("DEMO_API_KEY", "demo-key")                # in real life, set this
outside the code
headers = {"X-API-Key": os.environ["DEMO_API_KEY"], "Accept": "application/json"}
r = requests.get(f"{BASE}/orders", params={"page": 1, "size": 2}, headers=headers, timeout=10)
print(r.status_code, r.url.replace(BASE, "BASE"), r.headers["Content-Type"])
payload = r.json()
print(payload["page"], payload["next"], payload["data"][0]["customer"])
bad = requests.get(f"{BASE}/orders", timeout=10)
print(bad.status_code, bad.json())
try: bad.raise_for_status()
except requests.HTTPError as e: print("HTTPError:", str(e)[:40])
```

OUTPUT

```
200 BASE/orders?page=1&size=2 application/json
1 2 {'name': 'Asha', 'city': 'Pune'}
401 {'error': 'missing or bad API key'}
HTTPError: 401 Client Error: Unauthorized for url:
```

Always set a `timeout`, check the status (`raise_for_status()`), and keep keys in environment variables.

POST requests

Python

```
r = requests.post(f"{BASE}/orders", json={"customer": "Asha", "items": [{"sku": "P9", "qty":
1}]}, headers=headers, timeout=10)
print(r.status_code, r.json())
```

OUTPUT

```
201 {'created': True, 'echo': {'customer': 'Asha', 'items': [{'sku': 'P9', 'qty': 1}]}}
```

`json=` serialises the body and sets the `Content-Type` header for you. Use `data=` for form posts and `files=` for uploads.

Pagination, retries and rate limits

Python

```
def fetch_all(path, params=None, max_pages=50):
    rows, page = [], 1
    with requests.Session() as session:                          # re-uses the connection
        session.headers.update(headers)
        while page and page ⩽ max_pages:
            resp = session.get(f"{BASE}{path}", params={**(params or {}), "page": page, "size":
2}, timeout=10)
            resp.raise_for_status()
            body = resp.json(); rows += body["data"]; page = body["next"]
    return rows
orders = fetch_all("/orders"); print(len(orders), "orders fetched across pages")

def get_with_retry(url, tries=4, backoff=0.2, **kw):
    for attempt in range(1, tries + 1):
        resp = requests.get(url, timeout=10, **kw)
        if resp.status_code not in (429, 500, 502, 503, 504): return resp
        wait = float(resp.headers.get("Retry-After", backoff * 2 ** (attempt - 1)))     #
exponential backoff
        print(f"  attempt {attempt}: {resp.status_code}, retrying in {wait:.1f}s");
time.sleep(wait)
    resp.raise_for_status()
print(get_with_retry(f"{BASE}/flaky", headers=headers).status_code)
```

OUTPUT

```
5 orders fetched across pages
  attempt 1: 503, retrying in 0.2s
200
```

Respect rate limits: read the API's documented limits, honour `Retry-After`, cache results, and sleep between calls.

Flattening nested JSON

Python

```
import pandas as pd
pd.set_option("display.width", 160)
flat = pd.json_normalize(orders, sep="_")
print(flat.drop(columns="items"))
lines = pd.json_normalize(orders, record_path="items", meta=["id", "status", ["customer",
"city"]], sep="_")
lines["value"] = lines.qty * lines.price
print(lines)
```

OUTPUT

```
   id     status customer_name customer_city
0   1    shipped          Asha          Pune
1   2        new          Ravi         Delhi
2   3    shipped         Meera         Kochi
3   4  cancelled         Kabir           Goa
4   5        new          Zoya          Pune
  sku  qty  price id     status customer_city   value
0  P1    2  100.0  1    shipped          Pune   200.0
1  P2    3  200.0  2        new         Delhi   600.0
2  P3    1  300.0  3    shipped         Kochi   300.0
3  P4    2  400.0  4  cancelled           Goa   800.0
4  P5    3  500.0  5        new          Pune  1500.0
```

`record_path` explodes a nested list into rows; `meta` copies parent fields onto each row.

Real public APIs to practise with

API	Data	Notes
data.gov.in	Indian government open data	Free API key
RBI DBIE, World Bank, IMF	Economic indicators	`wbgapi` package for the World Bank
Open-Meteo	Weather forecasts and history	No key
GitHub REST API	Repositories, issues	Token raises limits
Alpha Vantage, yfinance	Stock prices	Free tiers, rate-limited
OpenAI, Anthropic, Gemini	LLMs	Paid keys, see GenAI track

Python

```python
# Example (needs internet): daily max temperature in Pune from Open-Meteo
r = requests.get("https://api.open-meteo.com/v1/forecast",
                 params={"latitude": 18.52, "longitude": 73.86, "daily": "temperature_2m_max",
"timezone": "Asia/Kolkata"}, timeout=10)
weather = pd.DataFrame(r.json()["daily"])
```

Web scraping

When there is no API, you can parse HTML. **BeautifulSoup** navigates the HTML tree; `pandas.read_html` grabs `<table>` elements directly.

Python

```
from bs4 import BeautifulSoup
from io import StringIO
html = requests.get(f"{BASE}/page", timeout=10).text
soup = BeautifulSoup(html, "html.parser")
print(soup.h1.text, "|", soup.find("p", class_="note").get_text(strip=True), "|", [a["href"] for
a in soup.find_all("a")])
rows = [[td.get_text(strip=True) for td in tr.find_all(["td", "th"])] for tr in
soup.select("table#p tr")]
print(rows)
t = pd.read_html(StringIO(html))[0]
t["Price"] = t["Price"].str.replace(r"[₹,]", "", regex=True).astype(int)
print(t, t.Price.sum())
srv.shutdown()
```

OUTPUT

```
Prices | Updated daily | ['/about']
[['Product', 'Price'], ['Chair', '₹4,500'], ['Desk', '₹12,000']]
  Product  Price
0   Chair   4500
1    Desk  12000 16500
```

BeautifulSoup	Finds
`soup.find("tag", class_="x")`	First match
`soup.find_all("a")`	All matches
`soup.select("div.card > h2")`	CSS selectors
`el.get_text(strip=True)` , `el["href"]`	Text and attributes

For pages built by JavaScript, use **Playwright** or Selenium to drive a real browser, or look in the browser's Network tab for the underlying JSON API.

WATCH OUT **Scrape responsibly and legally**

Check the site's **terms of use** and **robots.txt**, prefer official APIs, identify your scraper with a User-Agent, go slowly (one request every few seconds), cache pages, and never collect personal data without a lawful basis (India's DPDP Act 2023, GDPR). Logged-in or paywalled content is usually off-limits.

Summary

KEY TAKEAWAYS

- HTTP: methods (GET, POST…), status codes (200, 401, 404, 429, 5xx), headers and JSON bodies.
- `requests.get(url, params=, headers=, timeout=)` ; `r.json()` ; `r.raise_for_status()` ; keys from environment variables.
- Loop over pages with a `Session` ; retry 429/5xx with exponential backoff and honour `Retry-After` .
- `pd.json_normalize(data, record_path=, meta=)` flattens nested JSON.
- Scrape with BeautifulSoup or `pd.read_html` only when allowed; prefer APIs and be polite.

Quick quiz

6 questions · tap an answer to check it

1. What does status code 429 mean?

A Not found

B Too many requests (rate limit)

C Server error

D Success

Answer: B. Slow down and retry after the indicated time.

2. Why set `timeout=` on every request?

A It speeds up requests

B Without it, a request can hang forever

C It is required by APIs

D To retry automatically

Answer: B. Typical values are 5–30 seconds.

3. Where should an API key be kept?

- **A** In the URL in your notebook
- **B** In an environment variable or secrets store
- **C** In a GitHub README
- **D** In a comment

Answer: B. Read it with `os.getenv`.

4. What does `json_normalize(..., record_path="items")` do?

- **A** Deletes the items field
- **B** Creates one row per element of each record's items list
- **C** Converts to CSV
- **D** Sorts the data

Answer: B. `meta` keeps parent fields.

5. Which is the best first option to get data from a website?

- **A** An official API or data download
- **B** Scraping every page quickly
- **C** Copy-paste
- **D** A screenshot

Answer: A. Scraping is a fallback.

6. Which tool can read every `<table>` on a page into DataFrames?

A `requests.post`

B `pd.read_html`

C `json.loads`

D `soup.h1`

Answer: B. It returns a list of DataFrames.

EXERCISE **Practice**

(1) Using the Open-Meteo example (on your machine), fetch 7 days of forecasts for three cities and combine them into one DataFrame. (2) Write `fetch_all` for an API that uses `offset` / `limit` instead of pages. (3) Scrape the list of tables from a Wikipedia page of your choice with `read_html` and clean one of them.

Show solution (worked answer)

Python

```
cities = {"Pune": (18.52, 73.86), "Delhi": (28.61, 77.21), "Kochi": (9.93, 76.27)}
frames = []
for name, (lat, lon) in cities.items():
    r = requests.get("https://api.open-meteo.com/v1/forecast", params={"latitude": lat,
"longitude": lon, "daily": "temperature_2m_max", "timezone": "Asia/Kolkata"}, timeout=10)
    frames.append(pd.DataFrame(r.json()["daily"]).assign(city=name))
df = pd.concat(frames)
def fetch_offset(url, limit=100):
    out, offset = [], 0
    while True:
        batch = requests.get(url, params={"offset": offset, "limit": limit},
timeout=10).json()
        if not batch: return out
        out += batch; offset += limit
```

INTERVIEW CORNER

Questions you may be asked

Q: How do you collect data from a paginated API reliably? Use a session, loop until there is no next page/cursor, set timeouts, retry 429/5xx with backoff, respect rate limits, log progress, and make the job restartable.

Q: API vs web scraping? APIs are stable, documented and permitted; scraping depends on page structure, can break, and has legal and ethical limits.

Q: How do you turn nested JSON into a table? `pd.json_normalize` with `record_path` and `meta`, or explode list columns and expand dict columns.

PYTHON FOR DATA SCIENCE · LESSON 25

Performance and Bigger Data: Vectorisation, Memory, Parquet, Polars and DuckDB

Make Python data work fast and scale beyond memory. Measure with timeit, replace loops with vectorised code, shrink DataFrames with the right dtypes, store data as Parquet, use Polars lazy queries and DuckDB for out-of-core analytics, and know when to move to Spark and the cloud.

Python class 19 · ✦ pandas 3 · Polars 1.x/2.x · DuckDB · PyArrow · Intermediate · ⏱ 100 min

By the end of this lesson you can

- Measure speed and memory before optimising
- Replace row loops and apply with vectorised pandas and NumPy operations
- Reduce memory with categories, smaller numeric types and selected columns
- Store and read data efficiently with Parquet and explain columnar formats
- Write equivalent queries in pandas, Polars and DuckDB and choose between them

Measure first

"Premature optimisation is the root of all evil" (Donald Knuth). Profile, find the slow part, fix that.

Python

```
import pandas as pd, numpy as np, time, timeit
s = pd.read_csv("superstore_sales.csv", parse_dates=["order_date"])
big = pd.concat([s] * 20, ignore_index=True)                # ~126k rows to make timings visible
print(big.shape)
def t(fn, n=3): return min(timeit.repeat(fn, number=1, repeat=n)) * 1000
loop = lambda: [r.sales * (1 - r.discount) for r in big.itertuples()]
apply_ = lambda: big.apply(lambda r: r["sales"] * (1 - r["discount"]), axis=1)
vec = lambda: big["sales"] * (1 - big["discount"])
print(f"itertuples loop {t(loop, 1):8.1f} ms\napply(axis=1)   {t(apply_, 1):8.1f} ms\nvectorised {t(vec):8.2f} ms")
```

OUTPUT

```
(126460, 19)
itertuples loop   1070.5 ms
apply(axis=1)      659.5 ms
vectorised           0.65 ms
```

The vectorised version is more than a thousand times faster. `itertuples` and `apply(axis=1)` take about the same time because both are Python loops (`apply` with `axis=1` is a loop in disguise), and `iterrows` is slower still.

Vectorising conditional logic

Python

```python
def band_loop(d): return "high" if d > 0.2 else "mid" if d > 0 else "none"
f1 = lambda: big["discount"].map(band_loop)
f2 = lambda: np.select([big.discount > 0.2, big.discount > 0], ["high", "mid"], default="none")
f3 = lambda: pd.cut(big.discount, [-1, 0, 0.2, 1], labels=["none", "mid", "high"])
print(f"map(func) {t(f1):.1f} ms | np.select {t(f2):.1f} ms | pd.cut {t(f3):.1f} ms")
print((f1().values == f2()).all())
```

OUTPUT

```
map(func) 15.6 ms | np.select 1.0 ms | pd.cut 1.5 ms
True
```

Slow pattern	Faster pattern
`for` / `iterrows` over rows	Column arithmetic, `np.where` , `np.select`
`apply(axis=1)`	Vectorised expression, or `groupby().transform`
Growing a DataFrame in a loop (`concat` each time)	Collect a list of dicts, build once
String ops in Python loops	`.str` methods
Repeated `df[df.x == k]` in a loop	One `groupby`
Reading a big CSV many times	Read once, save as Parquet

Memory

Python

```
def mb(df): return df.memory_usage(deep=True).sum() / 1e6
print(f"original: {mb(big):.1f} MB")
small = big.copy()
for c in ["region", "segment", "category", "sub_category", "ship_mode", "state", "city"]:
small[c] = small[c].astype("category")
small["quantity"] = pd.to_numeric(small["quantity"], downcast="integer")
small[["sales", "profit", "discount"]] = small[["sales", "profit",
"discount"]].astype("float32")
print(f"categories + downcast: {mb(small):.1f} MB")
print(small.dtypes.astype(str).value_counts().to_dict())
```

OUTPUT

```
original: 34.7 MB
categories + downcast: 18.3 MB
{'category': 7, 'str': 6, 'float32': 3, 'int64': 1, 'datetime64[us]': 1, 'int8': 1}
```

Categories store each label once plus small integer codes: ideal for columns with few distinct values. Also read only needed columns (`usecols=`) and stream with `chunksize=` .

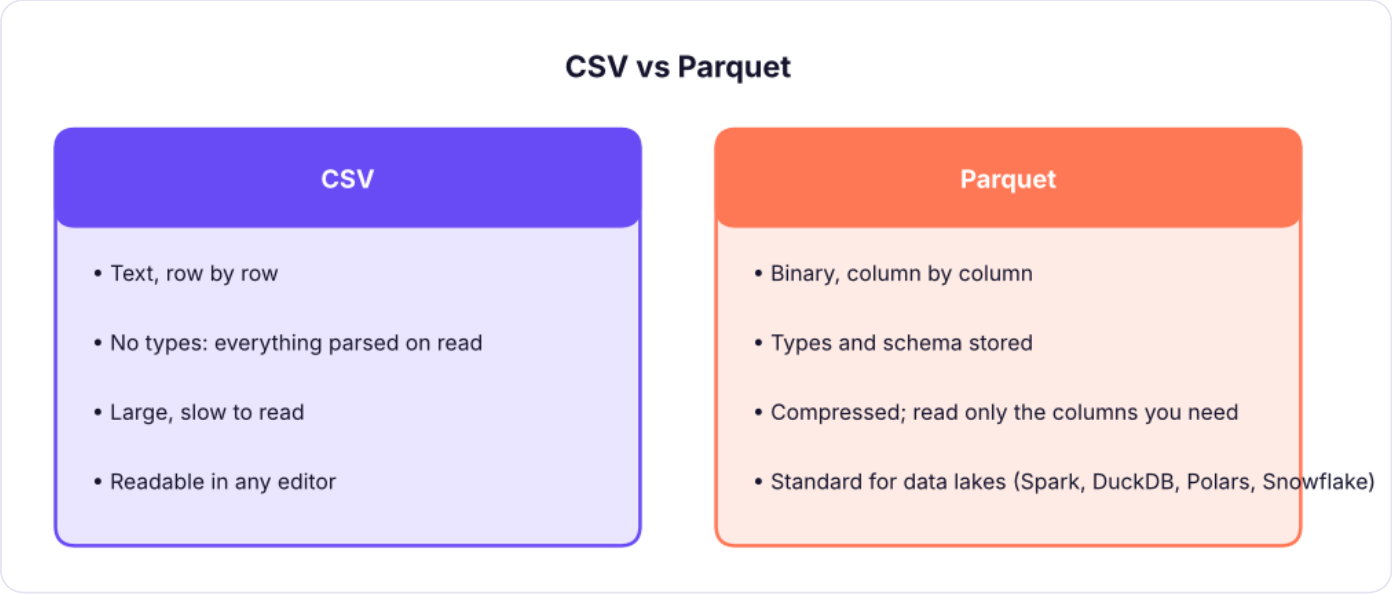
Python

```
total = 0.0
for chunk in pd.read_csv("superstore_sales.csv", usecols=["region", "sales"], chunksize=2000):
    total += chunk.loc[chunk.region == "West", "sales"].sum()
print(f"West sales via chunks: {total:,.0f}")
```

OUTPUT

```
West sales via chunks: 51,958,520
```

Parquet: the analytics file format



CSV is row-oriented text; Parquet stores each column together, typed and compressed.

Python

```
import os
big.to_csv("big.csv", index=False); big.to_parquet("big.parquet", index=False)
print(f"CSV {os.path.getsize('big.csv')/1e6:.1f} MB | Parquet
{os.path.getsize('big.parquet')/1e6:.1f} MB")
print(f"read CSV {t(lambda: pd.read_csv('big.csv')):.0f} ms | read Parquet {t(lambda:
pd.read_parquet('big.parquet')):.0f} ms | two columns {t(lambda: pd.read_parquet('big.parquet',
columns=['region', 'sales'])):.0f} ms")
```

OUTPUT

```
CSV 21.7 MB | Parquet 1.9 MB
read CSV 249 ms | read Parquet 15 ms | two columns 5 ms
```

Polars: fast DataFrames with lazy queries

Polars is a DataFrame library written in Rust. It uses all CPU cores, the Arrow memory format, and a **lazy** mode that optimises the whole query before running it.

Python

```
import polars as pl
lazy = (pl.scan_parquet("big.parquet")                          # nothing is read yet
          .filter(pl.col("discount") ⩽ 0.1)
          .group_by("region", "category")
          .agg(sales=pl.col("sales").sum(), profit=pl.col("profit").sum(), lines=pl.len())
          .with_columns(margin=(100 * pl.col("profit") / pl.col("sales")).round(1), sales_m=
(pl.col("sales") / 1e6).round(2))
          .drop("profit")
          .sort("sales", descending=True))
print(lazy.head(4).collect().drop("sales"))
pandas_way = lambda: (big[big.discount ⩽ 0.1].groupby(["region", "category"]).agg(sales=
("sales", "sum"), profit=("profit", "sum")))
print(f"pandas {t(pandas_way):.1f} ms | polars lazy {t(lambda: lazy.collect()):.1f} ms | polars
{pl.__version__}")
```

OUTPUT

```
shape: (4, 5)
┌─────────┬────────────┬───────┬────────┬─────────┐
│ region  ┆ category   ┆ lines ┆ margin ┆ sales_m │
│ ---     ┆ ---        ┆ ---   ┆ ---    ┆ ---     │
│ str     ┆ str        ┆ u32   ┆ f64    ┆ f64     │
╞═════════╪════════════╪═══════╪════════╪═════════╡
│ South   ┆ Technology ┆ 7220  ┆ 15.9   ┆ 642.79  │
│ West    ┆ Technology ┆ 7080  ┆ 16.1   ┆ 604.14  │
│ North   ┆ Technology ┆ 6720  ┆ 16.6   ┆ 598.63  │
│ East    ┆ Technology ┆ 6320  ┆ 17.5   ┆ 531.33  │
└─────────┴────────────┴───────┴────────┴─────────┘
pandas 22.0 ms | polars lazy 3.9 ms | polars 2.1.0
```

pandas	Polars
`df[df.x > 1]`	`df.filter(pl.col("x") > 1)`
`df.assign(y=df.x * 2)`	`df.with_columns(y=pl.col("x") * 2)`
`df.groupby("g").agg(s=("x", "sum"))`	`df.group_by("g").agg(s=pl.col("x").sum())`
`df.sort_values("x")`	`df.sort("x")`
`pd.read_csv`	`pl.read_csv` / lazy `pl.scan_csv`
`df.to_pandas()` ↔ `pl.from_pandas(df)`	Convert when a library needs pandas

DuckDB on files larger than memory

Python

```
import duckdb
q = """
SELECT region, category, ROUND(SUM(sales)/1e6, 2) AS sales_m, ROUND(100*SUM(profit)/SUM(sales),
1) AS margin
FROM 'big.parquet' WHERE discount ⩽ 0.1 GROUP BY ALL ORDER BY sales_m DESC LIMIT 4
"""
print(duckdb.sql(q).df())
print(f"duckdb {t(lambda: duckdb.sql(q).fetchall()):.1f} ms")
for f in ("big.csv", "big.parquet"): os.remove(f)
```

OUTPUT

```
  region    category  sales_m  margin
0  South  Technology   642.79    15.9
1   West  Technology   604.14    16.1
2  North  Technology   598.63    16.6
3   East  Technology   531.33    17.5
duckdb 7.0 ms
```

DuckDB and Polars stream data in batches, so they can process files bigger than RAM on a laptop. Folders of Parquet files (`'data/*.parquet'`) work the same way.

Going further

Need	Tool
Use all CPU cores for custom Python	`concurrent.futures` , `joblib.Parallel` , multiprocessing
Speed up numeric Python loops	Numba (`@njit`), Cython
Data bigger than one machine	Apache Spark (PySpark), Databricks, Dask, Ray
Warehouse-scale SQL	BigQuery, Snowflake, Redshift, Databricks SQL
GPU DataFrames	RAPIDS cuDF (`cudf.pandas`)

Rule of thumb: up to a few GB → pandas with good dtypes; up to ~100 GB on one machine → Polars or DuckDB with Parquet; beyond that or many users → Spark or a cloud warehouse.

Summary

KEY TAKEAWAYS

- Measure with `timeit` / `%timeit` before optimising; fix the bottleneck.
- Vectorise: column arithmetic, `np.where` / `np.select`, `pd.cut`, `.str`, `groupby`; avoid `iterrows` and `apply(axis=1)`.
- Save memory with categories, downcasting, `usecols` and `chunksize`.
- Parquet is columnar, typed and compressed: faster and smaller than CSV.
- Polars (lazy, multi-core) and DuckDB (SQL, out-of-core) handle data beyond pandas; Spark and warehouses go beyond one machine.

Quick quiz

6 questions · tap an answer to check it

1. Which is usually fastest for row-wise arithmetic?

A iterrows

B apply(axis=1)

C A vectorised column expression

D A while loop

Answer: C. Vectorised code runs in compiled loops.

2. Why convert a text column with 4 distinct values to `category`?

A It stores each label once with small integer codes, saving memory and speeding groupby

B It sorts the data

C It is required for charts

D It removes missing values

Answer: A. Ideal for low-cardinality columns.

3. What is the main advantage of Parquet over CSV?

A Human-readable

B Columnar, typed and compressed, so it is smaller and faster to read

C Supported only by Excel

D No schema

Answer: B. You can read just the columns you need.

4. What does Polars' `scan_parquet(...)` do?

A Reads the whole file immediately

B Creates a lazy query that is optimised and executed on `.collect()`

C Writes a file

D Converts to CSV

Answer: B. Lazy evaluation enables predicate and projection push-down.

5. Which tool queries Parquet files with SQL without a server?

A MySQL

B DuckDB

C Excel

D requests

Answer: B. It also queries pandas DataFrames.

6. When should you consider Spark or a cloud warehouse?

A For 10,000 rows

B When data or users exceed what one machine can handle

C Always

D Never

Answer: B. Start simple; scale when needed.

EXERCISE **Practice**

(1) Time `iterrows`, `apply(axis=1)` and a vectorised version of `profit / sales` on `big`. (2) Reduce the memory of `superstore_sales.csv` by at least 50% and report the percentage. (3) Rewrite a pandas groupby from Lesson 18 in Polars and in DuckDB SQL and check that the numbers match.

Show solution (worked answer)

Python

```
print(t(lambda: [r["profit"] / r["sales"] for _, r in big.head(20000).iterrows()], 1))
print(t(lambda: big.profit / big.sales))
before = mb(s); s2 = s.copy()
for c in s2.select_dtypes("object").columns.union(s2.select_dtypes("string").columns):
    if s2[c].nunique() < 100: s2[c] = s2[c].astype("category")
print(f"{100 * (1 - mb(s2) / before):.0f}% smaller")
pl_res = pl.from_pandas(s).group_by("region").agg(pl.col("sales").sum()).sort("region")
dd_res = duckdb.sql("SELECT region, SUM(sales) sales FROM s GROUP BY region ORDER BY
region").df()
```

INTERVIEW CORNER

Questions you may be asked

Q: Your pandas script is slow. What do you do? Profile to find the slow step; vectorise loops and `apply`; reduce data early (filter, `usecols`); fix dtypes; avoid repeated work; save intermediate results as Parquet; consider Polars/DuckDB or pushing work to SQL.

Q: Why is Parquet popular in data lakes? Columnar storage enables compression and reading only needed columns; it stores the schema; and every major engine reads it.

Q: pandas vs Polars? pandas has the biggest ecosystem; Polars is faster and more memory-efficient thanks to Rust, multi-threading and lazy optimisation. Many teams use both and convert where needed.

PYTHON FOR DATA SCIENCE · LESSON 26

DSA I: Big-O, Arrays, Strings, Hashing, Two Pointers and Sliding Window

Data structures and algorithms for data-analyst, data-scientist and data-engineer interviews. Learn Big-O time and space complexity, then the patterns that solve most array and string questions (hash maps and sets, prefix sums, two pointers, sliding window), with worked interview problems and timings.

DSA for data roles (added) · Python 3.13 · Intermediate · 120 min

By the end of this lesson you can

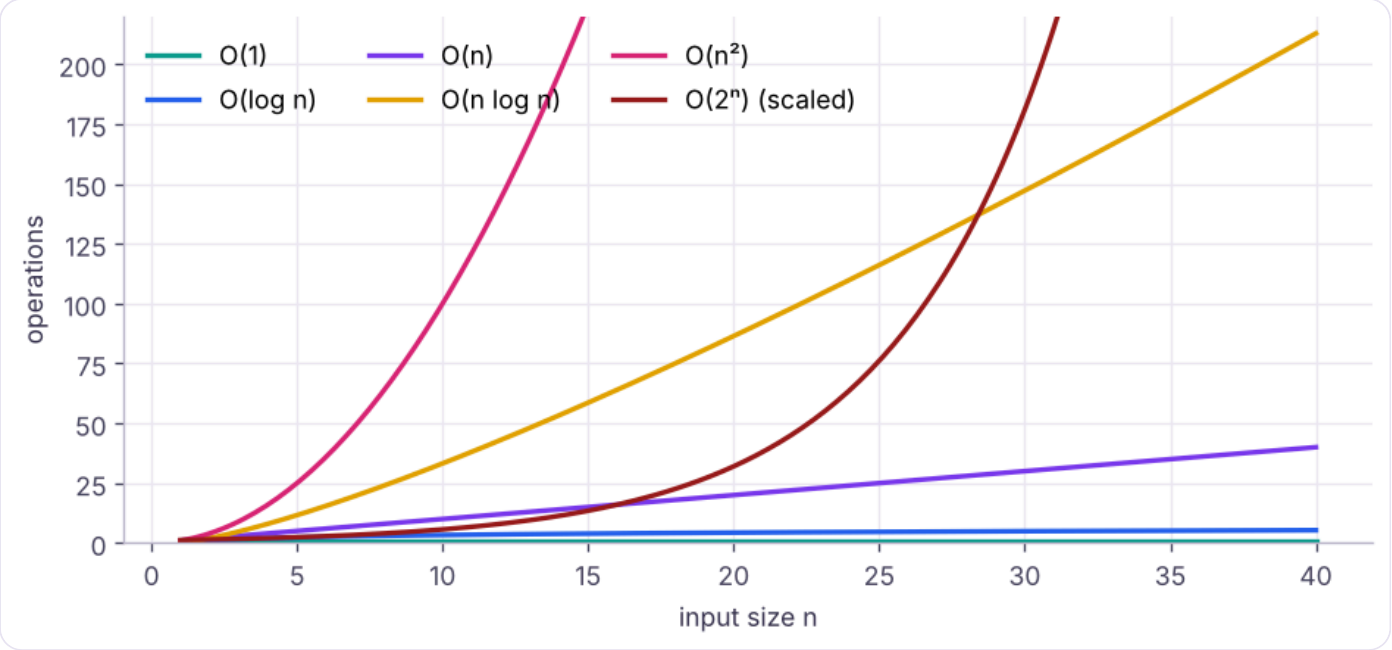
- Estimate time and space complexity with Big-O and compare common growth rates
- Know the cost of operations on lists, dicts, sets and deques
- Solve array and string problems with hash maps, counting and prefix sums
- Apply the two-pointer and sliding-window patterns
- Explain trade-offs out loud as in a coding interview

Why DSA for data roles?

Data interviews (Amazon, Flipkart, Walmart, fintechs, product companies) often include one or two coding rounds with easy–medium problems on arrays, strings, hash maps and SQL. Beyond interviews, complexity thinking explains why a join on a set is instant while a nested loop over two million rows never finishes.

Big-O notation

Big-O describes how running time (or memory) grows with input size *n*, ignoring constants.



Growth of common complexity classes. At n = 1,000,000 the difference between O(n log n) and O(n²) is days versus a second.

Big-O	Name	Example
O(1)	Constant	Dict lookup, list index, append
O(log n)	Logarithmic	Binary search, balanced-tree lookup
O(n)	Linear	One loop, `x in list`, `sum`
O(n log n)	Linearithmic	Sorting (`sorted`, Timsort), merge sort
O(n²)	Quadratic	Nested loops over the same data
O(2ⁿ)	Exponential	All subsets, naive recursion for Fibonacci

Rules: drop constants (O(2n) → O(n)); keep the dominant term (O(n² + n) → O(n²)); sequential steps add, nested steps multiply.

Python

```
import time
def has_duplicate_quadratic(a):
    for i in range(len(a)):
        for j in range(i + 1, len(a)):
            if a[i] == a[j]: return True
    return False
def has_duplicate_set(a): return len(set(a)) != len(a)
for n in (1000, 2000, 4000):
    data = list(range(n))
    t0 = time.perf_counter(); has_duplicate_quadratic(data); t1 = time.perf_counter();
has_duplicate_set(data); t2 = time.perf_counter()
    print(f"n={n:5d}: nested loops {1000*(t1-t0):7.1f} ms | set {1000*(t2-t1):.3f} ms")
```

OUTPUT

```
n= 1000: nested loops    15.1 ms | set 0.022 ms
n= 2000: nested loops    59.7 ms | set 0.042 ms
n= 4000: nested loops   261.3 ms | set 0.088 ms
```

Doubling *n* roughly quadruples the nested-loop time (O(n²)) but only doubles the set version (O(n)).

Cost of Python operations

Operation	list	dict / set	deque
Index `a[i]`	O(1)	—	O(n)
Append / pop at end	O(1)	add O(1)	O(1)
Insert / pop at front	**O(n)**	—	**O(1)**
`x in c`	**O(n)**	**O(1)**	O(n)
Sort	O(n log n)	—	—

Pattern 1: hash maps and counting

Turn "search inside a loop" (O(n²)) into "look up in a dict/set" (O(n)).

Python

```python
def two_sum(nums, target):
    """Return indices of two numbers adding up to target. O(n) time, O(n) space."""
    seen = {}                                   # value → index
    for i, x in enumerate(nums):
        if target - x in seen: return [seen[target - x], i]
        seen[x] = i
    return []
print(two_sum([2, 7, 11, 15], 9), two_sum([3, 2, 4], 6), two_sum([1, 2], 7))

from collections import Counter, defaultdict
def is_anagram(a, b): return Counter(a) == Counter(b)
def group_anagrams(words):
    g = defaultdict(list)
    for w in words: g["".join(sorted(w))].append(w)
    return list(g.values())
def first_unique_char(s):
    c = Counter(s)
    return next((i for i, ch in enumerate(s) if c[ch] == 1), -1)
def top_k_frequent(nums, k): return [x for x, _ in Counter(nums).most_common(k)]
print(is_anagram("listen", "silent"), group_anagrams(["eat", "tea", "tan", "ate", "nat",
"bat"]), first_unique_char("leetcode"), top_k_frequent([1, 1, 1, 2, 2, 3], 2))
```

OUTPUT

```
[0, 1] [1, 2] []
True [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']] 0 [1, 2]
```

Pattern 2: prefix sums

Precompute running totals so any range sum is O(1).

Python

```python
from itertools import accumulate
sales = [5, 3, 8, 6, 2, 7]
prefix = [0] + list(accumulate(sales))                # prefix[i] = sum of first i items
range_sum = lambda i, j: prefix[j + 1] - prefix[i]    # sum of sales[i..j]
print(prefix, range_sum(1, 3), range_sum(0, 5))

def subarray_sum_equals_k(nums, k):
    """Count contiguous subarrays summing to k. O(n) with prefix sums + a counter."""
    count, running, seen = 0, 0, Counter({0: 1})
    for x in nums:
        running += x
        count += seen[running - k]
        seen[running] += 1
    return count
print(subarray_sum_equals_k([1, 1, 1], 2), subarray_sum_equals_k([1, 2, 3, -3, 3], 3))
```

OUTPUT

```
[0, 5, 8, 16, 22, 24, 31] 17 31
2 5
```

Pattern 3: two pointers

Two indices move toward each other (or in the same direction) through a **sorted** array or string.

Python

```python
def is_palindrome(s):
    s = [c.lower() for c in s if c.isalnum()]
    i, j = 0, len(s) - 1
    while i < j:
        if s[i] != s[j]: return False
        i += 1; j -= 1
    return True
def pair_with_sum_sorted(a, target):
    i, j = 0, len(a) - 1
    while i < j:
        total = a[i] + a[j]
        if total == target: return a[i], a[j]
        if total < target: i += 1
        else: j -= 1
    return None
def remove_duplicates_sorted(a):              # in place, returns new length
    if not a: return 0
    w = 1
    for r in range(1, len(a)):
        if a[r] != a[w - 1]: a[w] = a[r]; w += 1
    return w
def merge_sorted(a, b):
    i = j = 0; out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]: out.append(a[i]); i += 1
        else: out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
arr = [1, 1, 2, 3, 3, 3, 4]; n = remove_duplicates_sorted(arr)
print(is_palindrome("A man, a plan, a canal: Panama"), pair_with_sum_sorted([1, 3, 4, 6, 9],
10), arr[:n], merge_sorted([1, 4, 9], [2, 3, 10]))
```

OUTPUT

```
True (1, 9) [1, 2, 3, 4] [1, 2, 3, 4, 9, 10]
```

Pattern 4: sliding window

Keep a window `[left, right]` and slide it, updating a running state instead of recomputing.



A sliding window adds the new element on the right and removes the old one on the left.

Python

```
def max_sum_window(a, k):
    window = best = sum(a[:k])
    for r in range(k, len(a)):
        window += a[r] - a[r - k]
        best = max(best, window)
    return best
def longest_unique_substring(s):
    last, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left: left = last[ch] + 1     # shrink past the repeat
        last[ch] = right; best = max(best, right - left + 1)
    return best
def min_window_at_least(nums, target):          # shortest subarray with sum >= target (positive
numbers)
    left = total = 0; best = float("inf")
    for right, x in enumerate(nums):
        total += x
        while total >= target:
            best = min(best, right - left + 1); total -= nums[left]; left += 1
    return 0 if best == float("inf") else best
print(max_sum_window([2, 1, 5, 1, 3, 2], 3), longest_unique_substring("abcabcbb"),
longest_unique_substring("pwwkew"), min_window_at_least([2, 3, 1, 2, 4, 3], 7))
```

OUTPUT

```
9 3 3 2
```

The same idea appears in SQL window functions and in pandas `rolling`: a 7-day moving average is a sliding window.

Classic array problems

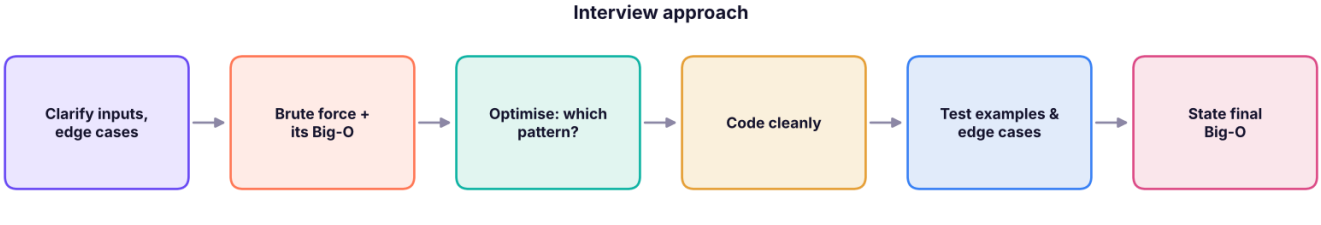
Python

```
def max_profit(prices):                          # best single buy then sell
    lo, best = float("inf"), 0
    for p in prices: lo = min(lo, p); best = max(best, p - lo)
    return best
def max_subarray(nums):                          # Kadane's algorithm
    cur = best = nums[0]
    for x in nums[1:]: cur = max(x, cur + x); best = max(best, cur)
    return best
def product_except_self(nums):
    n = len(nums); out = [1] * n; left = right = 1
    for i in range(n): out[i] = left; left *= nums[i]
    for i in range(n - 1, -1, -1): out[i] *= right; right *= nums[i]
    return out
def move_zeroes(a):
    w = 0
    for x in a:
        if x != 0: a[w] = x; w += 1
    a[w:] = [0] * (len(a) - w); return a
def rotate(a, k): k %= len(a); return a[-k:] + a[:-k]
print(max_profit([7, 1, 5, 3, 6, 4]), max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]),
product_except_self([1, 2, 3, 4]), move_zeroes([0, 1, 0, 3, 12]), rotate([1, 2, 3, 4, 5], 2))
```

OUTPUT

```
5 6 [24, 12, 8, 6] [1, 3, 12, 0, 0] [4, 5, 1, 2, 3]
```

How to answer in an interview



A calm, structured approach scores better than jumping into code.

Patterns to try, in order: **sort? hash map? two pointers? sliding window? prefix sum? stack? binary search? BFS/DFS? DP?** (the next two lessons cover the rest).

Summary

KEY TAKEAWAYS

- Big-O measures growth: O(1) < O(log n) < O(n) < O(n log n) < O(n^2) < O(2^n); drop constants, keep the dominant term.
- `in` on a list is O(n), on a set/dict O(1); inserting at the front of a list is O(n), use `deque`.
- Hash maps/Counters turn searches into lookups (two-sum, anagrams, frequency).
- Prefix sums give O(1) range sums; two pointers work on sorted data and palindromes; sliding windows handle subarray/substring problems in O(n).
- In interviews: clarify, brute force, optimise with a pattern, code, test, and state complexity.

Quick quiz

6 questions · tap an answer to check it

1. What is the time complexity of checking `x in my_set` ?

A O(1) on average

B O(n)

C O(log n)

D O(n^2)

Answer: A. Sets are hash tables.

2. Two nested loops over the same list of n items are...

A O(n)

B O(n log n)

C O(n^2)

D O(2n)

Answer: C. Nested steps multiply.

3. Which pattern solves "longest substring without repeating characters" in O(n)?

A Binary search

B Sliding window with a dictionary of last positions

C Sorting

D Prefix sums only

Answer: B. Shrink the window past repeated characters.

4. What does the two-sum hash map store?

A Each value seen so far mapped to its index

B All pairs

C The sorted list

D Only the target

Answer: A. Check whether target − x was already seen.

5. Which algorithm finds the maximum subarray sum in O(n)?

A Merge sort

B Kadane's algorithm

C Binary search

D Bubble sort

Answer: B. Keep the best sum ending at each position.

6. What is $O(3n + n^2 + 10)$ simplified?

A $O(3n)$

B $O(n^2)$

C $O(n)$

D $O(10)$

Answer: B. Keep the dominant term.

EXERCISE **Practice set (try before opening the solutions)**

(1) Return the indices of all pairs that sum to a target. (2) Check if a string has all unique characters. (3) Find the majority element (appears > n/2 times). (4) Longest run of consecutive 1s after flipping at most one 0 (sliding window). (5) Count customers whose total spend is in the top 10% (prefix sums not needed: think sorting).

Show solution (worked answer)

Python

```
def all_pairs(nums, t):
    seen, out = defaultdict(list), []
    for i, x in enumerate(nums):
        out += [(j, i) for j in seen[t - x]]; seen[x].append(i)
    return out
def all_unique(s): return len(set(s)) == len(s)
def majority(nums):                        # Boyer-Moore voting, O(1) space
    cand = cnt = 0
    for x in nums:
        if cnt == 0: cand = x
        cnt += 1 if x == cand else -1
    return cand
def longest_ones(a, k=1):
    left = zeros = best = 0
    for right, x in enumerate(a):
        zeros += x == 0
        while zeros > k: zeros -= a[left] == 0; left += 1
        best = max(best, right - left + 1)
    return best
print(all_pairs([1, 5, 3, 3, 7], 6), all_unique("data"), majority([2, 2, 1, 1, 2]),
longest_ones([1, 0, 1, 1, 0, 1]))
```

INTERVIEW CORNER

Questions you may be asked

Q: Find duplicates in a list of 10 million customer IDs. Use a set (or `Counter`) in one pass, O(n) time and O(n) space; in pandas `df.id.duplicated()`; in SQL `GROUP BY id HAVING COUNT(*) > 1`.

Q: Why is sorting O(n log n)? Comparison sorts split the problem in half repeatedly (log n levels) and do O(n) work per level; Python uses Timsort.

Q: Space vs time trade-off example? Two-sum: $O(n^2)$ time with O(1) space, or O(n) time with O(n) extra space for a hash map.

PYTHON FOR DATA SCIENCE · LESSON 27

DSA II: Stacks, Queues, Linked Lists, Recursion, Sorting and Binary Search

Learn linear data structures (stack, queue, deque, linked list), recursion and backtracking, the main sorting algorithms with their trade-offs, and binary search including "search on the answer", with interview problems such as valid parentheses, next greater element, reverse a linked list and search in a rotated array.

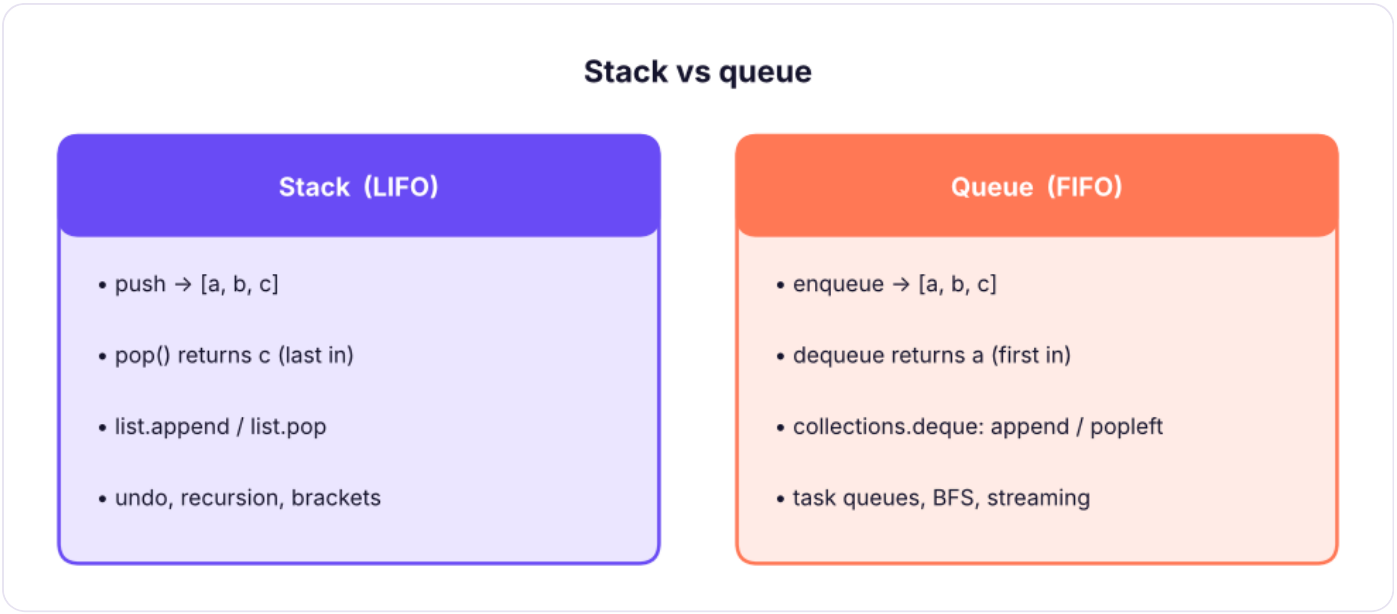
DSA for data roles (added) · Python 3.13 · Intermediate · 120 min

By the end of this lesson you can

- Implement and use stacks, queues and deques, and solve bracket and monotonic-stack problems
- Build and manipulate a singly linked list (reverse, detect a cycle, find the middle)
- Write recursive solutions with base cases and use backtracking for combinations
- Explain bubble, insertion, merge and quick sort and their complexities
- Apply binary search to sorted arrays, boundaries and answer spaces

Stacks: last in, first out

A **stack** adds and removes at the same end (LIFO). In Python a list works: `append` to push, `pop()` to pop, both O(1). Uses: undo, browser back, function calls, parsing brackets.



Stack (LIFO) vs queue (FIFO).

Python

```
def valid_parentheses(s):
    pairs, stack = {")": "(", "]": "[", "}": "{"}, []
    for ch in s:
        if ch in "([{": stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]: return False
    return not stack
print([valid_parentheses(x) for x in ["()[]{}", "([)]", "{[()]}", "((", "SUM(IF(a>1,(b),c))"]])

def eval_rpn(tokens):                        # reverse Polish notation: "3 4 + 2 *" = 14
    st = []
    for t in tokens:
        if t in "+-*/":
            b, a = st.pop(), st.pop()
            st.append({"+": a + b, "-": a - b, "*": a * b, "/": int(a / b)}[t])
        else: st.append(int(t))
    return st[0]
print(eval_rpn("3 4 + 2 *".split()))
```

OUTPUT

```
[True, False, True, False, True]
14
```

Monotonic stack

Keep the stack sorted to answer "next greater/smaller element" in O(n) instead of O(n²).

Python

```
def next_greater(nums):
    res, st = [-1] * len(nums), []          # st holds indices with decreasing values
    for i, x in enumerate(nums):
        while st and nums[st[-1]] < x: res[st.pop()] = x
        st.append(i)
    return res
def days_until_warmer(temps):
    res, st = [0] * len(temps), []
    for i, t in enumerate(temps):
        while st and temps[st[-1]] < t:
            j = st.pop(); res[j] = i - j
        st.append(i)
    return res
print(next_greater([2, 1, 2, 4, 3]), days_until_warmer([73, 74, 75, 71, 69, 72, 76, 73]))
```

OUTPUT

```
[4, 2, 4, -1, -1] [1, 1, 4, 2, 1, 1, 0, 0]
```

Queues and deques

Python

```
from collections import deque
q = deque(); q.append("job1"); q.append("job2"); q.appendleft("urgent")
print(q.popleft(), q.popleft(), list(q))
recent = deque(maxlen=3)                    # keeps only the last 3 items: a rolling buffer
for x in range(1, 7): recent.append(x)
print(list(recent))
def max_sliding_window(nums, k):            # monotonic deque, O(n)
    dq, out = deque(), []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] ⩽ x: dq.pop()
        dq.append(i)
        if dq[0] ⩽ i - k: dq.popleft()
        if i ⩾ k - 1: out.append(nums[dq[0]])
    return out
print(max_sliding_window([1, 3, -1, -3, 5, 3, 6, 7], 3))
```

OUTPUT

```
urgent job1 ['job2']
[4, 5, 6]
[3, 3, 5, 5, 6, 7]
```

Never use `list.pop(0)` for a queue: it is O(n). Use `deque.popleft()` (O(1)). `queue.Queue` adds thread safety; `heapq` gives a priority queue (next lesson).

Linked lists

A **linked list** stores nodes that point to the next node. Insert/delete at a known node is O(1); access by index is O(n). Python rarely needs them, but interviews love them.

Python

```
class Node:
    def __init__(self, val, nxt=None): self.val, self.next = val, nxt
def build(vals):
    head = None
    for v in reversed(vals): head = Node(v, head)
    return head
def to_list(head):
    out = []
    while head: out.append(head.val); head = head.next
    return out
def reverse(head):
    prev = None
    while head: head.next, prev, head = prev, head, head.next
    return prev
def middle(head):                          # slow/fast pointers
    slow = fast = head
    while fast and fast.next: slow, fast = slow.next, fast.next.next
    return slow.val
def has_cycle(head):                       # Floyd's tortoise and hare
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast: return True
    return False
def merge_two(a, b):
    dummy = tail = Node(0)
    while a and b:
        if a.val ⩽ b.val: tail.next, a = a, a.next
        else: tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b; return dummy.next
lst = build([1, 2, 3, 4, 5])
print(to_list(reverse(build([1, 2, 3, 4, 5]))), middle(lst), has_cycle(lst),
to_list(merge_two(build([1, 4, 7]), build([2, 3, 9]))))
cyc = build([1, 2, 3]); cyc.next.next.next = cyc.next; print(has_cycle(cyc))
```

OUTPUT

```
[5, 4, 3, 2, 1] 3 False [1, 2, 3, 4, 7, 9]
True
```

Recursion and backtracking

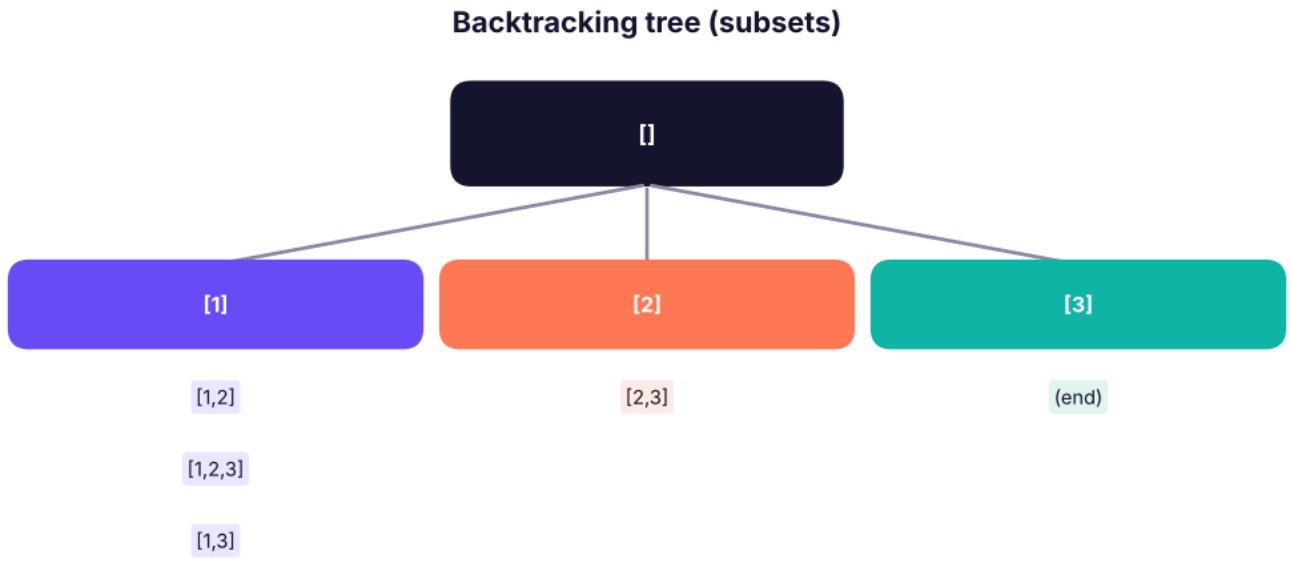
A recursive function solves a problem by calling itself on a smaller version. It needs a **base case** and must move toward it.

Python

```python
def power(x, n):                              # fast exponentiation, O(log n)
    if n == 0: return 1
    half = power(x, n // 2)
    return half * half * (x if n % 2 else 1)
def flatten(nested):
    out = []
    for item in nested: out.extend(flatten(item) if isinstance(item, list) else [item])
    return out
def subsets(nums):                            # backtracking: explore, choose, un-choose
    res = []
    def bt(start, path):
        res.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i]); bt(i + 1, path); path.pop()
    bt(0, []); return res
def permutations(s):
    if len(s) <= 1: return [s]
    return [ch + p for i, ch in enumerate(s) for p in permutations(s[:i] + s[i + 1:])]
print(power(2, 10), flatten([1, [2, [3, 4]], 5]), subsets([1, 2, 3]), permutations("abc"))
```

OUTPUT

```
1024 [1, 2, 3, 4, 5] [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]] ['abc',
'acb', 'bac', 'bca', 'cab', 'cba']
```



The recursion tree for subsets of [1, 2, 3]: each level decides whether to include the next item; $2^3 = 8$ leaves.

Sorting algorithms

Python

```
def bubble_sort(a):
    a = a[:]
    for i in range(len(a)):
        swapped = False
        for j in range(len(a) - 1 - i):
            if a[j] > a[j + 1]: a[j], a[j + 1] = a[j + 1], a[j]; swapped = True
        if not swapped: break
    return a
def insertion_sort(a):
    a = a[:]
    for i in range(1, len(a)):
        key, j = a[i], i - 1
        while j ⩾ 0 and a[j] > key: a[j + 1] = a[j]; j -= 1
        a[j + 1] = key
    return a
def merge_sort(a):
    if len(a) ⩽ 1: return a
    mid = len(a) // 2; left, right = merge_sort(a[:mid]), merge_sort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] ⩽ right[j]: out.append(left[i]); i += 1
        else: out.append(right[j]); j += 1
    return out + left[i:] + right[j:]
def quick_sort(a):
    if len(a) ⩽ 1: return a
    pivot = a[len(a) // 2]
    return quick_sort([x for x in a if x < pivot]) + [x for x in a if x == pivot] +
quick_sort([x for x in a if x > pivot])
data = [38, 27, 43, 3, 9, 82, 10]
print(bubble_sort(data), insertion_sort(data), merge_sort(data), quick_sort(data), sorted(data)
== merge_sort(data))
import random, time
big = [random.random() for _ in range(3000)]
for f in (insertion_sort, merge_sort, quick_sort, sorted):
    t0 = time.perf_counter(); f(big); print(f"{f.__name__:15s} {1000 * (time.perf_counter() -
t0):7.1f} ms")
```

OUTPUT

```
[3, 9, 10, 27, 38, 43, 82] [3, 9, 10, 27, 38, 43, 82] [3, 9, 10, 27, 38, 43, 82] [3, 9,
10, 27, 38, 43, 82] True
insertion_sort    133.5 ms
merge_sort          3.5 ms
quick_sort          3.5 ms
sorted              0.2 ms
```

Algorithm	Best	Average	Worst	Space	Stable	Notes
Bubble	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Teaching only
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Great for small / nearly sorted
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Predictable; external sorting
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No	Fast in practice
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No	
Timsort (Python `sorted`)	O(n)	O(n log n)	O(n log n)	O(n)	Yes	Merge + insertion hybrid

Stable means equal keys keep their original order, so you can sort by city, then stably by region. In practice, use `sorted(items, key=...)`.

Binary search

On **sorted** data, halve the search space each step: O(log n). One million items need at most 20 comparisons.

Python

```
def binary_search(a, target):
    lo, hi = 0, len(a) - 1
    while lo ⩽ hi:
        mid = (lo + hi) // 2
        if a[mid] == target: return mid
        if a[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1
import bisect
a = [3, 8, 15, 15, 15, 23, 42]
print(binary_search(a, 23), binary_search(a, 5), bisect.bisect_left(a, 15),
bisect.bisect_right(a, 15), bisect.bisect_right(a, 15) - bisect.bisect_left(a, 15))
def search_rotated(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo ⩽ hi:
        mid = (lo + hi) // 2
        if nums[mid] == target: return mid
        if nums[lo] ⩽ nums[mid]:                    # left half sorted
            if nums[lo] ⩽ target < nums[mid]: hi = mid - 1
            else: lo = mid + 1
        else:                                       # right half sorted
            if nums[mid] < target ⩽ nums[hi]: lo = mid + 1
            else: hi = mid - 1
    return -1
print(search_rotated([4, 5, 6, 7, 0, 1, 2], 0), search_rotated([4, 5, 6, 7, 0, 1, 2], 3))
def min_capacity_to_ship(weights, days):           # "binary search on the answer"
    def ok(cap):
        d, load = 1, 0
        for w in weights:
            if load + w > cap: d += 1; load = 0
            load += w
        return d ⩽ days
    lo, hi = max(weights), sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid): hi = mid
        else: lo = mid + 1
    return lo
print(min_capacity_to_ship([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5))
grades = ["F", "D", "C", "B", "A"]; cutoffs = [40, 50, 65, 80]
print([grades[bisect.bisect_right(cutoffs, s)] for s in (35, 50, 79, 92)])        # banding with
bisect
```

OUTPUT

```
5 -1 2 5 3
4 -1
15
['F', 'C', 'B', 'A']
```

`bisect` is how pandas `cut` and SQL range lookups work under the hood: find the band for a value in O(log n).

Summary

KEY TAKEAWAYS

- Stack = LIFO (list append/pop); queue = FIFO (`deque` append/popleft); monotonic stacks/deques solve next-greater and window-max in O(n).
- Linked lists: pointer manipulation; slow/fast pointers find the middle and detect cycles.
- Recursion needs a base case; backtracking = choose, explore, un-choose.
- Merge sort is O(n log n) and stable; quick sort is fast on average; insertion sort suits small inputs; use `sorted` .
- Binary search halves the space: O(log n). Use it on sorted arrays, boundaries (`bisect`) and "minimum value that works" answers.

Quick quiz

6 questions · tap an answer to check it

1. Which structure checks balanced brackets?

A Stack

B Queue

C Set

D Heap

Answer: A. Push openings, pop on closings.

2. Why use `deque.popleft()` instead of `list.pop(0)` ?

A It sorts the data

B It is O(1); list.pop(0) is O(n)

C It is thread safe

D No difference

Answer: B. Lists shift every element when removing from the front.

3. How do you find the middle of a linked list in one pass?

A Count then walk

B Slow and fast pointers

C Sort it

D Binary search

Answer: B. When fast reaches the end, slow is in the middle.

4. What is the worst-case complexity of quick sort?

A O(n log n)

B $O(n^2)$

C O(n)

D O(log n)

Answer: B. With bad pivots; randomised pivots make it unlikely.

5. How many comparisons does binary search need for about 1,000,000 sorted items?

A 1,000,000

B 1,000

C About 20

D 2

Answer: C. $\log_2(1{,}000{,}000) \approx 20$.

6. What does a "stable" sort guarantee?

A It never fails

B Items with equal keys keep their original relative order

C O(n log n) always

D It uses no memory

Answer: B. Merge sort and Timsort are stable.

EXERCISE **Practice set**

(1) Implement a queue using two stacks. (2) Remove adjacent duplicate letters repeatedly ("abbaca" → "ca"). (3) Find the first and last position of a target in a sorted array with `bisect`. (4) Return all combinations of k numbers from 1..n. (5) Compute the integer square root with binary search.

Show solution (worked answer)

Python

```
class QueueTwoStacks:
    def __init__(self): self.inbox, self.outbox = [], []
    def push(self, x): self.inbox.append(x)
    def pop(self):
        if not self.outbox:
            while self.inbox: self.outbox.append(self.inbox.pop())
        return self.outbox.pop()
def remove_adjacent(s):
    st = []
    for ch in s:
        if st and st[-1] == ch: st.pop()
        else: st.append(ch)
    return "".join(st)
def first_last(a, t):
    i, j = bisect.bisect_left(a, t), bisect.bisect_right(a, t) - 1
    return [i, j] if i <= j else [-1, -1]
def combine(n, k):
    res = []
    def bt(start, path):
        if len(path) == k: res.append(path[:]); return
        for i in range(start, n + 1): path.append(i); bt(i + 1, path); path.pop()
    bt(1, []); return res
def isqrt(x):
    lo, hi = 0, x
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if mid * mid <= x: lo = mid
        else: hi = mid - 1
    return lo
print(remove_adjacent("abbaca"), first_last([5, 7, 7, 8, 8, 10], 8), combine(4, 2),
isqrt(80))
```

INTERVIEW CORNER

Questions you may be asked

Q: Explain merge sort. Split the list in half recursively until single items remain, then merge sorted halves by repeatedly taking the smaller front item; O(n log n) time, O(n) extra space, stable.

Q: When is binary search applicable? When the data is sorted, or more generally when a yes/no condition is monotonic over the search space (search on the answer).

Q: Stack vs queue in real systems? Stacks: call stacks, undo, expression parsing, DFS. Queues: job and message queues (Kafka, SQS), BFS, rate-limited processing.

PYTHON FOR DATA SCIENCE · LESSON 28

DSA III: Trees, Heaps, Graphs (BFS/DFS) and Dynamic Programming

Finish the DSA toolkit. Binary trees and binary search trees with traversals, heaps and priority queues with heapq (top-k, merge k lists, running median), graphs with adjacency lists, BFS and DFS, shortest paths and topological sort (as used in data pipelines), and dynamic programming from memoisation to tables.

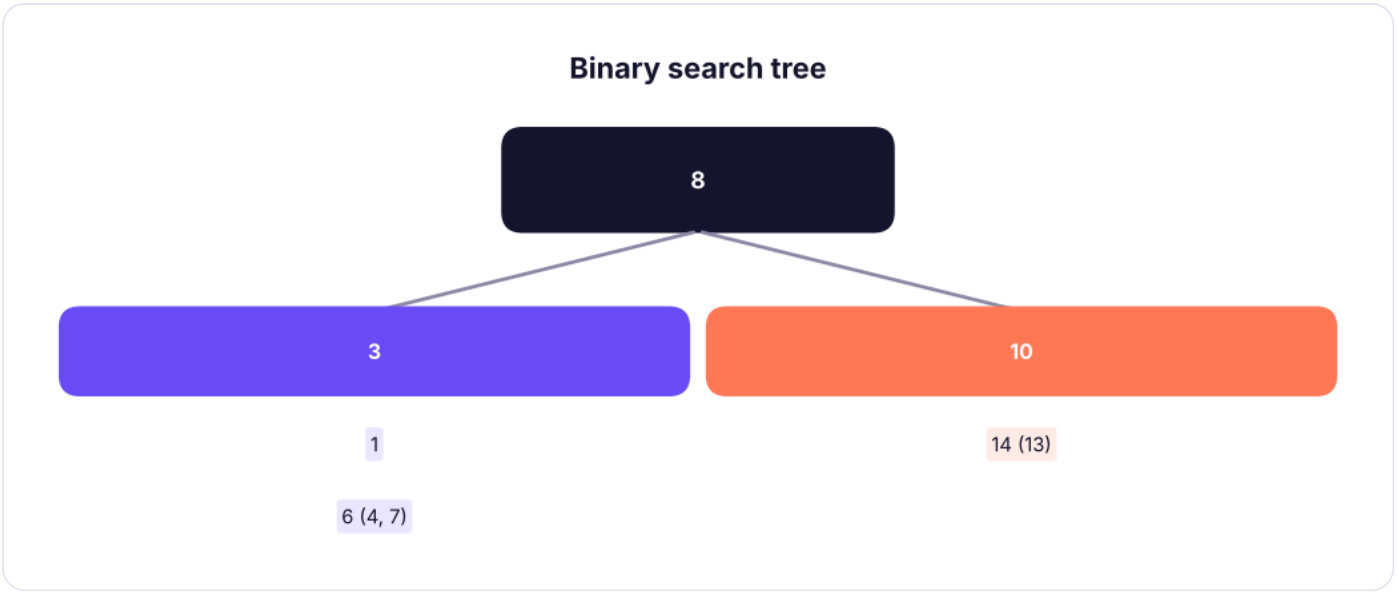
DSA for data roles (added) · Python 3.13 · Advanced · 130 min

By the end of this lesson you can

- Traverse binary trees (pre-, in-, post- and level-order) and use a binary search tree
- Use heapq for top-k, k-way merge, scheduling and running medians
- Represent graphs and apply BFS, DFS, Dijkstra and topological sort
- Recognise dynamic-programming problems and solve them with memoisation and tabulation
- Connect these structures to data tools (B-tree indexes, DAG schedulers, recommendation graphs)

Trees

A **tree** is a hierarchy of nodes: one root, each node with children, no cycles. Examples: folders, org charts, JSON documents, decision trees in ML, and B-tree indexes in databases.



A binary search tree: every left child is smaller and every right child larger than its parent.

Python

```
class TreeNode:
    def __init__(self, val, left=None, right=None): self.val, self.left, self.right = val, left,
right
def insert(root, val):                          # BST insert, O(height)
    if not root: return TreeNode(val)
    if val < root.val: root.left = insert(root.left, val)
    else: root.right = insert(root.right, val)
    return root
root = None
for v in [8, 3, 10, 1, 6, 14, 4, 7, 13]: root = insert(root, v)
def inorder(n): return inorder(n.left) + [n.val] + inorder(n.right) if n else []
def preorder(n): return [n.val] + preorder(n.left) + preorder(n.right) if n else []
def postorder(n): return postorder(n.left) + postorder(n.right) + [n.val] if n else []
from collections import deque
def level_order(n):
    out, q = [], deque([n])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft(); level.append(node.val)
            q.extend(c for c in (node.left, node.right) if c)
        out.append(level)
    return out
def height(n): return 1 + max(height(n.left), height(n.right)) if n else 0
def contains(n, x):
    while n:
        if x == n.val: return True
        n = n.left if x < n.val else n.right
    return False
print("in-order  :", inorder(root), "(sorted!)")
print("pre-order :", preorder(root)); print("post-order:", postorder(root)); print("levels
:", level_order(root))
print("height", height(root), "| contains 7?", contains(root, 7), "| contains 5?",
contains(root, 5))
```

OUTPUT

```
in-order  : [1, 3, 4, 6, 7, 8, 10, 13, 14] (sorted!)
pre-order : [8, 3, 1, 6, 4, 7, 10, 14, 13]
post-order: [1, 4, 7, 6, 3, 13, 14, 10, 8]
levels    : [[8], [3, 10], [1, 6, 14], [4, 7, 13]]
height 4 | contains 7? True | contains 5? False
```

Traversal	Order	Use
In-order	left, node, right	BST → sorted output
Pre-order	node, left, right	Copy/serialise a tree
Post-order	left, right, node	Delete, compute sizes bottom-up
Level-order (BFS)	level by level	Shortest path in unweighted trees, org levels

Python

```python
def is_valid_bst(n, lo=float("-inf"), hi=float("inf")):
    return not n or (lo < n.val < hi and is_valid_bst(n.left, lo, n.val) and
is_valid_bst(n.right, n.val, hi))
def lowest_common_ancestor(n, a, b):            # in a BST
    while n:
        if a < n.val and b < n.val: n = n.left
        elif a > n.val and b > n.val: n = n.right
        else: return n.val
def max_path_sum_root_to_leaf(n):
    if not n: return float("-inf")
    if not n.left and not n.right: return n.val
    return n.val + max(max_path_sum_root_to_leaf(n.left), max_path_sum_root_to_leaf(n.right))
print(is_valid_bst(root), lowest_common_ancestor(root, 4, 7), lowest_common_ancestor(root, 1,
14), max_path_sum_root_to_leaf(root))
```

OUTPUT

```
True 6 8 45
```

A BST is O(log n) only when **balanced**; inserting sorted data creates a "linked list" with O(n) lookups. Databases use balanced **B-trees**, which is why an index makes `WHERE id = 42` fast.

Heaps and priority queues

A **heap** always gives the smallest item in O(1) and inserts/removes in O(log n). Python's `heapq` is a min-heap on a list.

Python

```python
import heapq
h = []
for x in [5, 1, 8, 3, 9, 2]: heapq.heappush(h, x)
print(h[0], [heapq.heappop(h) for _ in range(3)])
sales = {"Pune": 120, "Delhi": 340, "Kochi": 90, "Goa": 210, "Jaipur": 175}
print(heapq.nlargest(3, sales.items(), key=lambda kv: kv[1]))
def top_k_stream(stream, k):                # keep a size-k min-heap: O(n log k), constant memory
    heap = []
    for x in stream:
        if len(heap) < k: heapq.heappush(heap, x)
        elif x > heap[0]: heapq.heapreplace(heap, x)
    return sorted(heap, reverse=True)
print(top_k_stream(iter([15, 3, 99, 42, 7, 64, 23, 88]), 3))
print(list(heapq.merge([1, 4, 9], [2, 3, 10], [0, 7])))            # k-way merge of sorted lists (external sort)
tasks = [(2, "send report"), (1, "fix pipeline"), (3, "update deck")]  # (priority, task)
heapq.heapify(tasks); print([heapq.heappop(tasks)[1] for _ in range(3)])

class RunningMedian:                        # two heaps: max-heap (negated) + min-heap
    def __init__(self): self.lo, self.hi = [], []
    def add(self, x):
        heapq.heappush(self.lo, -x); heapq.heappush(self.hi, -heapq.heappop(self.lo))
        if len(self.hi) > len(self.lo): heapq.heappush(self.lo, -heapq.heappop(self.hi))
    def median(self): return -self.lo[0] if len(self.lo) > len(self.hi) else (-self.lo[0] + self.hi[0]) / 2
rm = RunningMedian(); out = []
for x in [5, 15, 1, 3, 8, 7]: rm.add(x); out.append(rm.median())
print(out)
```

OUTPUT

```
1 [1, 2, 3]
[('Delhi', 340), ('Goa', 210), ('Jaipur', 175)]
[99, 88, 64]
[0, 1, 2, 3, 4, 7, 9, 10]
['fix pipeline', 'send report', 'update deck']
[5, 10.0, 5, 4.0, 5, 6.0]
```

Graphs

A **graph** has nodes (vertices) and edges, optionally directed and weighted. Social networks, road maps, product co-purchases, and data-pipeline dependencies are graphs.



A directed acyclic graph (DAG) of pipeline tasks: a task runs only after its dependencies. Airflow, dbt and Dagster schedule exactly this.

Python

```
from collections import defaultdict, deque
edges = [("A", "B"), ("A", "C"), ("B", "D"), ("C", "D"), ("D", "E"), ("F", "G")]
g = defaultdict(list)
for u, v in edges: g[u].append(v); g[v].append(u)              # undirected adjacency list
def bfs(start):
    seen, order, q = {start}, [], deque([start])
    while q:
        node = q.popleft(); order.append(node)
        for nb in g[node]:
            if nb not in seen: seen.add(nb); q.append(nb)
    return order
def dfs(node, seen=None):
    seen = seen if seen is not None else set(); seen.add(node); out = [node]
    for nb in g[node]:
        if nb not in seen: out += dfs(nb, seen)
    return out
def shortest_hops(a, b):                      # BFS gives shortest path in unweighted graphs
    q, prev = deque([a]), {a: None}
    while q:
        n = q.popleft()
        if n == b: break
        for nb in g[n]:
            if nb not in prev: prev[nb] = n; q.append(nb)
    path = []
    while b is not None: path.append(b); b = prev.get(b)
    return path[::-1]
def components():
    seen, comps = set(), []
    for n in list(g):
        if n not in seen: c = dfs(n, seen); comps.append(sorted(c))
    return comps
print("BFS from A:", bfs("A"), "| DFS from A:", dfs("A"), "| path A→E:", shortest_hops("A",
"E"), "| components:", components())
```

OUTPUT

```
BFS from A: ['A', 'B', 'C', 'D', 'E'] | DFS from A: ['A', 'B', 'D', 'C', 'E'] | path A→E:
['A', 'B', 'D', 'E'] | components: [['A', 'B', 'C', 'D', 'E'], ['F', 'G']]
```

Python

```
def number_of_islands(grid):                # classic grid DFS
    rows, cols, count = len(grid), len(grid[0]), 0
    def sink(r, c):
        if 0 <= r < rows and 0 <= c < cols and grid[r][c] == "1":
            grid[r][c] = "0"
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)): sink(r + dr, c + dc)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1": count += 1; sink(r, c)
    return count
print(number_of_islands([list("11000"), list("11000"), list("00100"), list("00011")]))
```

OUTPUT

```
3
```

Dijkstra's shortest path (weighted)

Python

```
def dijkstra(graph, src):
    dist = {src: 0}; pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist.get(u, float("inf")): continue
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, float("inf")): dist[v] = d + w; heapq.heappush(pq, (d + w,
v))
    return dist
roads = {"Mumbai": [("Pune", 150), ("Nashik", 170)], "Pune": [("Satara", 110), ("Nashik", 210)],
"Nashik": [("Aurangabad", 180)], "Satara": [("Kolhapur", 120)], "Aurangabad": [], "Kolhapur":
[]}
print(dijkstra(roads, "Mumbai"))
```

OUTPUT

```
{'Mumbai': 0, 'Pune': 150, 'Nashik': 170, 'Satara': 260, 'Aurangabad': 350, 'Kolhapur':
380}
```

Topological sort: ordering a pipeline

Python

```
deps = {"extract_orders": [], "extract_customers": [], "clean_orders": ["extract_orders"],
"join_customers": ["clean_orders", "extract_customers"],
        "build_marts": ["join_customers"], "refresh_dashboard": ["build_marts"], "train_model":
["join_customers"]}
def topo_sort(deps):                          # Kahn's algorithm
    indeg = {t: len(d) for t, d in deps.items()}; children = defaultdict(list)
    for t, ds in deps.items():
        for d in ds: children[d].append(t)
    q = deque(sorted(t for t, k in indeg.items() if k == 0)); order = []
    while q:
        t = q.popleft(); order.append(t)
        for c in sorted(children[t]):
            indeg[c] -= 1
            if indeg[c] == 0: q.append(c)
    if len(order) != len(deps): raise ValueError("cycle detected")
    return order
print(topo_sort(deps))
from graphlib import TopologicalSorter        # standard-library version
print(list(TopologicalSorter(deps).static_order())[:3])
```

OUTPUT

```
['extract_customers', 'extract_orders', 'clean_orders', 'join_customers', 'build_marts',
'train_model', 'refresh_dashboard']
['extract_orders', 'extract_customers', 'clean_orders']
```

Dynamic programming

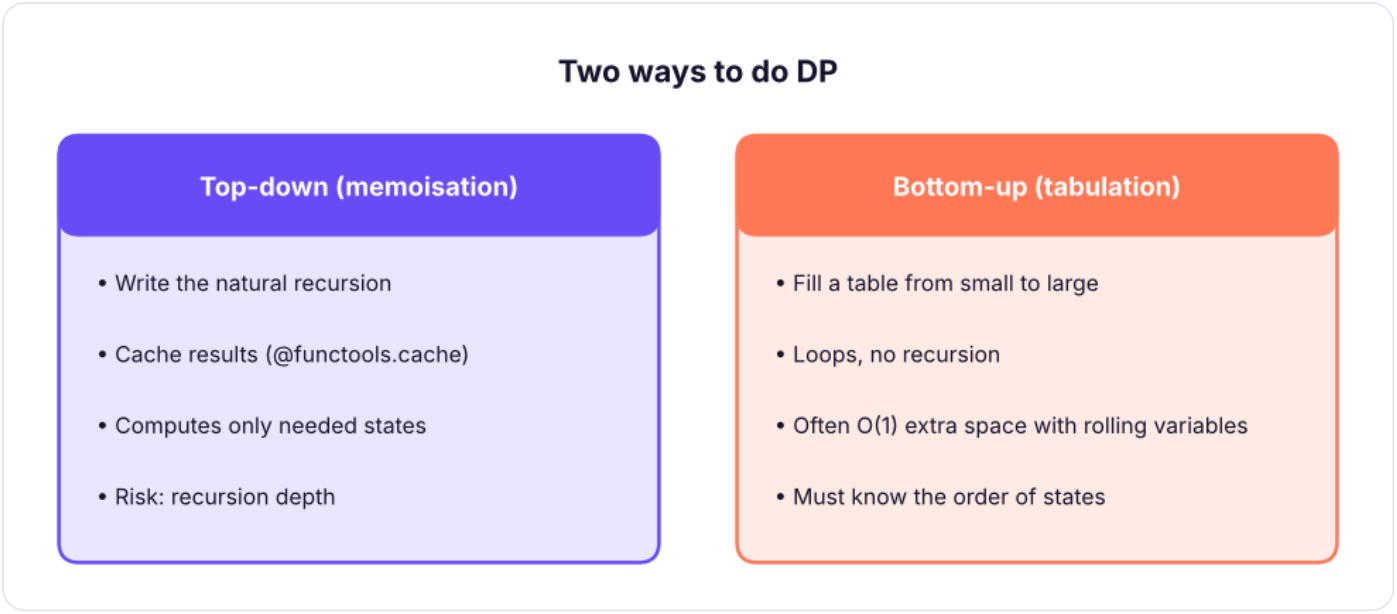
Dynamic programming (DP) solves problems with **overlapping subproblems** and **optimal substructure** by storing answers to subproblems instead of recomputing them.

Python

```
import functools, time
def fib_naive(n): return n if n < 2 else fib_naive(n - 1) + fib_naive(n - 2)
@functools.cache
def fib_memo(n): return n if n < 2 else fib_memo(n - 1) + fib_memo(n - 2)
def fib_table(n):
    a, b = 0, 1
    for _ in range(n): a, b = b, a + b
    return a
t0 = time.perf_counter(); fib_naive(27); t1 = time.perf_counter(); fib_memo(27); t2 =
time.perf_counter()
print(f"naive {1000*(t1-t0):.0f} ms (O(2^n)) | memoised {1000*(t2-t1):.3f} ms (O(n)) | values
equal: {fib_naive(20) == fib_memo(20) == fib_table(20)}")
```

OUTPUT

```
naive 31 ms (O(2^n)) | memoised 0.022 ms (O(n)) | values equal: True
```



Top-down (memoisation) vs bottom-up (tabulation) dynamic programming.

Python

```python
def climb_stairs(n):                          # 1 or 2 steps at a time
    a, b = 1, 1
    for _ in range(n): a, b = b, a + b
    return a
def coin_change(coins, amount):               # fewest coins
    dp = [0] + [float("inf")] * amount
    for x in range(1, amount + 1):
        dp[x] = min((dp[x - c] + 1 for c in coins if c <= x), default=float("inf"))
    return dp[amount] if dp[amount] != float("inf") else -1
def house_robber(nums):                       # no two adjacent
    take = skip = 0
    for x in nums: take, skip = skip + x, max(take, skip)
    return max(take, skip)
def lis(nums):                                # longest increasing subsequence, O(n log n)
    import bisect; tails = []
    for x in nums:
        i = bisect.bisect_left(tails, x)
        tails[i:i + 1] = [x]
    return len(tails)
def edit_distance(a, b):                      # Levenshtein: used in fuzzy matching of names
    dp = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        prev, dp[0] = dp[0], i
        for j in range(1, len(b) + 1):
            prev, dp[j] = dp[j], min(dp[j] + 1, dp[j - 1] + 1, prev + (a[i - 1] != b[j - 1]))
    return dp[-1]
def knapsack(items, cap):                     # 0/1 knapsack: (weight, value)
    dp = [0] * (cap + 1)
    for w, v in items:
        for c in range(cap, w - 1, -1): dp[c] = max(dp[c], dp[c - w] + v)
    return dp[cap]
print(climb_stairs(10), coin_change([1, 2, 5], 11), coin_change([2], 3), house_robber([2, 7, 9, 3, 1]), lis([10, 9, 2, 5, 3, 7, 101, 18]))
print(edit_distance("Bangalore", "Bengaluru"), edit_distance("kitten", "sitting"), knapsack([(1, 1), (3, 4), (4, 5), (5, 7)], 7))
```

OUTPUT

```
89 3 -1 12 4
3 3 9
```

How to spot DP: "minimum/maximum/number of ways", choices at each step, and the same subproblem appearing again. Define the **state** (`dp[x]` = best answer for amount x), the **transition**, and the **base case**.

Where these appear in data work

Structure	In data tools
B-trees	Database indexes
Hash tables	Hash joins, `GROUP BY`, Python dicts, Spark shuffles
Heaps	Top-N queries, schedulers, k-way merge in external sorting
DAGs + topological sort	Airflow, dbt, Dagster, Spark execution plans
Graphs + BFS	Recommendations ("people also bought"), fraud rings, knowledge graphs for GenAI
Edit distance (DP)	Fuzzy matching, deduplicating customer names
Trees	Decision trees, random forests, gradient boosting

Summary

KEY TAKEAWAYS

- Trees: four traversals; a BST's in-order traversal is sorted; balanced trees give O(log n) search (database B-tree indexes).
- Heaps (`heapq`) give the minimum in O(1) and push/pop in O(log n): top-k, k-way merge, priority scheduling, running median.
- Graphs as adjacency lists; BFS (queue) for levels and shortest unweighted paths, DFS (recursion/stack) for components and islands; Dijkstra for weighted paths; topological sort for DAG pipelines.
- DP = overlapping subproblems + optimal substructure; use memoisation (`@functools.cache`) or tabulation; define state, transition and base case.

Quick quiz

6 questions · tap an answer to check it

1. Which traversal of a BST returns the values in sorted order?

- **A** Pre-order
- **B** In-order
- **C** Post-order
- **D** Level-order

Answer: B. Left subtree, node, right subtree.

2. What does `heapq` give you in O(1)?

- **A** The smallest item
- **B** The largest item
- **C** The median
- **D** A sorted list

Answer: A. Negate values for a max-heap.

3. Which search finds the shortest path in an unweighted graph?

- **A** BFS
- **B** DFS
- **C** Binary search
- **D** Sorting

Answer: A. BFS explores level by level.

4. Which algorithm orders tasks so each runs after its dependencies?

A Dijkstra

B Topological sort

C Merge sort

D Kadane

Answer: B. It works on directed acyclic graphs.

5. Two signs that a problem suits dynamic programming are...

A Overlapping subproblems and optimal substructure

B Sorted input and unique values

C A graph and a heap

D Strings only

Answer: A. Store and reuse subproblem answers.

6. Why does a database index speed up lookups?

A It stores data in CSV

B It keeps keys in a balanced tree (B-tree) so lookups are O(log n)

C It caches every query

D It compresses the table

Answer: B. Without an index the database scans every row.

EXERCISE **Practice set**

(1) Return the maximum depth and the left view of a binary tree. (2) Find the k closest points to the origin with a heap. (3) Detect a cycle in a directed graph of task dependencies. (4) Count the number of ways to make change for an amount. (5) Longest common subsequence of two strings.

Show solution (worked answer)

Python

```
def left_view(n): return [lvl[0] for lvl in level_order(n)]
def k_closest(points, k): return heapq.nsmallest(k, points, key=lambda p: p[0] ** 2 + p[1]
** 2)
def has_cycle_directed(deps):
    try: topo_sort(deps); return False
    except ValueError: return True
def ways_to_change(coins, amount):
    dp = [1] + [0] * amount
    for c in coins:
        for x in range(c, amount + 1): dp[x] += dp[x - c]
    return dp[amount]
def lcs(a, b):
    dp = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            dp[i][j] = dp[i - 1][j - 1] + 1 if a[i - 1] == b[j - 1] else max(dp[i - 1][j],
dp[i][j - 1])
    return dp[-1][-1]
print(height(root), left_view(root), k_closest([(1, 3), (-2, 2), (5, 8)], 2),
has_cycle_directed({"a": ["b"], "b": ["a"]}), ways_to_change([1, 2, 5], 5), lcs("abcde",
"ace"))
```

INTERVIEW CORNER **Questions you may be asked**

Q: BFS vs DFS? BFS uses a queue, explores by distance and finds shortest unweighted paths; DFS uses recursion or a stack, goes deep first, and suits connectivity, cycle detection and topological ordering. Both are O(V + E).

Q: How would you get the top 10 products from a stream of billions of sales? Maintain a size-10 min-heap of running totals per product (or approximate with Count-Min Sketch), or aggregate in SQL/Spark and take the top 10.

Q: Memoisation vs tabulation? Memoisation caches results of a top-down recursion; tabulation fills a table bottom-up. Same complexity; tabulation avoids recursion limits and often allows space optimisation.

PYTHON FOR DATA SCIENCE · LESSON 29

Python Capstone: End-to-End Retail Sales Analytics Project

Bring the whole Python track together on the Superstore data. Structure a project, build a validated data pipeline, compute KPIs, growth and Pareto analysis, cohort retention and discount impact, create a chart pack and an Excel report, and write the README and executive summary you can put in your portfolio.

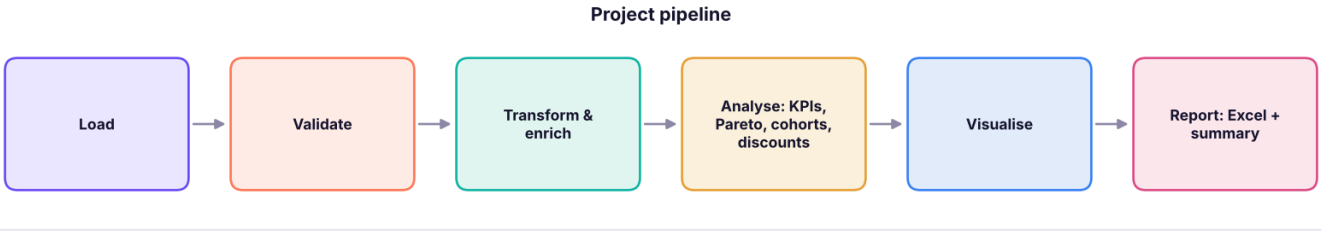
Python capstone | pandas 3 · Matplotlib 3.10 · openpyxl | Advanced | 180 min

By the end of this lesson you can

- Organise an analysis as a reproducible project with functions, checks and outputs
- Build a load → validate → transform pipeline with assertions
- Analyse KPIs, growth, Pareto concentration, cohorts and discount effects
- Produce a multi-sheet Excel report and a chart pack
- Write an executive summary and a portfolio-ready README

The brief

The sales director of an Indian office-supplies retailer asks: **How did 2025 go, where do we make and lose money, how loyal are our customers, and what should we change in 2026?** Deliver a short report with charts and an Excel workbook her team can filter.



Capstone pipeline.

Project layout (Lesson 13): `data/raw`, `src/` (functions below), `notebooks/` (exploration), `reports/` (outputs), `README.md`.

1. Load and validate

Python

```python
import pandas as pd, numpy as np, matplotlib.pyplot as plt
pd.set_option("display.max_columns", 20); pd.set_option("display.width", 160)

def load(path="superstore_sales.csv"):
    df = pd.read_csv(path, parse_dates=["order_date", "ship_date"])
    return df

def validate(df):
    checks = {
        "row_id unique": df["row_id"].is_unique,
        "no missing values": df.isna().sum().sum() == 0,
        "sales positive": (df["sales"] > 0).all(),
        "discount between 0 and 1": df["discount"].between(0, 1).all(),
        "ship after order": (df["ship_date"] ⩾ df["order_date"]).all(),
        "quantity 1-20": df["quantity"].between(1, 20).all(),
    }
    for name, ok in checks.items(): print(f"  {'PASS' if ok else 'FAIL'}  {name}")
    assert all(checks.values()), "validation failed"
    return df

s = validate(load())
print(s.shape, s.order_date.min().date(), "→", s.order_date.max().date())
```

OUTPUT

```
  PASS  row_id unique
  PASS  no missing values
  PASS  sales positive
  PASS  discount between 0 and 1
  PASS  ship after order
  PASS  quantity 1-20
(6323, 19) 2023-01-01 → 2025-12-31
```

2. Transform and enrich

Python

```
def enrich(df):
    out = df.copy()
    out["year"] = out.order_date.dt.year
    out["month"] = out.order_date.dt.to_period("M")
    out["margin_pct"] = 100 * out.profit / out.sales
    out["ship_days"] = (out.ship_date - out.order_date).dt.days
    out["discount_band"] = pd.cut(out.discount, [-0.01, 0, 0.1, 0.2, 1], labels=["0%", "1-10%",
"11-20%", ">20%"])
    return out
s = enrich(s)
print(s[["order_id", "year", "month", "sales", "profit", "margin_pct",
"discount_band"]].head(3))
```

OUTPUT

	order_id	year	month	sales	profit	margin_pct	discount_band
0	IN-2023-00001	2023	2023-01	36000.0	3472.55	9.645972	0%
1	IN-2023-00001	2023	2023-01	3850.0	1169.54	30.377662	0%
2	IN-2023-00001	2023	2023-01	62080.0	6044.47	9.736582	0%

3. Headline KPIs and growth

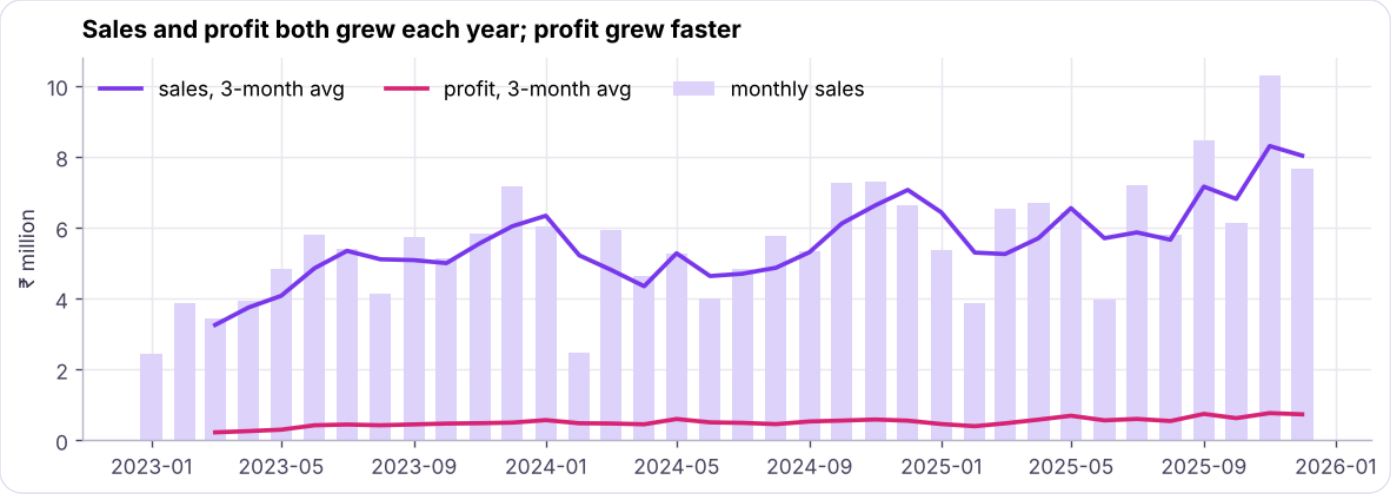
Python

```
def kpis(df):
    g = df.groupby("year").agg(sales=("sales", "sum"), profit=("profit", "sum"), orders=
("order_id", "nunique"), customers=("customer_id", "nunique"), units=("quantity", "sum"))
    g["margin_%"] = 100 * g.profit / g.sales
    g["aov"] = g.sales / g.orders
    g["sales_growth_%"] = 100 * g.sales.pct_change()
    g["profit_growth_%"] = 100 * g.profit.pct_change()
    return g
k = kpis(s)
view = k.assign(sales=k.sales / 1e6, profit=k.profit / 1e6, aov=k.aov / 1e3).round(1)
view.columns = ["sales ₹M", "profit ₹M", "orders", "customers", "units", "margin %", "AOV ₹K",
"sales growth %", "profit growth %"]
print(view)
```

OUTPUT

year	sales ₹M	profit ₹M	orders	customers	units	margin %	AOV ₹K	sales growth %	profit growth %
2023	57.6	4.7	883	540	4714	8.2	65.3	NaN	NaN
2024	65.4	6.1	1085	590	5404	9.4	60.3	13.5	29.9

```
2025        78.4         7.5     1232         623    6266        9.6     63.6              19.8
22.1
```



Monthly sales and profit, 2023–2025, with 3-month rolling averages.

4. Where we make and lose money

Python

```python
sub = s[s.year == 2025].groupby(["category", "sub_category"]).agg(sales=("sales", "sum"),
profit=("profit", "sum"), avg_disc=("discount", "mean"))
sub["margin_%"] = (100 * sub.profit / sub.sales).round(1)
sub = sub.sort_values("profit")
print((sub.assign(sales=sub.sales / 1e6, profit=sub.profit / 1e6)).round(2))
```

OUTPUT

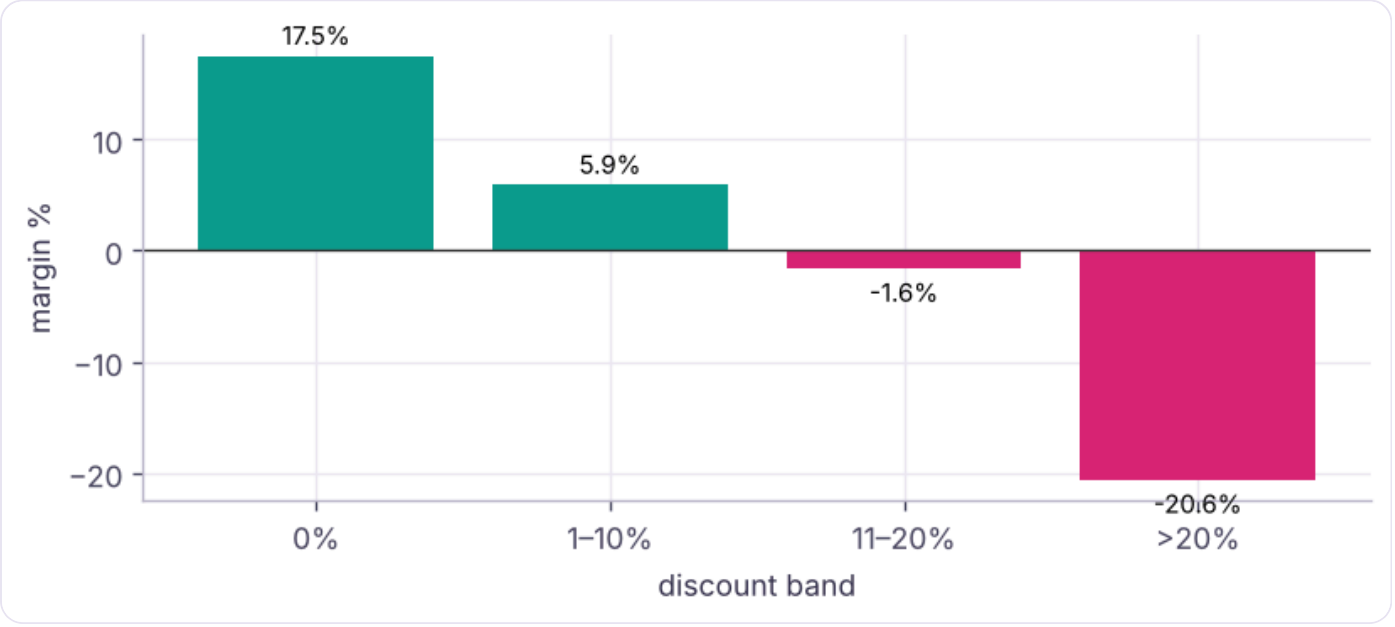
		sales	profit	avg_disc	margin_%
category	sub_category				
Furniture	Tables	5.30	-0.29	0.17	-5.4
	Bookcases	3.92	-0.17	0.19	-4.3
Office Supplies	Art	0.10	0.02	0.08	15.3
	Binders	0.16	0.02	0.08	14.2
Furniture	Furnishings	0.49	0.03	0.08	5.2
Office Supplies	Paper	0.19	0.03	0.07	15.8
Furniture	Chairs	3.05	0.13	0.08	4.3
Office Supplies	Storage	0.98	0.14	0.08	14.1
Technology	Accessories	1.22	0.15	0.07	11.9
Office Supplies	Appliances	2.54	0.38	0.07	14.9
Technology	Phones	12.94	1.71	0.06	13.2
	Machines	16.50	2.00	0.08	12.1
	Copiers	30.98	3.35	0.08	10.8

Python

```
db = s.groupby("discount_band", observed=True).agg(lines=("row_id", "size"), sales=("sales", "sum"), profit=("profit", "sum"))
db["margin_%"] = (100 * db.profit / db.sales).round(1); db["share_of_lines_%"] = (100 * db.lines / db.lines.sum()).round(1)
print(db.assign(sales=(db.sales / 1e6).round(1), profit=(db.profit / 1e6).round(2)))
```

OUTPUT

discount_band	lines	sales	profit	margin_%	share_of_lines_%
0%	3480	123.9	21.72	17.5	55.0
1–10%	726	21.9	1.29	5.9	11.5
11–20%	1217	35.9	-0.59	-1.6	19.2
>20%	900	19.7	-4.06	-20.6	14.2



Margin by discount band: deep discounts turn profitable lines into losses.

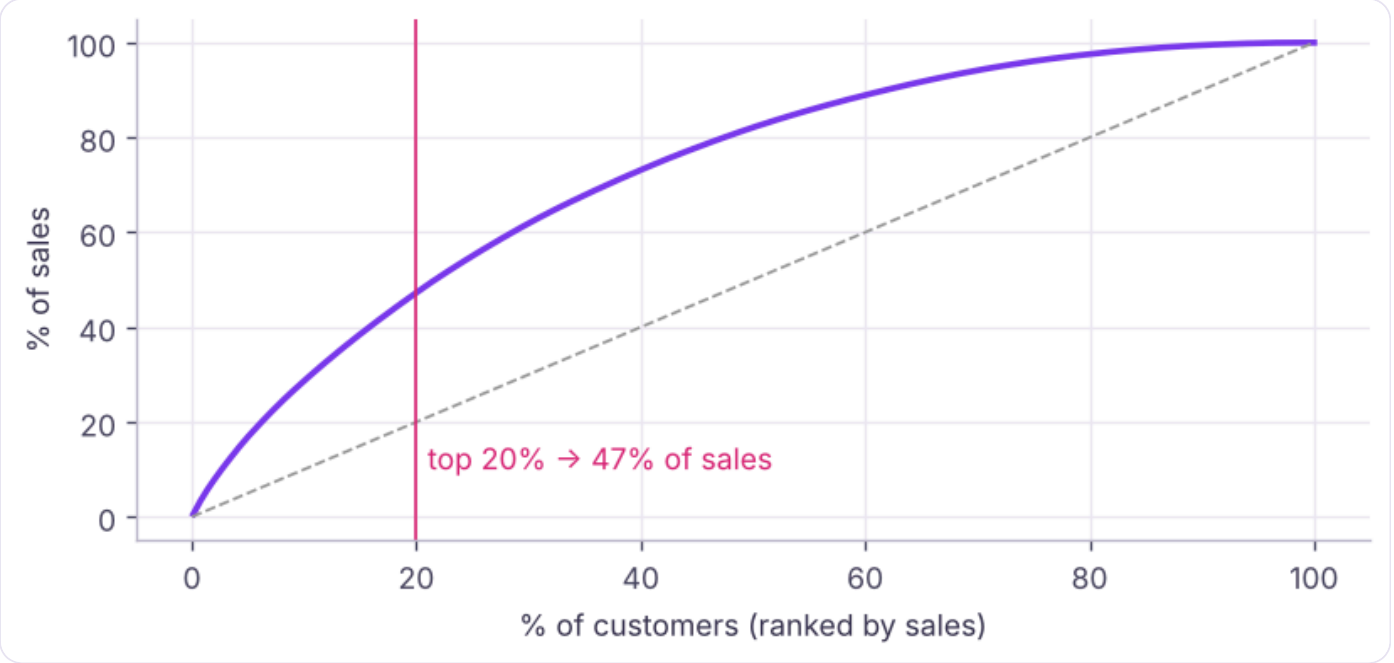
5. Customer concentration (Pareto)

Python

```
cust = s.groupby("customer_id")["sales"].sum().sort_values(ascending=False)
cum = cust.cumsum() / cust.sum()
top20 = int(len(cust) * 0.2)
print(f"{len(cust)} customers | top 20% ({top20}) bring {cum.iloc[top20 - 1]:.0%} of sales | customers needed for 50% of sales: {(cum < 0.5).sum() + 1}")
```

OUTPUT

```
785 customers | top 20% (157) bring 47% of sales | customers needed for 50% of sales: 171
```



Pareto curve of customer sales.

6. Cohort retention

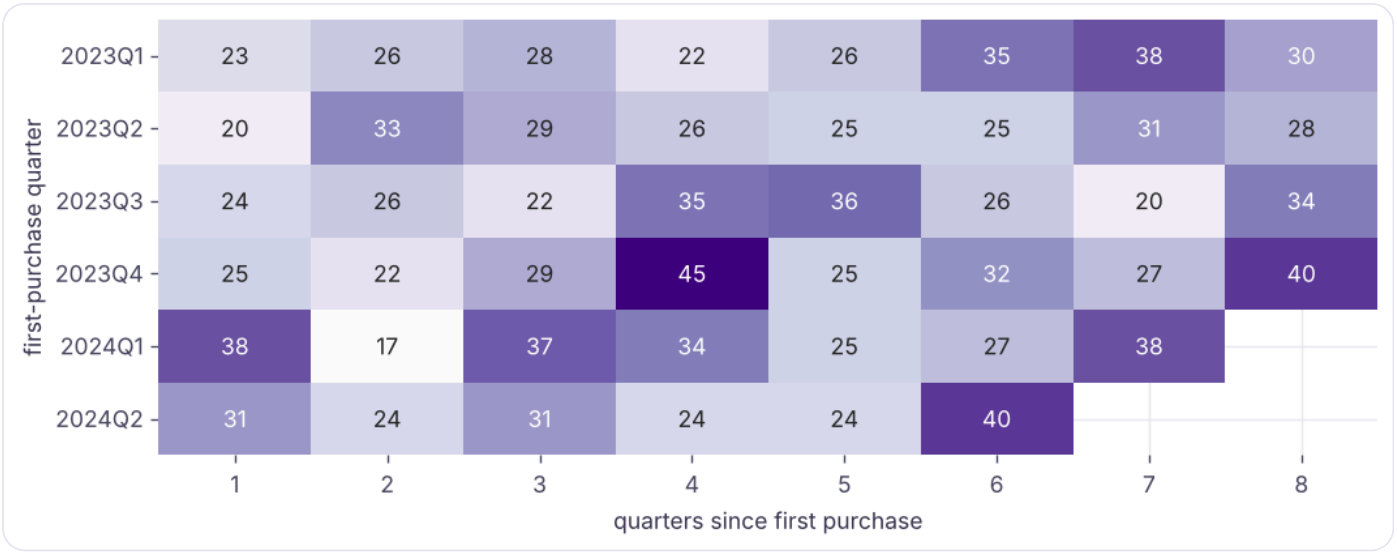
A **cohort** groups customers by the month (here: quarter) of their first purchase; retention tracks how many buy again later.

Python

```
s["quarter"] = s.order_date.dt.to_period("Q")
first = s.groupby("customer_id")["quarter"].min().rename("cohort")
c = s.join(first, on="customer_id")
c["age_q"] = (c.quarter - c.cohort).apply(lambda d: d.n)
active = c.groupby(["cohort", "age_q"])["customer_id"].nunique().unstack()
retention = (active.div(active[0], axis=0) * 100).round(0)
print(active[0].rename("cohort size").head(6).to_dict())
print(retention.iloc[:6, :8])
```

OUTPUT

```
{Period('2023Q1', 'Q-DEC'): 151.0, Period('2023Q2', 'Q-DEC'): 142.0, Period('2023Q3', 'Q-DEC'): 127.0, Period('2023Q4', 'Q-DEC'): 120.0, Period('2024Q1', 'Q-DEC'): 71.0, Period('2024Q2', 'Q-DEC'): 42.0}
age_q       0     1     2     3     4     5     6     7
cohort
2023Q1  100.0  23.0  26.0  28.0  22.0  26.0  35.0  38.0
2023Q2  100.0  20.0  33.0  29.0  26.0  25.0  25.0  31.0
2023Q3  100.0  24.0  26.0  22.0  35.0  36.0  26.0  20.0
2023Q4  100.0  25.0  22.0  29.0  45.0  25.0  32.0  27.0
2024Q1  100.0  38.0  17.0  37.0  34.0  25.0  27.0  38.0
2024Q2  100.0  31.0  24.0  31.0  24.0  24.0  40.0   NaN
```



Quarterly cohort retention (% of each cohort buying in later quarters).

7. Operations: shipping

Python

```
ship = s.groupby("ship_mode").agg(lines=("row_id", "size"), avg_days=("ship_days", "mean"),
sales=("sales", "sum"))
ship["share_%"] = (100 * ship.lines / ship.lines.sum()).round(1)
print(ship.round(1).sort_values("avg_days"))
reg = s[s.year == 2025].groupby("region").agg(sales=("sales", "sum"), profit=("profit", "sum"))
reg["margin_%"] = (100 * reg.profit / reg.sales).round(1); print((reg.assign(sales=reg.sales /
1e6, profit=reg.profit / 1e6)).round(2))
```

OUTPUT

```
                lines  avg_days        sales  share_%
ship_mode
Same Day          321       0.0   10082429.5      5.1
First Class       948       1.5   31455543.0     15.0
Second Class     1249       3.0   40858316.0     19.8
Standard Class   3805       5.6  119017280.5     60.2
        sales  profit  margin_%
region
East    18.91    2.07      10.9
North   16.43    1.56       9.5
South   20.80    1.77       8.5
West    22.23    2.10       9.4
```

8. Excel report for the business

Python

```
from pathlib import Path
out = Path("reports"); out.mkdir(exist_ok=True)
with pd.ExcelWriter(out / "sales_review_2025.xlsx", engine="openpyxl") as xw:
    view.to_excel(xw, sheet_name="KPIs")
    sub.reset_index().to_excel(xw, sheet_name="Sub-category 2025", index=False)
    db.reset_index().to_excel(xw, sheet_name="Discount bands", index=False)
    retention.reset_index().astype(str).to_excel(xw, sheet_name="Cohort retention", index=False)
    s[s.year == 2025].drop(columns=["month", "quarter"]).to_excel(xw, sheet_name="Data 2025",
index=False)
import openpyxl
wb = openpyxl.load_workbook(out / "sales_review_2025.xlsx")
for ws in wb.worksheets:
    ws.freeze_panes = "B2"; ws.auto_filter.ref = ws.dimensions
    for col in ws.columns: ws.column_dimensions[col[0].column_letter].width = 16
wb.save(out / "sales_review_2025.xlsx")
print(wb.sheetnames, f"{(out / 'sales_review_2025.xlsx').stat().st_size / 1e6:.1f} MB")
import shutil; shutil.rmtree(out)
```

OUTPUT

```
['KPIs', 'Sub-category 2025', 'Discount bands', 'Cohort retention', 'Data 2025'] 0.3 MB
```

9. Executive summary (template)

Write the summary **from the numbers above**; every claim should point to a table or chart.

KEY TAKEAWAYS

Fill each line from your outputs: 1. **Growth**: 2025 sales of **₹M (+**% on 2024), profit **₹M (+**%), margin __%. 2. **Profit engines**: __ and __ deliver __% of profit; __ and **lose mon**. 3. **Discounts**: lines discounted above 20% h**e a __% m**gin, against __% at no discount. 4. **Customers**: the top 20% of customers brin**__% of sales, an**__% of a cohort is still buying a year later. 5. **Actions for 2026**: cap discounts on __; review pricing of __; a loyalty programme for the top customers; a retention push in the first quarter after a customer's first purchase.

10. README for your portfolio

Text

```
# Superstore India: 2025 Sales Review (Python)
**Question:** How did 2025 perform, where is profit made and lost, how loyal are customers?
**Data:** 6,323 order lines, 2023-2025 (synthetic, modelled on the Superstore dataset).
**Tools:** pandas, Matplotlib, Seaborn, openpyxl.
**Method:** validation checks → enrichment → KPIs, Pareto, cohort retention, discount analysis.
**Key findings:** (3-5 bullets with numbers)
**Recommendations:** (3 bullets)
**How to run:** `pip install -r requirements.txt && python src/run_report.py`
**Repo:** data/ · src/ · notebooks/ · reports/ · README.md
```

Summary

KEY TAKEAWAYS

- A project is a pipeline: load → validate (assertions) → enrich → analyse → visualise → report.
- KPI tables with growth, sub-category profitability, discount bands, Pareto concentration and cohort retention answer most retail questions.
- `pd.ExcelWriter` + openpyxl create business-ready, filterable workbooks.
- Finish with an executive summary tied to numbers and a portfolio README.

Quick quiz

6 questions · tap an answer to check it

1. Why put validation checks at the start of the pipeline?

A To stop early if the data breaks assumptions, rather than producing wrong results

B To make the code longer

C Because pandas requires it

D To speed up loading

Answer: A. Assertions make failures loud.

2. What does a Pareto analysis of customers show?

A Average sales

B How concentrated sales are among the top customers

C Retention over time

D Discount impact

Answer: B. Often a minority of customers brings most of the revenue.

3. What is a cohort in retention analysis?

A Customers grouped by the period of their first purchase

B Customers in the same city

C All customers

D Products in a category

Answer: A. Retention follows each cohort over time.

4. Which tool writes several sheets to one Excel file?

A `df.to_csv`

B `pd.ExcelWriter`

C `openpyxl.load_workbook` only

D `df.to_json`

Answer: B. Then format with openpyxl.

5. What belongs in an executive summary?

A All code

B Key numbers, findings and recommended actions

C Raw data

D Library versions

Answer: B. Lead with the answer.

6. Why write analysis steps as functions?

A They can be tested, re-run on new data and reused

B They run faster in all cases

C Notebooks forbid loops

D To avoid variables

Answer: A. Reproducibility matters.

EXERCISE

Extend the capstone

(1) Add product-level ABC analysis (A = top 80% of sales). (2) Compute RFM scores for 2025 customers (see ML track lesson on segmentation). (3) Turn the chart cells into a function that saves PNGs to `reports/`. (4) Publish the project on GitHub with the README template and one chart in the README.

Show solution (worked answer)

Python

```
prod = s.groupby("product_name")["sales"].sum().sort_values(ascending=False)
share = prod.cumsum() / prod.sum()
abc = pd.cut(share, [0, .8, .95, 1], labels=["A", "B", "C"]); print(abc.value_counts())
snap = s.order_date.max() + pd.Timedelta(days=1)
rfm = s[s.year == 2025].groupby("customer_id").agg(R=("order_date", lambda d: (snap -
d.max()).days), F=("order_id", "nunique"), M=("sales", "sum"))
rfm["score"] = pd.qcut(-rfm.R, 4, labels=False) + pd.qcut(rfm.F.rank(method="first"), 4,
labels=False) + pd.qcut(rfm.M, 4, labels=False)
print(rfm.sort_values("score", ascending=False).head())
```

INTERVIEW CORNER

Questions you may be asked

Q: Walk me through a project you are proud of. Use STAR: the business situation and question, your task, the actions (data, cleaning, analysis, tools), and the result with numbers and what changed. Keep it to two minutes and be ready to go deeper on any step.

Q: How do you make sure your numbers are right? Validation checks, reconciling totals with a trusted source, spot-checking individual records, peer review, and versioned, re-runnable code.

Q: What would you do next with more time? Automate the refresh (schedule + Parquet/database), add a dashboard, build a churn or demand-forecast model, and run an experiment on discount policy.

TRACK 6

Statistics & Machine

Statistics & Machine Learning

Probability, CLT, hypothesis tests, the ML lifecycle, regression, classification, trees, ensembles, clustering, PCA, time series, NLP, deep learning basics, evaluation, explainability and deployment.

1. Descriptive Statistics: Centre, Spread, Shape and Relationships
2. Probability and Distributions: Rules, Bayes, Binomial, Poisson and Normal
3. Sampling, the Central Limit Theorem and Confidence Intervals
4. Hypothesis Testing I: The Framework, p-values, z-tests and t-tests
5. Hypothesis Testing II: Chi-Square, ANOVA, Non-Parametric Tests and Correlation Tests
6. A/B Testing and Experimentation: Design, Power, Analysis and Pitfalls
7. Introduction to Machine Learning: Types, Business Framing and the ML Lifecycle
8. The scikit-learn Workflow: Train/Test Split, Baselines, Pipelines and Saving Models
9. Data Preprocessing and Feature Engineering for ML
10. Linear Regression: Fitting, Interpreting, Assumptions and Diagnostics
11. Gradient Descent: How Models Learn (Built from Scratch in NumPy)
12. Overfitting, Polynomial Regression and Regularisation (Ridge, Lasso, Elastic Net)
13. Logistic Regression: Probabilities, Odds Ratios and Credit-Risk Scoring
14. Classification Metrics: Confusion Matrix, Precision, Recall, ROC, PR and Business Value
15. Cross-Validation and Hyperparameter Tuning: K-Fold, GridSearchCV, RandomizedSearchCV and Optuna
16. K-Nearest Neighbours: Classifier and Regressor (Bank Loans Case Study)
17. Decision Trees: Splits, Impurity, Pruning and Interpretable Rules
18. Ensembles I: Bagging and Random Forests
19. Ensembles II: Gradient Boosting, XGBoost, LightGBM and CatBoost
20. Support Vector Machines and Naive Bayes
21. Imbalanced Classification: Fraud Detection with Class Weights, Resampling and Thresholds
22. Clustering I: K-Means and RFM Customer Segmentation
23. Clustering II and Dimensionality Reduction: Hierarchical, DBSCAN, Gaussian Mixtures, PCA, t-SNE and UMAP
24. Time-Series Forecasting: Decomposition, Baselines, ETS, ARIMA and ML with Lag Features
25. NLP and Text Analytics: Cleaning, TF-IDF, Sentiment Classification, Topics and Embeddings
26. Neural Networks and Deep Learning: From a Neuron to Transformers
27. Explainable AI and MLOps: SHAP, Fairness, Deployment, Monitoring and Responsible ML
28. ML Capstone: End-to-End Churn Prediction and Retention Strategy

STATISTICS & MACHINE LEARNING · LESSON 1

Descriptive Statistics: Centre, Spread, Shape and Relationships

Describe data with numbers before modelling it. Types of data and measurement scales, mean, median and mode, range, variance, standard deviation, IQR and coefficient of variation, percentiles and z-scores, skewness and kurtosis, covariance and correlation (Pearson and Spearman), all on the house-price and HR datasets.

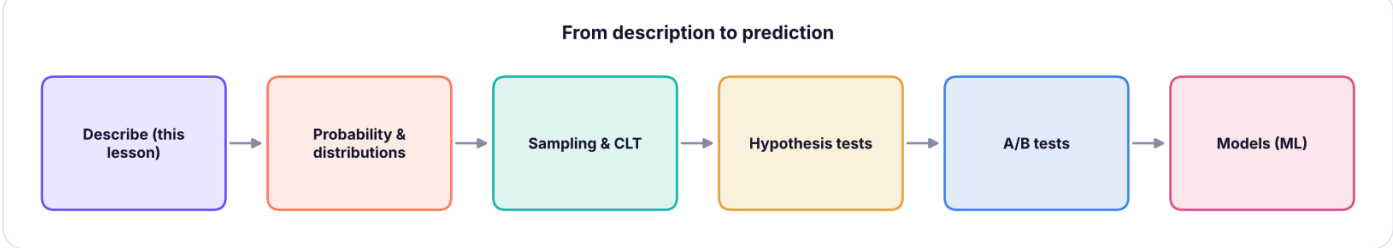
Python class 14 · pandas 3 · SciPy 1.1x · Beginner · 100 min

By the end of this lesson you can

- Classify variables as categorical (nominal, ordinal) or numeric (discrete, continuous; interval, ratio)
- Compute and interpret mean, median, mode, weighted mean and trimmed mean
- Compute and interpret range, IQR, variance, standard deviation and coefficient of variation
- Use percentiles and z-scores and describe skewness and kurtosis
- Measure association with covariance, Pearson and Spearman correlation, and explain why correlation is not causation

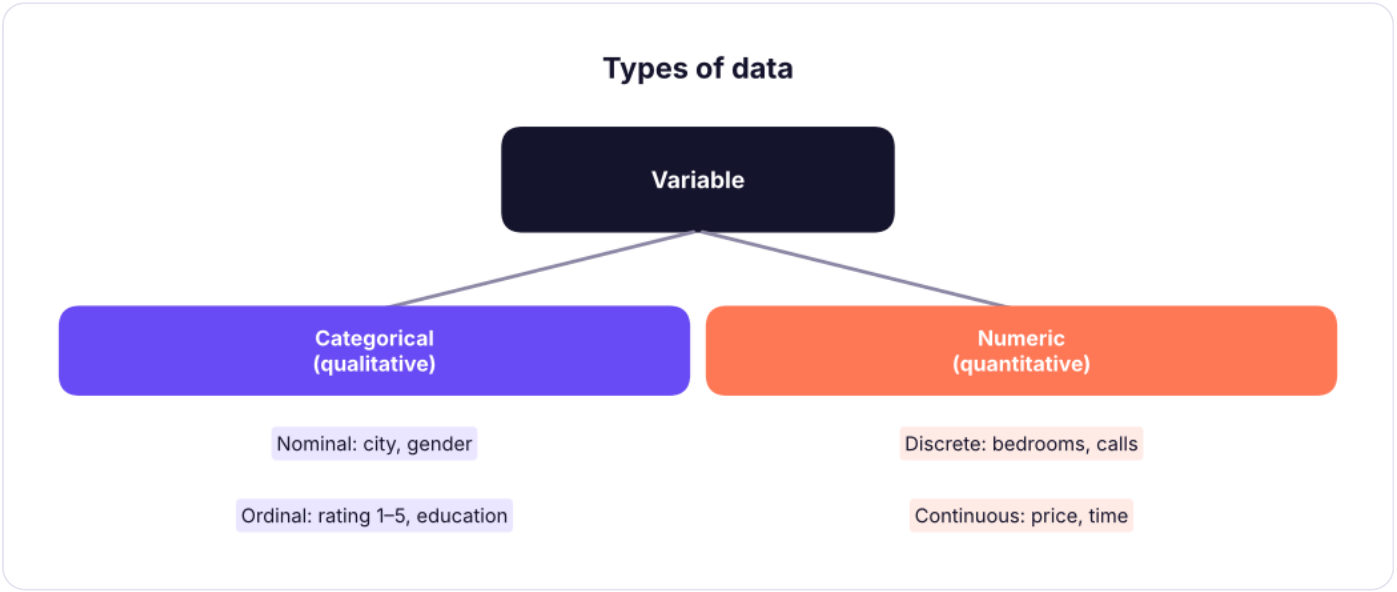
Statistics in data science

Descriptive statistics summarise the data you have. **Inferential statistics** use a sample to draw conclusions about a larger population (next lessons). Machine learning builds on both.



The statistics roadmap for this track.

Types of data



Variable types decide which statistics and charts make sense.

Scale	Has order?	Equal intervals?	True zero?	Example	Allowed statistics
Nominal	No	–	–	City	Mode, counts
Ordinal	Yes	No	–	Satisfaction 1–4	Median, percentiles
Interval	Yes	Yes	No	Temperature °C	Mean, sd (no ratios)
Ratio	Yes	Yes	Yes	Income, price	Everything, including ratios

Measures of centre

Python

```
import pandas as pd, numpy as np
from scipy import stats
h = pd.read_csv("house_prices.csv")
p = h["price_lakh"]
print(f"mean {p.mean():.1f} | median {p.median():.1f} | mode (rounded to 10)
{(p.round(-1)).mode()[0]:.0f} | trimmed mean 10% {stats.trim_mean(p, 0.1):.1f}")
print("bedrooms mode:", h["bedrooms"].mode().tolist(), "| city mode:", h["city"].mode()[0])
w = pd.DataFrame({"city_avg_price": [62, 88, 75], "houses": [100, 20, 50]})
print("simple mean of city averages:", w.city_avg_price.mean().round(1), "| weighted mean:",
np.average(w.city_avg_price, weights=w.houses).round(1))
```

OUTPUT

```
mean 80.4 | median 78.9 | mode (rounded to 10) 80 | trimmed mean 10% 78.9
bedrooms mode: [2] | city mode: Bengaluru
simple mean of city averages: 75.0 | weighted mean: 68.9
```

Measure	Use when	Sensitive to outliers?
Mean	Symmetric numeric data; totals matter	Yes
Median	Skewed data (income, prices), ordinal data	No
Mode	Categories, most common value	No
Trimmed mean	Want a mean but robust to extremes	Less
Weighted mean	Groups of different sizes	Depends

Python

```
salaries = pd.Series([40, 42, 45, 47, 50, 52, 55, 60, 65, 900])      # one CEO in thousands
print(f"mean {salaries.mean():.1f}K vs median {salaries.median():.1f}K: one extreme value moves the mean, not the median")
```

OUTPUT

```
mean 135.6K vs median 51.0K: one extreme value moves the mean, not the median
```

Measures of spread

Python

```
def spread(x):
    q1, q3 = x.quantile([.25, .75])
    return pd.Series({"range": x.max() - x.min(), "IQR": q3 - q1, "variance": x.var(), "std": x.std(), "CV_%": 100 * x.std() / x.mean(), "MAD": (x - x.median()).abs().median()})
print(pd.DataFrame({"price_lakh": spread(p), "area_sqft": spread(h["area_sqft"].dropna()), "age_years": spread(h["age_years"])}).round(1))
```

OUTPUT

```
          price_lakh  area_sqft  age_years
range          518.0     2130.0       29.0
IQR             42.2      580.0       15.0
variance      1261.9   158572.6       74.2
std             35.5      398.2        8.6
CV_%            44.2       32.1       59.8
MAD             21.0      290.0        7.0
```

- **Variance** = average squared distance from the mean; **standard deviation** (sd) = its square root, in the original units.
- pandas `.std()` uses the **sample** formula (divide by *n* − *1*, Bessel's correction); NumPy `np.std` defaults to population (*n*).
- **Coefficient of variation** (CV = sd ÷ mean) compares spread across variables with different units.

- **IQR** (Q3 − Q1) is the spread of the middle 50%, robust to outliers.

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Percentiles, z-scores and the five-number summary

Python

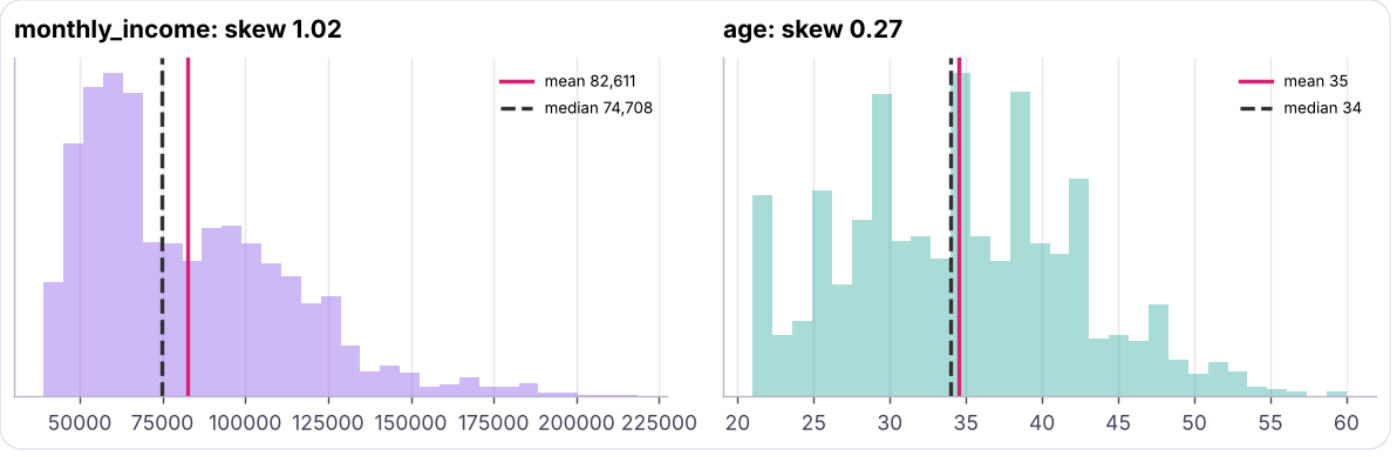
```
print(p.quantile([0, .1, .25, .5, .75, .9, 1]).round(1).to_dict())
z = (p - p.mean()) / p.std()
print(f"a ₹150 lakh house has z = {(150 - p.mean()) / p.std():.2f} | {(z.abs() > 2).mean():.1%}
of houses are beyond ±2 sd")
print(f"percentile rank of ₹150 lakh: {stats.percentileofscore(p, 150):.1f}")
```

OUTPUT

```
{0.0: 2.0, 0.1: 40.5, 0.25: 57.3, 0.5: 78.9, 0.75: 99.4, 0.9: 120.1, 1.0: 520.0}
a ₹150 lakh house has z = 1.96 | 1.7% of houses are beyond ±2 sd
percentile rank of ₹150 lakh: 98.2
```

A **z-score** says how many standard deviations a value is from the mean: it puts different variables on one scale (used in standardisation for ML).

Shape: skewness and kurtosis



Right-skewed (income), roughly symmetric (age) and the mean/median positions.

Python

```
for col in ["monthly_income", "age", "years_at_company", "distance_from_home_km"]:
    x = hr[col]; print(f"{col:24s} skew {x.skew():6.2f} | excess kurtosis {x.kurt():6.2f} | mean {x.mean():10,.1f} | median {x.median():10,.1f}")
print("log(income) skew:", round(np.log(hr.monthly_income).skew(), 2))
```

OUTPUT

```
monthly_income           skew   1.02 | excess kurtosis   0.95 | mean   82,610.7 | median 74,708.5
age                      skew   0.27 | excess kurtosis  -0.39 | mean       34.5 | median 34.0
years_at_company         skew   1.55 | excess kurtosis   3.00 | mean        3.8 | median 3.0
distance_from_home_km    skew   0.00 | excess kurtosis  -1.26 | mean       15.2 | median 15.0
log(income) skew: 0.28
```

Skewness	Shape	Mean vs median
≈ 0	Symmetric	Mean ≈ median
> 0	Right (positive) skew, long right tail	Mean > median
< 0	Left skew	Mean < median

Kurtosis describes tail heaviness (excess kurtosis 0 = normal; positive = more extreme values). A log transform often makes right-skewed money variables more symmetric.

Relationships: covariance and correlation

Python

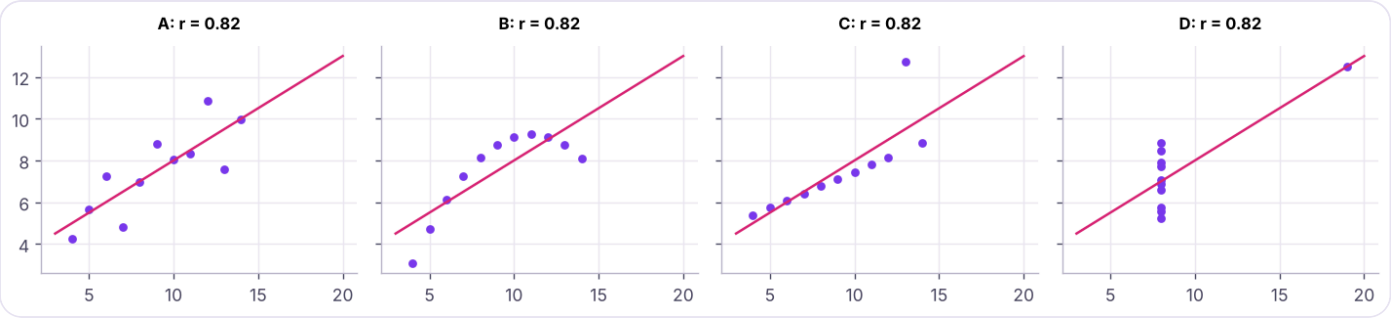
```
num = h[["price_lakh", "area_sqft", "bedrooms", "age_years", "distance_to_center_km"]].dropna()
print("covariance price-area:", round(num.price_lakh.cov(num.area_sqft), 1), "(units: lakh × sq ft, hard to interpret)")
print(num.corr().round(2)["price_lakh"].sort_values(ascending=False))
print("Spearman (rank):", num.corr(method="spearman").round(2)["price_lakh"].drop("price_lakh").to_dict())
r, pval = stats.pearsonr(num.area_sqft, num.price_lakh); print(f"Pearson r = {r:.3f}, p-value = {pval:.1e}")
```

OUTPUT

```
covariance price-area: 11182.8 (units: lakh × sq ft, hard to interpret)
price_lakh               1.00
area_sqft                0.79
bedrooms                 0.58
```

```
distance_to_center_km   -0.07
age_years               -0.11
Name: price_lakh, dtype: float64
Spearman (rank): {'area_sqft': 0.9, 'bedrooms': 0.68, 'age_years': -0.13,
'distance_to_center_km': -0.06}
Pearson r = 0.787, p-value = 3.3e-248
```

- **Pearson r** measures linear association (−1 to +1). Rough guide: |r| < 0.3 weak, 0.3–0.7 moderate, > 0.7 strong.
- **Spearman ρ** correlates ranks: robust to outliers and captures monotonic non-linear relations.
- **Cramér's V** (categorical × categorical) and point-biserial (binary × numeric) cover other pairs.



Anscombe's quartet: four datasets with the same mean, variance and correlation (r ≈ 0.82) but completely different shapes. Always plot your data.

WATCH OUT **Correlation is not causation**

Ice-cream sales and drowning both rise in summer (a **confounder**: temperature). Correlation can also come from reverse causation, selection bias or chance. Only experiments or careful causal methods show cause.

Summary

KEY TAKEAWAYS

- Know the variable type and scale first: it decides valid statistics.
- Centre: mean (sensitive), median (robust), mode (categories); use weighted means for groups of different sizes.
- Spread: range, IQR (robust), variance and sd (sample uses n − 1), CV to compare variables.
- Shape: skewness (mean vs median), kurtosis (tails); log transforms reduce right skew.
- z-scores standardise; percentiles rank. Correlation (Pearson linear, Spearman rank) measures association, not causation; always plot.

Quick quiz

6 questions · tap an answer to check it

1. Which is the best measure of centre for household income?

- **A** Mean
- **B** Median
- **C** Mode
- **D** Range

Answer: B. Income is right-skewed; the median resists extremes.

2. In a right-skewed distribution, which is usually true?

- **A** Mean > median
- **B** Mean < median
- **C** Mean = median
- **D** Mode > mean

Answer: A. The long right tail pulls the mean up.

3. Why does the sample standard deviation divide by n − 1?

- **A** To make it larger for fun
- **B** To correct the bias from estimating the mean from the same sample (Bessel's correction)
- **C** Because n is unknown
- **D** It is a convention with no reason

Answer: B. It gives an unbiased variance estimate.

4. A z-score of −1.5 means...

- **A** The value is 1.5 standard deviations below the mean
- **B** The value is negative
- **C** The value is in the 15th percentile exactly
- **D** An error

Answer: A. z = (x − mean) / sd.

5. Which correlation suits a monotonic but non-linear relationship?

- **A** Pearson
- **B** Spearman
- **C** Covariance
- **D** None

Answer: B. Spearman uses ranks.

6. What does Anscombe's quartet teach?

- **A** Correlation proves causation
- **B** Identical summary statistics can hide very different patterns, so plot the data
- **C** Means are useless
- **D** Outliers never matter

Answer: B. Visualise before you summarise.

EXERCISE

Practice (hr_employees.csv)

(1) Mean, median and sd of `monthly_income` by department; which department is most variable (CV)? (2) The 90th percentile of income. (3) The share of employees beyond ±3 sd of income. (4) Pearson and Spearman correlation between age and income. (5) Classify each column by type and scale.

Show solution (worked answer)

Python

```
g = hr.groupby("department")["monthly_income"].agg(["mean", "median", "std"]); g["cv"] =
g["std"] / g["mean"]; print(g.sort_values("cv"))
print(hr.monthly_income.quantile(.9))
zi = (hr.monthly_income - hr.monthly_income.mean()) / hr.monthly_income.std();
print((zi.abs() > 3).mean())
print(hr[["age", "monthly_income"]].corr().iloc[0, 1], hr[["age",
"monthly_income"]].corr("spearman").iloc[0, 1])
# gender, department, city: nominal; education, job_satisfaction, performance_rating:
ordinal; age, income: ratio
```

INTERVIEW CORNER

Questions you may be asked

Q: Mean vs median: when do you report each? Median for skewed data or outliers (income, house prices, delivery times); mean for symmetric data or when totals matter.

Q: What does a standard deviation of 0 mean? All values are identical.

Q: Correlation is 0. Are the variables independent? Not necessarily: Pearson only captures linear relationships; a perfect U-shape can have r = 0.

STATISTICS & MACHINE LEARNING · LESSON 2

Probability and Distributions: Rules, Bayes, Binomial, Poisson and Normal

The language of uncertainty. Probability rules, conditional probability and Bayes' theorem (with a fraud-alert example), random variables and expected value, and the distributions analysts use most (Bernoulli, binomial, Poisson, uniform, normal, exponential), with scipy.stats and simulation.

Python class 14 · SciPy 1.1x · NumPy 2 · Beginner · 110 min

By the end of this lesson you can

- Apply the addition, multiplication and complement rules and conditional probability
- Use Bayes' theorem to update probabilities and explain base-rate effects
- Compute expected value and variance of a random variable
- Model counts and measurements with binomial, Poisson, normal and exponential distributions using scipy.stats
- Use the empirical (68-95-99.7) rule and check normality with a Q-Q plot

Probability basics

Probability is a number from 0 (impossible) to 1 (certain). For equally likely outcomes, P(event) = favourable outcomes ÷ all outcomes. In data, it is a long-run frequency.

Python

```
import numpy as np, pandas as pd
from scipy import stats
rng = np.random.default_rng(7)
rolls = rng.integers(1, 7, 100_000)
print("P(six) theory 0.1667 | simulated", round((rolls == 6).mean(), 4))
print("P(even or > 4) theory", round(4 / 6, 4), "| simulated", round(((rolls % 2 == 0) | (rolls
> 4)).mean(), 4))
```

OUTPUT

```
P(six) theory 0.1667 | simulated 0.1682
P(even or > 4) theory 0.6667 | simulated 0.6679
```

Rule	Formula	Example
Complement	P(not A) = 1 − P(A)	P(no conversion) = 1 − 0.10
Addition	P(A or B) = P(A) + P(B) − P(A and B)	Even or > 4: 3/6 + 2/6 − 1/6
Multiplication (independent)	P(A and B) = P(A) × P(B)	Two sixes: 1/36
Conditional	P(A | B) = P(A and B) ÷ P(B)	P(churn | month-to-month)

Python

```
tc = pd.DataFrame({"contract": ["M2M"] * 1690 + ["1yr"] * 739 + ["2yr"] * 571,
                   "churn": [1] * 607 + [0] * 1083 + [1] * 125 + [0] * 614 + [1] * 40 + [0] *
531})   # counts from the churn EDA
joint = pd.crosstab(tc.contract, tc.churn, normalize="all").round(3)
print(joint)
print("P(churn) =", round(tc.churn.mean(), 3), "| P(churn | M2M) =", round(tc[tc.contract ==
"M2M"].churn.mean(), 3), "| P(M2M | churn) =", round((tc[tc.churn == 1].contract ==
"M2M").mean(), 3))
```

OUTPUT

```
churn          0      1
contract
1yr        0.205  0.042
2yr        0.177  0.013
M2M        0.361  0.202
P(churn) = 0.257 | P(churn | M2M) = 0.359 | P(M2M | churn) = 0.786
```

P(churn | M2M) and P(M2M | churn) are **different** questions. Mixing them up is one of the most common reasoning errors.

Bayes' theorem

$$P(A \mid B) = \frac{P(B \mid A)\,P(A)}{P(B)}$$

A fraud model flags 99% of fraudulent payments and wrongly flags 2% of genuine ones. Only 0.5% of payments are fraud. If a payment is flagged, how likely is it to be fraud?

Python

```
p_fraud, sens, fpr = 0.005, 0.99, 0.02
p_flag = sens * p_fraud + fpr * (1 - p_fraud)                # law of total probability
p_fraud_given_flag = sens * p_fraud / p_flag
print(f"P(flag) = {p_flag:.4f} | P(fraud | flag) = {p_fraud_given_flag:.1%}")
n = 100_000; fraud = rng.random(n) < p_fraud
flag = np.where(fraud, rng.random(n) < sens, rng.random(n) < fpr)
print(f"simulated: {fraud[flag].mean():.1%} of flagged payments are fraud ({flag.sum():,} flags,
{(fraud & flag).sum()} real fraud)")
```

OUTPUT

```
P(flag) = 0.0249 | P(fraud | flag) = 19.9%
simulated: 19.9% of flagged payments are fraud (2,485 flags, 495 real fraud)
```

Only about one flag in five is real fraud, because genuine payments vastly outnumber fraud (the **base rate**). This is why precision matters in imbalanced ML problems (lesson 21) and why naive Bayes classifiers work (lesson 20).

Random variables and expected value

A **random variable** assigns a number to each outcome. Its **expected value** is the probability-weighted average: the long-run mean.

Python

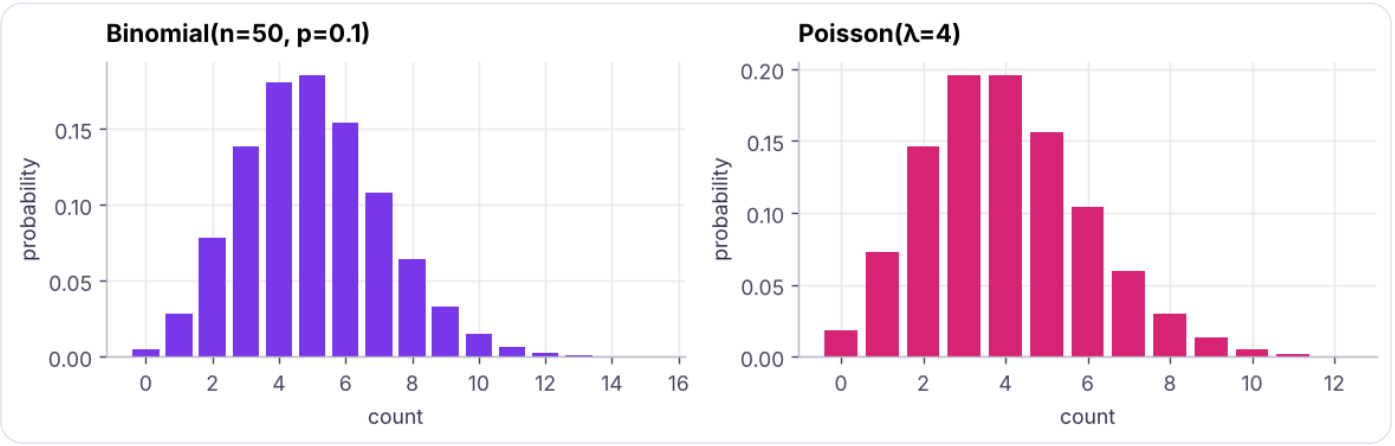
```
values = np.array([0, 500, 2000]); probs = np.array([0.80, 0.15, 0.05])      # customer spend
next month
ev = (values * probs).sum(); var = ((values - ev) ** 2 * probs).sum()
print(f"E[spend] = ₹{ev:.0f} | sd = ₹{var ** 0.5:.0f}")
cost, uplift = 60, 0.25                                                      # campaign: costs
₹60, raises spend by 25%
print(f"expected profit per customer from the campaign: ₹{ev * uplift - cost:.0f} → {'run it' if
ev * uplift > cost else 'do not run it'}")
```

OUTPUT

```
E[spend] = ₹175 | sd = ₹455
expected profit per customer from the campaign: ₹-16 → do not run it
```

Expected value drives decisions: pricing, insurance, campaign ROI, and the "expected loss" in credit risk (PD × LGD × EAD).

Discrete distributions



Binomial (number of conversions out of 50 visitors, p = 0.1) and Poisson (support tickets per hour, mean 4).

Distribution	Models	Parameters	Mean	Variance
Bernoulli	One yes/no trial (did this user convert?)	p	p	p(1 − p)
Binomial	Number of successes in n trials	n, p	np	np(1 − p)
Poisson	Count of events in an interval (calls/hour, defects/batch)	λ	λ	λ
Geometric	Trials until first success	p	1/p	(1 − p)/p²

Python

```
b = stats.binom(n=50, p=0.10)
print(f"P(exactly 5) = {b.pmf(5):.3f} | P(at most 2) = {b.cdf(2):.3f} | P(10 or more) =
{b.sf(9):.4f} | mean {b.mean()} sd {b.std():.2f}")
po = stats.poisson(mu=4)
print(f"tickets/hour: P(0) = {po.pmf(0):.3f} | P(more than 8) = {po.sf(8):.4f} | 95th percentile
= {po.ppf(0.95):.0f}")
ab = pd.read_csv("ab_test.csv"); ctrl = ab[ab.group == "A_control"]
print(f"A/B control: {ctrl.converted.sum()} conversions from {len(ctrl)} users, p̂ =
{ctrl.converted.mean():.3f}")
```

OUTPUT

```
P(exactly 5) = 0.185 | P(at most 2) = 0.112 | P(10 or more) = 0.0245 | mean 5.0 sd 2.12
tickets/hour: P(0) = 0.018 | P(more than 8) = 0.0214 | 95th percentile = 8
A/B control: 198 conversions from 2000 users, p̂ = 0.099
```

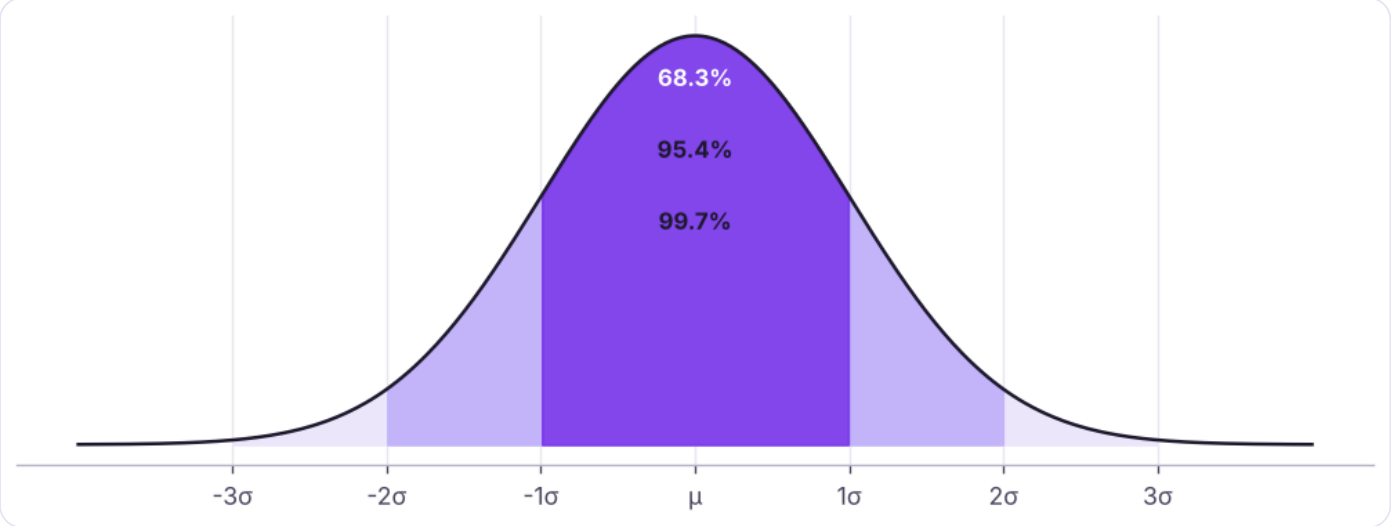
scipy.stats methods: `pmf` / `pdf` (probability or density at a point), `cdf` (P(X ≤ x)), `sf` (P(X > x) = 1 − cdf), `ppf` (inverse cdf: percentiles), `rvs` (random samples).

Continuous distributions

For continuous variables, probability is the **area under the density curve**; P(X = exact value) is 0.

The normal distribution

Bell-shaped, symmetric, described by mean μ and standard deviation σ. Heights, measurement errors and, crucially, **sample means** (next lesson) are approximately normal.



The empirical rule: about 68%, 95% and 99.7% of a normal distribution lie within 1, 2 and 3 standard deviations of the mean.

Python

```
d = pd.read_csv("delivery_times.csv"); pune = d[d.city == "Pune"].delivery_minutes
mu, sd = pune.mean(), pune.std()
nd = stats.norm(mu, sd)
print(f"Pune delivery: mean {mu:.1f} min, sd {sd:.1f}")
print(f"P(late, > 40 min) model {nd.sf(40):.1%} vs data {(pune > 40).mean():.1%}")
print(f"promise time covering 95% of orders: {nd.ppf(0.95):.1f} min | within ±1 sd: {((pune -
mu).abs() ⩽ sd).mean():.0%} of orders")
print("68-95-99.7 check:", [round(float(stats.norm.cdf(k) - stats.norm.cdf(-k)), 4) for k in (1,
2, 3)], "| z for 95% two-sided:", round(stats.norm.ppf(0.975), 3))
```

OUTPUT

```
Pune delivery: mean 31.7 min, sd 6.4
P(late, > 40 min) model 9.7% vs data 13.3%
promise time covering 95% of orders: 42.2 min | within ±1 sd: 68% of orders
68-95-99.7 check: [0.6827, 0.9545, 0.9973] | z for 95% two-sided: 1.96
```

The **standard normal** has μ = 0, σ = 1; any normal value converts with z = (x − μ)/σ. The number **1.96** (95% two-sided) appears in confidence intervals and tests.

Other continuous distributions

Distribution	Models	Example
Uniform(a, b)	Every value equally likely	Random number generators, A/B bucket assignment
Exponential(λ)	Waiting time between Poisson events	Time between support calls
Log-normal	Positive, right-skewed values	Income, house prices, order values
t, χ^2, F	Sampling distributions for tests	t-tests, chi-square tests, ANOVA

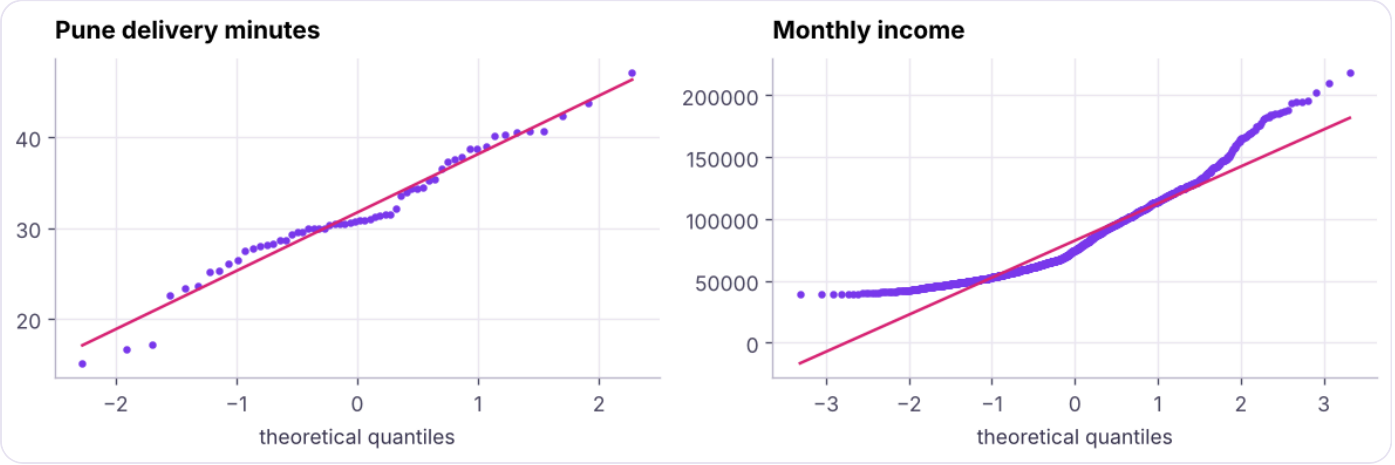
Python

```
wait = stats.expon(scale=1 / 4)                    # 4 calls per hour → mean wait 15 minutes
print(f"P(no call for 30+ minutes) = {wait.sf(0.5):.3f} | memoryless: P(wait > 45 | already 15)
= {wait.sf(0.75) / wait.sf(0.25):.3f}")
```

OUTPUT

```
P(no call for 30+ minutes) = 0.135 | memoryless: P(wait > 45 | already 15) = 0.135
```

Is it normal? Q-Q plots and tests



Q-Q plots compare data quantiles with normal quantiles: points on the line mean "approximately normal". Pune delivery times look normal; HR income is right-skewed.

Python

```
for name, data in [("delivery (Pune)", pune), ("income", hr.monthly_income), ("log income",
np.log(hr.monthly_income))]:
    print(f"{name:16s} Shapiro-Wilk p = {stats.shapiro(data.sample(min(len(data), 500),
random_state=0)).pvalue:.4f} | skew {stats.skew(data):.2f}")
```

OUTPUT

```
delivery (Pune)  Shapiro-Wilk p = 0.1629 | skew -0.15
income           Shapiro-Wilk p = 0.0000 | skew 1.02
log income       Shapiro-Wilk p = 0.0000 | skew 0.28
```

A small p-value (< 0.05) suggests the data are **not** normal. With large samples, tiny departures become "significant", so rely on plots and skewness too.

Summary

KEY TAKEAWAYS

- Probability rules: complement, addition, multiplication (independence), conditional P(A | B) ≠ P(B | A).
- Bayes' theorem updates beliefs; low base rates make most alerts false positives.
- Expected value = probability-weighted average, the basis of ROI and risk decisions.
- Binomial (successes in n trials), Poisson (counts per interval), normal (measurements, sample means), exponential (waiting times); use `scipy.stats` `pmf/pdf`, `cdf`, `sf`, `ppf`, `rvs`.
- Empirical rule 68-95-99.7; z = (x − μ)/σ; check normality with Q-Q plots and skewness.

Quick quiz

6 questions · tap an answer to check it

1. A fraud test is 99% sensitive with a 2% false-positive rate, and fraud is 0.5% of payments. Roughly what share of flagged payments are fraud?

A 99%

B 50%

C About 20%

D 2%

Answer: C. The low base rate means most flags are genuine payments.

2. Which distribution models the number of support calls per hour?

A Normal

B Poisson

C Uniform

D Binomial with n = 1

Answer: B. Counts of events in a fixed interval.

3. For a binomial with n = 50 and p = 0.1, what is the mean?

A 0.1

B 5

C 50

D 4.5

Answer: B. Mean = np.

4. About what share of a normal distribution lies within ±2 standard deviations?

A 68%

B 95%

C 99.7%

D 50%

Answer: B. The empirical rule.

5. Which scipy method gives P(X > x)?

A pdf

B cdf

C sf

D ppf

Answer: C. sf = survival function = 1 − cdf.

6. P(churn | month-to-month) equals P(month-to-month | churn).

A True

B False

Answer: B. They condition on different groups.

EXERCISE **Practice**

(1) Probability of at least one conversion from 20 visitors if p = 0.08. (2) A call centre averages 6 calls per 10 minutes; probability of more than 10 calls in 10 minutes. (3) Hyderabad delivery times: estimate P(> 45 min) with a normal model and compare with the data. (4) Simulate the Bayes fraud example with fraud at 2% instead of 0.5%. What changes?

Show solution (worked answer)

Python

```
print(1 - 0.92 ** 20, stats.binom.sf(0, 20, 0.08))
print(stats.poisson.sf(10, 6))
hyd = d[d.city == "Hyderabad"].delivery_minutes; print(stats.norm(hyd.mean(),
hyd.std()).sf(45), (hyd > 45).mean())
p_fraud = 0.02; print(sens * p_fraud / (sens * p_fraud + fpr * (1 - p_fraud)))     # about
50%: the base rate matters
```

INTERVIEW CORNER

Questions you may be asked

Q: Explain Bayes' theorem with an example. It updates a prior probability with new evidence. For a rare disease with an accurate test, a positive result can still be more likely a false positive because healthy people vastly outnumber sick ones.

Q: When would you use a Poisson distribution? For counts of independent events in a fixed time or space with a constant average rate, such as orders per minute, defects per batch or calls per hour.

Q: What is the difference between pdf and cdf? The pdf gives density at a point (probability for discrete pmf); the cdf gives the cumulative probability $P(X \le x)$.

STATISTICS & MACHINE LEARNING · LESSON 3

Sampling, the Central Limit Theorem and Confidence Intervals

Learn how a sample tells us about a population. Population vs sample, sampling methods and biases, sampling distributions and standard error, the Central Limit Theorem by simulation, confidence intervals for means and proportions (z, t and bootstrap), and how to choose a sample size.

Python classes 14–15 · ML class 1 | SciPy 1.1x · statsmodels 0.1x | Intermediate | 110 min

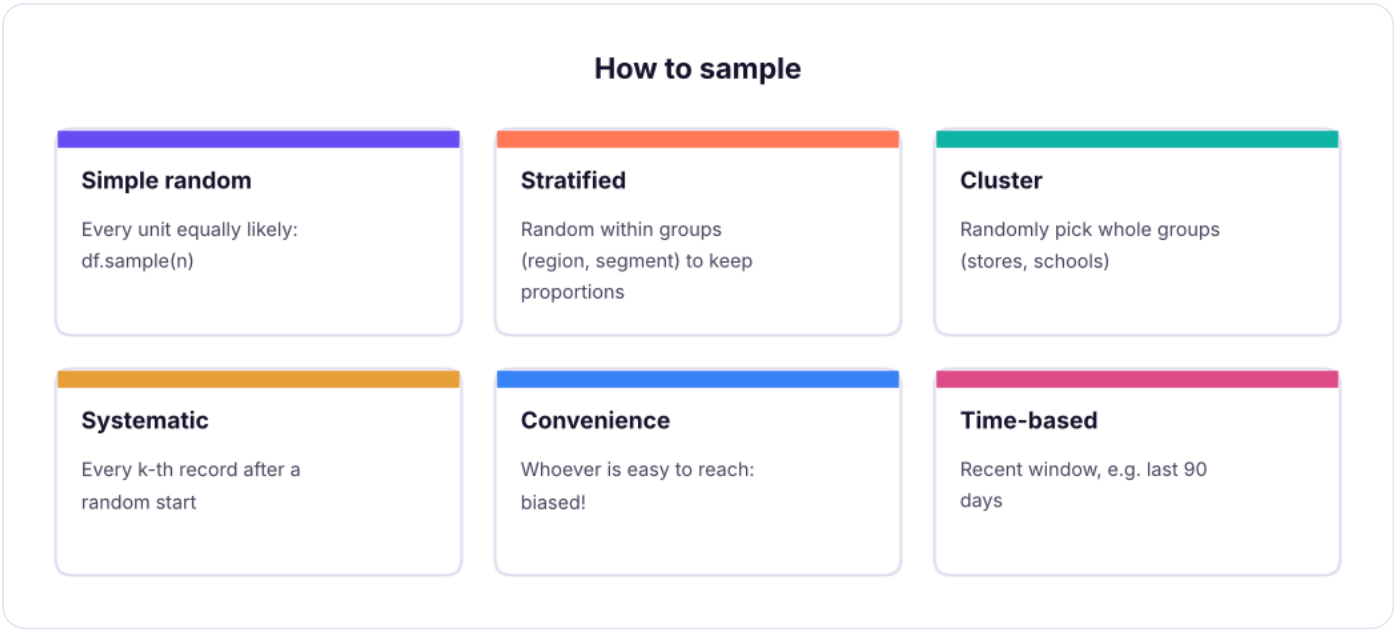
By the end of this lesson you can

- Distinguish population and sample, parameter and statistic, and name common sampling methods and biases
- Explain sampling distributions and standard error
- Demonstrate the Central Limit Theorem by simulation and state when it applies
- Compute and interpret confidence intervals for a mean and a proportion, including the bootstrap
- Estimate the sample size needed for a target margin of error

Population and sample

Term	Population	Sample
What	Everyone/everything of interest	The part you observe
Summary	**Parameter** (μ, σ, p)	**Statistic** ($\bar{x}$, s, $\hat{p}$)
Example	All 1,500 employees	100 surveyed employees

We almost never see the population (all customers, all future orders), so we estimate parameters from samples and state our **uncertainty**.



Sampling methods.

Python

```
import pandas as pd, numpy as np
from scipy import stats
hr = pd.read_csv("hr_employees.csv")
pop_mean, pop_rate = hr.monthly_income.mean(), (hr.attrition == "Yes").mean()
print(f"population: mean income {pop_mean:,.0f}, attrition {pop_rate:.1%}, N = {len(hr)}")
srs = hr.sample(150, random_state=1)
strat = hr.groupby("department", group_keys=False).sample(frac=0.1, random_state=1)     # stratified 10%
print(f"simple random n=150: mean {srs.monthly_income.mean():,.0f}, attrition {(srs.attrition == 'Yes').mean():.1%}")
print(f"stratified n={len(strat)}: mean {strat.monthly_income.mean():,.0f}; department shares kept:",
      (strat.department.value_counts(normalize=True) - hr.department.value_counts(normalize=True)).abs().max().round(3) < 0.01)
```

OUTPUT

```
population: mean income 82,611, attrition 25.9%, N = 1500
simple random n=150: mean 85,587, attrition 21.3%
stratified n=149: mean 81,315; department shares kept: True
```

WATCH OUT **Sampling bias beats sample size**

A huge biased sample is worse than a small random one. Watch for **selection bias** (only app users surveyed), **non-response bias** (unhappy customers ignore surveys), **survivorship bias** (studying only companies that still exist) and **time bias** (a festive week is not a normal week).

Sampling distributions and standard error

Take many samples and compute the mean of each: those means form the **sampling distribution**. Its standard deviation is the **standard error (SE)**.

$$SE(\bar{x}) = \frac{\sigma}{\sqrt{n}} \qquad SE(\hat{p}) = \sqrt{\frac{p(1-p)}{n}}$$

Python

```
rng = np.random.default_rng(0)
inc = hr.monthly_income.to_numpy()
for n in (10, 40, 160, 640):
    means = [rng.choice(inc, n).mean() for _ in range(3000)]
    print(f"n={n:4d}: sd of sample means {np.std(means):7,.0f} | formula σ/√n {inc.std() /
np.sqrt(n):7,.0f}")
```

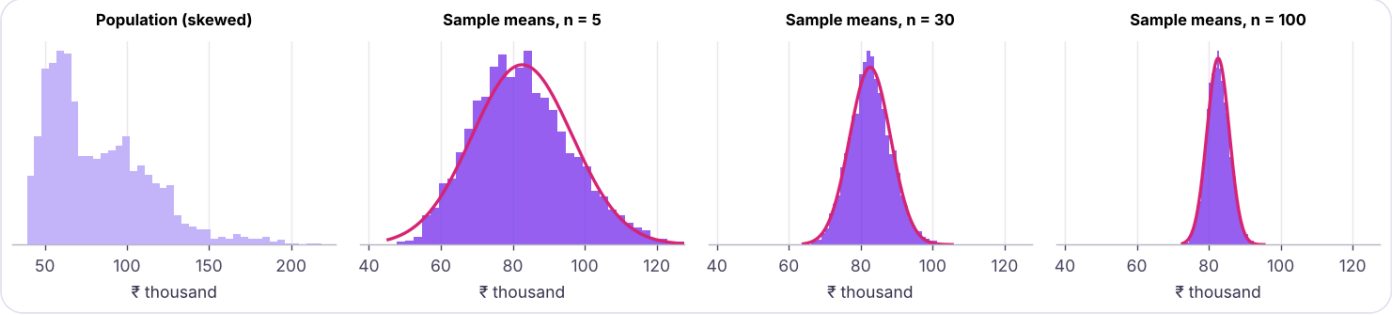
OUTPUT

```
n=  10: sd of sample means   9,854 | formula σ/√n   9,846
n=  40: sd of sample means   4,970 | formula σ/√n   4,923
n= 160: sd of sample means   2,492 | formula σ/√n   2,461
n= 640: sd of sample means   1,232 | formula σ/√n   1,231
```

Quadrupling the sample size halves the standard error: precision grows with √n, so doubling precision costs four times the data.

The Central Limit Theorem

CLT: for a large enough sample (rule of thumb n ≥ 30, more for very skewed data), the sampling distribution of the mean is approximately **normal**, whatever the shape of the population, with mean μ and SE σ/√n.



Income is right-skewed (left), yet the means of samples of 5, 30 and 100 employees become ever more normal and narrow.

The CLT is why z- and t-tests, confidence intervals and A/B tests work on non-normal business data such as revenue.

Confidence intervals

A **95% confidence interval** is a range built by a method that captures the true parameter in 95% of repeated samples.

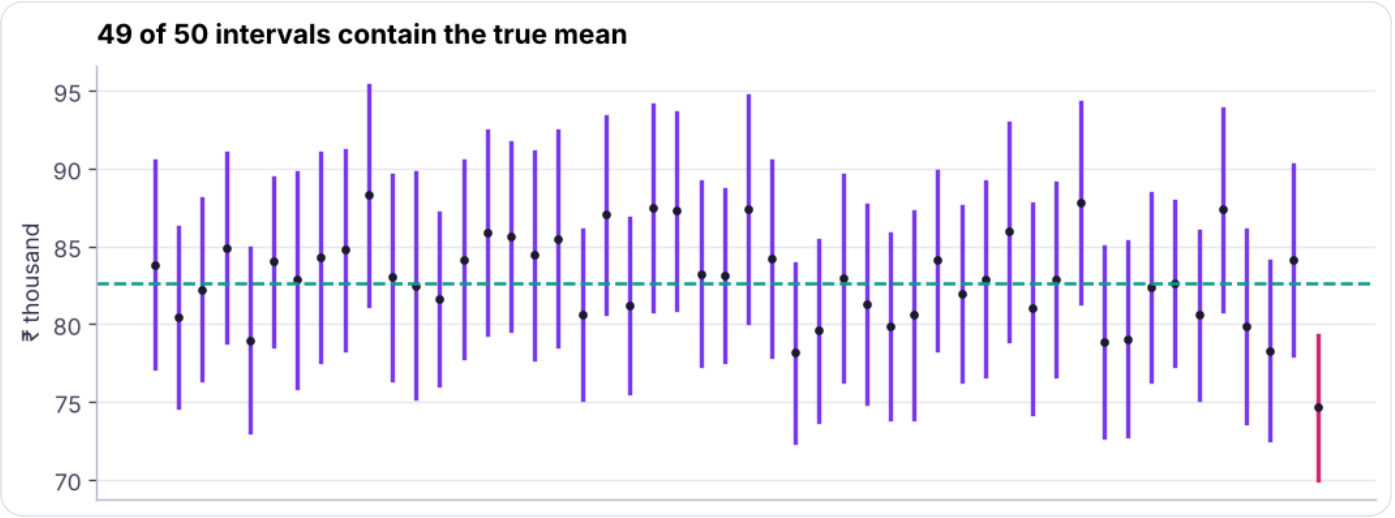
$$\bar{x} \pm t_{\alpha/2,\, n-1} \frac{s}{\sqrt{n}} \qquad \hat{p} \pm z_{\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

Python

```
sample = hr.sample(100, random_state=3)
x = sample.monthly_income
se = x.std() / np.sqrt(len(x)); tcrit = stats.t.ppf(0.975, df=len(x) - 1)
lo, hi = x.mean() - tcrit * se, x.mean() + tcrit * se
print(f"mean {x.mean():,.0f} | 95% CI {lo:,.0f} to {hi:,.0f} | t* = {tcrit:.3f} | true mean
{pop_mean:,.0f} inside? {lo ⩽ pop_mean ⩽ hi}")
print("scipy:", [round(v) for v in stats.t.interval(0.95, len(x) - 1, loc=x.mean(), scale=se)])
k = (sample.attrition == "Yes").sum(); n = len(sample); ph = k / n
z = stats.norm.ppf(0.975)
print(f"attrition p̂ = {ph:.2f}, 95% CI (Wald) {ph - z * np.sqrt(ph * (1 - ph) / n):.3f} to {ph +
z * np.sqrt(ph * (1 - ph) / n):.3f}")
from statsmodels.stats.proportion import proportion_confint
print("Wilson CI (better for small n or extreme p):", [round(v, 3) for v in
proportion_confint(k, n, method="wilson")])
```

OUTPUT

```
mean 82,653 | 95% CI 77,094 to 88,213 | t* = 1.984 | true mean 82,611 inside? True
scipy: [77094, 88213]
attrition p̂ = 0.32, 95% CI (Wald) 0.229 to 0.411
Wilson CI (better for small n or extreme p): [0.237, 0.417]
```



50 samples of 100 employees, each with its 95% CI. Most intervals (purple) contain the true mean; about 1 in 20 (pink) misses.

> **NOTE** **What a 95% CI does not mean**
>
> It does **not** mean "there is a 95% probability that the true value is in this particular interval": the true value is fixed. It means the *procedure* captures the true value 95% of the time. In practice, say "we are 95% confident the average income is between … and …".

Wider interval when…	Narrower when…
Higher confidence (99% vs 95%)	Larger sample size
More variable data	Less variable data
Smaller sample	Lower confidence (90%)

Bootstrap confidence intervals

When there is no simple formula (median, ratios, percentiles), **resample the sample with replacement** many times and take percentiles of the statistic.

Python

```
boot = np.array([rng.choice(x.to_numpy(), len(x), replace=True) for _ in range(5000)])
med = np.median(boot, axis=1)
print(f"sample median {x.median():,.0f} | 95% bootstrap CI {np.percentile(med, 2.5):,.0f} to
{np.percentile(med, 97.5):,.0f}")
res = stats.bootstrap((x.to_numpy(),), np.median, confidence_level=0.95, n_resamples=5000,
random_state=0, method="percentile")
print("scipy.stats.bootstrap:", [round(v) for v in (res.confidence_interval.low,
res.confidence_interval.high)])
```

OUTPUT

```
sample median 82,666 | 95% bootstrap CI 68,750 to 90,542
scipy.stats.bootstrap: [68746, 89750]
```

Sample size planning

Required n for a margin of error E (half-width of the CI):

$$n = \left(\frac{z\sigma}{E}\right)^2 \qquad n = \frac{z^2\, p(1-p)}{E^2}$$

Python

```python
def n_for_mean(sigma, E, conf=0.95): return int(np.ceil((stats.norm.ppf(1 - (1 - conf) / 2) *
sigma / E) ** 2))
def n_for_prop(E, p=0.5, conf=0.95): return int(np.ceil(stats.norm.ppf(1 - (1 - conf) / 2) ** 2
* p * (1 - p) / E ** 2))
print("income within ±₹5,000:", n_for_mean(inc.std(), 5000), "employees | ±₹2,000:",
n_for_mean(inc.std(), 2000))
print("survey proportion within ±5 points:", n_for_prop(0.05), "| ±3 points:", n_for_prop(0.03),
"| ±1 point:", n_for_prop(0.01))
```

OUTPUT

```
income within ±₹5,000: 149 employees | ±₹2,000: 931
survey proportion within ±5 points: 385 | ±3 points: 1068 | ±1 point: 9604
```

This is why opinion polls survey about 1,000 people (±3 points) and why A/B tests on small effects need tens of thousands of users (lesson 6).

Summary

KEY TAKEAWAYS

- Parameters describe populations; statistics estimate them from samples. Random (and stratified) sampling avoids bias; size cannot fix bias.
- The standard error σ/√n measures how much a statistic varies from sample to sample.
- The CLT makes sample means approximately normal for large n, whatever the population shape.
- CI for a mean: $\bar{x} \pm t \cdot s/\sqrt{n}$; *for a proportion:* $\hat{p} \pm z \cdot \sqrt{\hat{p}(1-\hat{p})/n}$ (Wilson for small samples); bootstrap for anything else.
- Plan sample size from the margin of error: halving the error needs four times the data.

Quick quiz

6 questions · tap an answer to check it

1. What happens to the standard error when the sample size quadruples?

- **A** It quadruples
- **B** It doubles
- **C** It halves
- **D** It stays the same

Answer: C. SE ∝ 1/√n.

2. What does the Central Limit Theorem say?

- **A** All data are normal
- **B** Sample means are approximately normal for large samples, whatever the population shape
- **C** Samples must be normal
- **D** Means equal medians

Answer: B. It underpins most tests on business data.

3. A 95% CI for mean spend is ₹820–₹910. Best interpretation?

- **A** We are 95% confident the true mean spend lies between ₹820 and ₹910 (the method captures the truth 95% of the time)
- **B** 95% of customers spend between ₹820 and ₹910
- **C** The mean is 865 with certainty
- **D** There is a 5% chance the sample is wrong

Answer: A. A CI is about the parameter, not individual values.

4. Which makes a confidence interval narrower?

A Higher confidence level

B Larger sample

C More variable data

D Smaller sample

Answer: B. More data, more precision.

5. How do you get a CI for a median without a formula?

A Use the z-table

B Bootstrap: resample with replacement and take percentiles

C Use the mean instead

D It is impossible

Answer: B. Works for almost any statistic.

6. Which bias occurs when only active app users answer a satisfaction survey?

A Selection bias

B Rounding bias

C Recall bias only

D No bias

Answer: A. The sample does not represent all customers.

EXERCISE **Practice**

(1) From `ab_test.csv`, compute a 95% CI for the conversion rate of each group. Do they overlap? (2) Bootstrap a 95% CI for the difference in mean order value between groups. (3) How many users per group would you need to estimate a conversion rate within ±1 point?

Show solution (worked answer)

Python

```
ab = pd.read_csv("ab_test.csv")
for g, d in ab.groupby("group"): print(g, [round(v, 4) for v in
proportion_confint(d.converted.sum(), len(d), method="wilson")])
a, b = ab[ab.group == "A_control"].order_value.to_numpy(), ab[ab.group ==
"B_new_checkout"].order_value.to_numpy()
diffs = [rng.choice(b, len(b)).mean() - rng.choice(a, len(a)).mean() for _ in range(3000)]
print(np.percentile(diffs, [2.5, 97.5]))
print(n_for_prop(0.01, p=0.12))
```

INTERVIEW CORNER **Questions you may be asked**

Q: Explain the Central Limit Theorem to a non-technical manager. If we average enough randomly chosen customers, those averages behave predictably (a bell curve) even when individual spending is wildly uneven, so we can say how confident we are in an average.

Q: What is the difference between standard deviation and standard error? Standard deviation measures spread of individual values; standard error measures spread of a statistic (like the mean) across samples: SE = sd/√n.

Q: How would you choose a sample size for a survey? Decide the margin of error and confidence level, estimate variability (use p = 0.5 for proportions if unknown), compute n, then inflate for expected non-response.

STATISTICS & MACHINE LEARNING · LESSON 4

Hypothesis Testing I: The Framework, p-values, z-tests and t-tests

Turn business questions into testable hypotheses. Null and alternative hypotheses, significance level, test statistics, p-values and critical values, one- vs two-tailed tests, Type I and II errors and power, then one-sample, two-sample (Welch) and paired t-tests and z-tests for proportions, on a delivery-time case study.

ML classes 1–2 · SciPy 1.1x · statsmodels 0.1x · Intermediate · 120 min

By the end of this lesson you can

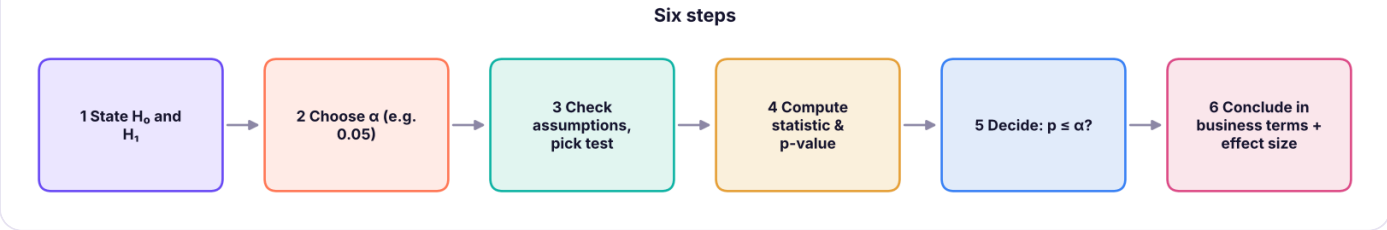
- Translate a business question into null and alternative hypotheses
- Explain significance level, test statistic, p-value and critical value, and decide with both
- Choose one- or two-tailed tests and describe Type I/II errors and power
- Run and interpret one-sample, independent (Welch) and paired t-tests and a z-test for proportions
- Report effect sizes and confidence intervals, not just p-values

From business question to hypothesis

> **Case.** A quick-commerce company promises "delivery in 30 minutes". Operations claim Pune meets the promise on average. Is the average delivery time in Pune really 30 minutes, or is it slower? Is Hyderabad different from Pune?

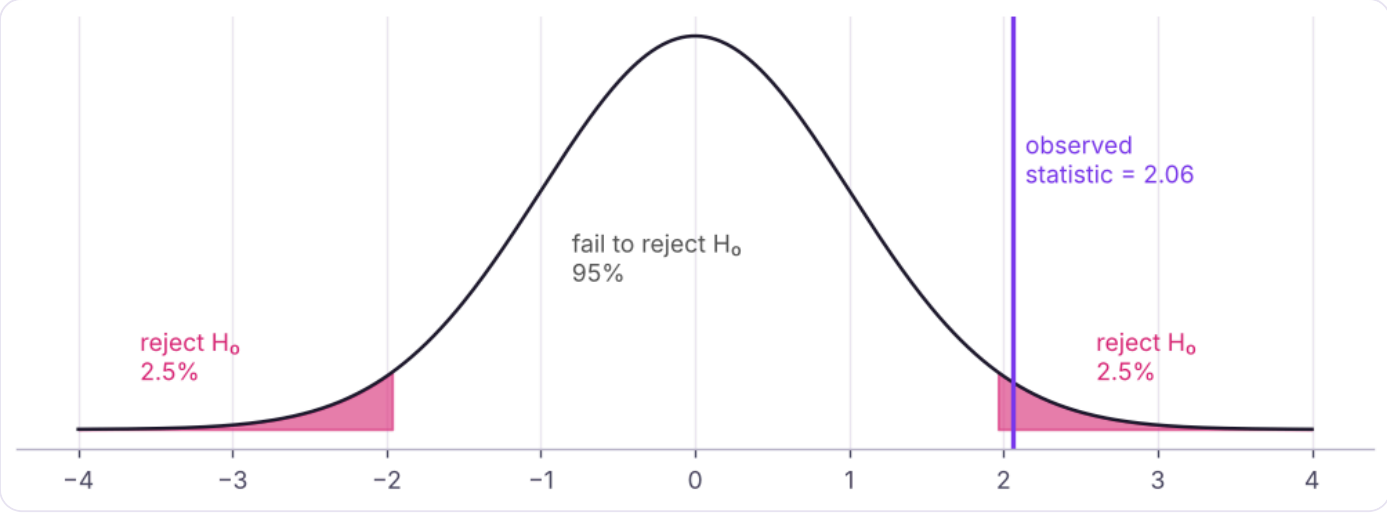
A **hypothesis test** checks whether data are consistent with a default claim.

Term	Meaning	Delivery case
H_0 (null)	No effect / status quo; assumed true until evidence	μ_Pune = 30 min
H_1 (alternative)	What we want evidence for	μ_Pune > 30 min (one-tailed) or ≠ 30 (two-tailed)
α (significance level)	Accepted risk of a false alarm	0.05
Test statistic	Distance from H_0 in standard-error units	$t = (\bar{x} - 30)/(s/\sqrt{n})$
p-value	P(data at least this extreme | H_0 true)	from the t distribution
Decision	$p \le \alpha \rightarrow$ reject H_0; $p > \alpha \rightarrow$ fail to reject H_0	



The hypothesis-testing procedure.

p-values and critical values



Two-tailed test at α = 0.05: the rejection regions are the 2.5% tails beyond ±1.96 (z). The p-value is the tail area beyond the observed statistic.

Python

```
print("critical values: two-tailed z ±", round(stats.norm.ppf(0.975), 3), "| one-tailed z",
round(stats.norm.ppf(0.95), 3), "| two-tailed t(df=59) ±", round(stats.t.ppf(0.975, 59), 3))
for z in (1.2, 1.7, 2.06, 3.1): print(f"z = {z}: two-tailed p = {2 * stats.norm.sf(abs(z)):.4f}
| one-tailed p = {stats.norm.sf(z):.4f}")
```

OUTPUT

```
critical values: two-tailed z ± 1.96 | one-tailed z 1.645 | two-tailed t(df=59) ± 2.001
z = 1.2: two-tailed p = 0.2301 | one-tailed p = 0.1151
z = 1.7: two-tailed p = 0.0891 | one-tailed p = 0.0446
z = 2.06: two-tailed p = 0.0394 | one-tailed p = 0.0197
z = 3.1: two-tailed p = 0.0019 | one-tailed p = 0.0010
```

WATCH OUT **What a p-value is not**

It is **not** the probability that H_0 is true, and 1 − p is not the probability that H_1 is true. A large p-value does not prove H_0 ("absence of evidence is not evidence of absence"). A tiny p-value does not mean a large or important effect.

One- vs two-tailed

	Two-tailed	One-tailed
H_1	μ ≠ 30 (any difference)	μ > 30 (only "slower")
α split	2.5% in each tail	5% in one tail
When	Default; direction unknown	Direction fixed **before** looking at data, and the other direction is irrelevant

Errors and power

	H_0 true	H_0 false
Reject H_0	**Type I error** (false positive), probability α	Correct: **power** = 1 − β
Fail to reject	Correct	**Type II error** (false negative), probability β

Fire alarm analogy: a Type I error rings without a fire; a Type II error stays silent during a fire. Lower α means fewer false alarms but more misses unless you collect more data. Power (commonly 80%) grows with sample size, effect size and α.

One-sample t-test: does Pune meet 30 minutes?

Python

```
import pandas as pd
d = pd.read_csv("delivery_times.csv")
pune, hyd = d[d.city == "Pune"].delivery_minutes, d[d.city == "Hyderabad"].delivery_minutes
print(f"Pune: n={len(pune)}, mean {pune.mean():.2f}, sd {pune.std():.2f}")
t_two = stats.ttest_1samp(pune, 30)
t_one = stats.ttest_1samp(pune, 30, alternative="greater")
se = pune.std() / np.sqrt(len(pune))
print(f"t = {t_two.statistic:.3f}, df = {len(pune) - 1} | two-tailed p = {t_two.pvalue:.4f} |
one-tailed (greater) p = {t_one.pvalue:.4f}")
print(f"by hand: t = ({pune.mean():.2f} - 30) / {se:.3f} = {(pune.mean() - 30) / se:.3f}")
ci = stats.t.interval(0.95, len(pune) - 1, loc=pune.mean(), scale=se)
print(f"95% CI for mean: {ci[0]:.2f} to {ci[1]:.2f} minutes (30 is {'outside' if not ci[0] ⩽ 30
⩽ ci[1] else 'inside'})")
print(f"effect size (Cohen's d) = {(pune.mean() - 30) / pune.std():.2f}")
```

OUTPUT

```
Pune: n=60, mean 31.70, sd 6.39
t = 2.060, df = 59 | two-tailed p = 0.0439 | one-tailed (greater) p = 0.0219
by hand: t = (31.70 - 30) / 0.825 = 2.060
95% CI for mean: 30.05 to 33.35 minutes (30 is outside)
effect size (Cohen's d) = 0.27
```

Conclusion. At $\alpha = 0.05$ we reject H_0: Pune's average delivery time (about 31.7 minutes) is statistically higher than the 30-minute promise ($p \approx 0.02$ one-tailed). The difference is small (under 2 minutes, $d \approx 0.27$), so the business question is whether 1–2 minutes matter to customers, not just whether $p < 0.05$.

Two-sample t-test: Pune vs Hyderabad

Python

```
print(f"Hyderabad: n={len(hyd)}, mean {hyd.mean():.2f}, sd {hyd.std():.2f}")
lev = stats.levene(pune, hyd); print(f"Levene test for equal variances p = {lev.pvalue:.3f} →
{'unequal' if lev.pvalue < 0.05 else 'equal'} variances")
w = stats.ttest_ind(hyd, pune, equal_var=False)             # Welch's t-test: safe default
print(f"Welch t = {w.statistic:.3f}, p = {w.pvalue:.4f}, difference = {hyd.mean() -
pune.mean():.2f} min")
pooled = np.sqrt(((len(pune) - 1) * pune.var() + (len(hyd) - 1) * hyd.var()) / (len(pune) +
len(hyd) - 2))
print(f"Cohen's d = {(hyd.mean() - pune.mean()) / pooled:.2f}")
```

OUTPUT

```
Hyderabad: n=55, mean 33.81, sd 7.60
Levene test for equal variances p = 0.047 → unequal variances
```

```
Welch t = 1.601, p = 0.1123, difference = 2.11 min
Cohen's d = 0.30
```

Conclusion. p ≈ 0.11 > 0.05: we **fail to reject** H_0. The data do not show a significant difference between the cities (Hyderabad is about 2 minutes slower in this sample, but that could be chance). Use **Welch's test** (`equal_var=False`) by default: it does not assume equal variances and loses almost nothing when they are equal.

Paired t-test: before and after

Use a **paired** test when the same units are measured twice (same riders before/after training, same stores before/after a layout change).

Python

```python
rng = np.random.default_rng(5)
before = rng.normal(34, 5, 30).round(1)
after = (before - rng.normal(1.5, 1.5, 30)).round(1)          # simulated route-training effect
pt = stats.ttest_rel(before, after)
it = stats.ttest_ind(before, after, equal_var=False)
print(f"mean improvement {np.mean(before - after):.2f} min | paired p = {pt.pvalue:.4f} |
(wrong) independent p = {it.pvalue:.4f}")
```

OUTPUT

```
mean improvement 1.17 min | paired p = 0.0003 | (wrong) independent p = 0.3342
```

Pairing removes rider-to-rider variation, so the same improvement is detected far more clearly.

z-test for proportions

Python

```python
from statsmodels.stats.proportion import proportions_ztest
hr = pd.read_csv("hr_employees.csv")
ot = hr[hr.overtime == "Yes"].attrition.eq("Yes"); no = hr[hr.overtime ==
"No"].attrition.eq("Yes")
stat, p = proportions_ztest([ot.sum(), no.sum()], [len(ot), len(no)])
print(f"attrition: overtime {ot.mean():.1%} (n={len(ot)}) vs no overtime {no.mean():.1%} (n=
{len(no)}) | z = {stat:.2f}, p = {p:.2e}")
stat1, p1 = proportions_ztest(hr.attrition.eq("Yes").sum(), len(hr), value=0.20,
alternative="larger")
print(f"is company attrition above the 20% industry benchmark? z = {stat1:.2f}, p = {p1:.2e}")
```

OUTPUT

```
attrition: overtime 36.7% (n=433) vs no overtime 21.6% (n=1067) | z = 6.07, p = 1.26e-09
is company attrition above the 20% industry benchmark? z = 5.24, p = 7.89e-08
```

Choosing a test (means)

Situation	Test	Python
One mean vs a target, σ unknown	One-sample t	`stats.ttest_1samp`
Two independent groups	Welch's t	`stats.ttest_ind(..., equal_var=False)`
Same units twice	Paired t	`stats.ttest_rel`
One or two proportions	z-test	`statsmodels proportions_ztest`
Non-normal small samples	Mann-Whitney U / Wilcoxon (next lesson)	`stats.mannwhitneyu`, `stats.wilcoxon`
3+ groups	ANOVA (next lesson)	`stats.f_oneway`

Assumptions for t-tests: independent observations, roughly normal data **or** large samples (CLT), and for the pooled version equal variances.

Reporting

> Pune deliveries averaged **31.7 minutes** (95% CI 30.1–33.4, n = 60), significantly above the 30-minute promise (one-sample t = 2.06, one-tailed p = 0.022). The gap is small (d = 0.27). **Recommendation:** monitor the share of orders above 40 minutes (13%) rather than the average.

Report the **estimate, CI, test, p-value, effect size and business meaning**.

Summary

KEY TAKEAWAYS

- H_o is the default (no effect); H_1 is what you want evidence for; fix α (usually 0.05) and the tails before looking.
- p-value = probability of data this extreme if H_o were true; reject H_o when $p \le \alpha$ (equivalently when the statistic passes the critical value).
- Type I (false positive, α) vs Type II (false negative, β); power = 1 − β grows with n and effect size.
- One-sample, Welch two-sample and paired t-tests compare means; z-tests compare proportions.
- Always report effect size and a confidence interval: statistical significance is not business significance.

Quick quiz

6 questions · tap an answer to check it

1. The p-value is 0.03 and α = 0.05. What do you conclude?

A Reject H_o: the result is statistically significant

B Accept H_1 with 97% probability

C Fail to reject H_o

D H_o is true

Answer: A. $p \le \alpha$.

2. Which is a Type I error?

A Concluding a new checkout works when it actually has no effect

B Missing a real improvement

C Using the wrong chart

D A small sample

Answer: A. A false positive.

3. Which test compares the same riders before and after training?

A Welch's t-test

B Paired t-test

C Chi-square test

D One-sample z-test

Answer: B. Measurements are paired by rider.

4. Why prefer Welch's t-test for two groups?

A It does not assume equal variances and works well either way

B It is always more significant

C It needs no data

D It is for proportions

Answer: A. `equal_var=False`.

5. p = 0.40. What can you say?

A H_o is proven true

B The data do not provide enough evidence against H_o

C The effect is 40%

D The test failed

Answer: B. Absence of evidence is not evidence of absence.

6. What increases the power of a test?

A Smaller sample

B Larger sample size or larger true effect

C Lower α only

D Using a two-tailed test always

Answer: B. Power = probability of detecting a real effect.

EXERCISE **Practice**

(1) Test whether Hyderabad's mean delivery time exceeds 30 minutes. (2) Test whether employees who left had lower monthly income than those who stayed (one-tailed Welch). Report the difference and d. (3) Test whether the share of overtime workers differs between Sales and IT.

Show solution (worked answer)

Python

```
print(stats.ttest_1samp(hyd, 30, alternative="greater"))
left, stay = hr[hr.attrition == "Yes"].monthly_income, hr[hr.attrition ==
"No"].monthly_income
r = stats.ttest_ind(left, stay, equal_var=False, alternative="less"); print(r, left.mean()
- stay.mean())
print((left.mean() - stay.mean()) / np.sqrt((left.var() + stay.var()) / 2))
s_, i_ = hr[hr.department == "Sales"].overtime.eq("Yes"), hr[hr.department ==
"IT"].overtime.eq("Yes")
print(proportions_ztest([s_.sum(), i_.sum()], [len(s_), len(i_)]))
```

INTERVIEW CORNER **Questions you may be asked**

Q: Explain a p-value to a business stakeholder. If there were truly no difference, how surprising would our data be? A p-value of 0.01 means a result this extreme would happen only 1% of the time by chance alone.

Q: What is the difference between statistical and practical significance? Statistical significance says an effect is unlikely to be zero; practical significance asks whether it is large enough to matter (effect size, cost, revenue).

Q: How do you choose α? By the cost of errors: 0.05 by convention; lower (0.01) when false positives are expensive (medical, financial launches); sometimes higher for early exploration.

STATISTICS & MACHINE LEARNING · LESSON 5

Hypothesis Testing II: Chi-Square, ANOVA, Non-Parametric Tests and Correlation Tests

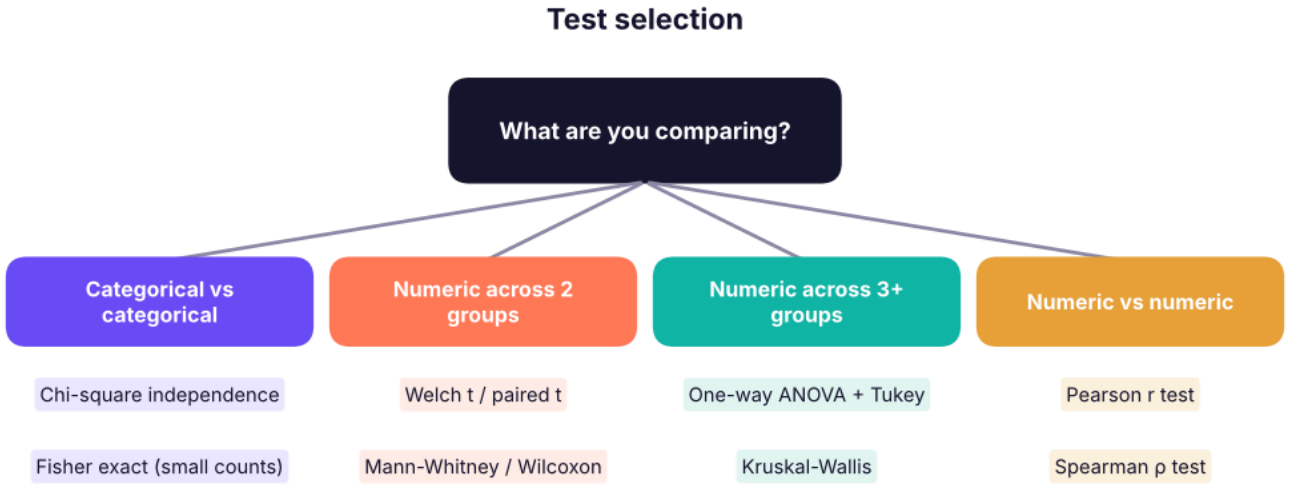
Test relationships between categories with chi-square tests, compare three or more groups with one-way ANOVA and Tukey post-hoc tests, use non-parametric alternatives (Mann-Whitney, Wilcoxon, Kruskal-Wallis) when assumptions fail, test correlations, and control false discoveries when running many tests.

● ML classes 1 and 3 · Python class 15 ✦ SciPy 1.1x · statsmodels 0.1x Intermediate ⏱ 110 min

By the end of this lesson you can

- Run chi-square tests of independence and goodness of fit and read expected counts
- Compare several group means with one-way ANOVA and locate differences with Tukey HSD
- Choose parametric vs non-parametric tests and run Mann-Whitney, Wilcoxon and Kruskal-Wallis
- Test correlations with Pearson and Spearman
- Adjust for multiple comparisons (Bonferroni, Benjamini-Hochberg)

Which test? A decision map



Choosing a test from the type of variables and the number of groups.

Parametric (assumes ~normal)	Non-parametric (ranks; fewer assumptions)
One-sample / paired t	Wilcoxon signed-rank
Independent t (Welch)	Mann-Whitney U
One-way ANOVA	Kruskal-Wallis
Pearson correlation	Spearman correlation

Parametric tests are more powerful when their assumptions hold; non-parametric tests are safer for small, skewed samples, ordinal data and outliers.

Chi-square test of independence

> **Case.** A retailer surveyed 600 customers: is the **preferred shopping channel** (App, Store, Website) related to **age group**?

Python

```
import pandas as pd, numpy as np
from scipy import stats
c = pd.read_csv("channel_survey.csv")
obs = pd.crosstab(c.age_group, c.preferred_channel)
print(obs, "\n")
print((pd.crosstab(c.age_group, c.preferred_channel, normalize="index") * 100).round(1))
res = stats.chi2_contingency(obs)
print(f"\nx² = {res.statistic:.2f}, df = {res.dof}, p = {res.pvalue:.2e}")
print("expected counts if independent:\n", pd.DataFrame(res.expected_freq, index=obs.index,
columns=obs.columns).round(1))
n = obs.values.sum(); cramers_v = np.sqrt(res.statistic / (n * (min(obs.shape) - 1)))
print(f"Cramér's V = {cramers_v:.2f} (effect size: 0.1 small, 0.3 medium, 0.5 large)")
```

OUTPUT

```
preferred_channel  App  Store  Website
age_group
18-25              106     47       65
26-40               72     70       90
41+                 31     56       63

preferred_channel   App  Store  Website
age_group
18-25              48.6   21.6     29.8
26-40              31.0   30.2     38.8
41+                20.7   37.3     42.0

x² = 33.67, df = 4, p = 8.71e-07
```

```
expected counts if independent:
 preferred_channel   App  Store  Website
age_group
18-25               75.9   62.9     79.2
26-40               80.8   66.9     84.3
41+                 52.2   43.2     54.5
Cramér's V = 0.17 (effect size: 0.1 small, 0.3 medium, 0.5 large)
```

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

The test compares **observed** counts with the counts **expected** if age and channel were independent. $p < 0.001$, so channel preference depends on age: 18–25-year-olds favour the app (49%), customers over 41 favour stores and the website. Cramér's V ≈ 0.17 shows a modest association.



Standardised residuals show which cells drive the result: positive = more than expected.

Assumptions: independent observations and **expected** counts of at least 5 in (almost) every cell; otherwise combine categories or use Fisher's exact test (`stats.fisher_exact` for 2 × 2).

Goodness of fit

Does an observed distribution match an expected one?

Python

```
shares_expected = np.array([0.35, 0.30, 0.35])                # last year's channel mix
observed = obs.sum().reindex(["App", "Store", "Website"]).to_numpy()
gof = stats.chisquare(observed, f_exp=shares_expected * observed.sum())
print(observed, (shares_expected * observed.sum()).round(0), f"x² = {gof.statistic:.2f}, p =
{gof.pvalue:.3f}")
```

OUTPUT

```
[209 173 218] [210. 180. 210.] x² = 0.58, p = 0.748
```

One-way ANOVA

> **Case.** 120 volunteers were randomly assigned to Keto, Low-fat or Mediterranean diets for 8 weeks. Do average weight losses differ?

Running three t-tests inflates the false-positive rate ($1 - 0.95^3 \approx 14\%$). **ANOVA** tests all means at once: H_0: $\mu_1 = \mu_2 = \mu_3$.

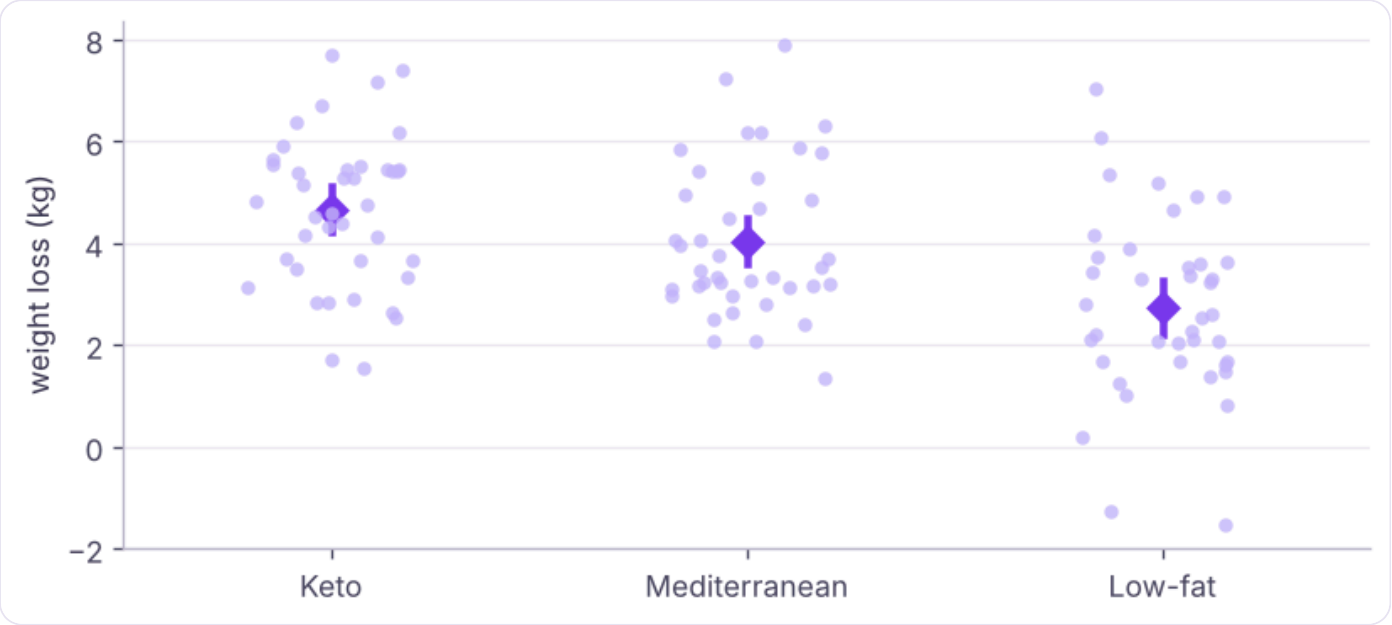
Python

```
d = pd.read_csv("diet_study.csv")
print(d.groupby("diet").weight_loss_kg.agg(["count", "mean", "std"]).round(2))
groups = [g.weight_loss_kg for _, g in d.groupby("diet")]
print("Levene equal variances p =", round(stats.levene(*groups).pvalue, 3), "| Shapiro p per
group:", [round(float(stats.shapiro(g).pvalue), 3) for g in groups])
f = stats.f_oneway(*groups)
print(f"F = {f.statistic:.2f}, p = {f.pvalue:.2e}")
grand = d.weight_loss_kg.mean()
ss_between = sum(len(g) * (g.mean() - grand) ** 2 for g in groups); ss_total =
((d.weight_loss_kg - grand) ** 2).sum()
print(f"n² (share of variance explained by diet) = {ss_between / ss_total:.2f}")
```

OUTPUT

```
               count  mean   std
diet
Keto              40  4.65  1.49
Low-fat           40  2.75  1.78
Mediterranean     40  4.03  1.50
Levene equal variances p = 0.615 | Shapiro p per group: [0.655, 0.698, 0.036]
F = 14.74, p = 1.96e-06
n² (share of variance explained by diet) = 0.20
```

ANOVA splits total variation into **between-group** and **within-group** parts; F = (between variance) ÷ (within variance). A large F means group means differ more than random noise would suggest.



Weight loss by diet with group means: Keto and Mediterranean beat Low-fat.

Post-hoc: which groups differ?

Python

```python
from statsmodels.stats.multicomp import pairwise_tukeyhsd
print(pairwise_tukeyhsd(d.weight_loss_kg, d.diet, alpha=0.05))
```

OUTPUT

```
    Multiple Comparison of Means - Tukey HSD, FWER=0.05
=============================================================
 group1     group2    meandiff p-adj   lower   upper  reject
-------------------------------------------------------------
   Keto       Low-fat  -1.9012    0.0 -2.7496 -1.0529   True
   Keto Mediterranean  -0.6152 0.2015 -1.4636  0.2331  False
Low-fat Mediterranean    1.286 0.0014  0.4376  2.1344   True
-------------------------------------------------------------
```

Tukey's HSD compares every pair while keeping the overall error at 5%. Keto and Mediterranean both beat Low-fat; Keto vs Mediterranean is not significant.

Two-way ANOVA (two factors and their interaction) uses `statsmodels.formula.api.ols("y ~ C(a) * C(b)")` with `anova_lm`.

Non-parametric alternatives

Python

```
kw = stats.kruskal(*groups); print(f"Kruskal-Wallis (diets): H = {kw.statistic:.2f}, p =
{kw.pvalue:.2e}")
hr = pd.read_csv("hr_employees.csv")
left, stay = hr[hr.attrition == "Yes"].monthly_income, hr[hr.attrition == "No"].monthly_income
print(f"income medians: left {left.median():,.0f} vs stayed {stay.median():,.0f} | Mann-Whitney
p = {stats.mannwhitneyu(left, stay).pvalue:.2e}")
sat_ot = hr[hr.overtime == "Yes"].job_satisfaction; sat_no = hr[hr.overtime ==
"No"].job_satisfaction
print(f"job satisfaction (ordinal 1-4), overtime vs not: Mann-Whitney p =
{stats.mannwhitneyu(sat_ot, sat_no).pvalue:.3f}")
rng = np.random.default_rng(1); before = rng.lognormal(3, .5, 20); after = before *
rng.uniform(.75, 1.05, 20)
print("Wilcoxon signed-rank (paired, skewed):", round(stats.wilcoxon(before, after).pvalue, 4))
```

OUTPUT

```
Kruskal-Wallis (diets): H = 23.29, p = 8.78e-06
income medians: left 67,917 vs stayed 77,750 | Mann-Whitney p = 3.81e-06
job satisfaction (ordinal 1-4), overtime vs not: Mann-Whitney p = 0.191
Wilcoxon signed-rank (paired, skewed): 0.001
```

Mann-Whitney compares whether values in one group tend to be larger than in the other (often described as comparing medians). It suits skewed income data and ordinal ratings.

Testing correlations

Python

```
for a, b in [("age", "monthly_income"), ("distance_from_home_km", "job_satisfaction"),
("years_at_company", "monthly_income")]:
    pr, sp = stats.pearsonr(hr[a], hr[b]), stats.spearmanr(hr[a], hr[b])
    print(f"{a:22s} vs {b:16s} Pearson r {pr.statistic:+.3f} (p {pr.pvalue:.3g}) | Spearman ρ
{sp.statistic:+.3f} (p {sp.pvalue:.3g})")
print("Department vs attrition: x² p =", round(stats.chi2_contingency(pd.crosstab(hr.department,
hr.attrition)).pvalue, 3))
```

OUTPUT

```
age                    vs monthly_income    Pearson r +0.135 (p 1.52e-07) | Spearman ρ
+0.108 (p 2.95e-05)
distance_from_home_km  vs job_satisfaction Pearson r +0.008 (p 0.764) | Spearman ρ +0.007
(p 0.776)
years_at_company       vs monthly_income    Pearson r +0.679 (p 1.63e-203) | Spearman ρ
+0.577 (p 7.98e-134)
Department vs attrition: x² p = 0.196
```

With 1,500 rows, even r = 0.1 can be "significant": look at the size of r, not only the p-value.

Many tests: the multiple-comparison problem

Test 100 segments where nothing really changed, at α = 0.05, and about five will look "significant" by chance.

Python

```
from statsmodels.stats.multitest import multipletests
rng = np.random.default_rng(42)
null_p = [stats.ttest_ind(rng.normal(0, 1, 50), rng.normal(0, 1, 50)).pvalue for _ in
range(100)]       # 100 segments with NO real effect
real_p = [stats.ttest_ind(rng.normal(0, 1, 50), rng.normal(0.8, 1, 50)).pvalue for _ in
range(5)]      # 5 segments with a real effect
pvals = np.array(null_p + real_p); truth = np.array([False] * 100 + [True] * 5)
for name, sig in [("raw p < 0.05", pvals < 0.05), ("Bonferroni", multipletests(pvals, 0.05,
"bonferroni")[0]), ("Benjamini-Hochberg", multipletests(pvals, 0.05, "fdr_bh")[0])]:
    print(f"{name:20s} significant {sig.sum():3d} | true effects found {(sig & truth).sum()} of
5 | false positives {(sig & ~truth).sum()}")
```

OUTPUT

```
raw p < 0.05         significant  12 | true effects found 5 of 5 | false positives 7
Bonferroni           significant   3 | true effects found 3 of 5 | false positives 0
Benjamini-Hochberg   significant   6 | true effects found 5 of 5 | false positives 1
```

Bonferroni (α ÷ number of tests) controls the chance of any false positive but is strict; **Benjamini-Hochberg** controls the false discovery rate and keeps more power. Use them for dashboards that test many segments, A/B tests with many metrics, and feature screening.

Summary

KEY TAKEAWAYS

- Chi-square tests whether two categorical variables are independent (expected counts ≥ 5); report Cramér's V and residuals.
- ANOVA compares 3+ means in one test (F = between/within variance); follow with Tukey HSD; report η^2.
- Non-parametric tests (Mann-Whitney, Wilcoxon, Kruskal-Wallis, Spearman) use ranks for skewed, ordinal or small data.
- Test correlations with Pearson (linear) or Spearman (monotonic); large n makes tiny r significant.
- Many tests → adjust p-values (Bonferroni, Benjamini-Hochberg).

Quick quiz

6 questions · tap an answer to check it

1. Which test checks whether preferred channel depends on age group?

- **A** ANOVA
- **B** Chi-square test of independence
- **C** Paired t-test
- **D** Pearson correlation

Answer: B. Both variables are categorical.

2. Why use ANOVA instead of several t-tests for 3 groups?

- **A** Multiple t-tests inflate the chance of a false positive
- **B** t-tests cannot handle numbers
- **C** ANOVA always gives smaller p-values
- **D** It is required by law

Answer: A. ANOVA tests all means together at α.

3. ANOVA gives p < 0.001 for three diets. What next?

- **A** Stop: all diets differ
- **B** Run a post-hoc test (Tukey HSD) to see which pairs differ
- **C** Run a chi-square test
- **D** Delete outliers

Answer: B. ANOVA only says at least one mean differs.

4. Which test compares two independent groups of skewed incomes?

A Paired t-test

B Mann-Whitney U

C Chi-square goodness of fit

D One-sample t-test

Answer: B. A rank-based alternative to the t-test.

5. What does the Benjamini-Hochberg method control?

A Type II errors

B The false discovery rate among significant results

C The sample size

D Normality

Answer: B. It is less strict than Bonferroni.

6. What is a chi-square assumption?

A Expected counts of at least about 5 per cell

B Normal data

C Equal variances

D Paired observations

Answer: A. Use Fisher's exact test for small tables.

EXERCISE

Practice (hr_employees.csv)

(1) Is overtime related to department? (chi-square + Cramér's V). (2) Does mean monthly income differ across departments (ANOVA + Tukey)? (3) Repeat (2) with Kruskal-Wallis. (4) Is education level related to attrition?

Show solution (worked answer)

Python

```
t = pd.crosstab(hr.department, hr.overtime); r = stats.chi2_contingency(t); print(r.pvalue,
np.sqrt(r.statistic / (t.values.sum() * (min(t.shape) - 1))))
gs = [g.monthly_income for _, g in hr.groupby("department")]; print(stats.f_oneway(*gs))
print(pairwise_tukeyhsd(hr.monthly_income, hr.department))
print(stats.kruskal(*gs))
print(stats.chi2_contingency(pd.crosstab(hr.education, hr.attrition)).pvalue)
```

INTERVIEW CORNER

Questions you may be asked

Q: When would you use a non-parametric test? Small samples that are clearly non-normal, ordinal data (ratings), or heavy outliers; for large samples the CLT usually makes parametric tests fine.

Q: What does a chi-square test tell you and what does it not? Whether two categorical variables are associated; not the direction, strength (use Cramér's V) or causation.

Q: Explain ANOVA's F-statistic. The ratio of variation between group means to variation within groups; large values mean group differences exceed what noise explains.

STATISTICS & MACHINE LEARNING · LESSON 6

A/B Testing and Experimentation: Design, Power, Analysis and Pitfalls

Run product experiments the way data teams at e-commerce and fintech companies do. Choose a metric and guardrails, size the test with power analysis, randomise, analyse conversion and revenue with z-tests, t-tests and bootstrap intervals, check sample-ratio mismatch, avoid peeking and novelty effects, and write the decision memo, using a new-checkout experiment.

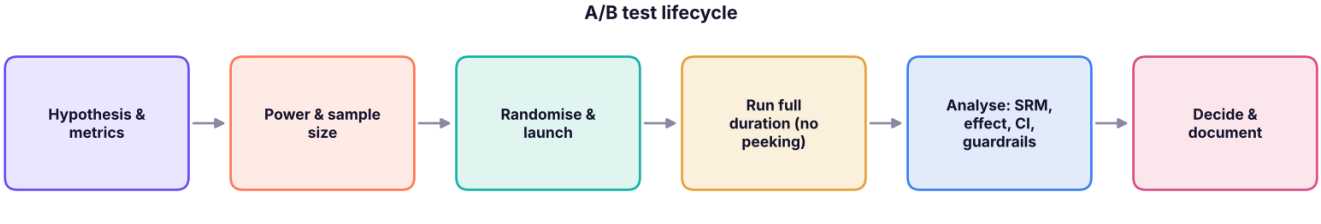
ML class 2 (case study) · ✦ statsmodels 0.1x · SciPy 1.1x · Intermediate · ⏱ 120 min

By the end of this lesson you can

- Design an experiment with a hypothesis, primary metric, guardrail metrics and randomisation unit
- Compute the required sample size with power analysis and a minimum detectable effect
- Analyse conversion (two-proportion z-test) and revenue per user (Welch t-test and bootstrap)
- Check sample-ratio mismatch and recognise peeking, novelty and network effects
- Write a launch/no-launch recommendation with uncertainty and business impact

What is an A/B test?

A **randomised controlled experiment**: users are randomly split into **control (A)** and **treatment (B)**; the only systematic difference is the change you test, so differences in outcomes are **caused** by the change (up to chance, which statistics quantifies).



The experiment lifecycle.

Case. An online store tests a **new one-page checkout (B)** against the current checkout (A). 4,000 users were split 50/50. **Hypothesis:** the new checkout increases the purchase conversion rate.

> **Primary metric:** conversion = purchasers ÷ users. **Secondary:** revenue per user, average order value. **Guardrails:** refund rate, page load time, support tickets.

Design choice	Good practice
Randomisation unit	User (not session or page view), so one person sees one version
Split	50/50 maximises power
Duration	At least one full business cycle (1–2 weeks, including weekends); fixed in advance
Metric	Defined before launch; one primary metric
Population	Who is eligible (new users only? all platforms?)

Sample size and power

Before launching: how many users do we need to detect the smallest effect worth acting on (the **minimum detectable effect**, MDE)?

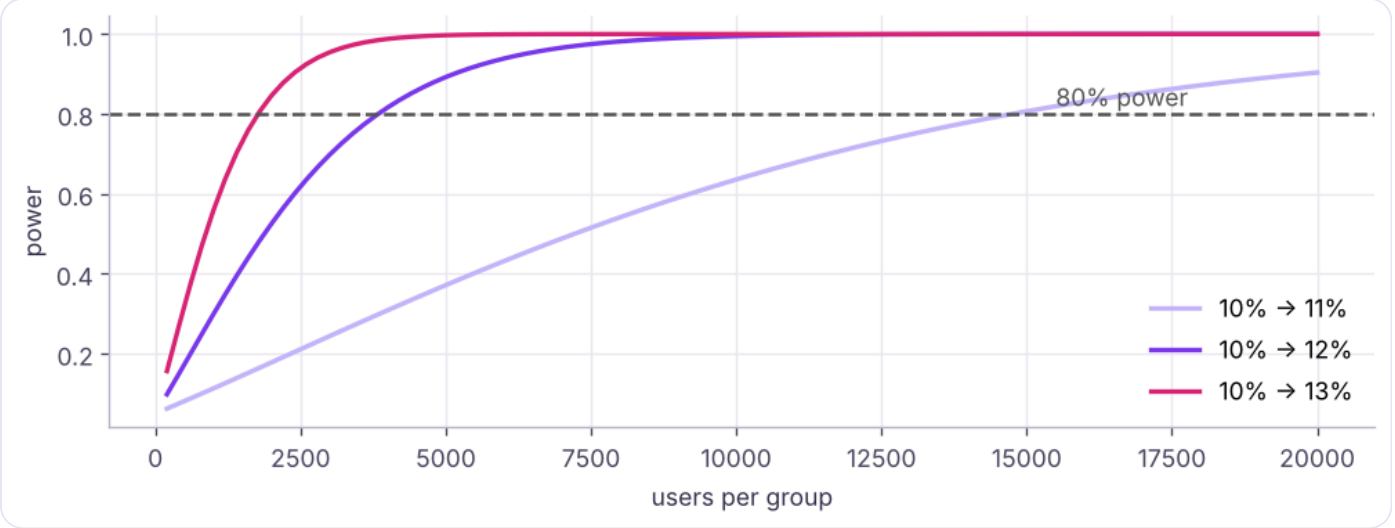
Python

```python
import numpy as np, pandas as pd
from scipy import stats
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize
base = 0.10
for target in (0.11, 0.12, 0.13):
    h = proportion_effectsize(target, base)                    # Cohen's h
    n = NormalIndPower().solve_power(effect_size=h, alpha=0.05, power=0.8, ratio=1,
alternative="two-sided")
    print(f"detect {base:.0%} → {target:.0%} (+{(target - base) / base:.0%} relative):
{int(np.ceil(n)):,} users per group")
```

OUTPUT

```
detect 10% → 11% (+10% relative): 14,745 users per group
detect 10% → 12% (+20% relative): 3,835 users per group
detect 10% → 13% (+30% relative): 1,769 users per group
```

Small lifts need very large samples. With 2,000 users per group, this test can reliably detect a lift from about 10% to 13%, not smaller ones.



Power curves: the chance of detecting a true lift grows with sample size and effect size.

Analysis step 1: sanity checks

Python

```
ab = pd.read_csv("ab_test.csv")
counts = ab.group.value_counts().sort_index(); print(counts.to_dict())
srm = stats.chisquare(counts.values)                                    # expected 50/50
print(f"sample-ratio mismatch test p = {srm.pvalue:.3f} → {'OK' if srm.pvalue > 0.01 else
'INVESTIGATE: randomisation or logging problem'}")
print("duplicate users:", ab.user_id.duplicated().sum(), "| missing:", ab.isna().sum().sum())
```

OUTPUT

```
{'A_control': 2000, 'B_new_checkout': 2000}
sample-ratio mismatch test p = 1.000 → OK
duplicate users: 0 | missing: 0
```

A **sample-ratio mismatch (SRM)** (for example 52/48 when you planned 50/50, with p < 0.01) signals a bug in assignment or tracking; do not trust results until it is explained.

Analysis step 2: conversion rate

Python

```
from statsmodels.stats.proportion import proportions_ztest, confint_proportions_2indep
g = ab.groupby("group").agg(users=("user_id", "size"), buyers=("converted", "sum"))
g["conversion"] = g.buyers / g.users
print(g)
z, p = proportions_ztest(g.buyers.values[::-1], g.users.values[::-1])          # B vs A
lo, hi = confint_proportions_2indep(g.buyers.iloc[1], g.users.iloc[1], g.buyers.iloc[0],
g.users.iloc[0], compare="diff")
pa, pb = g.conversion.iloc[0], g.conversion.iloc[1]
print(f"absolute lift {100 * (pb - pa):.2f} pts (95% CI {100 * lo:.2f} to {100 * hi:.2f}) |
relative lift {(pb - pa) / pa:.1%} | z = {z:.2f}, p = {p:.5f}")
```

OUTPUT

```
                users  buyers  conversion
group
A_control        2000     198      0.0990
B_new_checkout   2000     273      0.1365
absolute lift 3.75 pts (95% CI 1.75 to 5.75) | relative lift 37.9% | z = 3.68, p =
0.00023
```

Analysis step 3: revenue per user and order value

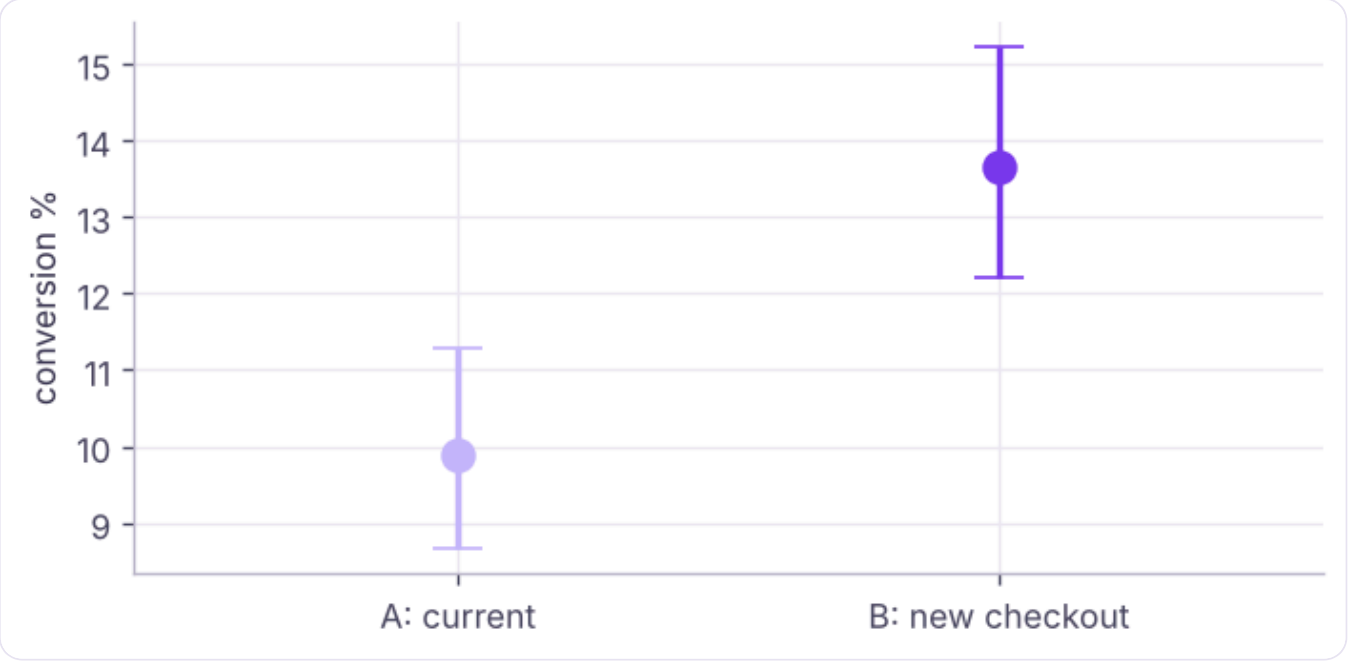
Python

```
a, b = ab[ab.group == "A_control"], ab[ab.group == "B_new_checkout"]
for name, col, fa, fb in [("revenue per user", "order_value", a, b), ("average order value
(buyers)", "order_value", a[a.converted == 1], b[b.converted == 1])]:
    t = stats.ttest_ind(fb[col], fa[col], equal_var=False)
    print(f"{name:30s} A {fa[col].mean():8.1f} | B {fb[col].mean():8.1f} | diff {fb[col].mean()
- fa[col].mean():7.1f} | Welch p = {t.pvalue:.4f}")
rng = np.random.default_rng(0)
boot = [rng.choice(b.order_value.values, len(b)).mean() - rng.choice(a.order_value.values,
len(a)).mean() for _ in range(5000)]
print(f"bootstrap 95% CI for revenue-per-user difference: {np.percentile(boot, 2.5):.1f} to
{np.percentile(boot, 97.5):.1f}")
```

OUTPUT

```
revenue per user               A    137.5 | B    202.8 | diff    65.3 | Welch p = 0.0000
average order value (buyers)   A   1389.3 | B   1485.6 | diff    96.4 | Welch p = 0.0082
bootstrap 95% CI for revenue-per-user difference: 35.5 to 94.9
```

Revenue per user mixes conversion and order size and has many zeros and a long tail; Welch's t-test (thanks to the CLT) and the bootstrap both work at this sample size.



Conversion rate with 95% confidence intervals for each variant.

Analysis step 4: business impact

Python

```
monthly_users = 300_000
extra_buyers = monthly_users * (pb - pa)
extra_rev = monthly_users * (b.order_value.mean() - a.order_value.mean())
print(f"at {monthly_users:,} users/month: about {extra_buyers:,.0f} extra orders and ₹{extra_rev / 1e5:,.1f} lakh extra revenue per month")
print(f"conservative (lower CI bound of conversion lift): {monthly_users * lo:,.0f} extra orders")
```

OUTPUT

```
at 300,000 users/month: about 11,250 extra orders and ₹195.8 lakh extra revenue per month
conservative (lower CI bound of conversion lift): 5,263 extra orders
```

Pitfalls

Pitfall	What goes wrong	Fix
Peeking	Checking daily and stopping at the first p < 0.05 inflates false positives to 20–30%	Fix the sample size; or use sequential tests / always-valid p-values
Too many metrics	Some will be "significant" by chance	One primary metric; adjust others (lesson 5)
Novelty / primacy effects	Users react to change itself, then revert	Run longer; analyse by week
SRM	Broken randomisation or logging	Chi-square check before analysis
Network effects	Treated users affect control users (marketplaces, social)	Cluster or geo randomisation
Simpson's paradox	Mix of segments differs between periods	Randomise; compare within segments
Underpowered tests	"No effect" just means "too small a test"	Power analysis first; report CIs
HARKing	Choosing the hypothesis after seeing results	Pre-register the plan

Python

```
rng = np.random.default_rng(11); false_wins = 0
for sim in range(400):                                          # A/A tests: there is NO real
difference
    xa, xb = rng.random(4000) < 0.10, rng.random(4000) < 0.10
    for day in range(1, 15):                                    # peek every "day" (≈285 users
per group per day)
        n = day * 285
        if proportions_ztest([xa[:n].sum(), xb[:n].sum()], [n, n])[1] < 0.05: false_wins += 1;
break
print(f"stopping at the first significant daily peek: {false_wins / 400:.0%} false positives
(should be 5%)")
```

OUTPUT

```
stopping at the first significant daily peek: 20% false positives (should be 5%)
```

Beyond the classic test

- **CUPED / regression adjustment:** use pre-experiment behaviour as a covariate to cut variance (and required sample size) by 20–50%.
- **Bayesian A/B testing:** reports "probability that B beats A" and expected loss; intuitive for stakeholders.

- **Multi-armed bandits:** shift traffic to the winning variant during the test (good for short campaigns, worse for clean inference).
- **Quasi-experiments** when you cannot randomise: difference-in-differences, synthetic control, regression discontinuity, propensity matching.
- Platforms: Optimizely, Statsig, GrowthBook, Eppo, LaunchDarkly, Firebase A/B Testing.

The decision memo

KEY TAKEAWAYS

Experiment: new one-page checkout, 4,000 users, 50/50, no sample-ratio mismatch. **Result:** conversion rose from 9.9% to 13.7% (+3.75 points, 95% CI about +1.7 to +5.8; $p < 0.001$). Revenue per user rose from about ₹138 to ₹203, and buyers' average order value also rose. **Guardrails:** check refunds, load time and support tickets before rollout. **Decision:** launch B to all users, with a 5% holdout for four weeks to confirm the effect lasts.

Summary

KEY TAKEAWAYS

- Randomisation makes A/B tests causal; define the hypothesis, one primary metric, guardrails and the duration before launch.
- Power analysis sets the sample size for a minimum detectable effect; small lifts need large samples.
- Check sample-ratio mismatch first; analyse conversion with a two-proportion z-test and revenue with Welch's t-test or the bootstrap; report CIs and business impact.
- Avoid peeking, metric fishing and underpowered tests; consider CUPED, Bayesian methods and quasi-experiments where appropriate.

Quick quiz

6 questions · tap an answer to check it

1. Why randomise users into A and B?

- **A** So the only systematic difference between groups is the change being tested
- **B** To make the test faster
- **C** To increase conversion
- **D** To avoid statistics

Answer: A. Randomisation balances all other factors on average.

2. What does a power analysis give you before the test?

- **A** The p-value
- **B** The sample size needed to detect a given minimum effect with a chosen probability
- **C** The winner
- **D** The conversion rate

Answer: B. Typical settings: $\alpha = 0.05$, power = 0.8.

3. The planned split was 50/50 but you observe 53/47 with SRM $p < 0.001$. What do you do?

- **A** Report the result
- **B** Investigate the assignment or logging bug before trusting the results
- **C** Increase α
- **D** Remove users until it is 50/50

Answer: B. SRM invalidates naive analysis.

4. Why is stopping as soon as p < 0.05 ("peeking") a problem?

- **A** It inflates the false-positive rate well above 5%
- **B** It wastes traffic
- **C** It makes p-values larger
- **D** It is fine

Answer: A. Use fixed horizons or sequential methods.

5. Which test suits comparing conversion rates of A and B?

- **A** Paired t-test
- **B** Two-proportion z-test (or chi-square)
- **C** ANOVA
- **D** Pearson correlation

Answer: B. Conversion is a proportion.

6. A test with 300 users per group shows no significant difference. Best conclusion?

- **A** The change has no effect
- **B** The test may be underpowered; look at the confidence interval for the plausible range of effects
- **C** Launch anyway
- **D** The metric is wrong

Answer: B. Absence of evidence is not evidence of absence.

EXERCISE

Practice

(1) Compute the users per group needed to detect a lift from 13.7% to 15% conversion. (2) Analyse average order value among buyers with a Mann-Whitney test as well. (3) Simulate an A/A test 1,000 times with a fixed horizon and confirm about 5% false positives. (4) Draft a one-paragraph memo for the case where the result had been p = 0.20.

Show solution (worked answer)

Python

```
print(NormalIndPower().solve_power(proportion_effectsize(0.15, 0.137), alpha=0.05,
power=0.8))
print(stats.mannwhitneyu(b[b.converted == 1].order_value, a[a.converted == 1].order_value))
fp = np.mean([proportions_ztest([(rng.random(4000) < .1).sum(), (rng.random(4000) <
.1).sum()], [4000, 4000])[1] < .05 for _ in range(1000)]); print(fp)
# Memo: "No significant difference (p = 0.20); CI ranges from -x to +y points, so we cannot
rule out a meaningful lift. Extend the test to N users or test a bolder change."
```

INTERVIEW CORNER

Questions you may be asked

Q: How would you design an A/B test for a new recommendation widget? State the hypothesis and primary metric (e.g. revenue per user), guardrails (latency, returns), randomise by user, size it with power analysis, run full weeks, check SRM, analyse with CIs, and decide with business impact.

Q: Your test is significant on day 3. Do you stop? No: stopping early inflates false positives and novelty effects can fade; finish the planned duration or use a sequential design.

Q: What if you cannot randomise? Use quasi-experimental methods: difference-in-differences with a comparison group, synthetic control, regression discontinuity or matching, and state the assumptions.

STATISTICS & MACHINE LEARNING · LESSON 7

Introduction to Machine Learning: Types, Business Framing and the ML Lifecycle

What machine learning is and is not, how it differs from rules and statistics, supervised vs unsupervised vs reinforcement learning, regression vs classification vs clustering, turning a business problem into an ML problem, the end-to-end ML lifecycle (CRISP-DM to MLOps), linear vs non-linear models and link functions, and where GenAI fits.

ML classes 3–4 | ✦ scikit-learn 1.7+ · 2026 practice | Beginner | ⏱ 100 min

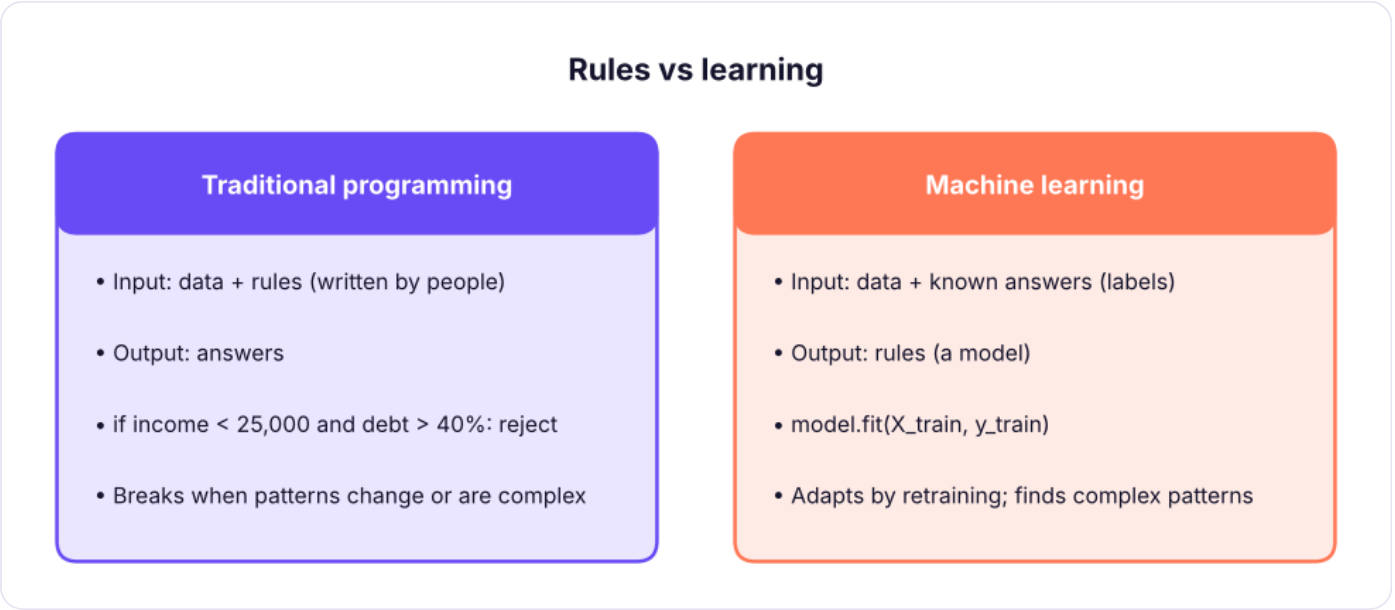
By the end of this lesson you can

- Define machine learning and distinguish it from rule-based systems and classical statistics
- Classify problems as supervised (regression, classification), unsupervised (clustering, dimensionality reduction) or reinforcement learning
- Translate a business question into target, features, unit of prediction, metric and decision
- Describe each stage of the ML lifecycle and common failure points
- Explain linear vs non-linear models, link functions, and when not to use ML

What is machine learning?

> "A computer program is said to learn from experience E with respect to some task T and performance measure P if its performance at T, as measured by P, improves with experience E." (Tom Mitchell)

Instead of writing rules by hand, we give the computer **examples** (data) and let an algorithm **learn the rules** (a model) that map inputs to outputs.

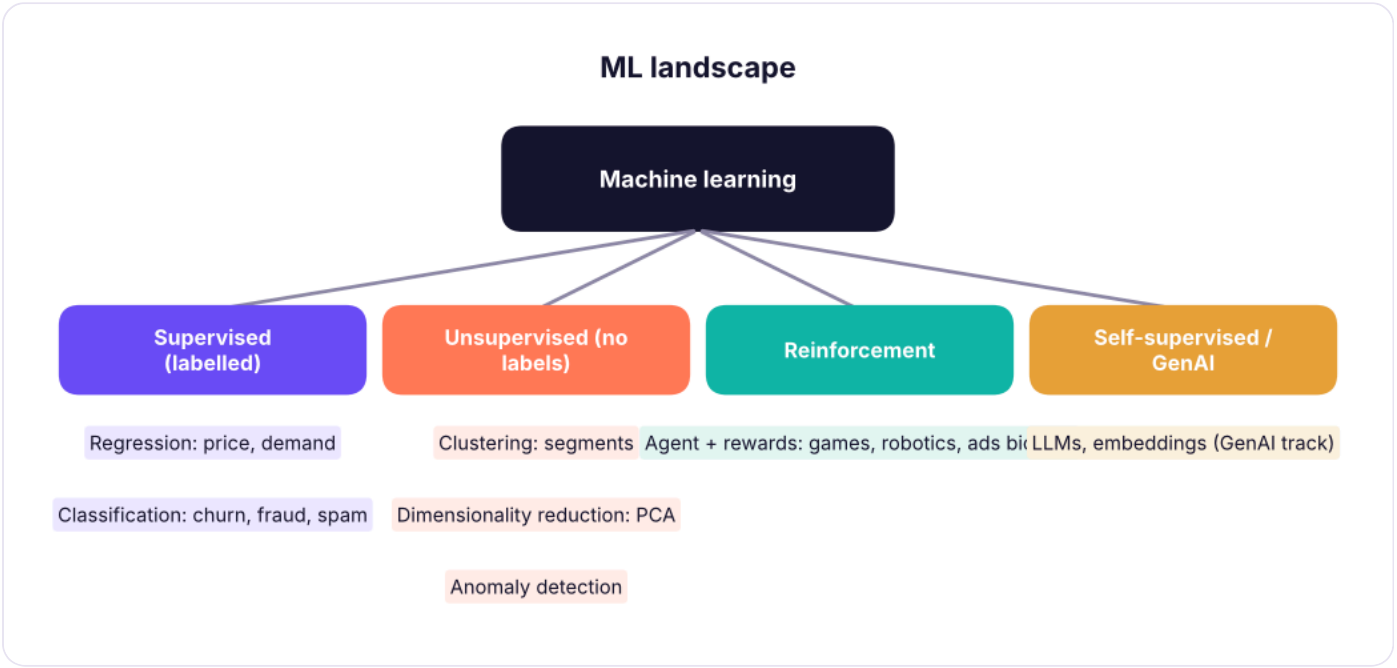


Traditional programming vs machine learning.

	Statistics	Machine learning
Main goal	Inference: understand relationships, test hypotheses	Prediction: accurate outputs on new data
Typical question	"Does overtime increase attrition, and by how much?"	"Which employees are likely to leave next quarter?"
Model choice	Interpretable, with assumptions	Whatever predicts best (with interpretability as needed)
Validation	p-values, confidence intervals	Held-out test data, cross-validation

They overlap heavily: linear and logistic regression belong to both.

Types of machine learning



The main families of machine learning.

Problem	Target type	Example	Algorithms (this track)
Regression	Number	House price, delivery time, demand	Linear, Ridge/Lasso, trees, boosting
Classification	Category	Churn yes/no, loan default, spam	Logistic, KNN, trees, RF, boosting, SVM, NB
Clustering	None	Customer segments	K-means, hierarchical, DBSCAN
Dimensionality reduction	None	Compress 50 features to 5	PCA
Time-series forecasting	Number over time	Next month's sales	Seasonal naive, ETS, ARIMA, ML with lags
Recommendation / ranking	Ranked list	Products to show	Collaborative filtering, learning to rank

Python

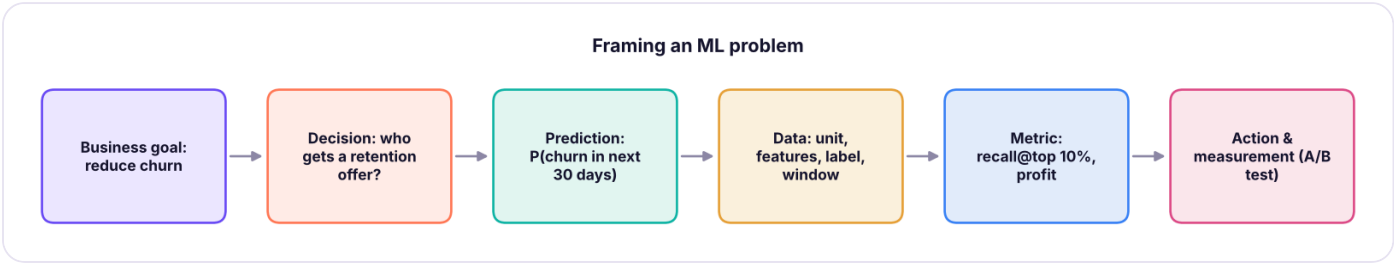
```
import pandas as pd
h = pd.read_csv("house_prices.csv"); tc = pd.read_csv("telecom_churn.csv")
print("Regression target:", "price_lakh", "| examples:", h.price_lakh.head(3).tolist(), "|
features:", [c for c in h.columns if c not in ("house_id", "price_lakh")])
print("Classification target: churn", tc.churn.value_counts().to_dict(), "| features:", [c for c
in tc.columns if c not in ("customer_id", "churn")])
```

OUTPUT

```
Regression target: price_lakh | examples: [60.2, 60.9, 43.7] | features: ['city',
'area_sqft', 'bedrooms', 'bathrooms', 'age_years', 'distance_to_center_km', 'parking']
Classification target: churn {'No': 2228, 'Yes': 772} | features: ['senior_citizen',
'tenure_months', 'contract', 'internet_service', 'payment_method', 'monthly_charges',
'total_charges', 'support_calls']
```

Vocabulary: a row is an **observation/sample**; input columns are **features** (X, predictors, independent variables); the column to predict is the **target** (y, label, dependent variable); the learned function is the **model**; using it on new data is **inference/prediction**.

From business problem to ML problem



Framing comes before algorithms.

Question	Churn example
What decision will the prediction change?	Which 10% of customers receive a ₹200 retention offer
Unit of prediction	One customer, scored monthly
Target definition	Cancelled within 30 days after the scoring date
Features (only information available **at prediction time**)	Tenure, contract, charges, support calls in the previous 90 days
Success metric	Recall in the top 10%, net profit of the campaign
Baseline	Current rule: target month-to-month customers with 3+ calls
Constraints	Explainability for the retention team, privacy, weekly batch scoring

WATCH OUT **Data leakage**

Using information that would not be available at prediction time (for example "number of cancellation calls" recorded **after** the customer decided to leave) gives brilliant test scores and useless production models. Define a clear cut-off date for features and a later window for the label.

When not to use ML

Simple rules work; there is little data or no labels; mistakes are unacceptable and must be explained exactly; the pattern changes faster than you can retrain; or nobody will act on the predictions. Always compare with a simple baseline.

The ML lifecycle



CRISP-DM extended with MLOps: an iterative cycle, not a straight line.

Stage	Key work	Typical tools
Business understanding	Goal, decision, metric, baseline, constraints	Workshops, documents
Data understanding	Sources, EDA, quality, label definition	SQL, pandas, plots (Python track)
Data preparation	Cleaning, encoding, scaling, feature engineering, train/test split	pandas, scikit-learn pipelines
Modelling	Train candidate models, tune hyperparameters	scikit-learn, XGBoost, LightGBM
Evaluation	Test-set metrics, error analysis, fairness, business value	metrics, SHAP
Deployment	Batch scoring or API, integration with apps	FastAPI, Docker, cloud ML platforms
Monitoring	Data drift, performance decay, retraining	MLflow, Evidently, dashboards

In practice, most time goes to framing, data preparation and deployment, not to choosing algorithms.

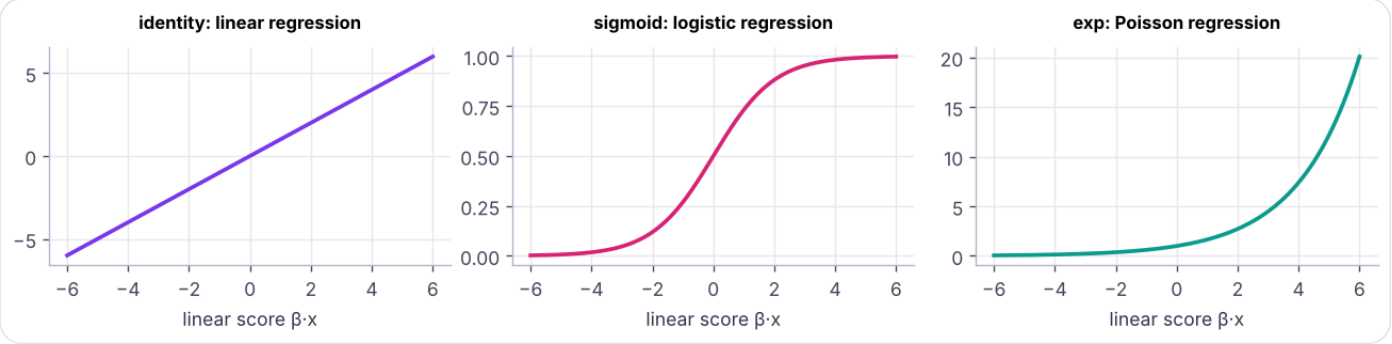
Linear vs non-linear models and link functions

A **linear model** predicts with a weighted sum of features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ \dots \ + \beta_p x_p$$

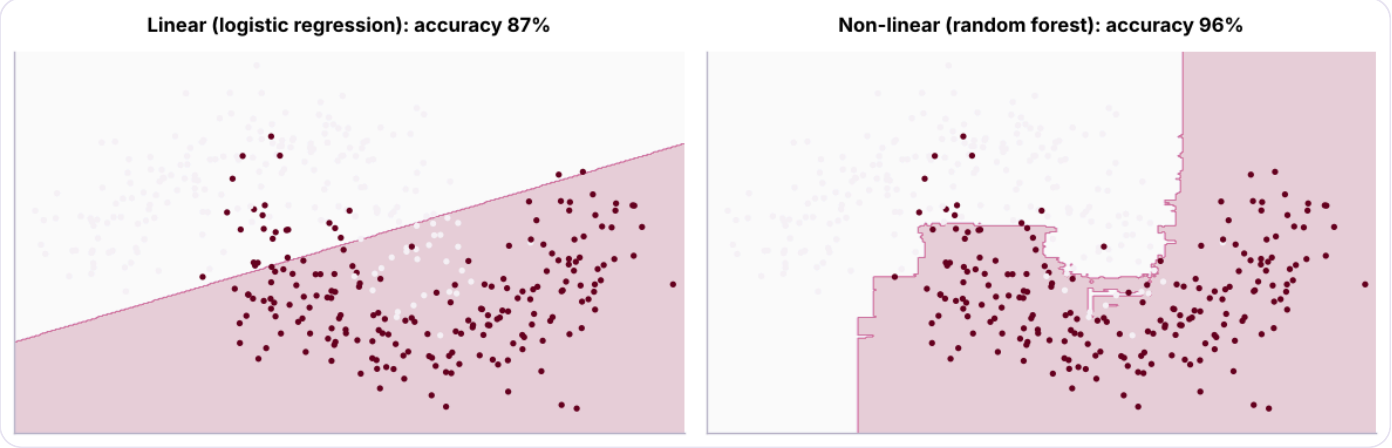
Generalised linear models pass that linear score through a **link function** so the output fits the target type:

Target	Model	Link (inverse)	Output
Any real number	Linear regression	Identity	ŷ = score
Probability (0–1)	Logistic regression	Logit ⇄ sigmoid 1/(1 + e^(−score))	P(y = 1)
Count (0, 1, 2…)	Poisson regression	Log ⇄ exp(score)	Expected count



The same linear score through three link functions: identity (regression), sigmoid (classification probability) and exp (counts).

Non-linear models (decision trees, random forests, gradient boosting, KNN, neural networks) capture thresholds and interactions automatically, usually at the cost of interpretability.



A linear model draws one straight boundary; a non-linear model can bend around the data.

Overfitting and generalisation (preview)

A model must work on **new** data. Memorising the training set (overfitting) fails in production; being too simple (underfitting) misses real patterns. We therefore always evaluate on data the model has not seen (lessons 8 and 15).

Where does GenAI fit?

Large language models are trained with self-supervised learning on huge text corpora, then adapted with instruction tuning and reinforcement learning from human feedback. In analytics they **complement** classical ML: LLMs read and write text, code and SQL; classical models remain best for structured, tabular prediction (churn, credit risk, demand), where gradient boosting usually wins. The GenAI track covers LLMs, RAG and agents.

Summary

KEY TAKEAWAYS

- ML learns rules from data; statistics focuses on inference, ML on prediction, with much overlap.
- Supervised (regression, classification), unsupervised (clustering, PCA, anomalies), reinforcement and self-supervised (LLMs) learning.
- Frame first: decision, unit, target and time windows, leak-free features, metric, baseline.
- Lifecycle: business → data → preparation → modelling → evaluation → deployment → monitoring, iteratively.
- Linear models use link functions (identity, sigmoid, exp); non-linear models capture thresholds and interactions.

Quick quiz

6 questions · tap an answer to check it

1. Predicting next month's sales in rupees is...

A Regression

B Classification

C Clustering

D Reinforcement learning

Answer: A. The target is a number.

2. Grouping customers into segments without labels is...

A Supervised learning

B Unsupervised learning (clustering)

C Regression

D A/B testing

Answer: B. No target column.

3. What is data leakage?

- **A** Using information in training that would not be available at prediction time
- **B** Losing data in a crash
- **C** Sharing data publicly
- **D** Missing values

Answer: A. It inflates test scores.

4. Which link function does logistic regression use to output a probability?

- **A** Identity
- **B** Sigmoid (inverse of the logit)
- **C** Exponential
- **D** Square root

Answer: B. It maps any score to 0–1.

5. Which stage often takes the most time in real ML projects?

- **A** Picking the algorithm
- **B** Data understanding and preparation
- **C** Writing the final slide
- **D** Installing Python

Answer: B. Data work dominates.

6. When might you NOT use ML?

- **A** When a simple rule works well or nobody will act on predictions
- **B** When you have lots of data
- **C** When the pattern is complex
- **D** When predictions save money

Answer: A. Start with a baseline.

EXERCISE **Frame these problems**

For each, state the ML type, target, unit, two leak-free features, the metric and the action: (1) predicting loan default at application; (2) forecasting daily units for each store; (3) grouping 5,000 products by sales pattern; (4) flagging suspicious card transactions in real time.

Show solution (worked answer)

(1) Classification; default within 12 months; applicant; income, debt-to-income; recall/precision at the approval threshold and expected loss; approve/decline/price. (2) Regression/forecasting; units per store-day; store-day; lagged sales, calendar/holidays; MAPE/WAPE; stock orders. (3) Clustering; none; product; weekly sales profile, price; silhouette + business sense; assortment strategy. (4) Classification (imbalanced); fraud label; transaction; amount vs customer average, merchant risk; precision/recall at alert capacity; block or step-up authentication.

INTERVIEW CORNER **Questions you may be asked**

Q: Explain machine learning to a non-technical person. Instead of writing rules ourselves, we show the computer many past examples with the right answers, and it learns patterns it can apply to new cases, like an employee learning from experience.

Q: Supervised vs unsupervised learning? Supervised learning uses labelled examples to predict a target; unsupervised learning finds structure such as groups or compressed representations without labels.

Q: How do you start an ML project? Clarify the business decision and success metric, define the target and prediction time, check data availability and leakage, set a simple baseline, then iterate on models.

STATISTICS & MACHINE LEARNING · LESSON 8

The scikit-learn Workflow: Train/Test Split, Baselines, Pipelines and Saving Models

Learn the standard machine-learning workflow in scikit-learn. The estimator API (fit, predict, transform), splitting data into train, validation and test sets with stratification and time order, baselines, ColumnTransformer and Pipeline to avoid leakage, comparing models, and saving and loading a trained pipeline.

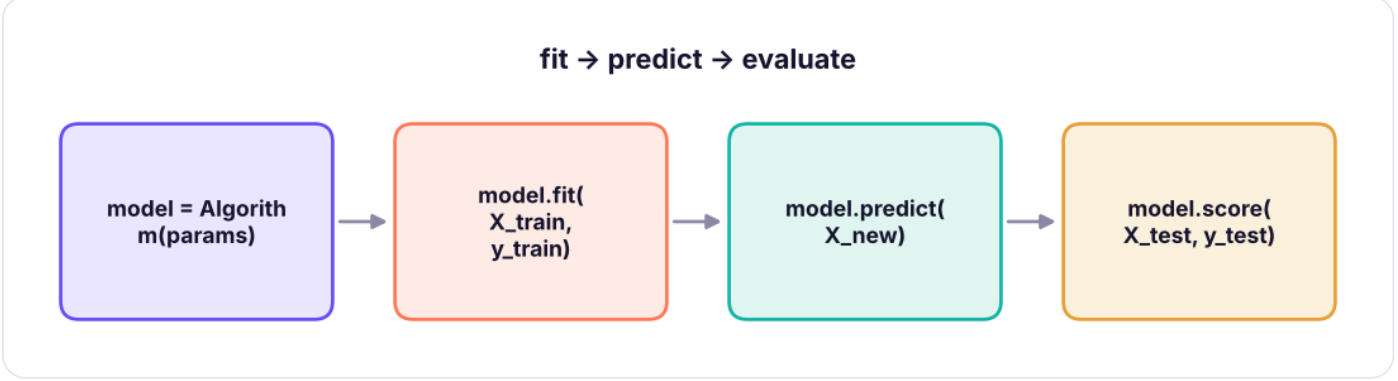
ML class 4 · scikit-learn 1.7+ · Intermediate · 110 min

By the end of this lesson you can

- Use the fit / predict / predict_proba / transform / score API consistently
- Split data into train and test (stratified, time-based) and explain why the test set is touched once
- Build dummy baselines and compare real models against them
- Combine preprocessing and a model in a Pipeline with ColumnTransformer
- Save, load and use a trained pipeline on new data

The estimator API

Every scikit-learn model follows the same pattern, which makes swapping algorithms easy.



One consistent interface for every estimator.

Method	Available on	Does
`fit(X, y)`	Every estimator	Learns parameters from training data
`predict(X)`	Models	Predicted labels or values
`predict_proba(X)`	Most classifiers	Class probabilities
`transform(X)`	Transformers (scalers, encoders, PCA)	Converts features using fitted parameters
`fit_transform(X)`	Transformers	Fit then transform (training data only!)
`score(X, y)`	Models	Default metric (accuracy or R^2)
`get_params()` / `set_params()`	All	Hyperparameters

X is a 2-D table of features (DataFrame or array, shape *n_samples* × *n_features*); **y** is a 1-D target.

Train/test split

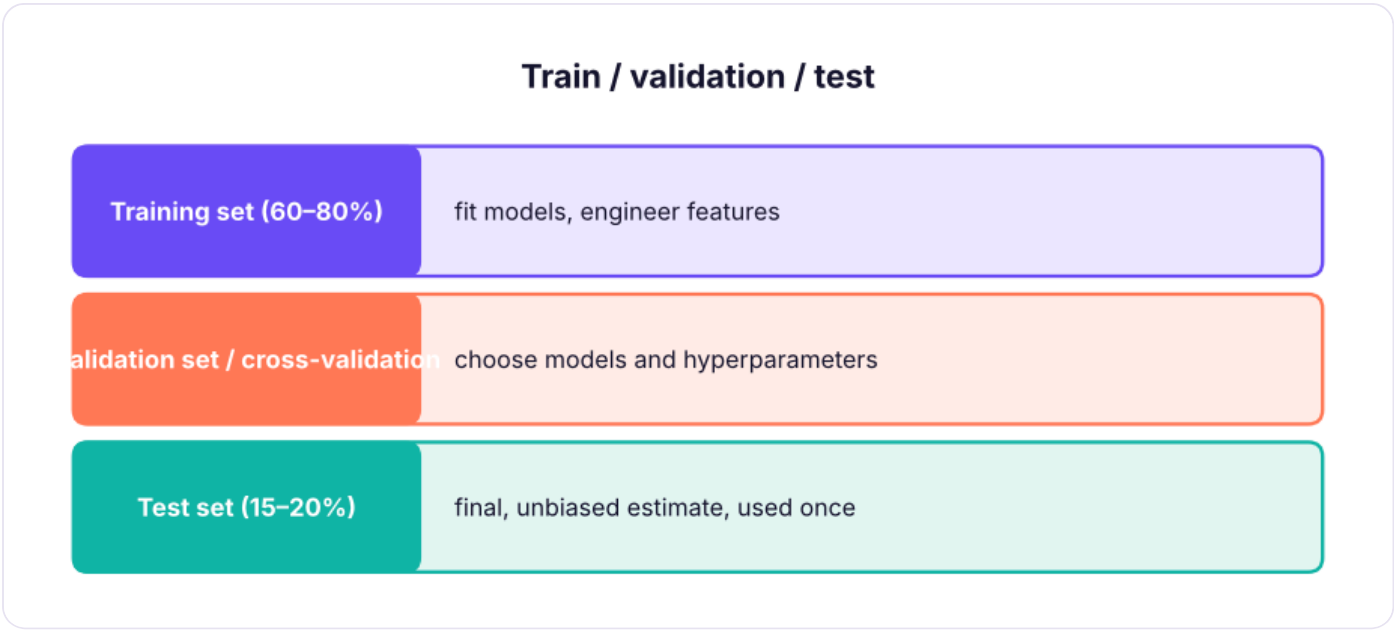
We judge a model on data it has **never seen**. Hold out a test set before doing anything else.

Python

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split
tc = pd.read_csv("telecom_churn.csv")
X = tc.drop(columns=["customer_id", "churn", "total_charges"])          # total_charges ≈ tenure
× monthly: redundant
y = (tc["churn"] == "Yes").astype(int)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=42)
print(X_train.shape, X_test.shape, f"churn rate train {y_train.mean():.3f} | test
{y_test.mean():.3f}")
```

OUTPUT

```
(2400, 7) (600, 7) churn rate train 0.258 | test 0.257
```



Typical data splits. The test set is a final exam: use it once, at the end.

Option	Use
`test_size=0.2`	Hold out 20%
`stratify=y`	Keep the class balance the same in both sets (classification)
`random_state=42`	Reproducible split
Time-based split	For time series and anything predicted in the future: train on the past, test on the most recent period
Group split (`GroupShuffleSplit`)	Keep all rows of a customer/store in one set

WATCH OUT **Never tune on the test set**

If you try many models and keep the one with the best **test** score, the test score is no longer an unbiased estimate. Choose with cross-validation on the training set (lesson 15); look at the test set once.

Baselines

A model is only useful if it beats a simple baseline.

Python

```
from sklearn.dummy import DummyClassifier, DummyRegressor
from sklearn.metrics import accuracy_score, recall_score
base = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
pred = base.predict(X_test)
print(f"always 'no churn': accuracy {accuracy_score(y_test, pred):.3f} | recall of churners
{recall_score(y_test, pred):.3f}")
rule = ((X_test.contract == "Month-to-month") & (X_test.tenure_months <= 12)).astype(int)
# business rule from the EDA
print(f"rule 'new month-to-month': accuracy {accuracy_score(y_test, rule):.3f} | recall
{recall_score(y_test, rule):.3f}")
```

OUTPUT

```
always 'no churn': accuracy 0.743 | recall of churners 0.000
rule 'new month-to-month': accuracy 0.777 | recall 0.240
```

74% accuracy by predicting "no churn" for everyone shows why **accuracy alone is misleading** for imbalanced problems (lesson 14).

Preprocessing with ColumnTransformer

Models need numbers on comparable scales. Numeric and categorical columns need different treatment.

Python

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline, make_pipeline
num_cols = ["senior_citizen", "tenure_months", "monthly_charges", "support_calls"]
cat_cols = ["contract", "internet_service", "payment_method"]
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), num_cols),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
OneHotEncoder(handle_unknown="ignore")), cat_cols),
])
Xt = prep.fit_transform(X_train)
print(Xt.shape, list(prep.get_feature_names_out())[:7], "...")
```

OUTPUT

```
(2400, 14) ['num__senior_citizen', 'num__tenure_months', 'num__monthly_charges',
'num__support_calls', 'cat__contract_Month-to-month', 'cat__contract_One year',
'cat__contract_Two year'] ...
```

Pipelines: preprocessing + model as one object

Python

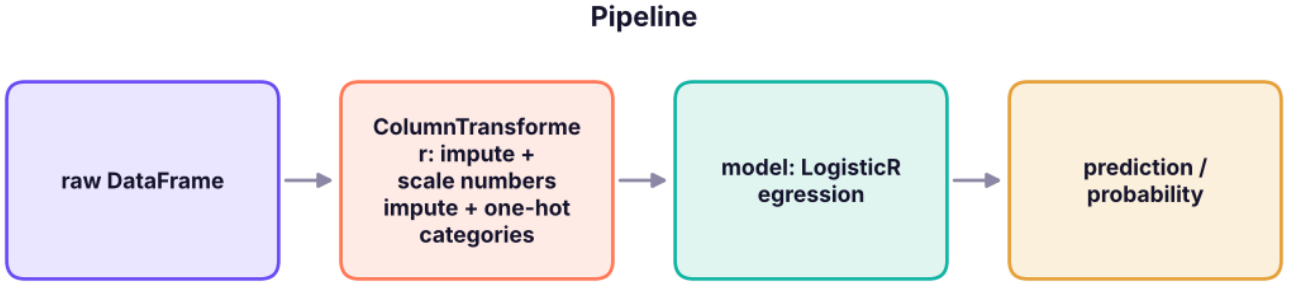
```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import roc_auc_score
models = {"logistic regression": LogisticRegression(max_iter=1000), "k-nearest neighbours":
KNeighborsClassifier(25),
          "random forest": RandomForestClassifier(300, min_samples_leaf=5, random_state=0),
"gradient boosting": HistGradientBoostingClassifier(random_state=0)}
for name, model in models.items():
    pipe = Pipeline([("prep", prep), ("model", model)]).fit(X_train, y_train)
    proba = pipe.predict_proba(X_test)[:, 1]
    print(f"{name:22s} accuracy {pipe.score(X_test, y_test):.3f} | ROC-AUC
{roc_auc_score(y_test, proba):.3f} | recall {recall_score(y_test, pipe.predict(X_test)):.3f}")
```

OUTPUT

```
logistic regression    accuracy 0.790 | ROC-AUC 0.803 | recall 0.396
k-nearest neighbours   accuracy 0.785 | ROC-AUC 0.759 | recall 0.344
random forest          accuracy 0.798 | ROC-AUC 0.793 | recall 0.403
gradient boosting      accuracy 0.760 | ROC-AUC 0.755 | recall 0.455
```

Why pipelines?

1. **No leakage:** scalers and imputers learn their statistics from the training data only, inside each fit (and inside every cross-validation fold).
2. **One object** to tune, save and deploy: raw data in, predictions out.
3. **Less code** and fewer mistakes when switching models.



A pipeline chains steps; fit runs fit_transform on each step, predict runs transform then predict.

A regression example

Python

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
h = pd.read_csv("house_prices.csv")
Xh, yh = h.drop(columns=["house_id", "price_lakh"]), h["price_lakh"]
Xh_tr, Xh_te, yh_tr, yh_te = train_test_split(Xh, yh, test_size=0.2, random_state=1)
prep_h = ColumnTransformer([("num", make_pipeline(SimpleImputer(strategy="median"),
StandardScaler()), ["area_sqft", "bedrooms", "bathrooms", "age_years", "distance_to_center_km",
"parking"]),
                            ("cat", OneHotEncoder(handle_unknown="ignore"), ["city"])])
for name, m in [("mean baseline", DummyRegressor()), ("linear regression", LinearRegression())]:
    p = Pipeline([("prep", prep_h), ("m", m)]).fit(Xh_tr, yh_tr).predict(Xh_te)
    print(f"{name:18s} MAE {mean_absolute_error(yh_te, p):6.2f} lakh | R² {r2_score(yh_te,
p):.3f}")
```

OUTPUT

```
mean baseline      MAE  23.53 lakh | R² -0.005
linear regression  MAE   7.18 lakh | R² 0.900
```

Saving and loading a model

Python

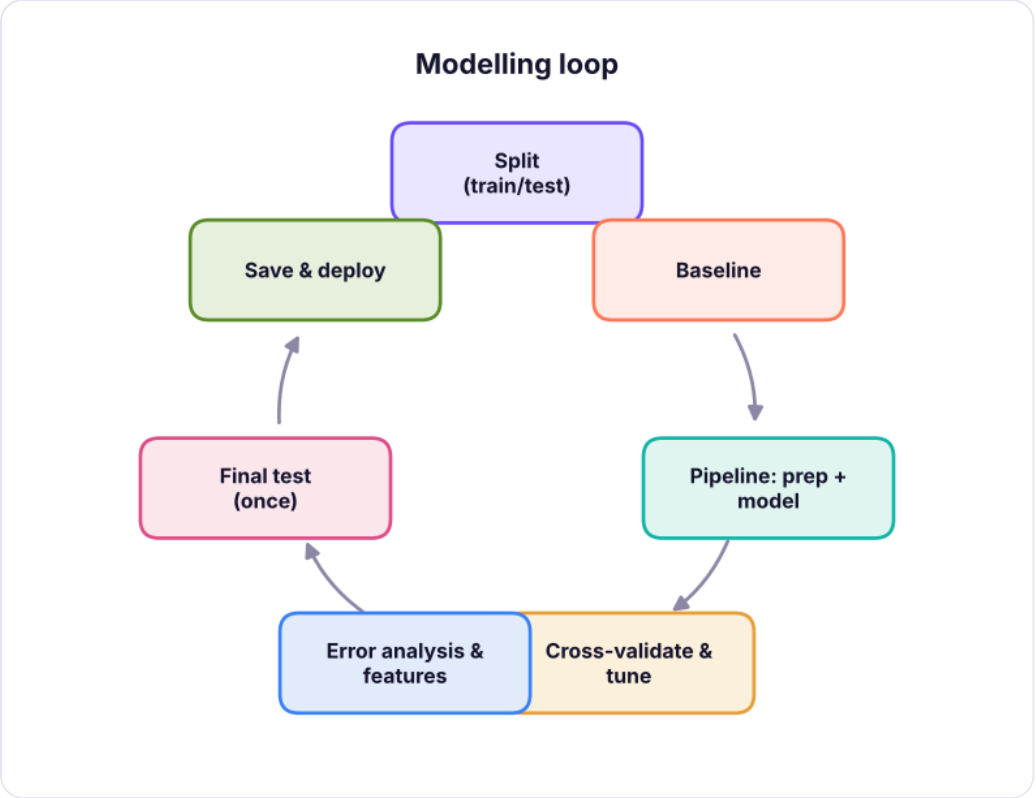
```
import joblib, os
best = Pipeline([("prep", prep), ("model", LogisticRegression(max_iter=1000))]).fit(X_train,
y_train)
joblib.dump(best, "churn_model.joblib")
loaded = joblib.load("churn_model.joblib")
new_customers = pd.DataFrame([{"senior_citizen": 0, "tenure_months": 3, "contract": "Month-to-
month", "internet_service": "Fiber", "payment_method": "UPI", "monthly_charges": 95.0,
"support_calls": 3},
                              {"senior_citizen": 1, "tenure_months": 60, "contract": "Two year",
"internet_service": "DSL", "payment_method": "Auto-debit", "monthly_charges": 55.0,
"support_calls": 0}])
print(loaded.predict_proba(new_customers)[:, 1].round(3), f"| file size
{os.path.getsize('churn_model.joblib') / 1024:.0f} KB")
os.remove("churn_model.joblib")
```

OUTPUT

```
[0.864 0.008] | file size 5 KB
```

Save the **whole pipeline**, note the library versions (`sklearn.__version__`), and keep a model card (data, date, metrics, intended use). Load models only from sources you trust: joblib/pickle files can run code.

The workflow, end to end



The modelling loop you will repeat in every lesson of this track.

Summary

KEY TAKEAWAYS

- Every estimator uses `fit`, then `predict` / `predict_proba` or `transform`; X is 2-D, y is 1-D.
- Split first (stratified for classification, by time for forecasting); the test set is used once.
- Always compare with a baseline (`DummyClassifier`, `DummyRegressor`, or a business rule).
- `ColumnTransformer + Pipeline` apply the right preprocessing to each column type and prevent leakage.
- Save the fitted pipeline with joblib and document versions and metrics.

Quick quiz

6 questions · tap an answer to check it

1. Why use `stratify=y` in `train_test_split` ?

- **A** To keep the same class proportions in train and test sets
- **B** To shuffle the data
- **C** To scale features
- **D** To remove outliers

Answer: A. Important for imbalanced classes.

2. Where should a StandardScaler learn its mean and standard deviation?

- **A** On the full dataset
- **B** On the training data only (inside the pipeline)
- **C** On the test data
- **D** It does not matter

Answer: B. Fitting on all data leaks test information.

3. A churn model has 74% accuracy and churn is 26%. What can you say?

- **A** It is good
- **B** It may be no better than predicting "no churn" for everyone; check recall and AUC
- **C** It is perfect
- **D** It is overfitting

Answer: B. Compare with a baseline.

4. What does `ColumnTransformer` do?

- **A** Applies different preprocessing to different columns and combines the results
- **B** Trains several models
- **C** Splits the data
- **D** Removes columns with missing values

Answer: A. e.g. scale numbers, one-hot encode categories.

5. Which split suits forecasting next month's demand?

- **A** Random split
- **B** Time-based: train on earlier periods, test on the latest period
- **C** Stratified split
- **D** No split

Answer: B. Random splits leak the future into training.

6. What should you save for deployment?

- **A** Only the model coefficients
- **B** The entire fitted pipeline (preprocessing + model) with version notes
- **C** The test set
- **D** The notebook only

Answer: B. One object turns raw data into predictions.

EXERCISE **Practice**

(1) Build a pipeline for `hr_employees.csv` that predicts attrition with logistic regression; compare it with a dummy baseline using ROC-AUC. (2) Add `job_title` as a categorical feature: does AUC change? (3) Save and reload the pipeline and score two made-up employees.

Show solution (worked answer)

Python

```
hr = pd.read_csv("hr_employees.csv")
y2 = (hr.attrition == "Yes").astype(int); X2 = hr.drop(columns=["emp_id", "attrition",
"hire_date"])
nums = X2.select_dtypes("number").columns.tolist(); cats = [c for c in X2.columns if c not
in nums]
p2 = Pipeline([("prep", ColumnTransformer([("n", StandardScaler(), nums), ("c",
OneHotEncoder(handle_unknown="ignore"), cats)])), ("m",
LogisticRegression(max_iter=2000))])
a, b, c, d = train_test_split(X2, y2, test_size=.2, stratify=y2, random_state=0)
print(roc_auc_score(d, p2.fit(a, c).predict_proba(b)[:, 1]), roc_auc_score(d,
DummyClassifier().fit(a, c).predict_proba(b)[:, 1]))
```

INTERVIEW CORNER **Questions you may be asked**

Q: Why do we need a train/test split? To estimate how the model performs on new, unseen data; training performance is optimistic because the model has seen those examples.

Q: What is a pipeline and why use one? A chain of preprocessing steps and a model trained and applied as one object; it prevents leakage, simplifies tuning and makes deployment reliable.

Q: What is the difference between fit_transform and transform? `fit_transform` learns parameters (e.g. mean, categories) and applies them, used on training data; `transform` applies already-learned parameters, used on validation, test and new data.

STATISTICS & MACHINE LEARNING · LESSON 9

Data Preprocessing and Feature Engineering for ML

Prepare features that models can learn from. Imputation strategies, outlier handling, encoding categorical variables (one-hot, ordinal, target encoding), scaling (standard, min-max, robust), transformations (log, Box-Cox/Yeo-Johnson), binning, interactions and polynomial features, date and text features, and feature selection, all inside leak-free pipelines.

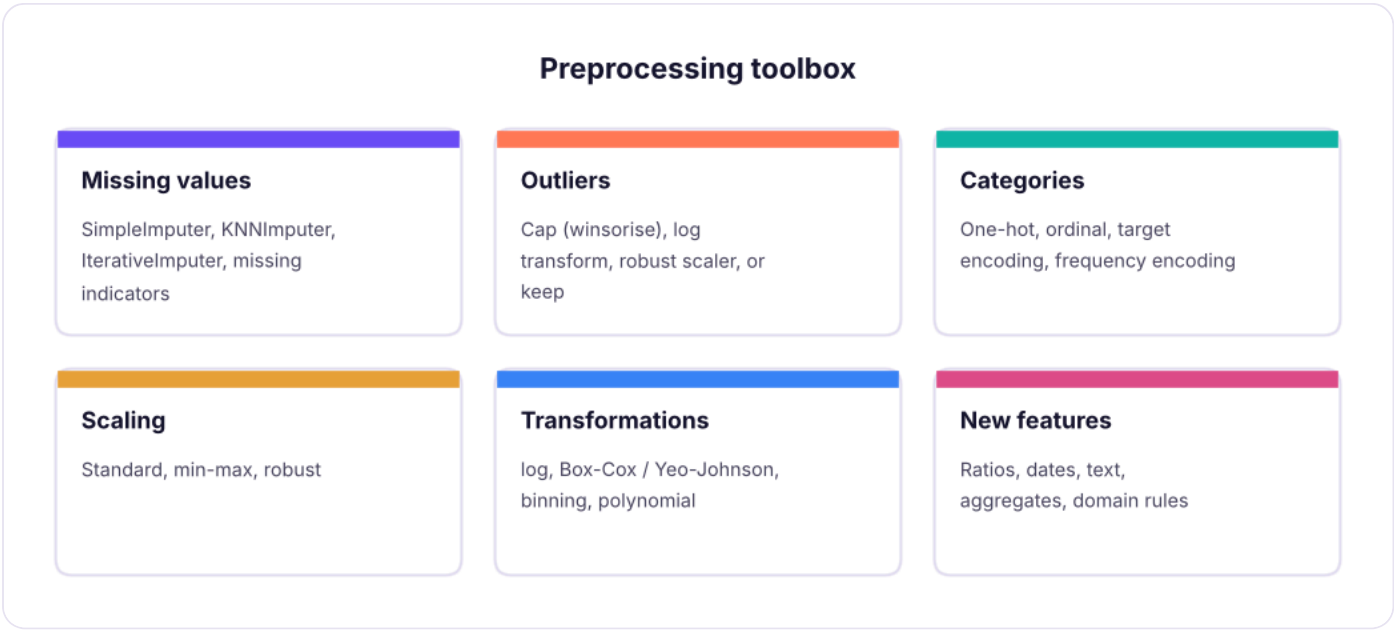
ML class 6 | scikit-learn 1.7+ | Intermediate | 120 min

By the end of this lesson you can

- Impute missing values with simple, group-aware and model-based strategies and add missing indicators
- Encode nominal and ordinal categories correctly, including high-cardinality columns
- Choose a scaler and know which algorithms need scaling
- Engineer features from numbers, dates, text and domain knowledge
- Select features with filter, wrapper and embedded methods without leaking test data

Why preprocessing matters

Algorithms only see numbers. How you represent the data often matters more than which algorithm you choose ("better data beats fancier models").



The preprocessing toolbox, applied inside a pipeline.

Missing values

Python

```
import pandas as pd, numpy as np
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.experimental import enable_iterative_imputer  # noqa: F401
from sklearn.impute import IterativeImputer
h = pd.read_csv("house_prices.csv")
num = h[["area_sqft", "bedrooms", "bathrooms", "age_years", "price_lakh"]]
mask = num.area_sqft.isna(); print("missing area:", mask.sum())
for name, imp in [("median", SimpleImputer(strategy="median")), ("KNN (5 neighbours)",
KNNImputer(n_neighbors=5)), ("iterative (regression)", IterativeImputer(random_state=0))]:
    filled = pd.DataFrame(imp.fit_transform(num), columns=num.columns)
    print(f"{name:24s} imputed areas: mean {filled.loc[mask, 'area_sqft'].mean():7.0f}, sd
{filled.loc[mask, 'area_sqft'].std():6.0f}")
print("SimpleImputer(add_indicator=True) adds a column:", SimpleImputer(strategy="median",
add_indicator=True).fit(num[["area_sqft"]]).get_feature_names_out().tolist())
```

OUTPUT

```
missing area: 25
median                   imputed areas: mean    1240, sd      0
KNN (5 neighbours)       imputed areas: mean    1396, sd    294
iterative (regression)   imputed areas: mean    1384, sd    310
SimpleImputer(add_indicator=True) adds a column: ['area_sqft',
'missingindicator_area_sqft']
```

The median fills every gap with one value (sd 0); KNN and iterative imputers use the other columns (bedrooms, price), so they produce realistic, varied values. In real projects compare strategies with cross-validation, and never use the target to impute features at prediction time.

Encoding categorical variables

Python

```
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, TargetEncoder
hr = pd.read_csv("hr_employees.csv")
print(hr.education.unique().tolist(), hr.department.nunique(), "departments |",
hr.job_title.nunique(), "job titles")
oh = OneHotEncoder(handle_unknown="ignore", sparse_output=False).fit(hr[["department"]])
print("one-hot:", oh.get_feature_names_out().tolist())
order = [["Diploma", "Bachelor's", "Master's", "PhD"]]
present = [e for e in order[0] if e in set(hr.education)]
oe = OrdinalEncoder(categories=[present]).fit(hr[["education"]])
print("ordinal:", dict(zip(present, oe.transform(pd.DataFrame({"education":
present})).ravel())))
y = (hr.attrition == "Yes").astype(int)
te = TargetEncoder(target_type="binary", random_state=0).fit(hr[["job_title"]], y)
print("target encoding (smoothed attrition rate per job title, first 4):",
dict(zip(te.categories_[0][:4], te.encodings_[0][:4].round(3))))
print("frequency encoding:",
hr.job_title.map(hr.job_title.value_counts(normalize=True)).round(3).head(3).tolist())
```

OUTPUT

```
["Bachelor's", "Master's", 'Diploma', 'PhD'] 6 departments | 18 job titles
one-hot: ['department_Finance', 'department_HR', 'department_IT',
'department_Operations', 'department_R&D', 'department_Sales']
ordinal: {'Diploma': np.float64(0.0), "Bachelor's": np.float64(1.0), "Master's":
np.float64(2.0), 'PhD': np.float64(3.0)}
target encoding (smoothed attrition rate per job title, first 4): {'Accountant':
np.float64(0.256), 'Analyst': np.float64(0.28), 'Data Scientist': np.float64(0.247),
'Developer': np.float64(0.295)}
frequency encoding: [0.049, 0.063, 0.063]
```

Encoding	For	Notes
One-hot	Nominal, few categories	One column per category; `handle_unknown="ignore"`; `drop="first"` for linear models if needed
Ordinal	Ordered categories (education, size S/M/L)	Give the order explicitly
Target encoding	High-cardinality nominal (job title, city, merchant)	Uses the target: must be cross-fitted (sklearn's `TargetEncoder` does this)
Frequency / count	High cardinality	Simple, leak-free
Hashing, embeddings	Very high cardinality (IDs, text)	Fixed size; embeddings in deep learning

WATCH OUT **Label encoding nominal data**

Mapping cities to 0, 1, 2... tells linear models and KNN that "Pune (2) > Delhi (1)", a false order. Use one-hot encoding for nominal variables (tree models tolerate ordinal codes better).

Scaling

Python

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
inc = hr[["monthly_income"]]
out = pd.DataFrame({"raw": inc.monthly_income})
for name, sc in [("standard", StandardScaler()), ("minmax", MinMaxScaler()), ("robust",
RobustScaler())]: out[name] = sc.fit_transform(inc).ravel()
print(out.describe().loc[["mean", "std", "min", "50%", "max"]].round(2))
```

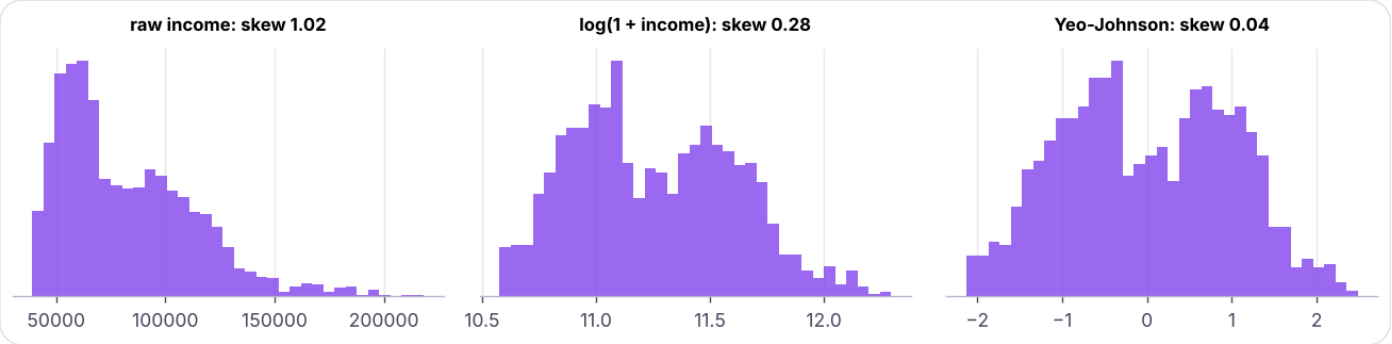
OUTPUT

```
            raw  standard  minmax  robust
mean   82610.67     -0.00    0.24    0.18
std    31145.80      1.00    0.17    0.71
min    39167.00     -1.40    0.00   -0.81
50%    74708.50     -0.25    0.20    0.00
max   218417.00      4.36    1.00    3.28
```

Scaler	Result	Use
StandardScaler	mean 0, sd 1	Default for linear models, SVM, PCA, neural networks
MinMaxScaler	range 0–1	Bounded inputs (neural networks, images)
RobustScaler	median 0, IQR 1	Data with outliers

Needs scaling: KNN, K-means, SVM, PCA, regularised linear/logistic regression, neural networks, gradient descent. **Does not:** decision trees, random forests, gradient boosting (they split on thresholds).

Transformations



Log and Yeo-Johnson transforms make right-skewed income closer to symmetric.

Log transforms help linear models with multiplicative effects (prices, incomes) and can be applied to the **target** too (`TransformedTargetRegressor`). Binning (`KBinsDiscretizer` , `pd.cut`) trades detail for robustness and interpretability.

Feature engineering

Good features encode domain knowledge the algorithm cannot easily find.

Python

```
hr["hire_date"] = pd.to_datetime(hr.hire_date)
fe = pd.DataFrame({
    "income_per_year_tenure": hr.monthly_income / (hr.years_at_company + 1),       # ratio
    "is_long_commute": (hr.distance_from_home_km > 20).astype(int),                # threshold
from domain knowledge
    "overtime_x_low_sat": ((hr.overtime == "Yes") & (hr.job_satisfaction ⩽ 2)).astype(int),   #
interaction
    "hire_month": hr.hire_date.dt.month, "hired_in_q4":
hr.hire_date.dt.quarter.eq(4).astype(int),   # date parts
    "income_vs_dept_median": hr.monthly_income /
hr.groupby("department").monthly_income.transform("median"),   # group-relative
    "title_is_manager": hr.job_title.str.contains("Manager").astype(int),          # text
pattern
})
fe["left"] = (hr.attrition == "Yes").astype(int)
print(fe.corr()["left"].drop("left").round(3).sort_values())
print(fe.groupby("overtime_x_low_sat").left.mean().round(3).to_dict())
```

OUTPUT

```
income_vs_dept_median    -0.109
title_is_manager         -0.048
hire_month                0.038
hired_in_q4               0.043
income_per_year_tenure    0.057
is_long_commute           0.094
overtime_x_low_sat        0.165
```

```
Name: left, dtype: float64
{0: 0.229, 1: 0.43}
```

Source	Feature ideas
Numbers	Ratios, differences, logs, bins, polynomial and interaction terms
Dates	Day of week, month, holiday flags, days since last event, tenure, seasonality (sin/cos)
Transactions	Recency, frequency, monetary (RFM), averages over 7/30/90 days, trends
Text	Length, keyword flags, TF-IDF, embeddings (lesson 25 and GenAI track)
Geography	Distance to centre/store, region aggregates
Groups	Value relative to group mean/median (income vs department)

Python

```python
from sklearn.preprocessing import PolynomialFeatures
pf = PolynomialFeatures(degree=2, include_bias=False).fit(pd.DataFrame({"area": [1], "age":
[1]}))
print(pf.get_feature_names_out().tolist())
```

OUTPUT

```
['area', 'age', 'area^2', 'area age', 'age^2']
```

Feature selection

More features are not always better: irrelevant ones add noise, cost and overfitting.

Python

```python
from sklearn.feature_selection import mutual_info_classif, SelectKBest, RFE
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
Xn = pd.get_dummies(hr[["age", "monthly_income", "years_at_company", "job_satisfaction",
"distance_from_home_km", "performance_rating", "overtime", "department"]],
drop_first=True).astype(float)
mi = pd.Series(mutual_info_classif(Xn, y, random_state=0),
index=Xn.columns).sort_values(ascending=False)
print("filter (mutual information):", mi.round(3).head(5).to_dict())
rfe = RFE(LogisticRegression(max_iter=2000),
n_features_to_select=4).fit(StandardScaler().fit_transform(Xn), y)
print("wrapper (RFE, logistic):", Xn.columns[rfe.support_].tolist())
rf = RandomForestClassifier(300, random_state=0, min_samples_leaf=5).fit(Xn, y)
print("embedded (random forest importance):", pd.Series(rf.feature_importances_,
Xn.columns).sort_values(ascending=False).round(3).head(5).to_dict())
```

OUTPUT

```
filter (mutual information): {'years_at_company': 0.025, 'job_satisfaction': 0.019,
'department_HR': 0.017, 'monthly_income': 0.009, 'overtime_Yes': 0.007}
wrapper (RFE, logistic): ['years_at_company', 'job_satisfaction',
'distance_from_home_km', 'overtime_Yes']
embedded (random forest importance): {'monthly_income': 0.242, 'distance_from_home_km':
0.18, 'age': 0.155, 'years_at_company': 0.117, 'job_satisfaction': 0.108}
```

The three methods disagree. That is normal: they measure different things. Random-forest (impurity) importances also favour continuous features with many distinct values, so check them with permutation importance (lesson 27).

Method	How	Pros / cons
Filter	Score each feature (correlation, chi-square, mutual information)	Fast; ignores interactions
Wrapper	Search subsets with a model (RFE, forward selection)	Strong; slow, risk of overfitting
Embedded	Selection during training (Lasso L1, tree importances)	Efficient; model-specific

Do feature selection **inside** cross-validation (in the pipeline) so the test data never influences which features are chosen.

Putting it together: a leak-free pipeline

Python

```python
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.model_selection import cross_val_score
X = hr.drop(columns=["emp_id", "attrition", "hire_date"])
pre = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), ["age",
"years_at_company", "monthly_income", "job_satisfaction", "distance_from_home_km",
"performance_rating"]),
    ("ord", OrdinalEncoder(categories=[present]), ["education"]),
    ("ohe", OneHotEncoder(handle_unknown="ignore"), ["gender", "department", "overtime",
"city"]),
    ("te", TargetEncoder(target_type="binary", random_state=0), ["job_title"]),
])
pipe = Pipeline([("pre", pre), ("sel", SelectKBest(mutual_info_classif, k=12)), ("model",
LogisticRegression(max_iter=2000))])
scores = cross_val_score(pipe, X, y, cv=5, scoring="roc_auc")
print(f"5-fold ROC-AUC {scores.mean():.3f} ± {scores.std():.3f}")
```

OUTPUT

```
5-fold ROC-AUC 0.661 ± 0.013
```

Summary

KEY TAKEAWAYS

- Impute with medians/modes, KNN or iterative imputers, and add missing-value indicators when missingness carries information.
- One-hot for nominal, ordinal encoding with an explicit order, target or frequency encoding for high-cardinality columns.
- Scale for distance- and gradient-based models (standard, min-max, robust); trees do not need scaling.
- Transform skewed variables (log, Yeo-Johnson); engineer ratios, interactions, date and group-relative features from domain knowledge.
- Select features with filter, wrapper or embedded methods, always inside the cross-validated pipeline.

Quick quiz

6 questions · tap an answer to check it

1. Which encoding suits "education level" (Diploma < Bachelor's < Master's < PhD)?

A Ordinal encoding with the order given explicitly

B One-hot only

C Target encoding only

D Leave as text

Answer: A. The categories have a natural order.

2. Which algorithm does NOT need feature scaling?

A KNN

B SVM

C Random forest

D K-means

Answer: C. Trees split on thresholds, unaffected by scale.

3. Why must target encoding be cross-fitted?

A Using a row's own target to encode it leaks the answer and overfits

B It is slow otherwise

C It needs scaling

D It only works on numbers

Answer: A. sklearn's TargetEncoder handles this internally.

4. What does RobustScaler use?

A Mean and standard deviation

B Median and interquartile range

C Min and max

D Logarithms

Answer: B. Robust to outliers.

5. Where should feature selection happen?

A On the full dataset before splitting

B Inside the pipeline, within cross-validation on training data

C On the test set

D After deployment

Answer: B. Otherwise test information leaks into the choice.

6. Which is an interaction feature?

A monthly_income

B overtime AND low job satisfaction

C log(income)

D hire_month

Answer: B. It combines two features.

EXERCISE

Practice (house_prices.csv)

(1) Engineer `price_per_sqft`, `rooms = bedrooms + bathrooms`, `is_new = age_years ⩽ 5`. Which correlate most with price? (Careful: price_per_sqft uses the target, so it cannot be a feature!) (2) Build a pipeline with KNN imputation, one-hot city, standard scaling and Ridge regression; report 5-fold R^2. (3) Try a log-transformed target with `TransformedTargetRegressor`.

Show solution (worked answer)

Python

```
from sklearn.linear_model import Ridge
from sklearn.compose import TransformedTargetRegressor
h["rooms"] = h.bedrooms + h.bathrooms; h["is_new"] = (h.age_years ⩽ 5).astype(int)
Xh = h.drop(columns=["house_id", "price_lakh"]); yh = h.price_lakh
pre_h = ColumnTransformer([("num", make_pipeline(KNNImputer(), StandardScaler()),
["area_sqft", "bedrooms", "bathrooms", "age_years", "distance_to_center_km", "parking",
"rooms", "is_new"]), ("cat", OneHotEncoder(handle_unknown="ignore"), ["city"])])
print(cross_val_score(Pipeline([("p", pre_h), ("m", Ridge())]), Xh, yh, cv=5,
scoring="r2").mean())
print(cross_val_score(Pipeline([("p", pre_h), ("m", TransformedTargetRegressor(Ridge(),
func=np.log1p, inverse_func=np.expm1))]), Xh, yh, cv=5, scoring="r2").mean())
```

INTERVIEW CORNER

Questions you may be asked

Q: How do you handle categorical variables with 1,000 levels? Target encoding (cross-fitted), frequency encoding, grouping rare levels into "Other", hashing, or learned embeddings; tree-based libraries like CatBoost handle them natively.

Q: Normalisation vs standardisation? Normalisation (min-max) rescales to 0–1; standardisation centres to mean 0 and sd 1. Use standardisation by default; min-max for bounded inputs.

Q: Give an example of a feature you engineered that helped. Use STAR: e.g. "days since last purchase" and "spend in the last 30 days vs previous 30" for churn, which raised recall in the top decile from X to Y.

STATISTICS & MACHINE LEARNING · LESSON 10

Linear Regression: Fitting, Interpreting, Assumptions and Diagnostics

Predict a number and explain it. Simple and multiple linear regression with ordinary least squares, interpreting coefficients, dummy variables and log targets, R^2, adjusted R^2, RMSE and MAE, the LINE assumptions and residual diagnostics, multicollinearity and VIF, outliers and influence, with statsmodels for inference and scikit-learn for prediction on the house-price data.

Python class 16 · ML class 4 | ✦ statsmodels 0.1x · scikit-learn 1.7+ | Intermediate | ⏱ 130 min

By the end of this lesson you can

- Fit simple and multiple linear regression with statsmodels and scikit-learn
- Interpret intercepts, slopes, dummy variables and log-transformed coefficients
- Evaluate models with R^2, adjusted R^2, MAE, RMSE and MAPE on test data
- Check linearity, independence, normality and equal variance of residuals
- Detect multicollinearity with VIF and influential points with Cook's distance

Simple linear regression

Find the straight line that best predicts *y* from *x*:

$$\hat{y} = \beta_0 + \beta_1 x$$

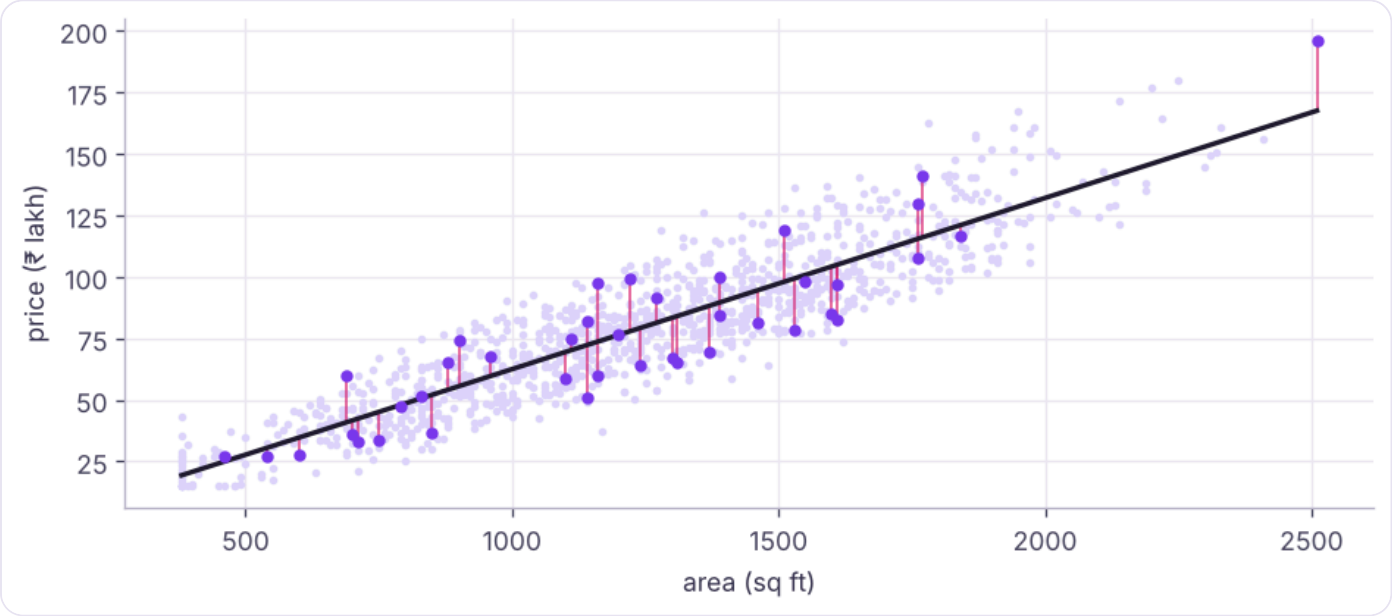
Ordinary least squares (OLS) chooses β_0 and β_1 to minimise the **sum of squared residuals**, where a residual is actual − predicted.

Python

```
import pandas as pd, numpy as np
import statsmodels.formula.api as smf
h = pd.read_csv("house_prices.csv").dropna(subset=["area_sqft"])
pps = h.price_lakh * 1e5 / h.area_sqft                          # price per sq ft
h = h[pps.between(pps.median() / 3, pps.median() * 3)]          # drop 4 impossible prices
(data-entry errors found in EDA)
print(len(h), "houses")
m1 = smf.ols("price_lakh ~ area_sqft", data=h).fit()
print(m1.params.round(4).to_dict(), "| R² =", round(m1.rsquared, 3))
b0, b1 = m1.params
print(f"price ≈ {b0:.2f} + {b1:.4f} × area → each extra 100 sq ft adds about ₹{100 * b1:.1f}
lakh")
print("prediction for 1,500 sq ft:", round(m1.predict(pd.DataFrame({"area_sqft": [1500]}))[0],
1), "lakh")
```

OUTPUT

```
1171 houses
{'Intercept': -7.0873, 'area_sqft': 0.0696} | R² = 0.827
price ≈ -7.09 + 0.0696 × area → each extra 100 sq ft adds about ₹7.0 lakh
prediction for 1,500 sq ft: 97.3 lakh
```



The least-squares line and the residuals (vertical distances) it minimises, for 40 sample houses.

Multiple linear regression

Python

```
m2 = smf.ols("price_lakh ~ area_sqft + bedrooms + bathrooms + age_years + distance_to_center_km
+ parking + C(city)", data=h).fit()
print(m2.summary().tables[1])
print(f"R² {m2.rsquared:.3f} | adjusted R² {m2.rsquared_adj:.3f} | F-test p {m2.f_pvalue:.1e} |
n = {int(m2.nobs)}")
```

OUTPUT

```
==============================================================================================
                             coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------------------
Intercept                 15.2624      1.057     14.446      0.000      13.190      17.335
C(city)[T.Chennai]       -24.2893      0.625    -38.870      0.000     -25.515     -23.063
C(city)[T.Hyderabad]     -15.6931      0.637    -24.641      0.000     -16.943     -14.444
C(city)[T.Pune]          -20.0518      0.624    -32.122      0.000     -21.277     -18.827
area_sqft                  0.0686      0.001     81.096      0.000       0.067       0.070
bedrooms                   0.9105      0.555      1.641      0.101      -0.178       1.999
bathrooms                 -0.3698      0.472     -0.783      0.434      -1.296       0.557
age_years                 -0.4218      0.026    -16.187      0.000      -0.473      -0.371
distance_to_center_km     -0.1990      0.037     -5.407      0.000      -0.271      -0.127
parking                    0.5099      0.487      1.046      0.296      -0.446       1.466
==============================================================================================
R² 0.937 | adjusted R² 0.937 | F-test p 0.0e+00 | n = 1171
```

Reading the output

Column	Meaning
`coef`	Change in predicted price for +1 unit of the feature, **holding the others constant**
`std err`	Uncertainty of the coefficient
`t`, `P>|t|`	Test of H_0: coefficient = 0
`[0.025 0.975]`	95% confidence interval
`C(city)[T.X]`	Dummy variable: difference from the **reference city** (the first alphabetically)

Python

```
coef = m2.params; ci = m2.conf_int()
for f in ["area_sqft", "age_years", "distance_to_center_km", "parking"]:
    print(f"{f:22s} {coef[f]:+8.3f} lakh per unit (95% CI {ci.loc[f, 0]:+.3f} to {ci.loc[f,
1]:+.3f}), p = {m2.pvalues[f]:.3g}")
ref = sorted(h.city.unique())[0]
print(f"city effects vs {ref}:", {k.split('T.')[1][:-1]: round(v, 1) for k, v in coef.items() if
k.startswith("C(city)")})
```

OUTPUT

```
area_sqft              +0.069 lakh per unit (95% CI +0.067 to +0.070), p = 0
age_years              -0.422 lakh per unit (95% CI -0.473 to -0.371), p = 2.69e-53
distance_to_center_km  -0.199 lakh per unit (95% CI -0.271 to -0.127), p = 7.76e-08
parking                +0.510 lakh per unit (95% CI -0.446 to +1.466), p = 0.296
city effects vs Bengaluru: {'Chennai': -24.3, 'Hyderabad': -15.7, 'Pune': -20.1}
```

Coefficients are **associations conditional on the other features**, not proof that renovating (changing age) would cause that price change.

Log target: percentage effects

Prices are right-skewed and effects are often multiplicative. Modelling log(price) turns coefficients into approximate percentage effects.

Python

```
m3 = smf.ols("np.log(price_lakh) ~ np.log(area_sqft) + bedrooms + bathrooms + age_years +
distance_to_center_km + parking + C(city)", data=h).fit()
print(f"elasticity: +1% area → +{m3.params['np.log(area_sqft)']:.2f}% price | each year of age →
{100 * (np.exp(m3.params['age_years']) - 1):+.2f}% | R² (log scale) {m3.rsquared:.3f}")
```

OUTPUT

```
elasticity: +1% area → +1.12% price | each year of age → -0.66% | R² (log scale) 0.927
```

Evaluating predictions

Python

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score,
mean_absolute_percentage_error
train, test = train_test_split(h, test_size=0.2, random_state=0)
fit = smf.ols("price_lakh ~ area_sqft + bedrooms + bathrooms + age_years + distance_to_center_km
+ parking + C(city)", data=train).fit()
p_train, p_test = fit.predict(train), fit.predict(test)
for name, yt, yp in [("train", train.price_lakh, p_train), ("test", test.price_lakh, p_test)]:
    print(f"{name:5s} R² {r2_score(yt, yp):.3f} | MAE {mean_absolute_error(yt, yp):.2f} | RMSE
{mean_squared_error(yt, yp) ** 0.5:.2f} | MAPE {100 * mean_absolute_percentage_error(yt,
yp):.1f}%")
```

OUTPUT

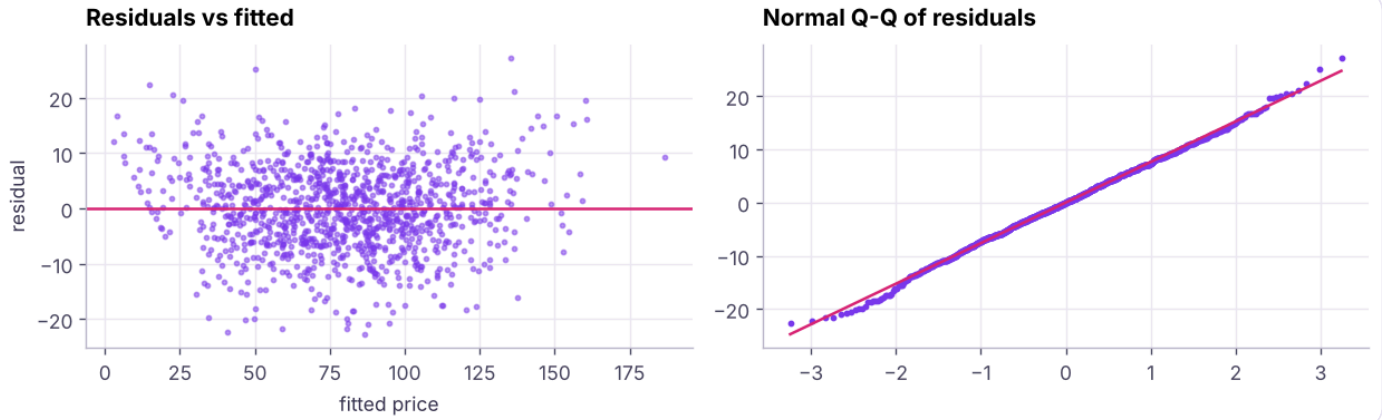
```
train R² 0.938 | MAE 5.95 | RMSE 7.51 | MAPE 9.4%
test  R² 0.936 | MAE 6.47 | RMSE 8.06 | MAPE 10.7%
```

Metric	Formula idea	Read as
MAE	mean |error|	Typical error in target units; robust
RMSE	√mean(error²)	Penalises large errors more
MAPE	mean |error| / actual	Percentage error; unstable near 0
R^2	1 − SSE/SST	Share of variance explained (0–1 on test; can be negative)
Adjusted R^2	R^2 penalised for number of features	Compare models with different feature counts

Similar train and test scores mean the model generalises; a large gap signals overfitting.

Assumptions (LINE) and diagnostics

Assumption	Meaning	Check	If violated
Linearity	Mean of y is linear in features	Residuals vs fitted: no curve	Transform, add polynomial terms, use trees
Independence	Errors not related to each other	Durbin-Watson ≈ 2; study design	Time-series models, clustered errors
Normality	Residuals ~ normal	Q-Q plot, histogram	Matters for small-sample inference; transform y
Equal variance	Constant spread of residuals	Residuals vs fitted: no funnel; Breusch-Pagan	log(y), robust (HC3) standard errors



Residual diagnostics for the multiple regression. Left: residuals vs fitted values. Right: Q-Q plot of residuals.

Python

```
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.stattools import durbin_watson
bp = het_breuschpagan(m2.resid, m2.model.exog)
print(f"Breusch-Pagan p = {bp[1]:.2e} ({'unequal variance (heteroscedastic)' if bp[1] < 0.05
else 'equal variance OK'}) | Durbin-Watson {durbin_watson(m2.resid):.2f} | residual skew
{stats.skew(m2.resid):.2f}")
robust = smf.ols(m2.model.formula, data=h).fit(cov_type="HC3")
print("area_sqft p-value: ordinary", f"{m2.pvalues['area_sqft']:.2e}", "| robust HC3", f"
{robust.pvalues['area_sqft']:.2e}")
```

OUTPUT

```
Breusch-Pagan p = 1.05e-01 (equal variance OK) | Durbin-Watson 1.99 | residual skew -0.05
area_sqft p-value: ordinary 0.00e+00 | robust HC3 0.00e+00
```

Multicollinearity

When features are strongly correlated with each other, coefficients become unstable and hard to interpret (predictions can still be fine).

Python

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
Xv = h[["area_sqft", "bedrooms", "bathrooms", "age_years", "distance_to_center_km",
"parking"]].assign(const=1.0)
vif = pd.Series([variance_inflation_factor(Xv.values, i) for i in range(Xv.shape[1] - 1)],
index=Xv.columns[:-1]).round(2)
print(vif.sort_values(ascending=False).to_dict())
print(h[["area_sqft", "bedrooms", "bathrooms"]].corr().round(2))
```

OUTPUT

```
{'bedrooms': 5.53, 'bathrooms': 4.09, 'area_sqft': 2.26, 'age_years': 1.01,
'distance_to_center_km': 1.01, 'parking': 1.0}
            area_sqft  bedrooms  bathrooms
area_sqft        1.00      0.74       0.63
bedrooms         0.74      1.00       0.87
bathrooms        0.63      0.87       1.00
```

VIF above about 5–10 signals a problem. Fixes: drop or combine correlated features, use regularisation (Ridge, lesson 12), or PCA.

Outliers and influential points

Python

```
infl = m2.get_influence(); cooks = infl.cooks_distance[0]
top = np.argsort(cooks)[-3:][::-1]
print("largest Cook's distances:", cooks[top].round(3), "| threshold 4/n =", round(4 / len(h),
4))
print(h.iloc[top][["city", "area_sqft", "bedrooms", "price_lakh"]])
```

OUTPUT

```
largest Cook's distances: [0.011 0.009 0.009] | threshold 4/n = 0.0034
          city  area_sqft  bedrooms  price_lakh
464  Bengaluru     1780.0         3       162.4
113  Hyderabad     1430.0         2        66.8
863       Pune      470.0         1        37.3
```

Investigate influential points: data errors should be fixed; genuine extremes may need robust methods (Huber regression) or separate treatment.

scikit-learn version

Python

```
from sklearn.linear_model import LinearRegression
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import make_pipeline
feats = ["area_sqft", "bedrooms", "bathrooms", "age_years", "distance_to_center_km", "parking",
"city"]
pipe = make_pipeline(make_column_transformer((OneHotEncoder(drop="first"), ["city"]),
remainder="passthrough"), LinearRegression())
pipe.fit(train[feats], train.price_lakh)
print(f"sklearn test R² {pipe.score(test[feats], test.price_lakh):.3f} (same model as
statsmodels: {np.isclose(pipe.score(test[feats], test.price_lakh), r2_score(test.price_lakh,
p_test))})")
```

OUTPUT

```
sklearn test R² 0.936 (same model as statsmodels: True)
```

Use **statsmodels** for inference (p-values, CIs, diagnostics) and **scikit-learn** for prediction pipelines, cross-validation and deployment.

Summary

KEY TAKEAWAYS

- OLS minimises squared residuals; coefficients are the change in y per unit of x, holding other features constant.
- Dummy variables compare categories with a reference; log targets give percentage effects.
- Evaluate on test data with MAE, RMSE, MAPE and R^2; adjusted R^2 penalises extra features.
- Check LINE assumptions with residual plots, Q-Q plots, Breusch-Pagan and Durbin-Watson; use robust standard errors or transformations when needed.
- VIF detects multicollinearity; Cook's distance finds influential points. Coefficients show association, not causation.

Quick quiz

6 questions · tap an answer to check it

1. What does OLS minimise?

- **A** The sum of absolute errors
- **B** The sum of squared residuals
- **C** The number of features
- **D** The p-values

Answer: B. Hence "least squares".

2. The coefficient of `age_years` is −0.6. Interpretation?

- **A** Each additional year of age is associated with ₹0.6 lakh lower price, holding other features constant
- **B** Older houses cost 60% less
- **C** Age causes prices to fall
- **D** Age is not significant

Answer: A. Conditional association, in target units.

3. Residuals fan out as fitted values grow. Which assumption is violated?

- **A** Linearity
- **B** Independence
- **C** Equal variance (homoscedasticity)
- **D** Normality only

Answer: C. Try log(y) or robust standard errors.

4. What does a VIF of 12 indicate?

- **A** Strong multicollinearity with other features
- **B** A great feature
- **C** Non-normal residuals
- **D** An outlier

Answer: A. Coefficients become unstable.

5. Test R^2 is 0.90 and train R^2 is 0.91. What does that suggest?

- **A** The model generalises well (no strong overfitting)
- **B** Severe overfitting
- **C** Data leakage for sure
- **D** Underfitting

Answer: A. Similar scores on unseen data are good news.

6. Why use a log target for prices?

- **A** Prices are skewed and effects are multiplicative; coefficients become percentage effects
- **B** To make R^2 larger always
- **C** Because statsmodels requires it
- **D** To remove outliers

Answer: A. It also stabilises variance.

EXERCISE **Practice**

(1) Add `area_sqft:C(city)` interaction: does the price per sq ft differ by city? (2) Compare test MAE of the plain and log-target models (back-transform with `np.exp`). (3) Remove the feature with the highest VIF and compare adjusted R^2. (4) Plot actual vs predicted on the test set.

Show solution (worked answer)

Python

```
mi = smf.ols("price_lakh ~ area_sqft * C(city) + bedrooms + bathrooms + age_years +
distance_to_center_km + parking", data=h).fit()
print(mi.params.filter(like="area_sqft:"))
lg = smf.ols("np.log(price_lakh) ~ area_sqft + bedrooms + bathrooms + age_years +
distance_to_center_km + parking + C(city)", data=train).fit()
print(mean_absolute_error(test.price_lakh, np.exp(lg.predict(test))),
mean_absolute_error(test.price_lakh, p_test))
m_novif = smf.ols("price_lakh ~ area_sqft + bathrooms + age_years + distance_to_center_km +
parking + C(city)", data=h).fit(); print(m_novif.rsquared_adj, m2.rsquared_adj)
plt.scatter(test.price_lakh, p_test, s=6); plt.plot([0, 300], [0, 300])
```

INTERVIEW CORNER **Questions you may be asked**

Q: What are the assumptions of linear regression? Linearity, independence of errors, normality of residuals (for inference), equal variance of residuals, and no perfect multicollinearity.

Q: R^2 vs adjusted R^2? R^2 never decreases when you add features; adjusted R^2 penalises the number of features, so it can drop when a feature adds little.

Q: How do you interpret a coefficient in a log-log model? As an elasticity: a 1% change in x is associated with a β% change in y.

STATISTICS & MACHINE LEARNING · LESSON 11

Gradient Descent: How Models Learn (Built from Scratch in NumPy)

Understand the optimisation engine behind linear regression, logistic regression and neural networks. Cost functions, gradients and the update rule, implementing batch gradient descent for linear regression in NumPy, the learning rate, feature scaling, stochastic and mini-batch variants, convergence checks, and comparison with the closed-form OLS solution and scikit-learn's SGDRegressor.

ML class 5 · NumPy 2 · scikit-learn 1.7+ · Intermediate · 110 min

By the end of this lesson you can

- Define a cost function (MSE) and explain what its gradient tells us
- Implement batch gradient descent for linear regression with NumPy
- Choose a learning rate and diagnose divergence and slow convergence with cost curves
- Explain why feature scaling speeds up gradient descent
- Compare batch, stochastic and mini-batch gradient descent and use SGDRegressor

Learning = minimising a cost

A model has **parameters** (weights). Learning means finding the weights that make a **cost function** as small as possible. For linear regression the usual cost is the **mean squared error**:

$$J(\mathbf{w}, b) = \frac{1}{2n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2, \qquad \hat{y}_i = \mathbf{w} \cdot \mathbf{x}_i + b$$

(The ½ simply makes the derivative tidier.) OLS has a formula for the minimum, but most models (logistic regression, neural networks) do not, so we **search** for it.

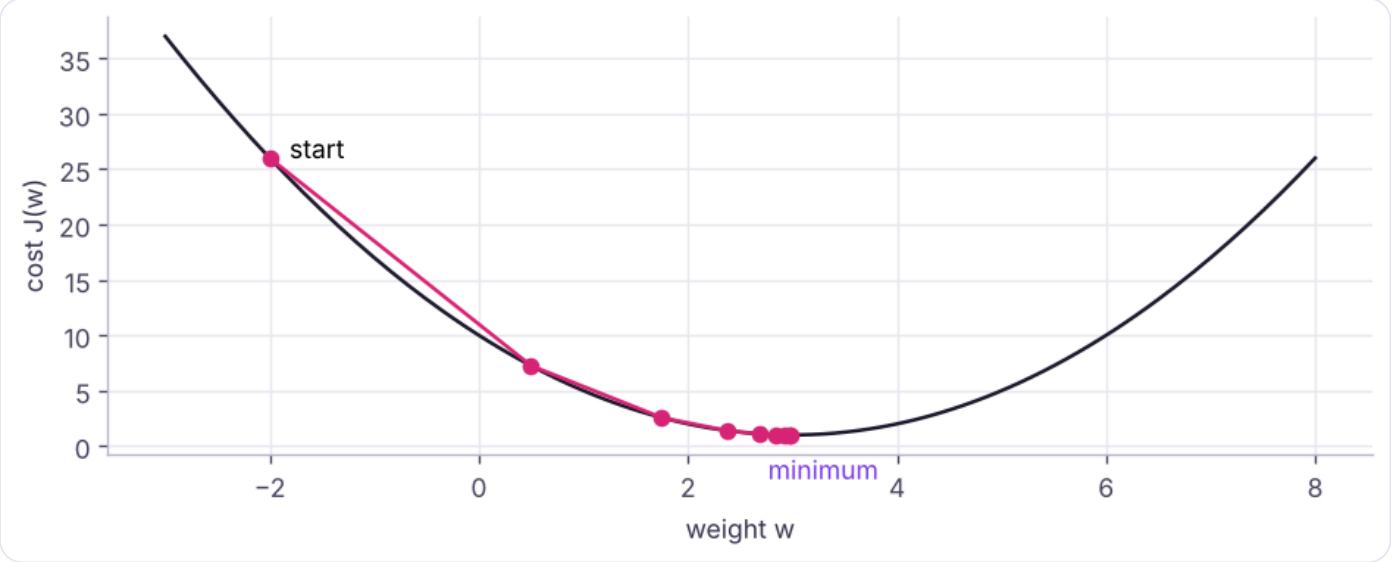
The idea: walk downhill

Imagine standing on a foggy hillside: feel the slope under your feet and step downhill; repeat until the ground is flat. The **gradient** is the slope; we step in the opposite direction.

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w}, \qquad b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

α is the **learning rate** (step size). For linear regression the gradients are:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{n} X^T(\hat{\mathbf{y}} - \mathbf{y}) \qquad \frac{\partial J}{\partial b} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)$$



Gradient descent on a one-parameter cost curve: each step moves against the slope; steps shrink as the slope flattens near the minimum.

Implementation in NumPy

Python

```
import pandas as pd
h = pd.read_csv("house_prices.csv").dropna(subset=["area_sqft"])
pps = h.price_lakh * 1e5 / h.area_sqft; h = h[pps.between(pps.median() / 3, pps.median() * 3)]
feats = ["area_sqft", "bedrooms", "bathrooms", "age_years", "distance_to_center_km"]
X_raw = h[feats].to_numpy(float); y = h.price_lakh.to_numpy(float)
mu, sd = X_raw.mean(axis=0), X_raw.std(axis=0); X = (X_raw - mu) / sd          # standardise
features

def gradient_descent(X, y, lr=0.1, epochs=300):
    n, p = X.shape; w, b = np.zeros(p), 0.0; history = []
    for _ in range(epochs):
        y_hat = X @ w + b
        err = y_hat - y
        history.append((err ** 2).mean() / 2)                                  # cost J
        w -= lr * (X.T @ err) / n                                              # gradient step
for weights
        b -= lr * err.mean()                                                   # gradient step
for bias
    return w, b, history

w, b, hist = gradient_descent(X, y)
print("cost: start", round(hist[0], 1), "→ after 10", round(hist[9], 1), "→ end",
round(hist[-1], 2))
print("weights (per 1 sd of each feature):", dict(zip(feats, w.round(2).tolist())), "| bias",
round(b, 2))
```

OUTPUT

```
cost: start 3604.0 → after 10 592.8 → end 72.59
weights (per 1 sd of each feature): {'area_sqft': 26.94, 'bedrooms': 1.29, 'bathrooms':
-0.48, 'age_years': -3.81, 'distance_to_center_km': -1.12} | bias 79.25
```

Check against the exact OLS solution

Python

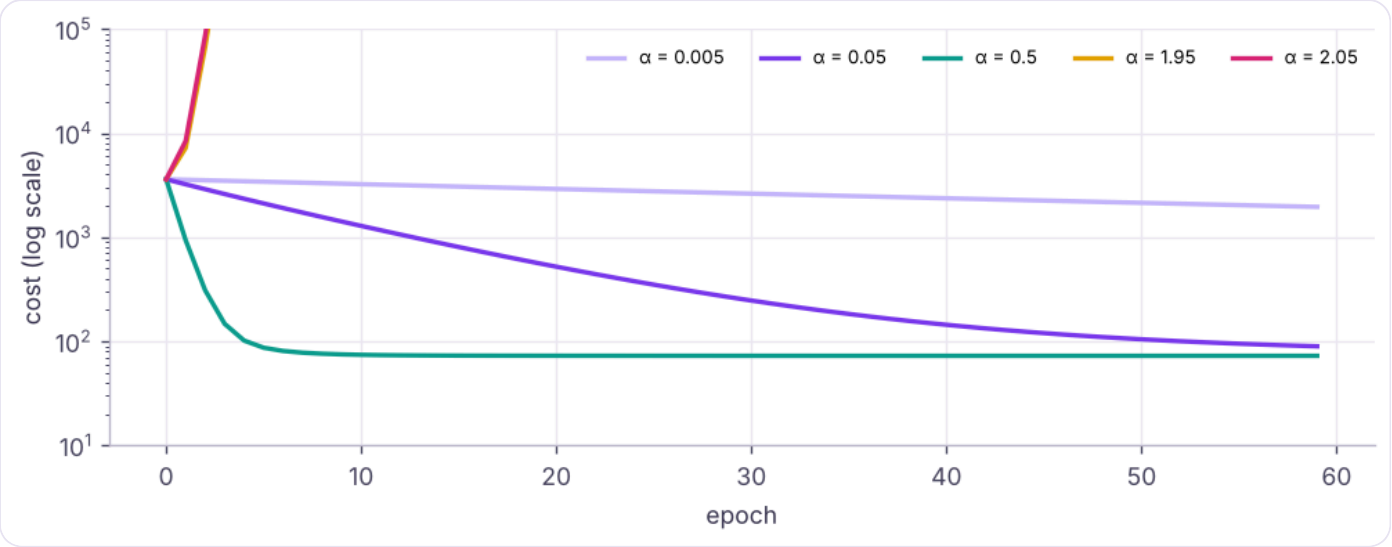
```
X1 = np.c_[np.ones(len(X)), X]
theta = np.linalg.lstsq(X1, y, rcond=None)[0]                                  # closed form
(normal equation)
print("OLS bias", theta[0].round(2), "weights", theta[1:].round(2))
print("max difference from gradient descent:", np.abs(theta[1:] - w).max().round(4))
w_orig = w / sd; b_orig = b - (w_orig * mu).sum()                              # convert back to
original units
print(f"per sq ft: {w_orig[0]:.4f} lakh (≈ ₹{w_orig[0] * 1e5:,.0f}) | per year of age:
{w_orig[3]:.3f} lakh")
```

OUTPUT

```
OLS bias 79.25 weights [26.98  1.16 -0.38 -3.81 -1.12]
max difference from gradient descent: 0.1303
per sq ft: 0.0677 lakh (≈ ₹6,769) | per year of age: -0.442 lakh
```

Gradient descent reached the same answer as the exact formula. The **normal equation** $\theta = (X^TX)^{-1}X^Ty$ is fine for a few thousand features, but gradient descent scales to millions of rows and parameters and to models without closed-form solutions.

The learning rate



Cost curves for different learning rates: too small is slow, a good rate converges quickly, too large diverges.

Symptom	Cause	Fix
Cost falls very slowly	α too small	Increase α (×3 steps: 0.001, 0.003, 0.01…)
Cost goes up or oscillates, becomes NaN	α too large	Decrease α
Cost flattens high	Stuck / underfitting / unscaled features	Scale features, more epochs, better model

Plot the cost every time: it should go down and flatten.

Why scale features?

Python

```
best = ((X1 @ theta - y) ** 2).mean() / 2                                    # the lowest
possible cost (OLS)
with np.errstate(all="ignore"):
    for name, Xs, lr in [("raw features", X_raw, 1e-7), ("raw features", X_raw, 1e-6),
("standardised", X, 0.1)]:
        _, _, hh = gradient_descent(Xs, y, lr=lr, epochs=300)
        gap = "diverged" if not np.isfinite(hh[-1]) or hh[-1] > 1e6 else f"{100 * (hh[-1] / best
- 1):.1f}% above the optimum"
        print(f"{name:13s} lr={lr:<6} cost after 300 epochs: {gap}")
```

OUTPUT

```
raw features  lr=1e-07  cost after 300 epochs: 14.0% above the optimum
raw features  lr=1e-06  cost after 300 epochs: 13.3% above the optimum
standardised  lr=0.1    cost after 300 epochs: 0.0% above the optimum
```

With raw features, area (hundreds to thousands) and bedrooms (1–5) differ hugely in scale, so the cost surface is a long narrow valley: a learning rate small enough for area is far too small for bedrooms. Standardising makes the valley round, so one learning rate suits all weights.

Batch, stochastic and mini-batch

Variant	Gradient from	Pros	Cons
Batch GD	All n rows each step	Smooth, exact	Slow for big data
Stochastic GD (SGD)	One random row	Fast updates, escapes shallow minima	Noisy
Mini-batch GD	32–1,024 rows	Best of both; GPU friendly	Batch size to tune

Python

```
def minibatch_gd(X, y, lr=0.05, epochs=20, batch=32, seed=0):
    rng = np.random.default_rng(seed); n, p = X.shape; w, b = np.zeros(p), 0.0; costs = []
    for _ in range(epochs):
        idx = rng.permutation(n)
        for start in range(0, n, batch):
            j = idx[start:start + batch]; err = X[j] @ w + b - y[j]
            w -= lr * X[j].T @ err / len(j); b -= lr * err.mean()
        costs.append(((X @ w + b - y) ** 2).mean() / 2)
    return w, b, costs
wm, bm, cm = minibatch_gd(X, y)
print("mini-batch cost per epoch:", [round(float(c), 1) for c in cm[:5]], "...",
round(float(cm[-1]), 2), "| weights close to OLS:", np.allclose(wm, theta[1:], atol=0.5))
```

OUTPUT

```
mini-batch cost per epoch: [170.3, 79.1, 74.1, 73.5, 73.0] ... 72.62 | weights close to
OLS: True
```

Deep learning uses mini-batch gradient descent with improved optimisers: **momentum** (keeps velocity), **RMSProp** and **Adam** (adaptive learning rates per parameter), plus learning-rate schedules.

scikit-learn: SGDRegressor

Python

```python
from sklearn.linear_model import SGDRegressor, LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
Xtr, Xte, ytr, yte = train_test_split(h[feats], y, test_size=0.2, random_state=0)
sgd = make_pipeline(StandardScaler(), SGDRegressor(learning_rate="invscaling", eta0=0.01,
max_iter=2000, tol=1e-6, random_state=0)).fit(Xtr, ytr)
ols = make_pipeline(StandardScaler(), LinearRegression()).fit(Xtr, ytr)
print(f"test R²: SGDRegressor {sgd.score(Xte, yte):.4f} | LinearRegression {ols.score(Xte,
yte):.4f} | SGD epochs used {sgd[-1].n_iter_}")
```

OUTPUT

```
test R²: SGDRegressor 0.8513 | LinearRegression 0.8514 | SGD epochs used 23
```

`SGDRegressor` / `SGDClassifier` also support `partial_fit` for data that arrives in streams or does not fit in memory, and penalties (L2, L1, elastic net) for regularisation.

Where gradient descent appears

Logistic regression (minimising log loss, lesson 13), neural networks and LLMs (backpropagation computes gradients for billions of weights, lesson 26), gradient boosting (fits each new tree to the gradient of the loss, lesson 19), and matrix factorisation in recommenders.

Summary

KEY TAKEAWAYS

- Learning = minimising a cost function; gradient descent repeatedly steps against the gradient: $w \leftarrow w - \alpha \cdot \partial J/\partial w$.
- For linear regression: gradient = $X^T(\hat{y} - y)/n$; it converges to the same solution as OLS.
- The learning rate controls step size: too small is slow, too large diverges; always plot the cost.
- Scale features so one learning rate suits all weights.
- Batch, stochastic and mini-batch variants trade accuracy of each step for speed; Adam and momentum power deep learning.

Quick quiz

6 questions · tap an answer to check it

1. In the update $w \leftarrow w - \alpha \cdot \partial J/\partial w$, what is α?

A The learning rate (step size)

B The gradient

C The cost

D The bias

Answer: A. It is a hyperparameter you choose.

2. The cost increases every epoch and becomes NaN. Most likely cause?

A Learning rate too small

B Learning rate too large

C Too much data

D Features are scaled

Answer: B. Steps overshoot the minimum.

3. Why standardise features before gradient descent?

A Features on very different scales create a narrow valley that slows convergence

B To change the final OLS solution

C To remove outliers

D It is required by NumPy

Answer: A. Scaling makes one learning rate work for all weights.

4. What does mini-batch gradient descent use for each update?

A All rows

B One row

C A small random subset of rows

D Only the test set

Answer: C. Common sizes: 32–512.

5. Gradient descent and the normal equation on the same linear regression should give...

A Approximately the same weights

B Completely different weights

C The same only without scaling

D No relation

Answer: A. Both minimise the same convex cost.

6. Which algorithm fits each new tree to the gradient of the loss?

- **A** K-means
- **B** Gradient boosting
- **C** KNN
- **D** PCA

Answer: B. Hence the name.

EXERCISE **Practice**

(1) Add an early-stopping rule: stop when the cost improves by less than 0.001. How many epochs are needed at α = 0.1? (2) Implement gradient descent for a model with only `area_sqft` and plot the fitted line. (3) Try mini-batch sizes 8, 64 and 512 and compare the final cost after 10 epochs.

Show solution (worked answer)

Python

```
def gd_early(X, y, lr=0.1, tol=1e-3, max_epochs=10_000):
    n, p = X.shape; w, b = np.zeros(p), 0.0; prev = np.inf
    for e in range(max_epochs):
        err = X @ w + b - y; cost = (err ** 2).mean() / 2
        if prev - cost < tol: return w, b, e
        prev = cost; w -= lr * X.T @ err / n; b -= lr * err.mean()
    return w, b, max_epochs
print(gd_early(X, y)[2])
w1, b1, _ = gradient_descent(X[:, [0]], y)
for bs in (8, 64, 512): print(bs, minibatch_gd(X, y, batch=bs, epochs=10)[2][-1])
```

INTERVIEW CORNER

Questions you may be asked

Q: Explain gradient descent intuitively. Start with random weights, compute how wrong the model is, find the direction that reduces the error fastest (the negative gradient), take a small step, and repeat until the error stops falling.

Q: Batch vs stochastic gradient descent? Batch uses all data per step (stable, slow); stochastic uses one example (fast, noisy); mini-batch is the practical compromise.

Q: Can gradient descent get stuck? For convex costs like linear and logistic regression there is one global minimum. For neural networks it can land in local minima or saddle points; momentum, Adam and random restarts help.

STATISTICS & MACHINE LEARNING · LESSON 12

Overfitting, Polynomial Regression and Regularisation (Ridge, Lasso, Elastic Net)

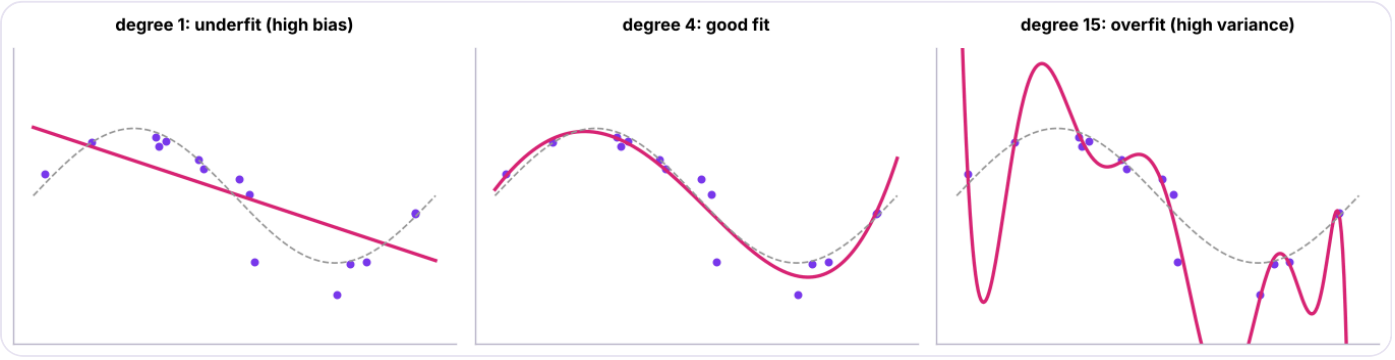
See overfitting and underfitting happen, learn the bias-variance trade-off with polynomial regression, and control model complexity with L2 (Ridge), L1 (Lasso) and Elastic Net penalties, including choosing the penalty strength with cross-validation, feature selection with Lasso, and learning curves.

ML class 6 · scikit-learn 1.7+ · Intermediate · 110 min

By the end of this lesson you can

- Recognise underfitting and overfitting from train vs test errors and plots
- Explain the bias-variance trade-off and how model complexity affects it
- Fit polynomial regression and control its degree
- Apply Ridge, Lasso and Elastic Net and tune alpha with RidgeCV/LassoCV
- Use learning and validation curves to decide between more data and a different model

Underfitting and overfitting



Polynomial models of degree 1, 4 and 15 on the same noisy data. Degree 1 underfits, degree 4 fits well, degree 15 chases noise.

	Underfitting	Good fit	Overfitting
Training error	High	Low	Very low
Test error	High	Low	High
Cause	Model too simple, missing features		Model too complex, too little data, leakage
Fix	More features, more flexible model, less regularisation		More data, simpler model, regularisation, feature selection, early stopping

The bias-variance trade-off

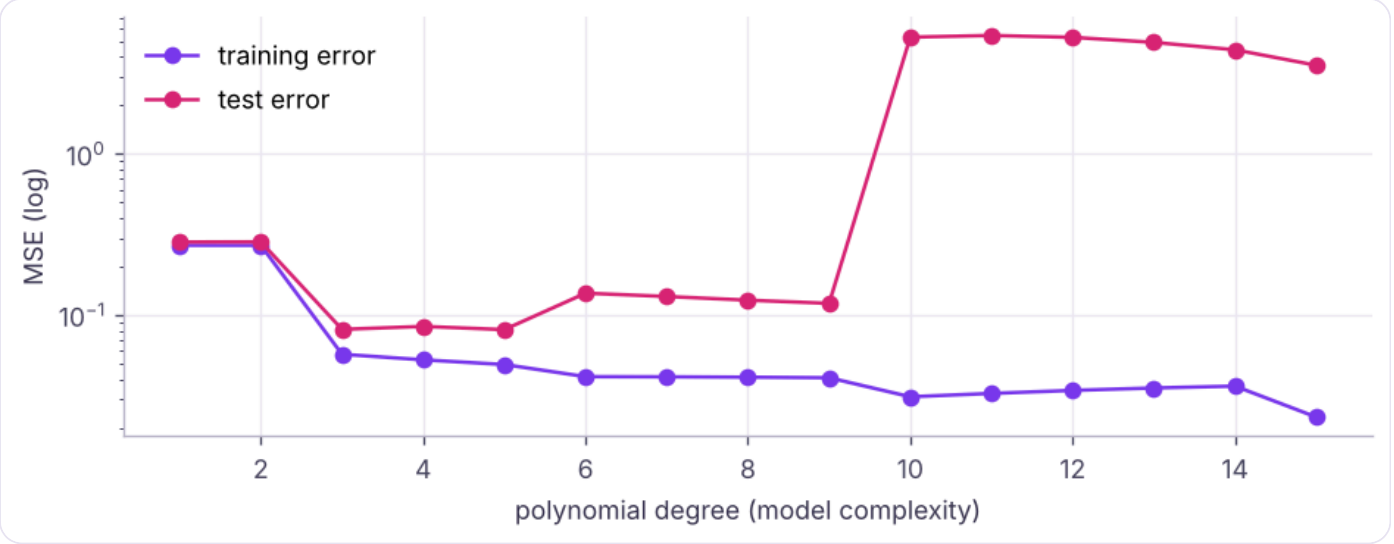
Bias is the error from wrong assumptions (a line for a curve). **Variance** is the error from sensitivity to the particular training sample. Total expected error = bias² + variance + irreducible noise.

Python

```
from sklearn.metrics import mean_squared_error
x_test = rng.uniform(0, 1, 500); y_test = np.sin(2 * np.pi * x_test) + rng.normal(0, .25, 500)
for d in (1, 2, 3, 4, 6, 9, 12, 15):
    m = make_pipeline(PolynomialFeatures(d), StandardScaler(), LinearRegression()).fit(x[:, None], y)
    tr = mean_squared_error(y, m.predict(x[:, None])); te = mean_squared_error(y_test, m.predict(x_test[:, None]))
    print(f"degree {d:2d}: train MSE {tr:7.3f} | test MSE {te:10.3f}")
```

OUTPUT

```
degree  1: train MSE   0.271 | test MSE      0.284
degree  2: train MSE   0.271 | test MSE      0.285
degree  3: train MSE   0.057 | test MSE      0.082
degree  4: train MSE   0.053 | test MSE      0.085
degree  6: train MSE   0.042 | test MSE      0.137
degree  9: train MSE   0.041 | test MSE      0.118
degree 12: train MSE   0.034 | test MSE      5.285
degree 15: train MSE   0.023 | test MSE      3.548
```



As complexity grows, training error keeps falling while test error falls then rises: the sweet spot is in between.

Regularisation

Instead of choosing fewer features by hand, add a **penalty** on large coefficients to the cost:

$$\text{Ridge (L2): } J = \text{MSE} + \alpha \sum_j w_j^2 \qquad \text{Lasso (L1): } J = \text{MSE} + \alpha \sum_j |w_j|$$

Elastic Net mixes both (`l1_ratio`). Larger α means stronger shrinkage, simpler models, more bias and less variance. Always **scale features** first, so the penalty treats them fairly.

Python

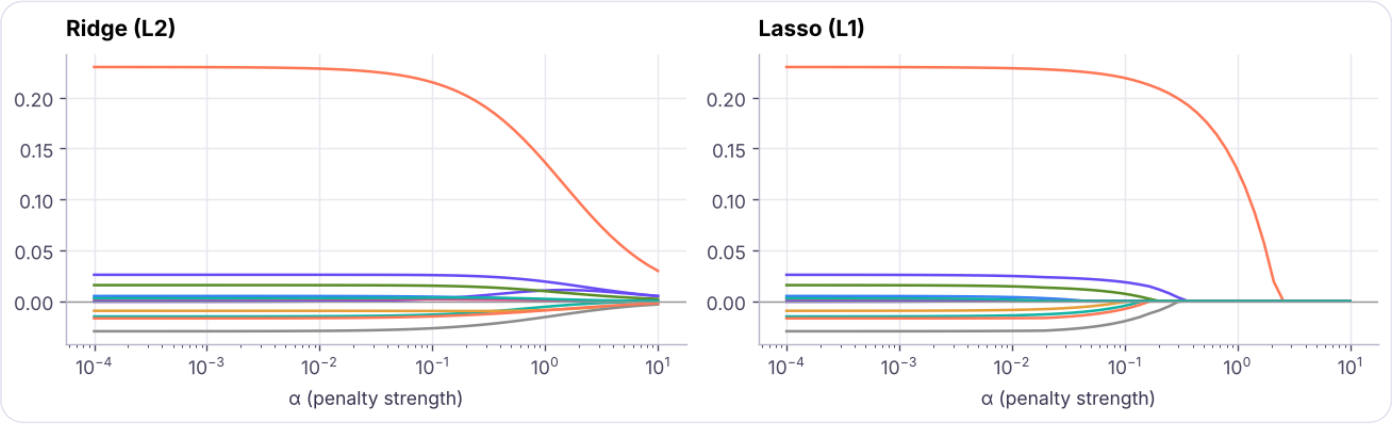
```
for name, model in [("no penalty", LinearRegression()), ("Ridge α=0.01", Ridge(alpha=0.01)),
("Lasso α=0.001", Lasso(alpha=0.001, max_iter=100000))]:
    m = make_pipeline(PolynomialFeatures(15), StandardScaler(), model).fit(x[:, None], y)
    coefs = m[-1].coef_
    print(f"degree-15 {name:12s} test MSE {mean_squared_error(y_test, m.predict(x_test[:,
None])):9.3f} | largest |coef| {np.abs(coefs).max():12.1f} | zero coefs {(np.abs(coefs) < 1e-
8).sum():2d} of {len(coefs)}")
```

OUTPUT

```
degree-15 no penalty   test MSE     3.548 | largest |coef|       9560.9 | zero coefs  0
of 16
degree-15 Ridge α=0.01 test MSE     0.113 | largest |coef|          2.2 | zero coefs  1
of 16
degree-15 Lasso α=0.001 test MSE     0.089 | largest |coef|          4.1 | zero coefs 12
of 16
```

The same degree-15 model becomes well behaved once its coefficients are penalised. **Lasso** sets some coefficients exactly to zero (automatic feature selection); **Ridge** shrinks all of them smoothly and handles

correlated features well.



Coefficient paths: as α grows, Ridge shrinks every coefficient gradually while Lasso drops features to exactly zero one by one.

Choosing α with cross-validation

Python

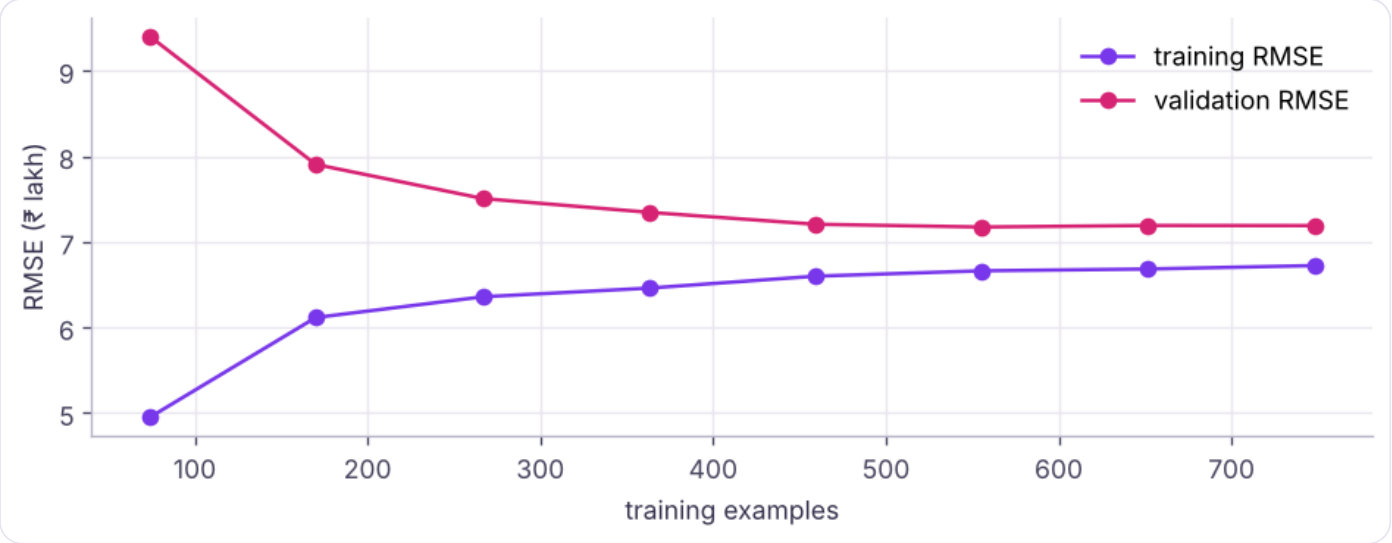
```
from sklearn.model_selection import train_test_split, cross_val_score
h = pd.read_csv("house_prices.csv").dropna(subset=["area_sqft"])
pps = h.price_lakh * 1e5 / h.area_sqft; h = h[pps.between(pps.median() / 3, pps.median() * 3)]
X = pd.get_dummies(h.drop(columns=["house_id", "price_lakh"]), columns=["city"],
drop_first=True).astype(float); yv = h.price_lakh
Xtr, Xte, ytr, yte = train_test_split(X, yv, test_size=0.2, random_state=0)
poly = lambda est: make_pipeline(PolynomialFeatures(2, include_bias=False), StandardScaler(),
est)
models = {"linear (degree 2)": poly(LinearRegression()), "RidgeCV":
poly(RidgeCV(alphas=np.logspace(-3, 3, 30))),
          "LassoCV": poly(LassoCV(cv=5, random_state=0, max_iter=50000)), "ElasticNet":
poly(ElasticNet(alpha=0.05, l1_ratio=0.5, max_iter=50000))}
for name, m in models.items():
    m.fit(Xtr, ytr); cv = cross_val_score(m, Xtr, ytr, cv=5,
scoring="neg_root_mean_squared_error").mean()
    extra = f" | α = {m[-1].alpha_:.4g}" if hasattr(m[-1], "alpha_") else ""
    nz = f" | non-zero coefs {(m[-1].coef_ ≠ 0).sum()} of {len(m[-1].coef_)}"
    print(f"{name:18s} CV RMSE {-cv:6.2f} | test RMSE {mean_squared_error(yte, m.predict(Xte))
** .5:6.2f}{extra}{nz}")
```

OUTPUT

```
linear (degree 2)  CV RMSE   7.21 | test RMSE   7.72 | non-zero coefs 51 of 54
RidgeCV            CV RMSE   7.20 | test RMSE   7.75 | α = 0.788 | non-zero coefs 53 of
54
LassoCV            CV RMSE   7.08 | test RMSE   7.64 | α = 0.07792 | non-zero coefs 25 of
54
ElasticNet         CV RMSE   7.41 | test RMSE   8.05 | non-zero coefs 45 of 54
```

With about 900 training rows and 54 polynomial features, all models perform similarly here; Lasso reaches the same accuracy with far fewer terms. Regularisation matters most when features are many relative to rows, or strongly correlated.

Learning curves: more data or a better model?



Learning curves for a degree-2 Ridge model on house prices: the training and validation errors converge as data grows, so more data would help little; a more flexible model or better features would help more.

Learning-curve pattern	Diagnosis	Action
Both errors high and close	High bias (underfitting)	More features, more complex model
Big gap: low train, high validation	High variance (overfitting)	More data, regularisation, simpler model
Gap closing as data grows	Variance falling	Collect more data

Regularisation beyond linear models

Model	Regularisation knobs
Logistic regression	`C` = 1/α (smaller C = stronger penalty), `penalty="l1"/"l2"/"elasticnet"`
Decision trees	`max_depth`, `min_samples_leaf`, pruning (`ccp_alpha`)
Random forests / boosting	Tree depth, number of trees, learning rate, subsampling
Neural networks	Weight decay (L2), dropout, early stopping, data augmentation
KNN	Larger k = smoother

Summary

KEY TAKEAWAYS

- Underfitting: high error everywhere (bias). Overfitting: low training error, high test error (variance).
- Polynomial features add flexibility; the degree controls the bias-variance trade-off.
- Ridge (L2) shrinks coefficients; Lasso (L1) zeroes some (feature selection); Elastic Net combines them. Scale features first.
- Pick α by cross-validation (RidgeCV, LassoCV, GridSearchCV).
- Learning curves tell you whether more data or a different model will help.

Quick quiz

6 questions · tap an answer to check it

1. Training error 2%, test error 25%. Diagnosis?

A Underfitting

B Overfitting (high variance)

C Perfect model

D Data too large

Answer: B. The model memorised the training data.

2. Which penalty can set coefficients exactly to zero?

A Ridge (L2)

B Lasso (L1)

C No penalty

D Both equally

Answer: B. L1 performs feature selection.

3. What happens as α increases in Ridge regression?

- **A** Coefficients shrink, bias rises and variance falls
- **B** Coefficients grow
- **C** The model overfits more
- **D** Nothing

Answer: A. Stronger penalty = simpler model.

4. Why scale features before regularisation?

- **A** So the penalty treats all coefficients fairly regardless of units
- **B** To speed up printing
- **C** To remove outliers
- **D** It is optional

Answer: A. Otherwise large-unit features are under-penalised.

5. In logistic regression in scikit-learn, a smaller C means...

- **A** Stronger regularisation
- **B** Weaker regularisation
- **C** More features
- **D** No effect

Answer: A. C is the inverse of α.

6. Learning curves show training and validation errors both high and close together. What helps?

A More data

B A more flexible model or better features

C Stronger regularisation

D Fewer features

Answer: B. This is high bias.

EXERCISE **Practice**

(1) Plot validation curves (`validation_curve`) of Ridge alpha for the house data. (2) Fit `LogisticRegressionCV` with L1 penalty on the HR attrition data: which features survive? (3) Compare degree 1, 2 and 3 polynomial Ridge models with 5-fold CV.

Show solution (worked answer)

Python

```
from sklearn.model_selection import validation_curve
from sklearn.linear_model import LogisticRegressionCV
al = np.logspace(-3, 3, 13)
tr_, va_ = validation_curve(poly(Ridge()), Xtr, ytr, param_name="ridge__alpha",
param_range=al, cv=5, scoring="neg_root_mean_squared_error")
yb = (hr.attrition == "Yes").astype(int)
lr1 = LogisticRegressionCV(penalty="l1", solver="saga", Cs=10, cv=5,
max_iter=5000).fit(StandardScaler().fit_transform(Xh), yb)
print(Xh.columns[lr1.coef_[0] != 0].tolist())
for d in (1, 2, 3): print(d, cross_val_score(make_pipeline(PolynomialFeatures(d,
include_bias=False), StandardScaler(), Ridge(1)), Xtr, ytr, cv=5, scoring="r2").mean())
```

INTERVIEW CORNER

Questions you may be asked

Q: Explain the bias-variance trade-off. Simple models have high bias (systematic error) and low variance; complex models have low bias but high variance (sensitivity to the training sample). We choose complexity that minimises total error on new data.

Q: L1 vs L2 regularisation? L2 shrinks coefficients towards zero and handles correlated features; L1 can set coefficients exactly to zero, selecting features; Elastic Net combines both.

Q: How do you detect overfitting? Compare training and validation/test performance (large gap), use cross-validation, and inspect learning curves.

STATISTICS & MACHINE LEARNING · LESSON 13

Logistic Regression: Probabilities, Odds Ratios and Credit-Risk Scoring

Classify with probabilities. The sigmoid and log-odds, maximum likelihood and log loss, fitting logistic regression with statsmodels (inference, odds ratios) and scikit-learn (pipelines, regularisation), thresholds and decisions, multiclass logistic regression, and scoring new applicants in the Bank Loans case study.

Python class 16 · ML class 4 | statsmodels 0.1x · scikit-learn 1.7+ | Intermediate | 120 min

By the end of this lesson you can

- Explain why linear regression fails for binary targets and how the sigmoid fixes it
- Interpret coefficients as log-odds and odds ratios
- Fit, evaluate and regularise logistic regression in statsmodels and scikit-learn
- Choose a probability threshold from business costs
- Score new applicants and explain a prediction

The case: who will default?

A bank has 700 past borrowers with a known outcome (`default` = 1 if they failed to repay) and 150 new applicants with unknown outcomes. Features: age, education level (`ed` 1–5), years with current employer (`employ`), years at current address (`address`), income (thousands), debt-to-income ratio % (`debtinc`), credit-card debt and other debt (thousands).

Python

```
import pandas as pd, numpy as np
b = pd.read_csv("bank_loans.csv")
known, new = b[b.default.notna()].copy(), b[b.default.isna()].copy()
known["default"] = known.default.astype(int)
print(len(known), "labelled |", len(new), "new applicants | default rate",
round(known.default.mean(), 3))
print(known.groupby("default")[["age", "employ", "address", "income", "debtinc", "creddebt",
"othdebt"]].mean().round(2))
```

OUTPUT

```
700 labelled | 150 new applicants | default rate 0.321
           age  employ  address  income  debtinc  creddebt  othdebt
```

```
default
0        39.69    7.61     6.29    50.94     8.55      1.47     2.96
1        32.67    3.45     3.29    46.30    13.17      2.31     3.99
```

Defaulters are younger, have shorter employment and address histories, and much higher debt-to-income ratios.

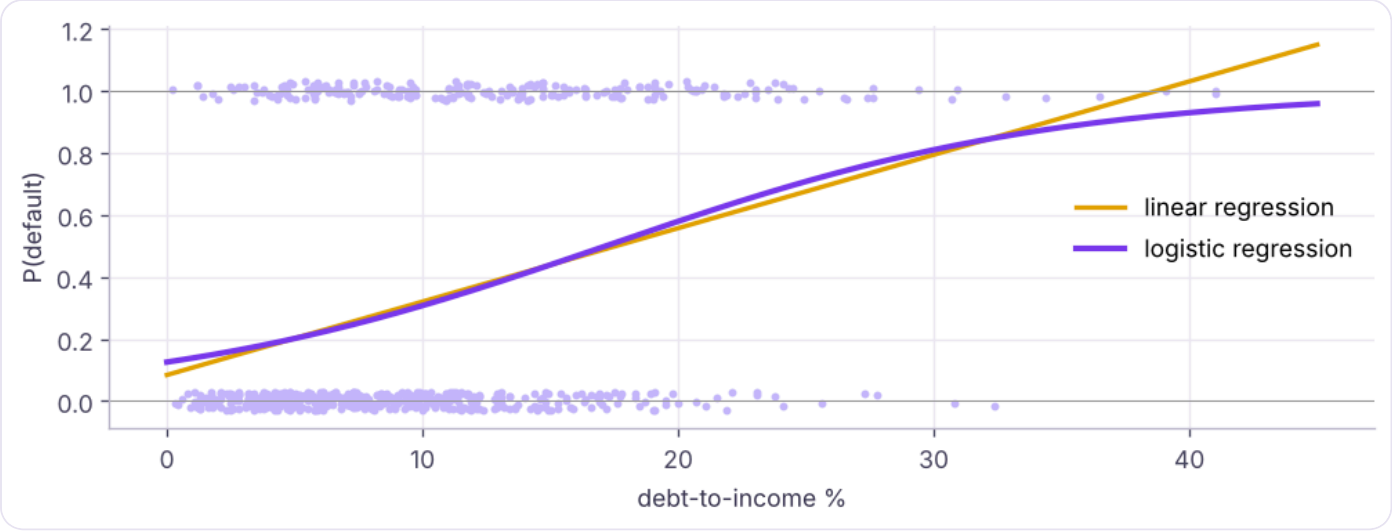
Why not linear regression?

A straight line predicts values below 0 and above 1, which are not probabilities. **Logistic regression** passes a linear score through the **sigmoid**:

$$p = P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \ \dots \ + \beta_k x_k)}}$$

Equivalently, the **log-odds** (logit) is linear in the features:

$$\log \frac{p}{1 - p} = \beta_0 + \beta_1 x_1 + \ \dots \ + \beta_k x_k$$



Linear regression vs logistic regression on default vs debt-to-income: the sigmoid keeps predictions between 0 and 1.

Fitting and interpreting with statsmodels

Logistic regression is fitted by **maximum likelihood**: choose coefficients that make the observed outcomes most probable, equivalently minimising **log loss** (cross-entropy).

Python

```
import statsmodels.formula.api as smf
m = smf.logit("default ~ age + ed + employ + address + income + debtinc + creddebt + othdebt",
data=known).fit(disp=0)
print(m.summary2().tables[1][["Coef.", "Std.Err.", "P>|z|"]].round(4))
print(f"pseudo R² (McFadden) {m.prsquared:.3f} | log-likelihood {m.llf:.1f}")
```

OUTPUT

```
            Coef.  Std.Err.   P>|z|
Intercept -0.5739    0.5772  0.3201
age       -0.0099    0.0137  0.4688
ed         0.0066    0.0989  0.9465
employ    -0.1586    0.0287  0.0000
address   -0.1021    0.0292  0.0005
income    -0.0014    0.0084  0.8687
debtinc    0.1067    0.0330  0.0012
creddebt   0.3246    0.1227  0.0081
othdebt   -0.0563    0.0579  0.3304
pseudo R² (McFadden) 0.244 | log-likelihood -332.3
```

Python

```
odds = pd.DataFrame({"odds_ratio": np.exp(m.params), "ci_low": np.exp(m.conf_int()[0]),
"ci_high": np.exp(m.conf_int()[1]), "p": m.pvalues}).drop("Intercept").round(3)
print(odds.sort_values("odds_ratio"))
```

OUTPUT

```
          odds_ratio  ci_low  ci_high      p
employ         0.853   0.807    0.903  0.000
address        0.903   0.853    0.956  0.000
othdebt        0.945   0.844    1.059  0.330
age            0.990   0.964    1.017  0.469
income         0.999   0.982    1.015  0.869
ed             1.007   0.829    1.222  0.946
debtinc        1.113   1.043    1.187  0.001
creddebt       1.384   1.088    1.760  0.008
```

Coefficient	Odds ratio e^β	Reading
β = 0	1	No association
β > 0	> 1	Higher odds of default per unit
β < 0	< 1	Lower odds of default per unit

For example, the odds ratio of about 0.85 for `employ` means each extra year with the current employer multiplies the odds of default by 0.85 (about 15% lower odds), holding the other variables constant. The odds ratio of about 1.38 for `creddebt` means each extra unit of card debt (₹1,000s) multiplies the odds by 1.38, and each extra point of debt-to-income raises the odds by about 11%. Age, education, income and other debt are not significant once the others are included.

> **NOTE** **Odds are not probabilities**
>
> Odds = p / (1 − p). A probability of 0.2 is odds of 0.25 ("1 to 4"). An odds ratio describes a *multiplicative* change in odds; the change in probability depends on where you start.

A reduced model keeps the significant predictors (a common credit-scoring practice: fewer, stable variables):

Python

```
m2 = smf.logit("default ~ employ + address + debtinc + creddebt", data=known).fit(disp=0)
print(m2.params.round(3).to_dict(), "| AIC full", round(m.aic, 1), "vs reduced", round(m2.aic,
1))
```

OUTPUT

```
{'Intercept': -0.829, 'employ': -0.175, 'address': -0.113, 'debtinc': 0.099, 'creddebt':
0.25} | AIC full 682.6 vs reduced 677.0
```

Lower AIC is better (it balances fit and complexity), so the four-variable model is preferred.

scikit-learn pipeline with regularisation

Python

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score, accuracy_score, log_loss, classification_report
feats = ["age", "ed", "employ", "address", "income", "debtinc", "creddebt", "othdebt"]
X, y = known[feats], known.default
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=7)
for C in (0.01, 0.1, 1, 100):
    pipe = make_pipeline(StandardScaler(), LogisticRegression(C=C, max_iter=1000))
    cv = cross_val_score(pipe, Xtr, ytr, cv=5, scoring="roc_auc").mean()
    print(f"C = {C:<5} 5-fold ROC-AUC {cv:.3f}")
clf = make_pipeline(StandardScaler(), LogisticRegression(C=1, max_iter=1000)).fit(Xtr, ytr)
proba = clf.predict_proba(Xte)[:, 1]
print(f"\ntest: ROC-AUC {roc_auc_score(yte, proba):.3f} | accuracy {accuracy_score(yte,
clf.predict(Xte)):.3f} | log loss {log_loss(yte, proba):.3f}")
print(classification_report(yte, clf.predict(Xte), target_names=["repaid", "default"],
digits=3))
```

OUTPUT

```
C = 0.01  5-fold ROC-AUC 0.791
C = 0.1   5-fold ROC-AUC 0.791
C = 1     5-fold ROC-AUC 0.793
C = 100   5-fold ROC-AUC 0.792

test: ROC-AUC 0.865 | accuracy 0.817 | log loss 0.431
              precision    recall  f1-score   support

      repaid      0.822     0.933     0.874       119
     default      0.800     0.571     0.667        56

    accuracy                          0.817       175
   macro avg      0.811     0.752     0.770       175
weighted avg      0.815     0.817     0.808       175
```

`C` is the inverse regularisation strength (lesson 12). Standardised coefficients from the pipeline are comparable across features:

Python

```
print(pd.Series(clf[-1].coef_[0], index=feats).round(3).sort_values().to_dict())
```

OUTPUT

```
{'employ': -0.875, 'address': -0.409, 'age': -0.148, 'othdebt': -0.025, 'income': -0.018,
'ed': 0.014, 'creddebt': 0.339, 'debtinc': 0.59}
```

Choosing the threshold

`predict` uses a 0.5 threshold, but the right cut-off depends on costs. Suppose a missed defaulter costs the bank ₹50,000 and rejecting a good customer loses ₹10,000 of profit.

Python

```
from sklearn.metrics import confusion_matrix
cost_fn, cost_fp = 50_000, 10_000
rows = []
for t in np.arange(0.1, 0.91, 0.1):
    tn, fp, fn, tp = confusion_matrix(yte, (proba ⩾ t).astype(int)).ravel()
    rows.append({"threshold": round(t, 1), "approved": tn + fn, "defaults_caught": tp,
"defaults_missed": fn, "good_rejected": fp, "cost_₹_lakh": (fn * cost_fn + fp * cost_fp) / 1e5})
th = pd.DataFrame(rows); print(th.to_string(index=False))
print("lowest-cost threshold:", th.loc[th["cost_₹_lakh"].idxmin(), "threshold"])
```

OUTPUT

```
 threshold  approved  defaults_caught  defaults_missed  good_rejected  cost_₹_lakh
       0.1        40               55                1             80          8.5
       0.2        72               51                5             52          7.7
       0.3        92               46               10             37          8.7
       0.4       118               41               15             16          9.1
       0.5       135               32               24              8         12.8
       0.6       152               19               37              4         18.9
       0.7       162               11               45              2         22.7
       0.8       167                8               48              0         24.0
       0.9       173                2               54              0         27.0
lowest-cost threshold: 0.2
```

When missing a defaulter is five times as costly as rejecting a good customer, the best threshold is well below 0.5. Lesson 14 formalises this with precision, recall and ROC curves.

Scoring new applicants

Python

```
final = make_pipeline(StandardScaler(), LogisticRegression(C=1, max_iter=1000)).fit(X, y)
# refit on all labelled data
new["p_default"] = final.predict_proba(new[feats])[:, 1].round(3)
best_t = th.loc[th["cost_₹_lakh"].idxmin(), "threshold"]
new["decision"] = np.where(new.p_default ⩾ best_t, "decline / review", "approve")
print(new.decision.value_counts().to_dict())
print(new.sort_values("p_default", ascending=False)[["customer_id", "age", "employ", "debtinc",
"creddebt", "p_default", "decision"]].head(5).to_string(index=False))
```

OUTPUT

```
{'decline / review': 84, 'approve': 66}
customer_id  age  employ  debtinc  creddebt  p_default         decision
      L0782   36       0     32.9      4.73      0.961 decline / review
      L0774   42       8     39.1      7.26      0.932 decline / review
      L0736   32       5     28.8      7.60      0.898 decline / review
      L0702   41       6     27.9      4.55      0.883 decline / review
      L0753   36       6     29.7      6.09      0.876 decline / review
```

Explaining one decision

Python

```python
one = new.sort_values("p_default", ascending=False).iloc[0]
z = (one[feats].astype(float) - final[0].mean_) / final[0].scale_
contrib = pd.Series(z.values * final[-1].coef_[0], index=feats).sort_values(ascending=False)
print(f"{one.customer_id}: P(default) {one.p_default:.2f}; log-odds contributions:",
contrib.round(2).to_dict())
```

OUTPUT

```
L0782: P(default) 0.96; log-odds contributions: {'debtinc': 2.37, 'employ': 0.97,
'creddebt': 0.96, 'address': 0.13, 'income': 0.03, 'age': 0.02, 'ed': -0.01, 'othdebt':
-0.17}
```

Each contribution is a coefficient × the applicant's standardised value: positive values push towards default. This additive explanation is a big reason banks like logistic regression (and regulators require explanations of adverse decisions).

Multiclass logistic regression

With more than two classes, scikit-learn fits a **multinomial** (softmax) model automatically: one score per class, converted to probabilities that sum to 1.

Python

```python
from sklearn.datasets import load_iris
iris = load_iris(as_frame=True)
mc = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(iris.data,
iris.target)
print(mc.predict_proba(iris.data.iloc[[0, 60, 120]]).round(3), "| classes:",
iris.target_names.tolist())
```

OUTPUT